Universidad Mariano Gálvez de Guatemala

Curso: Desarrollo Web

Sistema Distribuidora

Documentación técnica y manual de usuario

Catedrático: **Elmer Josué Esteban Chilel**

Integrantes: Oswaldo Guanche Juárez
Carlos Daniel Rojas
Kimberly Alejandra Yol Ixpata

Fecha: Octubre de 2026

Tienda: https://distribuidora-tienda.onrender.com · Panel: https://distribuidora-panel.onrender.com
Repositorio: https://github.com/kredlml/distribuidora-backend

Índice

1. **Presentación del proyecto**
   - Datos académicos
   - Sistema publicado
   - Capturas
   - Funcionalidades principales
   - Tecnologías
   - Estructura del repositorio
   - Inicio rápido en local
   - Comandos útiles
   - Documentación
   - Licencia y uso académico
2. **Arquitectura del sistema**
   1. Visión general
   2. Diagrama general
   3. Capas del backend
   4. Organización de los frontends
   5. Flujo de una compra con pago por tarjeta
   6. Integración con servicios externos
   7. Tareas periódicas
   8. Migraciones al arrancar
   9. Decisiones técnicas clave
   10. Documentos relacionados
3. **Modelo de datos**
   1. Visión general
   2. Diagrama entidad-relación
   3. Convenciones de tipos
   4. Tablas del modelo
   5. Tablas creadas por migraciones sin modelo
   6. Llaves foráneas y acciones referenciales
   7. Índices únicos e índices adicionales
   8. Cálculo del total del pedido
   9. Copia del cupón en el pedido
   10. Evolución del esquema (migraciones)

4. Roles y permisos
1. Principios generales
2. Roles del sistema
3. Autenticación y controles de rol
4. Alcance por sucursal
5. Gestión de empleados
6. Matriz de permisos
7. Menú del panel por rol
8. Invalidación de sesiones (token_version)
9. Códigos de respuesta de autorización

5. Reglas de negocio y flujos
1. Parámetros configurables
2. Pedidos
3. Stock por sucursal
4. Pagos
5. Vencimiento de pedidos pendientes
6. Devoluciones y cambios
7. Reembolsos
8. Cupones de descuento
9. Envío
10. IVA incluido
11. Trazabilidad
12. Correos a la clienta

6. Referencia de la API REST
1. URL base
2. Recorrido de una petición
3. Autenticación
4. Formato de errores
5. Paginación y búsqueda
6. Límite de intentos
7. Convenciones de las tablas de endpoints
8. Endpoints por módulo
9. Ejemplos de petición y respuesta
10. Colección de Postman
Documentos relacionados

7. Manual de usuario

Conceptos básicos
A1. Crear una cuenta
A2. Activar la cuenta con el código de caja
A3. Olvidé mi contraseña
A4. Catálogo y ficha de la prenda
A5. Carrito
A6. Finalizar compra: entrega y cupón
A7. Pagar con tarjeta
A8. Seguimiento del pedido y plazo de pago
A9. Solicitar un cambio o una devolución
A10. Seguimiento de la solicitud y reembolsos
A11. Perfil y cambio de contraseña
B1. Puestos y secciones visibles
B2. Iniciar sesión y cambiar la contraseña
B3. Registrar una venta de mostrador
B4. Cobrar y confirmar pagos
B5. Avanzar el estado de un pedido
B6. Procesar una devolución o un cambio
B7. Clientes y código web
B8. Catálogo, tipos de prenda y fotos
B9. Inventario y trazabilidad
B10. Sucursales y ajustes de stock
B11. Proveedores
B12. Empleados
B13. Promociones y cupones
B14. Reportes de ventas y PDF
B15. Estado del sistema
Preguntas frecuentes

8. Seguridad
1. Principios generales
2. Contraseñas
3. Sesiones con JWT
4. Autorización y alcance por sucursal
5. Validación de entradas y consultas a la base de datos
6. Límite de intentos
7. Cuentas de clientas: inicio de sesión, activación y restablecimiento
8. CORS y cuerpo de las peticiones
9. Pagos con Stripe
10. Fotos en Cloudinary
11. Concurrencia: transacciones y bloqueos
12. Idempotencia de reembolsos
13. Correos electrónicos
14. Manejo y registro de errores
15. Auditoría
16. Secretos y configuración
17. Dependencias con avisos de seguridad pendientes
18. Limitaciones conocidas
19. Resumen: riesgo, medida y archivo
20. Verificación en las pruebas automatizadas

9. Pruebas
1. Objetivo y alcance
2. Estrategia de pruebas
3. Requisitos
4. Ejecución
5. Resultados y artefactos
6. Suites de API
7. Suites de navegador
8. Último resultado
9. Revisión adversarial de código
10. Cómo agregar una suite
11. Documentos relacionados

10. Despliegue y operación
1. Arquitectura de despliegue
2. Requisitos previos
3. Publicación en Render paso a paso
4. Variables de entorno
5. Limitaciones del plan gratuito
6. Migraciones de la base de datos
7. Respaldos y restauración
8. Monitoreo
9. Tareas periódicas
10. Solución de problemas frecuentes
11. Documentos relacionados

Distribuidora

Sistema web para una distribuidora de ropa para dama con varias sucursales. Integra una API REST, un panel de empleados para la operación diaria (ventas en mostrador, inventario, pedidos, devoluciones, reportes) y una tienda en línea donde las clientas compran, pagan con tarjeta y solicitan cambios o devoluciones. Este documento es la portada del repositorio: resume el sistema, indica cómo ejecutarlo en un equipo local y enlaza la documentación técnica completa ubicada en `docs/`.

Contenido

- Datos académicos
- Sistema publicado
- Capturas
- Funcionalidades principales
- Tecnologías
- Estructura del repositorio
- Inicio rápido en local
- Comandos útiles
- Documentación
- Licencia y uso académico

Datos académicos

Campo	Valor
Universidad	Universidad Mariano Gálvez de Guatemala
Curso	Desarrollo Web
Catedrático	Elmer Josué Esteban Chilel
Integrantes	Oswaldo Guanche Juárez · Carlos Daniel Rojas · Kimberly Alejandra Yol Ixpata
Fecha	Octubre de 2026

Sistema publicado

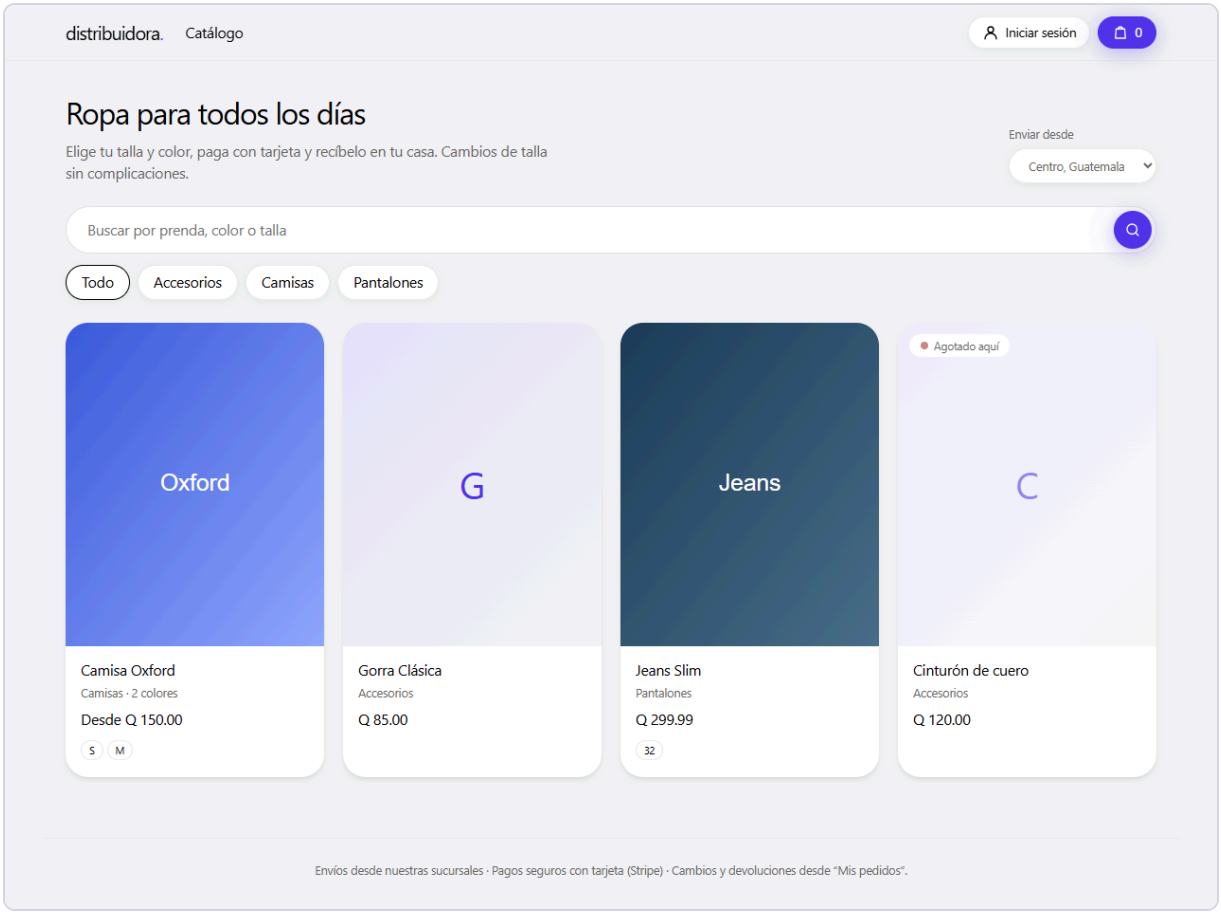
Las tres partes del sistema están desplegadas en Render:

Parte	Dirección
Tienda en línea (clientas)	https://distribuidora-tienda.onrender.com
Panel de empleados	https://distribuidora-panel.onrender.com
API REST	https://distribuidora-api-r6rw.onrender.com

Parte	Dirección
Verificación de salud de la API	https://distribuidora-api-r6rw.onrender.com/api/salud

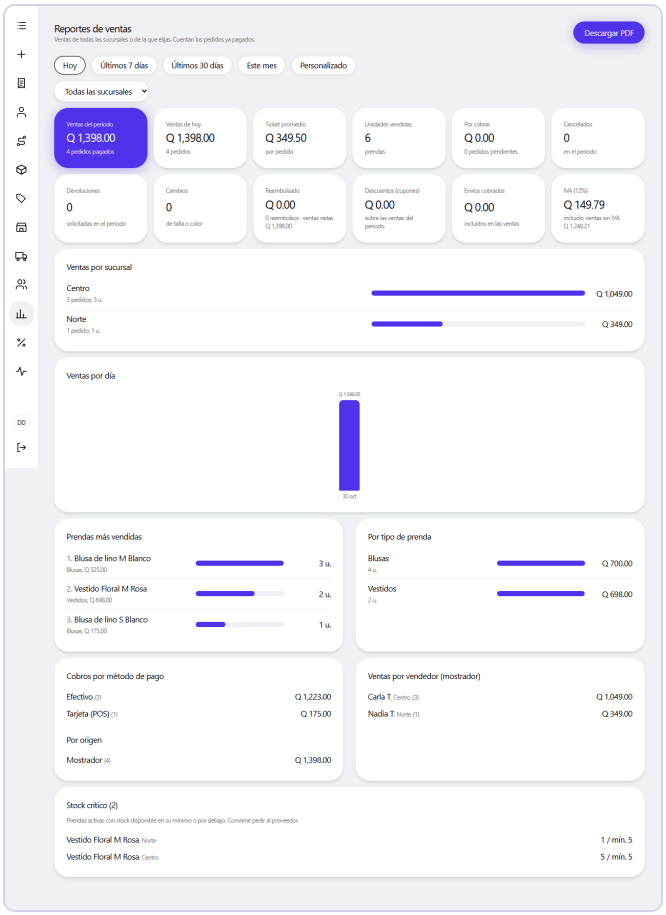
El endpoint `/api/salud` es público e informa si la API y la base de datos responden; el detalle del sistema y los últimos errores registrados (`/api/salud/detalle` , `/api/salud/errores`) están reservados al puesto DISTRITAL.

Capturas



Catálogo de la tienda en línea con buscador, filtros por tipo de prenda y selector de sucursal de envío

Catálogo de la tienda en línea: búsqueda por prenda, color o talla, filtros por tipo de prenda, selector de la sucursal desde la que se envía y aviso de prendas agotadas en esa sucursal.



Pantalla de reportes de ventas del panel de empleados

Reportes de ventas del panel: indicadores del periodo (ventas, ticket promedio, devoluciones, reembolsos, descuentos, envíos e IVA incluido), ventas por sucursal, por día, por prenda, por tipo de prenda, por método de pago y por vendedor, stock crítico y descarga en PDF.

Funcionalidades principales

- **Multi-sucursal con roles.** Cuatro puestos de empleado (DISTRITAL, ADMIN, CAJERO, BODEGUERO). El DISTRITAL opera sobre todas las sucursales; los demás puestos trabajan dentro de la sucursal asignada. Las clientas tienen una sesión independiente en la tienda.
- **Venta en tienda en línea y en mostrador.** La clienta compra desde la tienda eligiendo la sucursal de envío; el cajero registra ventas presenciales desde el panel y cobra en efectivo o con tarjeta (POS).
- **Pagos con Stripe.** Cobro con tarjeta en la tienda mediante Stripe.js, confirmación del cobro desde el backend y webhook firmado (`/api/webhook`). La clave secreta solo existe en el backend.
- **Devoluciones y cambios con logística inversa.** Una sola solicitud por pedido dentro del plazo configurado (24 horas desde la entrega por defecto). La solicitud recorre una máquina de estados (SOLICITADA, APROBADA, RECIBIDA, EN_REVISION, ACEPTADA y cierre como REINTEGRADA o CAMBIADA, o rechazo) que impide saltarse pasos; los movimientos de inventario y la bitácora de auditoría dejan la trazabilidad de cada operación.

- **Reembolsos.** Registro de reembolsos por Stripe o por otros métodos, con estados PENDIENTE, COMPLETADO y FALLIDO; los reembolsos a tarjeta pendientes se reintentan de forma periódica.
- **Cupones, envío e IVA.** Cupones por porcentaje o por monto, costo de envío a domicilio con umbral de envío gratis, e IVA incluido en los precios y desglosado en pedidos y reportes (valores configurables en `.env`).
- **Reportes con PDF.** Indicadores y desgloses de ventas por periodo y sucursal, exportables a PDF desde el panel.
- **Fotos de prendas en Cloudinary.** Carga y eliminación de fotografías de prendas almacenadas en Cloudinary.
- **Correos a clientas.** Avisos de pedido pagado, enviado, entregado y cancelado, de devoluciones y reembolsos, y recuperación de contraseña, enviados por SMTP (nodemailer).
- **Vencimiento de pedidos.** Los pedidos pendientes de pago se cancelan automáticamente al vencer su plazo (24 horas por defecto); la revisión se ejecuta cada cierto número de minutos.
- **Monitoreo.** Identificador por petición (cabecera `X-Request-Id`), registro de toda respuesta 5xx en la tabla `ErrorServidor`, manejador de errores en JSON sin exponer la pila y envío opcional a Sentry.
- **Migraciones y respaldos.** Esquema versionado con migraciones (Umzug) que el servidor aplica al arrancar, y scripts de respaldo y restauración con `pg_dump` / `pg_restore`.

Tecnologías

Versiones tomadas de los archivos `package.json` de cada parte.

Parte	Tecnología	Versión
API (raíz)	Express	^5.2.1
	Sequelize (ORM)	^6.37.8
	pg / pg-hstore (controlador PostgreSQL)	^8.22.0 / ^2.3.4
	Umzug (migraciones)	3.8.3
	jsonwebtoken (sesiones JWT)	^9.0.3
	bcryptjs (contraseñas)	^3.0.3
	stripe (SDK de pagos)	^22.6.0
	nodemailer (correos)	10.1.0
	cors · body-parser · dotenv	^2.8.6 · ^2.3.0 · ^17.4.2
	playwright-core (pruebas de interfaz, desarrollo)	1.63.0
Panel (`frontend/`)	React · React DOM	^19.3.1
	React Router DOM	^7.18.4
	jsPDF · jspdf-autotable (reportes PDF)	4.2.1 · 5.0.8
	Vite · @vitejs/plugin-react	^8.3.4 · ^6.1.2
Tienda (`tienda/`)	React · React DOM	^19.3.1

Parte	Tecnología	Versión
	React Router DOM	^7.18.4
	@stripe/stripe-js · @stripe/react-stripe-js	9.17.0 · 6.12.0
	Vite · @vitejs/plugin-react	^8.3.4 · ^6.1.2
Servicios externos	PostgreSQL en Neon (con pooler PgBouncer), Stripe, Cloudinary, SMTP	—
Hospedaje	Render (API, panel y tienda)	—

Estructura del repositorio

```
distribuidora-backend/
├── server.js               Punto de entrada de la API (Express): CORS, rutas, monitoreo y
migraciones al arrancar
├── app/
│   ├── config/             Configuración de base de datos, JWT, Stripe y Cloudinary
│   ├── models/             Modelos Sequelize (sucursal, empleado, cliente, producto, inventario,
pedido, pago, devolución, reembolso, cupón, auditoría…)
│   ├── controllers/        Lógica de cada recurso de la API
│   ├── routes/             Definición de endpoints bajo /api
│   ├── middlewares/        Autenticación y roles (authJwt), límite de intentos, validación de ids
│   ├── utils/              Reglas compartidas: totales, precios, cupones, plazos, estados de
devolución, reembolsos, vencimientos, correo, monitoreo, migrador
│   └── migraciones/        Migraciones versionadas del esquema (0001 a 0005)
├── frontend/               Panel de empleados (React + Vite, puerto 8081)
├── tienda/                 Tienda en línea para clientas (React + Vite, puerto 8082)
├── tests/
│   ├── correr.js           Ejecutor de pruebas sobre un PostgreSQL desechable
│   ├── api/                Pruebas de la API
│   ├── navegador/          Pruebas de interfaz con Chrome/Edge (playwright-core)
│   └── soporte/            Arranque de PostgreSQL de prueba, ubicación de Chrome, Cloudinary
simulado
├── postman/                Colección de Postman y su generador
├── scripts/                migrar.js, respaldo.js, restaurar.js
├── docs/                   Documentación técnica y capturas (docs/img/)
└── .env.example            Plantilla de variables de entorno (sin valores reales)
```

Inicio rápido en local

Requisitos

- Node.js y npm. Express 5 (API) requiere Node.js 18 o superior y Vite 8 (panel y tienda) requiere Node.js 20.19 o superior dentro de la versión 20, o 22.12 o superior.
- Una base de datos PostgreSQL accesible (local o en Neon).

- Para las pruebas: binarios de PostgreSQL instalados (`pg_ctl` , `initdb` ; se detectan en `C:/Program Files/PostgreSQL` o en el PATH, o se indican con `PG_BIN`) y, para las pruebas de interfaz, Chrome o Edge (ruta configurable con `CHROME_PATH`).
- Para respaldos: `pg_dump` y `pg_restore` .
- Opcional: cuentas de Stripe (modo prueba), Cloudinary y un servidor SMTP.

1. Instalar dependencias

```
npm install                # API (raíz)
cd frontend && npm install  # panel de empleados
cd ../tienda && npm install # tienda en línea
```

2. Configurar variables de entorno

```
cp .env.example .env.development           # API: base de datos, JWT_SECRET, Stripe, Cloudinary,
SMTP
cp frontend/.env.example frontend/.env     # VITE_API_URL
cp tienda/.env.example tienda/.env         # VITE_API_URL y VITE_STRIPE_PUBLISHABLE_KEY
```

La API carga `.env.development` , o `.env.production` cuando `NODE_ENV=production` . Ningún archivo `.env` real debe subirse al repositorio (están excluidos en `.gitignore`). Aspectos relevantes de la plantilla:

- `CORS_ORIGIN` admite por defecto `http://localhost:8081` y `http://localhost:8082` .
- Con `STRIPE_SECRET_KEY=sk_test_clave_de_prueba_universidad` el backend simula los cobros (pago aprobado al instante, sin llamar a Stripe). Con claves reales de prueba se usa la tarjeta `4242 4242 4242 4242` , cualquier fecha futura y cualquier CVC.
- Sin `SMTP_HOST` no se envían correos; solo se muestran en consola.
- Para una base local sin SSL se define `DB_SSL=false` .

3. Ejecutar

```
npm start                   # API en http://localhost:8080 (aplica las migraciones pendientes
antes de escuchar)
cd frontend && npm run dev   # panel en http://localhost:8081
cd tienda && npm run dev     # tienda en http://localhost:8082
```

Con la base vacía, el primer empleado se registra sin sesión (`POST /api/auth/registro`) y queda automáticamente con el puesto DISTRITAL; a partir de ese momento solo un ADMIN o el DISTRITAL autenticados registran empleados desde la pantalla Empleados del panel.

Comandos útiles

Ejecutados desde la raíz del repositorio:

Comando	Descripción
`npm start`	Inicia la API (`node server.js`).
`npm test`	Pruebas de la API sobre un PostgreSQL desechable (`tests/.pgdata` , puerto 54329); nunca usa la base de `.env.development` .
`npm run test:navegador`	Pruebas de interfaz del panel y la tienda con Chrome o Edge.
`npm run test:todo`	Todas las pruebas (API e interfaz).
`node tests/correr.js api/ventas`	Solo las suites cuyo nombre contiene el filtro indicado.
`npm run migrar`	Aplica las migraciones pendientes (el servidor también lo hace al arrancar).
`npm run migrar:estado`	Lista migraciones aplicadas y pendientes y avisa si un modelo tiene columnas sin migración.
`npm run respaldo`	Respaldo con `pg_dump` en `respaldos/<base>-AAAAMMDD-HHMMSS.dump` ; conserva los últimos `RESPALDOS_MAX` (14 por defecto).
`npm run restaurar -- respaldos/<archivo>.dump --base <nombre>`	Restaura un respaldo, reemplazando el contenido de la base; antes crea un respaldo de seguridad. El servidor debe estar detenido.

En `frontend/` y `tienda/` están disponibles `npm run dev` , `npm run build` y `npm run preview` .

Documentación

Documento	Contenido
01 · Arquitectura	Componentes del sistema, servicios externos y capas del backend.
02 · Modelo de datos	Diagrama entidad-relación y descripción de cada tabla.
03 · Roles y permisos	Puestos de empleado, clienta y alcance por sucursal.
04 · Reglas de negocio	Pedidos, pagos, devoluciones, reembolsos, cupones, envío, IVA y vencimientos.
05 · API	Endpoints por módulo, autenticación, errores y colección de Postman.
06 · Manual de usuario	Guía paso a paso por rol en el panel y en la tienda.
07 · Seguridad	Contraseñas, sesiones, límite de intentos y protección de pagos.
08 · Pruebas	Estrategia, ejecución y cobertura de las pruebas.
09 · Despliegue y operación	Publicación en Render, variables, migraciones, respaldos y monitoreo.

Licencia y uso académico

Este proyecto fue desarrollado como trabajo académico del curso Desarrollo Web de la Universidad Mariano Gálvez de Guatemala. El archivo `package.json` declara la licencia ISC. Los datos que aparecen en las capturas son de prueba, y los servicios externos (Stripe, Cloudinary, SMTP) se utilizan en modo de prueba o con credenciales propias que no forman parte del repositorio. Su uso, copia o adaptación fuera del ámbito académico requiere la autorización de los integrantes del equipo.

Arquitectura del sistema

Este documento describe la arquitectura del sistema Distribuidora: los componentes que lo forman, la manera en que se comunican entre sí y con los servicios externos, la organización interna del backend y de las dos aplicaciones web, el flujo completo de una compra en línea con pago por tarjeta, las tareas que el servidor ejecuta de forma periódica, el mecanismo de migraciones de la base de datos y las decisiones técnicas que dan forma al conjunto. Todo lo descrito corresponde al código del repositorio.

Contenido

1. Visión general
2. Diagrama general
3. Capas del backend
4. Organización de los frontends
5. Flujo de una compra con pago por tarjeta
6. Integración con servicios externos
7. Tareas periódicas
8. Migraciones al arrancar
9. Decisiones técnicas clave
10. Documentos relacionados

---

1. Visión general

El sistema sigue una arquitectura cliente-servidor de tres niveles: dos aplicaciones web de una sola página (SPA) consumen una única API REST, y la API es el único componente que accede a la base de datos y a los servicios externos con credenciales secretas.

Componente	Carpeta	Tecnología	Usuarios	Publicación en Render
API REST	raíz del repositorio (`server.js` , `app/`)	Node.js, Express 5, Sequelize 6, PostgreSQL	Ambas aplicaciones web y Stripe (webhook)	https://distribuidora-api-r6rw.onrender.com (salud: `/api/salud`)
Panel de empleados	`frontend/`	React 19, Vite, React Router	Empleados con puesto DISTRITAL, ADMIN, CAJERO o BODEGUERO	https://distribuidora-panel.onrender.com
Tienda en línea	`tienda/`	React 19, Vite, React Router, Stripe.js	Clientas	https://distribuidora-tienda.onrender.com
Base de datos	—	PostgreSQL en Neon, con pooler PgBouncer	Solo la API	Servicio administrado (Neon)

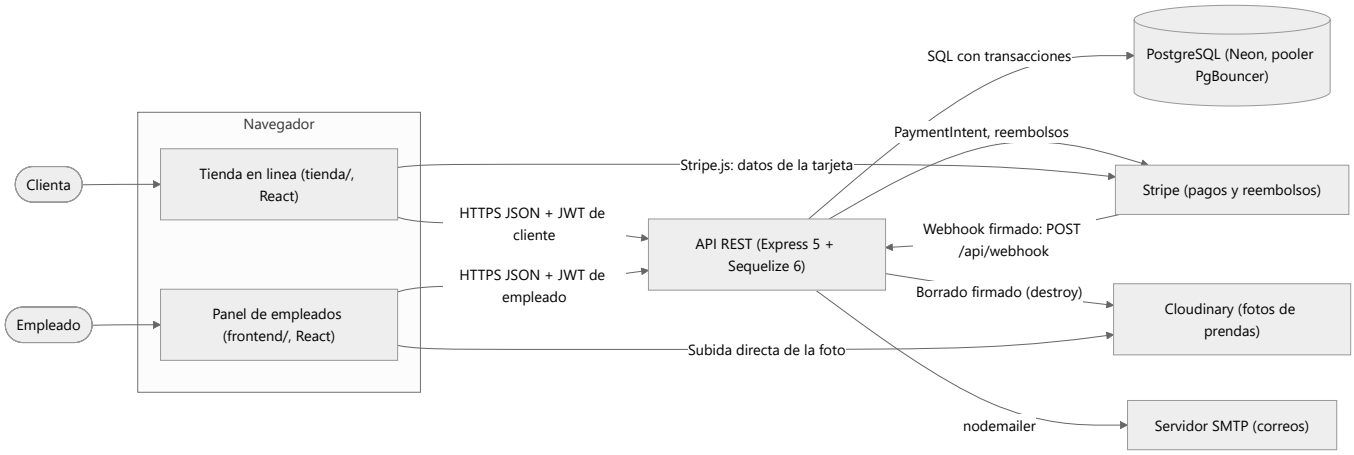
Componente	Carpeta	Tecnología	Usuarios	Publicación en Render
Pruebas	`tests/`	Ejecutor propio (`tests/correr.js`), `playwright-core` para el navegador	Equipo de desarrollo	—
Colección de API	`postman/`	Postman	Equipo de desarrollo	—
Scripts de operación	`scripts/`	Node.js	Equipo de desarrollo	—

Servicios externos que utiliza la API:

Servicio	Uso en el sistema	Variables de entorno (ver `.env.example`)
Stripe	Cobro con tarjeta en la tienda (PaymentIntent) y reembolsos a tarjeta	`STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` (API); `VITE_STRIPE_PUBLISHABLE_KEY` (tienda, en `tienda/.env.example`)
Cloudinary	Almacenamiento de las fotos de las prendas	`CLOUDINARY_CLOUD_NAME` , `CLOUDINARY_API_KEY` , `CLOUDINARY_API_SECRET`
Servidor SMTP (nodemailer)	Correos a las clientas: pago recibido, envío, entrega, cancelación, devoluciones, reembolsos y restablecimiento de contraseña	`SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASS` , `CORREO_REMITENTE` , `CORREO_MODO` , `TIENDA_URL`
Neon	PostgreSQL en la nube	`DB_HOST` , `DB_PORT` , `DB_USER` , `DB_PASSWORD` , `DB_NAME` , `DB_SSL` , `DB_POOL_MAX` , `DB_HOST_DIRECTO`
Sentry (opcional)	Envío de errores del servidor, solo si se define `SENTRY_DSN`	`SENTRY_DSN`

2. Diagrama general

El diagrama muestra quién se comunica con quién. Las flechas indican el sentido en que se inicia cada comunicación.



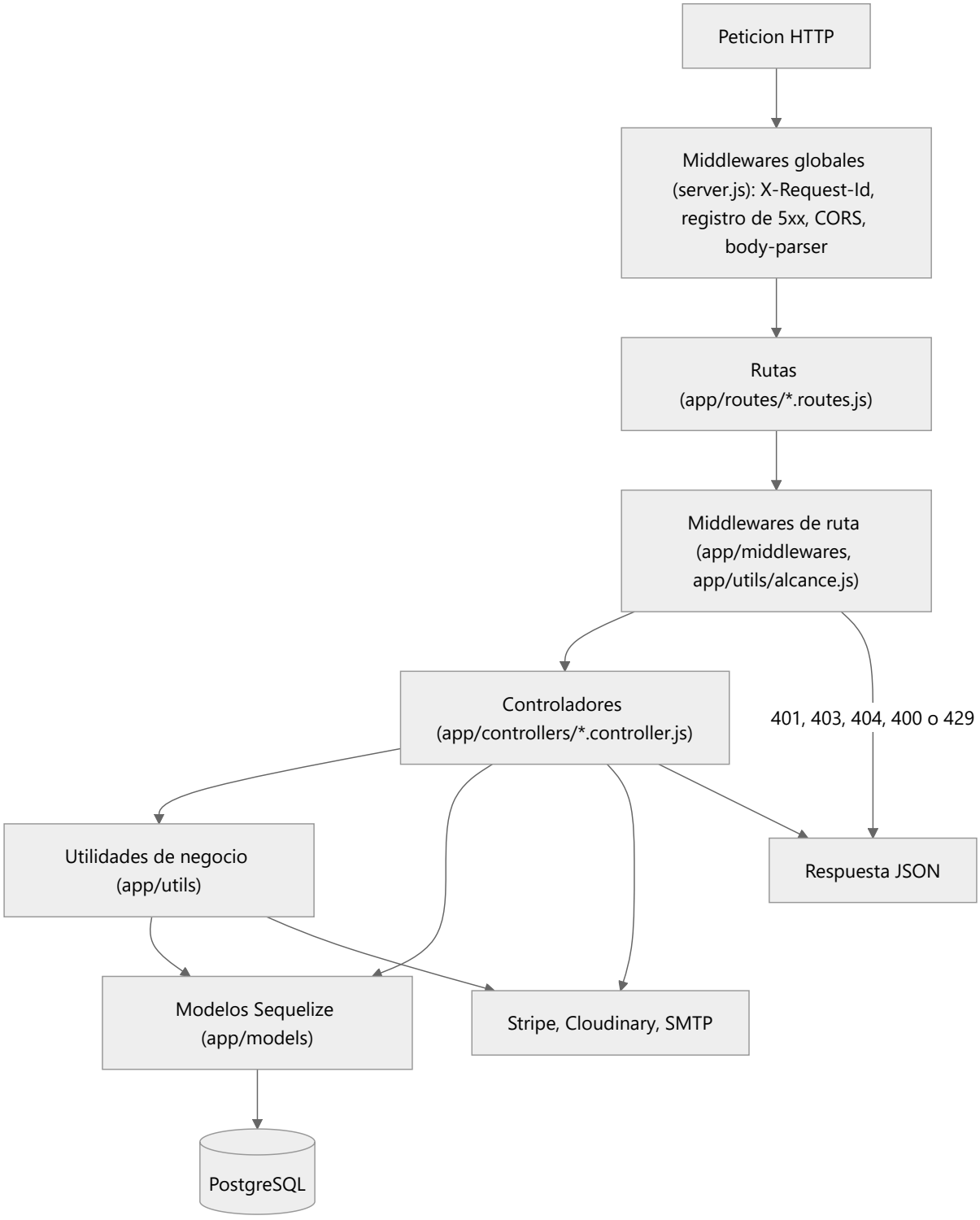
Aspectos que conviene destacar del diagrama:

- **Ninguna aplicación web accede a la base de datos.** Toda lectura y escritura pasa por la API, que valida el token, el puesto y la sucursal del usuario en cada petición.
- **Los datos de la tarjeta nunca pasan por la API.** La tienda los entrega directamente a Stripe mediante Stripe.js (`CardElement`); la API solo crea el PaymentIntent y recibe su resultado.
- **Las fotos tampoco pasan por la API.** La API entrega una firma de subida y el navegador envía el archivo directamente a Cloudinary (ver la sección 6).
- **Stripe se comunica con la API por dos vías:** las respuestas a las llamadas que hace la API y las notificaciones asíncronas del webhook (`/api/webhook`), cuya firma se valida con `STRIPE_WEBHOOK_SECRET` .
- **CORS:** la API solo acepta peticiones de navegador desde los orígenes listados en `CORS_ORIGIN` (por defecto `http://localhost:8081` para el panel y `http://localhost:8082` para la tienda) y expone la cabecera `X-Request-Id` .

3. Capas del backend

3.1 Recorrido de una petición

Cada petición atraviesa las mismas capas, en este orden:



3.2 Arranque del servidor (`server.js`)

El archivo `server.js` configura la aplicación en un orden deliberado:

Paso	Acción	Motivo
1	Carga `.env.production` o `.env.development` según `NODE_ENV`.	Separar la configuración de cada ambiente.
2	Inicia Sentry si existe `SENTRY_DSN` (`app/utils/monitoreo.js`).	Debe iniciarse antes que Express.
3	`app.set("trust proxy", 1)`.	Detrás del proxy de Render, `req.ip` debe ser la IP real del cliente; la usa el límite de intentos.
4	Middlewares `idDePeticion` y `registrarRespuestas5xx`.	Cada respuesta lleva `X-Request-Id` y toda respuesta 5xx queda registrada en la tabla `ErrorServidor`.
5	CORS con los orígenes de `CORS_ORIGIN`.	Solo el panel y la tienda pueden llamar a la API desde el navegador.
6	Ruta del webhook de Stripe (`webhook.routes.js`).	Se registra **antes** de `bodyParser.json` porque la firma se valida sobre el cuerpo crudo (`express.raw`).
7	`bodyParser.json` y `bodyParser.urlencoded` con `inflate: false`.	No se aceptan cuerpos comprimidos (mitigación de GHSA-27r3-jxmh-p828).
8	Registro de las rutas de cada módulo.	Ver la tabla de la sección 3.3.
9	`noEncontrado` para `/api` y `manejadorDeErrores`.	Toda ruta inexistente o error no controlado responde JSON, nunca la pila de llamadas.
10	`prepararBase(db)` y luego `app.listen(PORT)` (8080 por defecto).	El servidor solo atiende con la base al día (ver la sección 8).
11	`iniciarVencimientos()`.	Arranca las tareas periódicas (ver la sección 7).

3.3 Rutas

Cada archivo de `app/routes` recibe la aplicación de Express, crea un `Router` y lo monta bajo un prefijo `/api/...`. Los prefijos registrados son:

Prefijo	Archivo de rutas	Módulo
`/api/salud`	`salud.routes.js`	Estado de la API y de la base; detalle y errores para el DISTRITAL
`/api/auth`	`auth.routes.js`	Inicio de sesión y registro de empleados
`/api/sucursales`, `/api/clientes`, `/api/proveedores`, `/api/categorias`, `/api/empleados`, `/api/productos`	archivos homónimos	Catálogos y administración
`/api/inventario`	`inventario.routes.js`	Existencias por sucursal y movimientos

Prefijo	Archivo de rutas	Módulo
`/api/pedidos`, `/api/detalles` y `/api/pagos`	`pedido.routes.js`, `detalle_pedido.routes.js`, `pago.routes.js`	Ventas de mostrador, líneas y cobros
`/api/devoluciones`, `/api/reembolsos`	`devolucion.routes.js`, `reembolso.routes.js`	Logística inversa y devolución de dinero
`/api/cupones`	`cupon.routes.js`	Cupones de descuento
`/api/reportes`, `/api/auditoria`	`reporte.routes.js`, `auditoria.routes.js`	Reportes de ventas y bitácora
`/api/imagenes`	`imagen.routes.js`	Firma de subida y descarte de fotos en Cloudinary
`/api/tienda`	`tienda.routes.js`	Cuenta, catálogo, pedidos, pagos y devoluciones de la clienta
`/api/webhook`	`webhook.routes.js`	Notificaciones de Stripe

El detalle de cada endpoint se encuentra en API.

3.4 Middlewares

Los middlewares se encadenan por ruta, por ejemplo `[verificarToken, isCajero]` o `[verificarToken, deMiSucursal]`.

Middleware	Archivo	Función
`verificarToken`	`app/middlewares/authJwt.js`	Valida el JWT de un **empleado**, rechaza los tokens de clientas, comprueba que la cuenta exista y esté activa, y que `token_version` coincida (un cambio de contraseña o una desactivación invalida las sesiones anteriores). Deja el empleado en `req.empleado`.
`verificarCliente`	`app/middlewares/authJwt.js`	Equivalente para la tienda: exige un token de tipo `CLIENTE` de una cuenta activa y con `token_version` vigente. Deja `req.clienteId`.
`isDistrital`, `isAdmin`, `isCajero`, `isBodeguero`	`app/middlewares/authJwt.js`	Controles de rol construidos con `exigirPuesto`. El puesto se lee de la base en cada petición, de modo que un cambio de puesto aplica de inmediato. `isAdmin` admite ADMIN y DISTRITAL; `isCajero` admite CAJERO, ADMIN y DISTRITAL; `isBodeguero` admite BODEGUERO, ADMIN y DISTRITAL.
`permitirRegistro`	`app/middlewares/authJwt.js`	Con la tabla de empleados vacía permite registrar al primer empleado sin token (queda como DISTRITAL, sin sucursal); después exige un empleado autenticado con puesto ADMIN o DISTRITAL (`isAdmin`).
`exigirAlcance(cargador)`	`app/utils/alcance.js`	Alcance por sucursal: si el registro solicitado pertenece a otra sucursal responde 404 ("No se encontró el registro en tu sucursal"). El DISTRITAL ve todas las sucursales; ADMIN, CAJERO y BODEGUERO solo la propia.

Middleware	Archivo	Función
`validarId`	`app/middlewares/validarId.js`	Se registra con `router.param("id", ...)`. Rechaza con 400 los identificadores no numéricos o fuera del rango de `INTEGER` de PostgreSQL antes de consultar la base.
`limitarIntentos({ max, ventanaMs })`	`app/middlewares/limitador.js`	Límite de intentos por IP y ruta (ventana de 15 minutos por defecto). Los contadores se guardan en la tabla `IntentoLimite`, de modo que sobreviven a reinicios y se comparten entre instancias; si la base no responde se usa un contador en memoria. Al superarse responde 429 con la cabecera `Retry-After`.

El detalle de qué puede hacer cada puesto se encuentra en Roles y permisos, y las medidas de seguridad asociadas en Seguridad.

3.5 Controladores

Los controladores (`app/controllers/*.controller.js`) reciben la petición ya autenticada, validan los datos de entrada, abren la transacción cuando la operación modifica varias tablas, delegan las reglas compartidas en `app/utils` y construyen la respuesta. Existe un controlador por módulo; la tienda tiene dos propios: `tienda_auth.controller.js` (cuentas de clientas) y `tienda.controller.js` (catálogo, pedidos, pagos y devoluciones), que reutilizan la misma lógica de negocio que el mostrador. El webhook de Stripe se atiende en `webhook.controller.js` y el estado del sistema en `salud.controller.js`.

3.6 Utilidades de negocio (`app/utils`)

Las reglas que comparten varios controladores viven en módulos de utilidades, de modo que cada regla tiene una sola implementación:

Módulo	Responsabilidad
`facturacion.js`	Núcleo de ventas: `crearPedido` (pedido PENDIENTE, líneas con precio del catálogo y descuento de stock por FIFO con bloqueo), `cancelarPedido`, `descontarStockVenta`, `reintegrarStockVenta`, `totalPagado`, `procesarPedidoSiEstaPagado` (paso automático a PROCESADO) y `registrarCobroExitoso`.
`totales.js`	`recalcularPedido`: fórmula única del total, `max(0, líneas − descuento) + envío`, usada por los hooks de `DetallePedido` y por la creación de pedidos.
`precios.js`	Costo de envío (`ENVIO_COSTO`, 25 por defecto; gratis desde `ENVIO_GRATIS_DESDE`, 300 por defecto), descuento de un cupón y desglose del IVA (`IVA_PORCENTAJE`, 12 por defecto) incluido en el precio.
`cupones.js`	Validación de cupones (vigencia, usos, sucursal, compra mínima), registro de su uso y liberación al cancelar.
`cotizacion.js`	Cotización de un carrito antes de crear el pedido (subtotal, descuento, envío, total e IVA) sin reservar nada.
`reembolsos.js`	Cálculo del monto a devolver por una devolución aceptada, creación de reembolsos (primero a la tarjeta por Stripe y el resto en caja), procesamiento en Stripe y reintento de los pendientes.
`pagosStripe.js`	Ayudas de pago con tarjeta: identificar un pago real de Stripe, conversión a centavos, anular un pago pendiente y cancelar un PaymentIntent consultando su estado real.

Módulo	Responsabilidad
`vencimientos.js`	Vencimiento automático de pedidos pendientes de pago y ciclo de tareas periódicas.
`notificaciones.js`	Avisos por correo a la clienta. Se invocan después del `COMMIT` (`t.afterCommit`) y nunca lanzan excepciones.
`correo.js`	Envío de correos en modo `smtp`, `consola` o `archivo` (`CORREO_MODO`); un correo fallido nunca interrumpe una venta ni un pago.
`monitoreo.js`	Identificador por petición, registro de errores 5xx y excepciones no controladas en `ErrorServidor`, manejador final de errores y Sentry opcional.
`migrador.js`	Migraciones con Umzug, candado de transacción `conCandado` y `prepararBase`.
`esquema.js`	Herramientas de esquema de "solo agregar" para las migraciones y detección de columnas de los modelos que faltan en la base.
`mantenimiento.js`	Correcciones de datos idempotentes que se ejecutan en cada arranque.
`alcance.js`	Alcance por sucursal de cada empleado (`sucursalDe`, `whereSucursal`, `exigirAlcance` y sus cargadores).
`estadosDevolucion.js`	Máquina de estados de devoluciones y cambios (transiciones válidas).
`plazos.js`	Plazo para solicitar un cambio o devolución (`HORAS_PLAZO_DEVOLUCION`, 24 horas por defecto desde la entrega).
`auditoria.js`	Registro en la tabla de auditoría dentro de la misma transacción de la operación.
`empleados.js`, `paginacion.js`	Reglas de alta y edición de empleados; paginación y búsqueda de listados del panel.

3.7 Modelos

Los modelos Sequelize se definen en `app/models/*.model.js` y se registran, junto con sus asociaciones, en `app/models/index.js`: Sucursal, Cliente, Empleado, Proveedor, Categoria, Producto (con variantes mediante `id_producto_padre`), Inventario, Pedido, DetallePedido, Pago, Devolucion, DetalleDevolucion, MovimientoInventario, Auditoria, Reembolso y Cupon. La conexión usa SSL salvo `DB_SSL=false` y un pool de hasta `DB_POOL_MAX` conexiones (5 por defecto). El modelo `DetallePedido` contiene hooks que calculan el subtotal de cada línea y recalculan el total del pedido al crear, modificar o eliminar líneas. El diagrama entidad-relación completo está en Modelo de datos.

4. Organización de los frontends

Las dos aplicaciones web comparten la misma estructura de carpetas dentro de `src/`, lo que facilita moverse de una a otra:

Carpeta	Panel de empleados (`frontend/src`)	Tienda en línea (`tienda/src`)
`pages/`	Una carpeta por pantalla: Devoluciones, DevolucionDetalle, NuevaSolicitud, Trazabilidad, Inventario, Catalogo, Reportes, Pedidos, NuevaVenta, PedidoDetalle, Clientes, Proveedores, Sucursales, Promociones, Sistema, Empleados y Login. Los diálogos de cada pantalla viven junto a ella.	Catalogo, Prenda, Carrito, Checkout, Pedido (incluye `PagoTarjeta.jsx`), Pedidos, Devoluciones y Cuenta (inicio de sesión, registro, activación, recuperación de contraseña y perfil).
`components/`	Piezas reutilizables: `AppLayout` , `Rail` (navegación lateral), `RoleRoute` (muestra la pantalla solo a los puestos permitidos), `Modal` , `Paginador` , `Timeline` , `FotoPrenda` , `Reembolsos` , entre otras.	`Layout` , `RutaPrivada` (exige sesión de clienta), `SelectorSucursal` , `PasosPedido` , `Desglose` , `ReembolsosCliente` , `Cantidad` , entre otras.
`services/`	`api.js` (cliente HTTP común: agrega el token, convierte errores en mensajes y cierra la sesión ante un 401), `session.js` y un servicio por módulo de la API (pedidos, pagos, inventario, imágenes, reportes, salud, etc.).	`api.js` , `session.js` , `authService.js` , `catalogoService.js` , `pedidosService.js` y `devolucionesService.js` .
`context/`	`AuthContext` (sesión y puesto del empleado) y `ToastContext` (avisos).	`AuthContext` , `ToastContext` y `CartContext` (carrito guardado en `localStorage` , ligado a una sola sucursal de despacho).
`hooks/`	`useFetch` (carga de datos con recarga), `useSubmit` (envío de formularios) y `useDebounced` (búsqueda).	`useFetch` , `useSubmit` y `useRevisionCarrito` (contrasta el carrito con el catálogo vigente: precios, existencias y prendas retiradas).
`utils/`	`permisos.js` (puestos permitidos por acción, reflejo de los roles del backend), `constants.js` , `format.js` , `cambios.js` (envía solo los campos modificados) y `reportePdf.js` (reportes en PDF con jsPDF).	`constants.js` y `format.js` .
`config/env.js`	`API_URL` desde `VITE_API_URL` .	`API_URL` desde `VITE_API_URL` y la clave publicable de Stripe desde `VITE_STRIPE_PUBLISHABLE_KEY` .
`styles/`	Hojas de estilo globales.	Hojas de estilo globales.

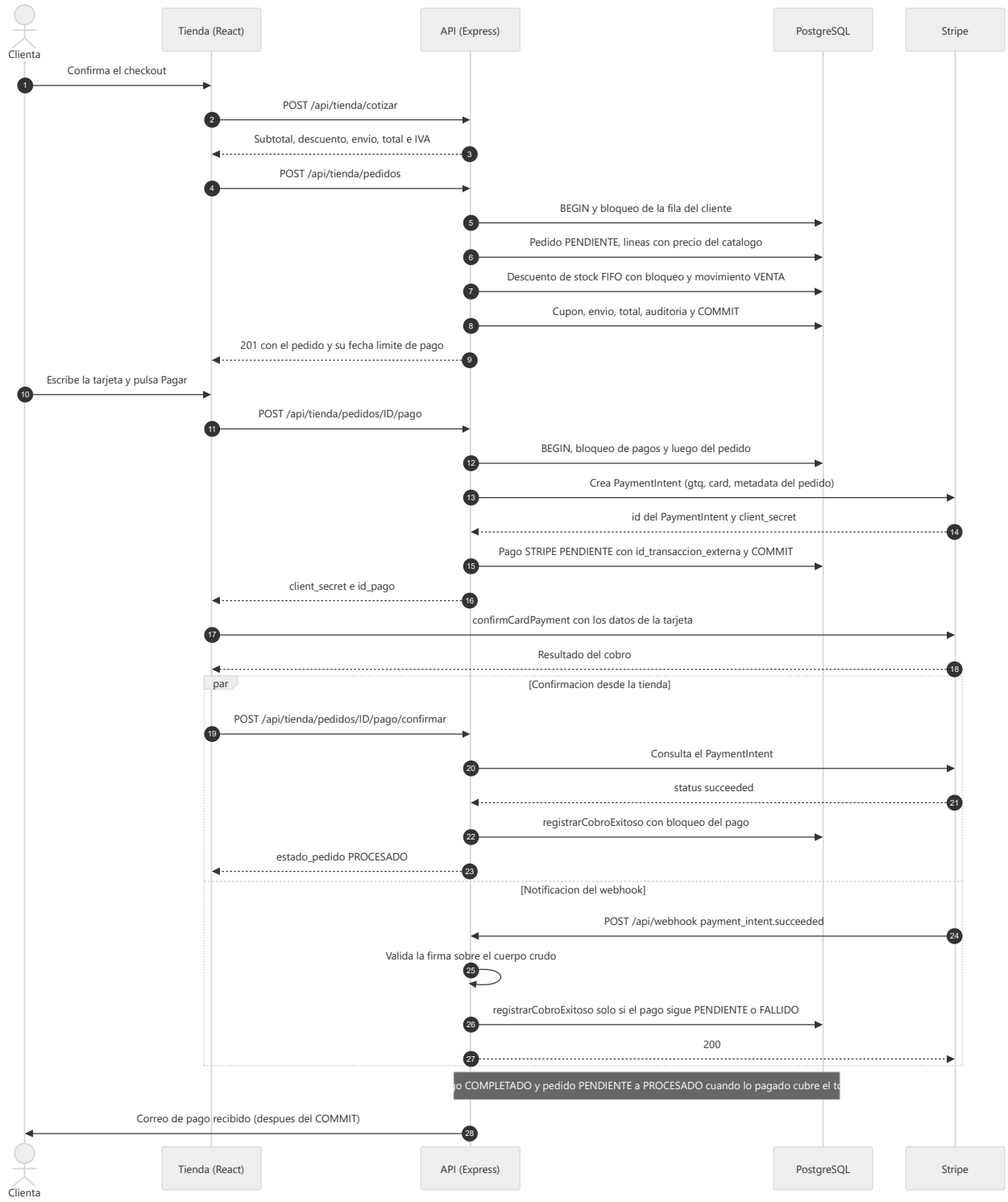
Principios comunes a ambas aplicaciones:

- **El servidor siempre decide.** Los permisos de `frontend/src/utils/permisos.js` solo ocultan opciones; la API vuelve a validar el puesto y la sucursal en cada petición.

- **Los precios los calcula el backend.** El carrito de la tienda guarda una copia de los datos visibles de cada prenda, pero el total definitivo se obtiene de `POST /api/tienda/cotizar` y de la creación del pedido.
- **Sesiones separadas.** El panel guarda su sesión bajo la clave `sesion` y la tienda bajo `sesion_tienda`, de modo que pueden convivir en el mismo dominio.

5. Flujo de una compra con pago por tarjeta

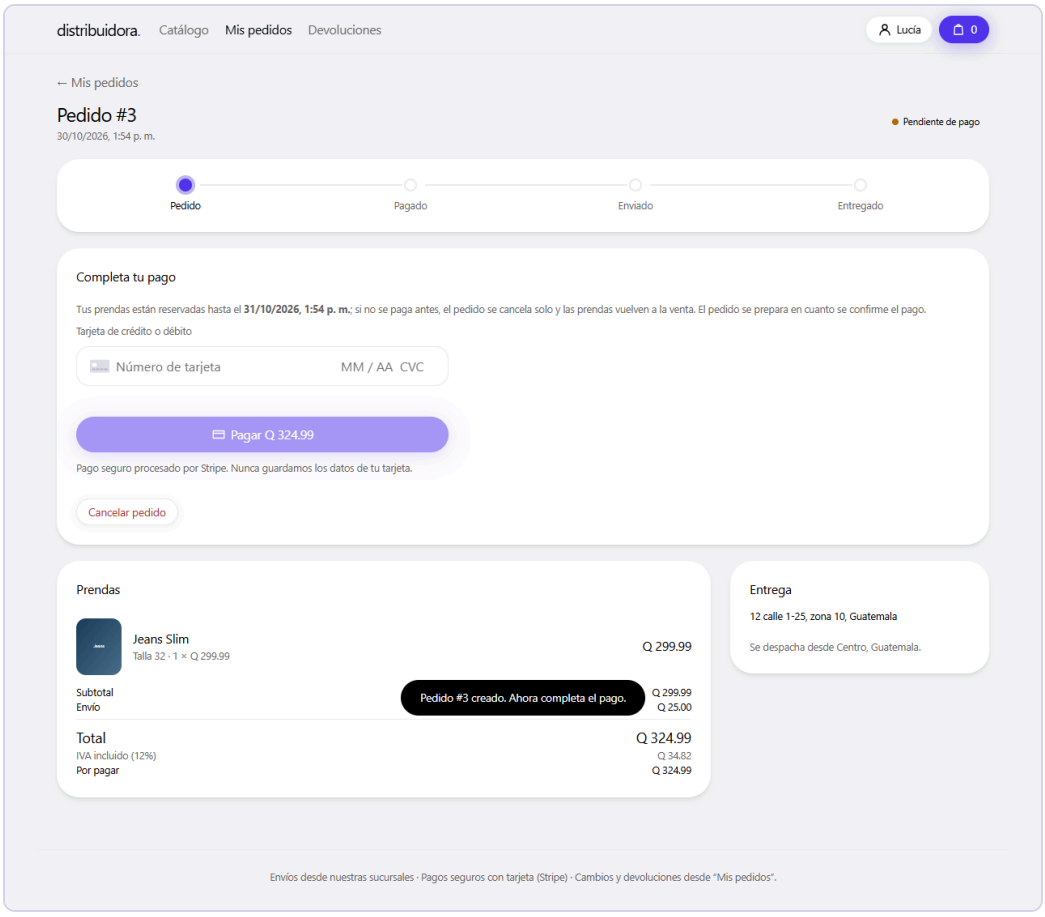
El siguiente diagrama de secuencia describe una compra en la tienda pagada con tarjeta. Las rutas son las de `app/routes/tienda.routes.js` y la lógica corresponde a `tienda.controller.js`, `webhook.controller.js` y `app/utils/facturacion.js`.



Detalles del flujo:

- **Reserva de prendas.** Al crear el pedido el stock se descuenta de inmediato, por lotes en orden FIFO y con bloqueo de filas, por lo que las prendas quedan apartadas para la clienta. Si el pedido no se paga dentro del plazo, la tarea de vencimientos lo cancela y el stock vuelve a estar disponible (ver la sección 7).

- **Límites de la tienda.** Una clienta puede tener como máximo 3 pedidos pendientes de pago y comprar como máximo 20 unidades de cada prenda por pedido; un pedido admite hasta 50 prendas distintas.
- **Reanudación del pago.** Si ya existe un PaymentIntent pendiente que cobra exactamente el saldo, `iniciarPago` lo reutiliza (doble clic, recarga de la página). Si Stripe ya está cobrando, responde `procesando` y la tienda solo consulta la confirmación.
- **Doble confirmación idempotente.** La confirmación llega por dos caminos (la tienda y el webhook). Ambos bloquean la fila del pago y solo registran el cobro si el pago no está ya COMPLETADO, por lo que el cobro nunca se cuenta dos veces. El camino de la tienda permite operar en local sin recibir webhooks; el webhook garantiza el registro aunque la clienta cierre el navegador.
- **Cobros que ya no corresponden.** Si Stripe confirma un cobro cuando el pedido ya fue cancelado, vencido o cubierto, `registrarCobroExitoso` registra el pago, deja constancia en la auditoría (`PAGO_REQUIERE_REEMBOLSO`) y solicita automáticamente el reembolso a la tarjeta.
- **Tarjeta rechazada.** El evento `payment_intent.payment_failed` no es definitivo: el pago sigue PENDIENTE para que la clienta reintente con otra tarjeta. El evento `payment_intent.canceled` marca el pago como FALLIDO.
- **Modo simulado.** Cuando `STRIPE_SECRET_KEY` contiene la clave de pruebas reservada que reconoce `app/config/stripe.config.js`, el pago se registra como COMPLETADO al instante sin contactar a Stripe; lo usan las pruebas automatizadas.



Pantalla de pago con tarjeta de un pedido pendiente en la tienda

Pantalla de pago de un pedido pendiente: indica hasta cuándo están reservadas las prendas, muestra el campo de tarjeta de Stripe, el desglose con envío e IVA incluido, y la opción de cancelar el pedido.

Los estados del pedido y del pago, así como las reglas de cancelación y devolución, se detallan en Reglas de negocio.

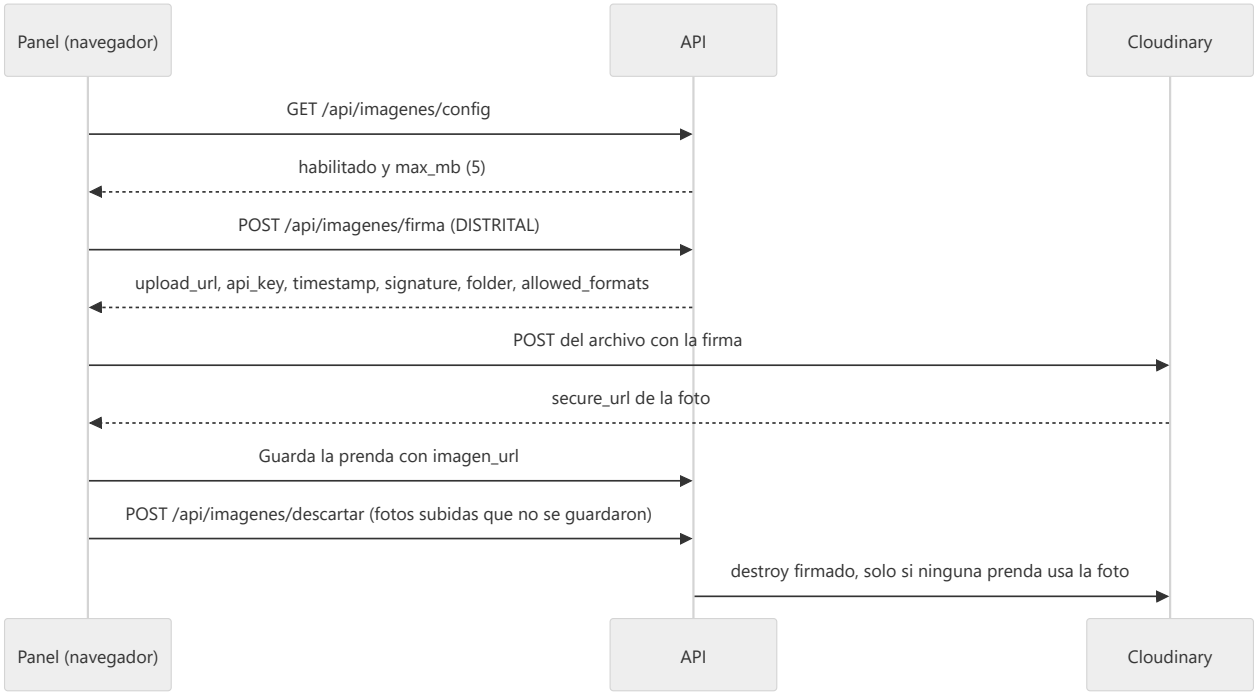
6. Integración con servicios externos

6.1 Stripe

El cliente de Stripe es único para todo el backend (`app/config/stripe.config.js`) y se configura con un tiempo de espera de 15 segundos y un solo reintento de red, porque varias llamadas a Stripe ocurren mientras la transacción mantiene filas bloqueadas y una conexión de la base ocupada. La API usa Stripe para crear, consultar y cancelar PaymentIntents y para crear y consultar reembolsos. El webhook atiende los eventos `payment_intent.succeeded` , `payment_intent.payment_failed` , `payment_intent.canceled` y los eventos de reembolso (`refund.created` , `refund.updated` , `refund.failed` , `charge.refund.updated`); ante un error de base responde 500 para que Stripe reintente la notificación.

6.2 Cloudinary

La subida de fotos de prendas está reservada al DISTRITAL y sigue este flujo:



La firma restringe la subida a la carpeta `distribuidora/prendas` y a los formatos jpg, jpeg, png y webp; el panel verifica además el tamaño máximo de 5 MB. El secreto de Cloudinary nunca sale del backend. La API solo borra fotos de su propia nube y carpeta, y solo cuando ninguna prenda las utiliza (las variantes pueden

compartir foto). Si las variables `CLOUDINARY_*` no están definidas, el panel permite únicamente pegar un enlace a la imagen. Para mostrar las fotos, el panel solicita a Cloudinary versiones optimizadas (`f_auto,q_auto` y ancho limitado).

6.3 Correo (SMTP)

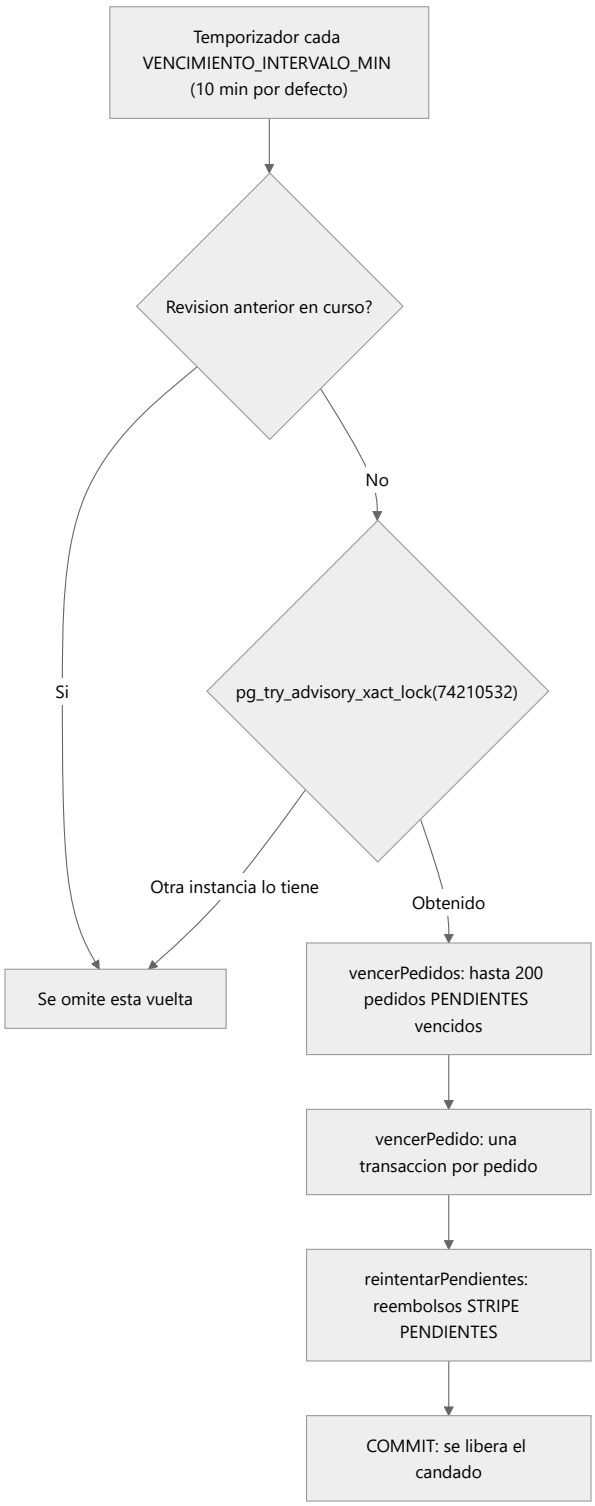
`app/utils/correo.js` envía los correos con nodemailer cuando existe `SMTP_HOST` (modo `smtp`); en otro caso los muestra en consola (modo `consola`), y las pruebas los guardan como archivos JSON (modo `archivo`). En producción, la ausencia de SMTP o de `TIENDA_URL` produce un aviso al arrancar. Los avisos de `notificaciones.js` se programan con `t.afterCommit`, de modo que solo se envían si la operación se confirmó, y además verifican que el pedido esté realmente en el estado que anuncian.

6.4 Base de datos en Neon

La API se conecta a PostgreSQL en Neon con SSL a través del endpoint con pooler (PgBouncer en modo transacción). Esta característica condiciona el diseño de los candados descritos en las secciones 7 y 8. Los scripts de respaldo y restauración (`npm run respaldo`, `npm run restaurar`) requieren en cambio la conexión directa, que se toma de `DB_HOST_DIRECTO` o, en Neon, del mismo host sin el sufijo `-pooler` (`scripts/pg.js`).

7. Tareas periódicas

El módulo `app/utils/vencimientos.js` ejecuta, dentro del mismo proceso del servidor, una revisión periódica que se inicia con `iniciarVencimientos()` inmediatamente después de invocar `app.listen`, una vez que `prepararBase` terminó.



Vencimiento de pedidos sin pago. Un pedido PENDIENTE vence cuando transcurren `PLAZO_PEDIDO_PENDIENTE_HORAS` (24 por defecto; el valor 0 desactiva el vencimiento) desde su creación. Para cada candidato, `vencerPedido` abre su propia transacción, bloquea los pagos pendientes y luego el pedido (el mismo orden que usan la tienda y el webhook), vuelve a validar todo con las filas ya bloqueadas y entonces:

Situación del pedido	Resultado
Tiene algún pago COMPLETADO (pago parcial)	Se omite; lo resuelve caja.
Tiene un pago de caja pendiente de confirmar	Se omite.
Tiene un cobro con tarjeta iniciado hace menos de 30 minutos	Se omite; la clienta podría estar escribiendo la tarjeta.
Stripe ya había cobrado el PaymentIntent	Se registra el cobro y el pedido pasa a PROCESADO.
Stripe está procesando el cobro	Se omite hasta la siguiente revisión.
Ninguna de las anteriores	Se cancela el PaymentIntent en Stripe, el pago queda FALLIDO, el pedido pasa a CANCELADO, el stock vuelve a estar disponible, se registra la auditoría y, tras el `COMMIT`, se envía un correo a la clienta.

Un error en un pedido no detiene a los demás: se revierte su transacción y se reintenta en la siguiente vuelta.

Reintento de reembolsos. En la misma vuelta, `reintentarPendientes` (de `app/utils/reembolsos.js`) vuelve a consultar o solicitar en Stripe hasta 50 reembolsos a tarjeta que siguen PENDIENTES desde hace más de 5 minutos, sin esperar a que caja pulse "Reintentar en Stripe".

Coordinación entre instancias. Toda la revisión se ejecuta dentro de `conCandado` con la clave 74210532 en modo `intentar`: si hay varias instancias del servidor, solo una revisa y las demás no esperan, lo intentan en la siguiente vuelta. Dentro del proceso, una bandera evita que dos revisiones se solapen. La primera revisión se ejecuta pocos segundos después del arranque (el menor entre 5 segundos y el intervalo) y los temporizadores no impiden que el proceso termine (`unref`).

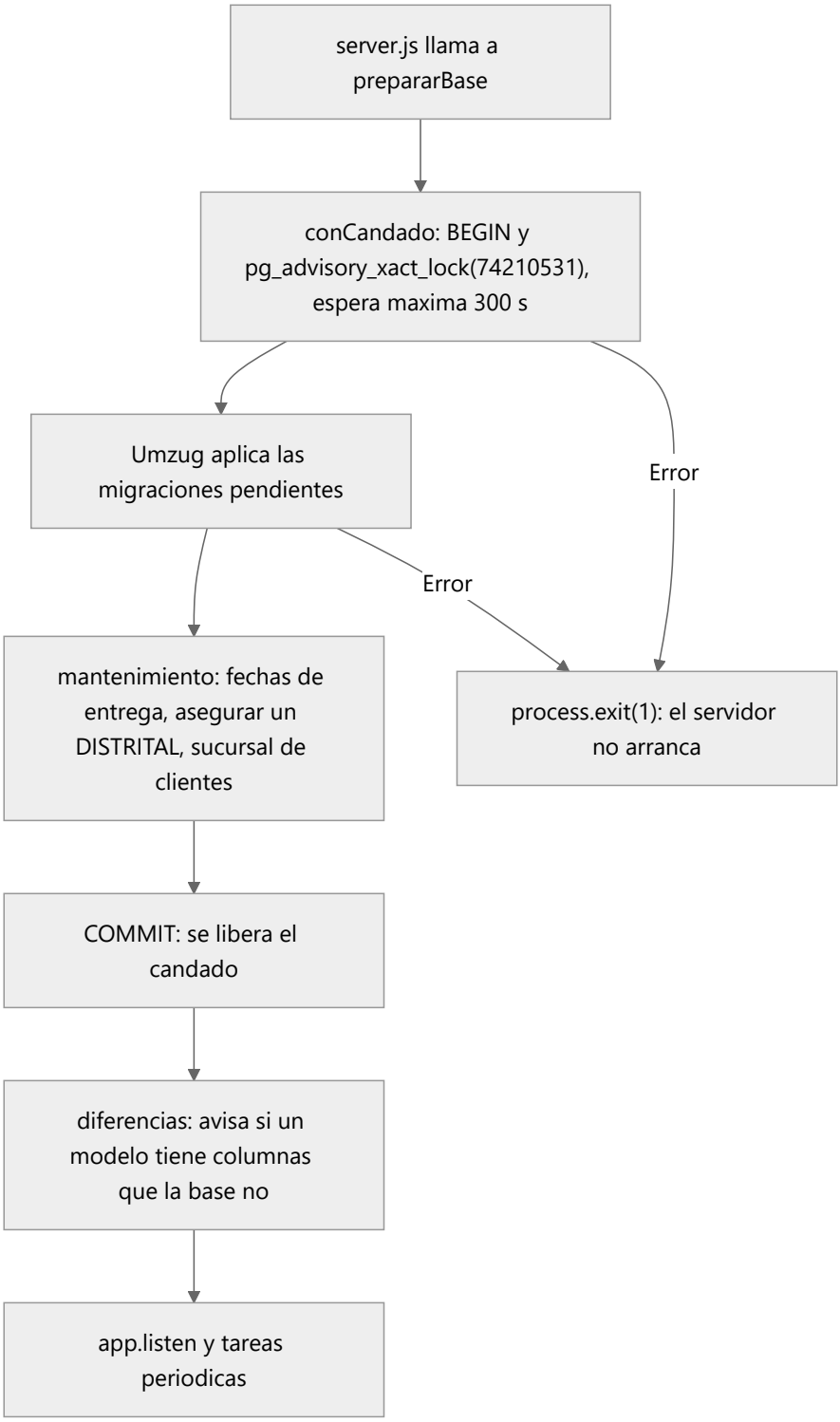
8. Migraciones al arrancar

El esquema de la base se gestiona con migraciones controladas mediante Umzug (`app/utils/migrador.js`). Cada archivo de `app/migraciones` cuyo nombre sigue el patrón `NNNN-nombre.js` se aplica una sola vez, en orden, y queda anotado en la tabla `SequelizeMeta`.

Migración	Contenido
`0001-esquema-base.js`	Línea base: crea las tablas, columnas y valores de ENUM que falten según los modelos actuales y los índices únicos sin distinguir mayúsculas (correos de empleados y clientes, nombres de tipos de prenda). Es idempotente y solo agrega.
`0002-intentos-limite.js`	Tabla `IntentoLimite` para el límite de intentos (clave SHA-256 de IP y ruta).

Migración	Contenido
`0003-errores-servidor.js`	Tabla `ErrorServidor` para el panel "Estado del sistema"; nunca guarda el cuerpo de la petición.
`0004-envio-cupones.js`	Tabla `Cupon` y columnas `costo_envio`, `descuento` e `id_cupon` en `Pedido`.
`0005-cupon-en-pedido.js`	Copia del cupón en el pedido (tipo, valor y compra mínima) para que editar un cupón no altere pedidos existentes.

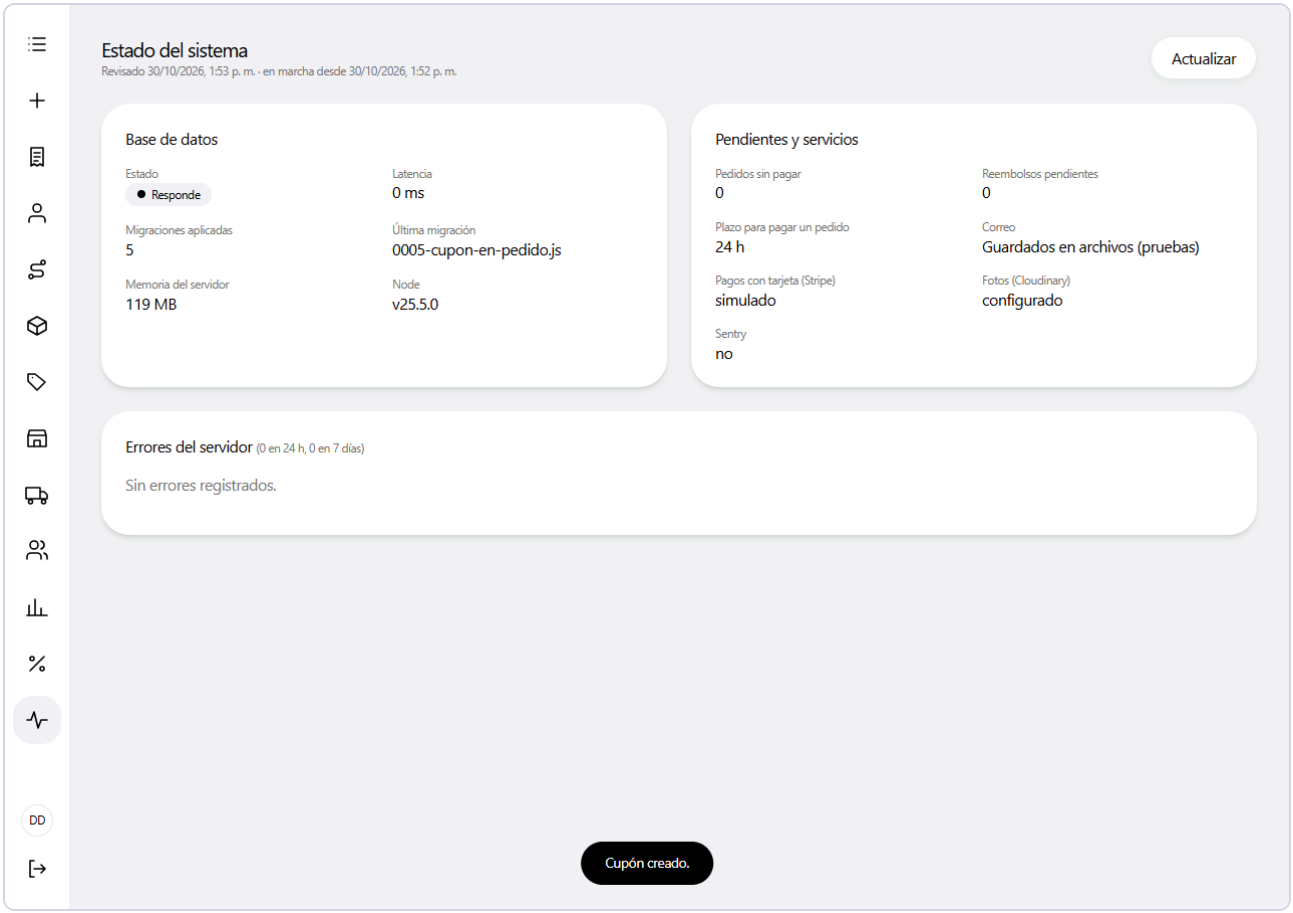
Secuencia de `prepararBase(db)`, invocada por `server.js` antes de `app.listen`:



- **Candado de arranque.** Si dos instancias arrancan a la vez, la segunda espera a que la primera termine; así las migraciones no se ejecutan dos veces ni se promueven dos DISTRITAL.
- **Mantenimiento.** Después de las migraciones se ejecutan correcciones de datos idempotentes: completar `fecha_entrega` de pedidos entregados antiguos a partir de la auditoría, si no existe ningún DISTRITAL activo, promover a DISTRITAL a los ADMIN activos sin sucursal o, si no hay ninguno, al

ADMIN activo más antiguo, y asignar sucursal a clientes de mostrador antiguos.

- **Detección de olvidos.** Si un modelo tiene columnas o valores de ENUM que la base no tiene, se muestra una advertencia indicando que falta escribir una migración; la base no se modifica.
- **Falla segura.** Si una migración falla, el proceso termina con código 1: el servidor no atiende con un esquema a medias.
- **Uso manual.** `npm run migrar` aplica las pendientes, `npm run migrar:estado` lista las aplicadas y pendientes, y `node scripts/migrar.js deshacer` revierte la última (si tiene `down`).



Panel Estado del sistema con migraciones aplicadas y servicios

Pantalla "Estado del sistema" del panel (solo DISTRITAL): muestra si la base responde, su latencia, el número de migraciones aplicadas y la última, los pedidos sin pagar, los reembolsos pendientes, el plazo para pagar un pedido, el modo de correo, Stripe, Cloudinary, Sentry y los errores recientes del servidor.

9. Decisiones técnicas clave

Decisión	Alternativa descartada	Justificación
Transacciones y bloqueos de fila (`lock: t.LOCK.UPDATE`) en ventas, pagos, cancelaciones, vencimientos y webhook, con un orden de bloqueo fijo: pagos y luego pedido; inventario por `id_producto` ascendente.	Operaciones sueltas sin transacción.	Una venta modifica varias tablas (pedido, líneas, inventario, movimientos, cupón, auditoría) que deben confirmarse o revertirse juntas. Los bloqueos impiden vender el mismo stock dos veces, contar dos veces un cobro que llega por la tienda y por el webhook, o superar el límite de pedidos pendientes con dos checkouts simultáneos. El orden fijo evita interbloqueos (deadlocks).
Migraciones con Umzug y herramientas de "solo agregar".	`sequelize.sync({ alter: true })` en cada arranque.	`sync alter` compara modelos y base en cada arranque y puede borrar columnas o cambiar tipos sin aviso, con pérdida de datos. Las migraciones son explícitas, se aplican una sola vez, quedan registradas en `SequelizeMeta` , se revisan en el código y pueden consultarse con `migrar:estado` .
Subida directa del navegador a Cloudinary con firma emitida por la API.	Subir el archivo a la API y reenviarlo a Cloudinary.	El secreto de Cloudinary permanece en el backend; la API no recibe ni procesa archivos binarios (solo interpreta cuerpos JSON o de formulario) ni consume memoria y ancho de banda del servidor; la firma limita carpeta y formatos.
Candados de transacción (`pg_advisory_xact_lock` y `pg_try_advisory_xact_lock`) en una conexión dedicada.	Candados de sesión (`pg_advisory_lock` y `pg_advisory_unlock`).	Neon se usa a través de PgBouncer en modo transacción: entre transacciones el pooler puede cambiar de proceso servidor, de modo que el `unlock` de un candado de sesión podía ejecutarse en otro proceso y dejar el candado tomado. Un candado de transacción se libera solo con `COMMIT` o `ROLLBACK` , y durante la transacción el pooler no cambia de proceso.
Webhook registrado antes de `bodyParser.json` .	Webhook con el cuerpo ya interpretado.	La firma de Stripe se valida sobre el cuerpo crudo; si el JSON ya fue interpretado la validación siempre falla.
Confirmación del pago por dos caminos idempotentes (tienda y webhook).	Depender solo del webhook.	El pedido se actualiza en cuanto la clienta paga, incluso en desarrollo local sin webhooks, y el webhook asegura el registro si la clienta cierra la página. El bloqueo del pago y la verificación de su estado evitan el doble registro.
Precios y totales calculados solo en el backend, con una fórmula única en `totales.js` y la copia del cupón guardada en el pedido.	Aceptar precios o totales enviados por el navegador.	Impide manipular precios desde el cliente y garantiza que el panel, la tienda y los reportes muestren el mismo total; editar un cupón no altera pedidos existentes.
Puesto, estado y `token_version` leídos de la base en cada petición.	Confiar solo en los datos del JWT.	Desactivar a un empleado, cambiar su puesto o su contraseña surte efecto de inmediato, aunque su token aún no haya expirado.
Límite de intentos en la tabla `IntentoLimite` con respaldo en memoria.	Contador únicamente en memoria.	Los contadores sobreviven a reinicios y valen para varias instancias del servidor; si la base no responde, la ruta sigue protegida.

Decisión	Alternativa descartada	Justificación
Correos después del `COMMIT` y que nunca lanzan excepciones.	Enviar el correo dentro de la operación.	No se notifica algo que finalmente se revirtió, y una falla del servidor de correo no interrumpe una venta ni un pago.
Tiempo de espera corto para Stripe (15 segundos, un reintento).	Valores por defecto de la librería (80 segundos y 2 reintentos).	Varias llamadas a Stripe ocurren con filas bloqueadas y una conexión ocupada; una caída de Stripe no debe agotar el pool de conexiones.

10. Documentos relacionados

- Modelo de datos: diagrama entidad-relación y descripción de las tablas.
- Roles y permisos: qué puede hacer cada puesto y el alcance por sucursal.
- Reglas de negocio: estados de pedidos, pagos, devoluciones y reembolsos.
- API: endpoints, autenticación y errores.
- Seguridad: contraseñas, sesiones, límite de intentos y protección de pagos.
- Pruebas: estrategia y ejecución de las pruebas.
- Despliegue y operación: publicación en Render, variables de entorno, respaldos y monitoreo.

Modelo de datos

Este documento describe la estructura de la base de datos PostgreSQL del sistema Distribuidora: las entidades definidas como modelos de Sequelize en `app/models/`, sus relaciones (declaradas en `app/models/index.js`), las tablas auxiliares que crean las migraciones de `app/migraciones/` sin tener un modelo asociado, los índices únicos y la fórmula con la que se calcula el total de un pedido. Toda la información se obtuvo del código fuente del proyecto.

Contenido

- 1. Visión general
- 2. Diagrama entidad-relación
- 3. Convenciones de tipos
- 4. Tablas del modelo
  - 4.1 Sucursal
  - 4.2 Cliente
  - 4.3 Empleado
  - 4.4 Proveedor
  - 4.5 Categoria
  - 4.6 Producto
  - 4.7 Inventario
  - 4.8 Pedido
  - 4.9 DetallePedido
  - 4.10 Pago
  - 4.11 Devolucion
  - 4.12 DetalleDevolucion
  - 4.13 MovimientoInventario
  - 4.14 auditoria
  - 4.15 Reembolso
  - 4.16 Cupon
- 5. Tablas creadas por migraciones sin modelo
- 6. Llaves foráneas y acciones referenciales
- 7. Índices únicos e índices adicionales
- 8. Cálculo del total del pedido
- 9. Copia del cupón en el pedido
- 10. Evolución del esquema (migraciones)

1. Visión general

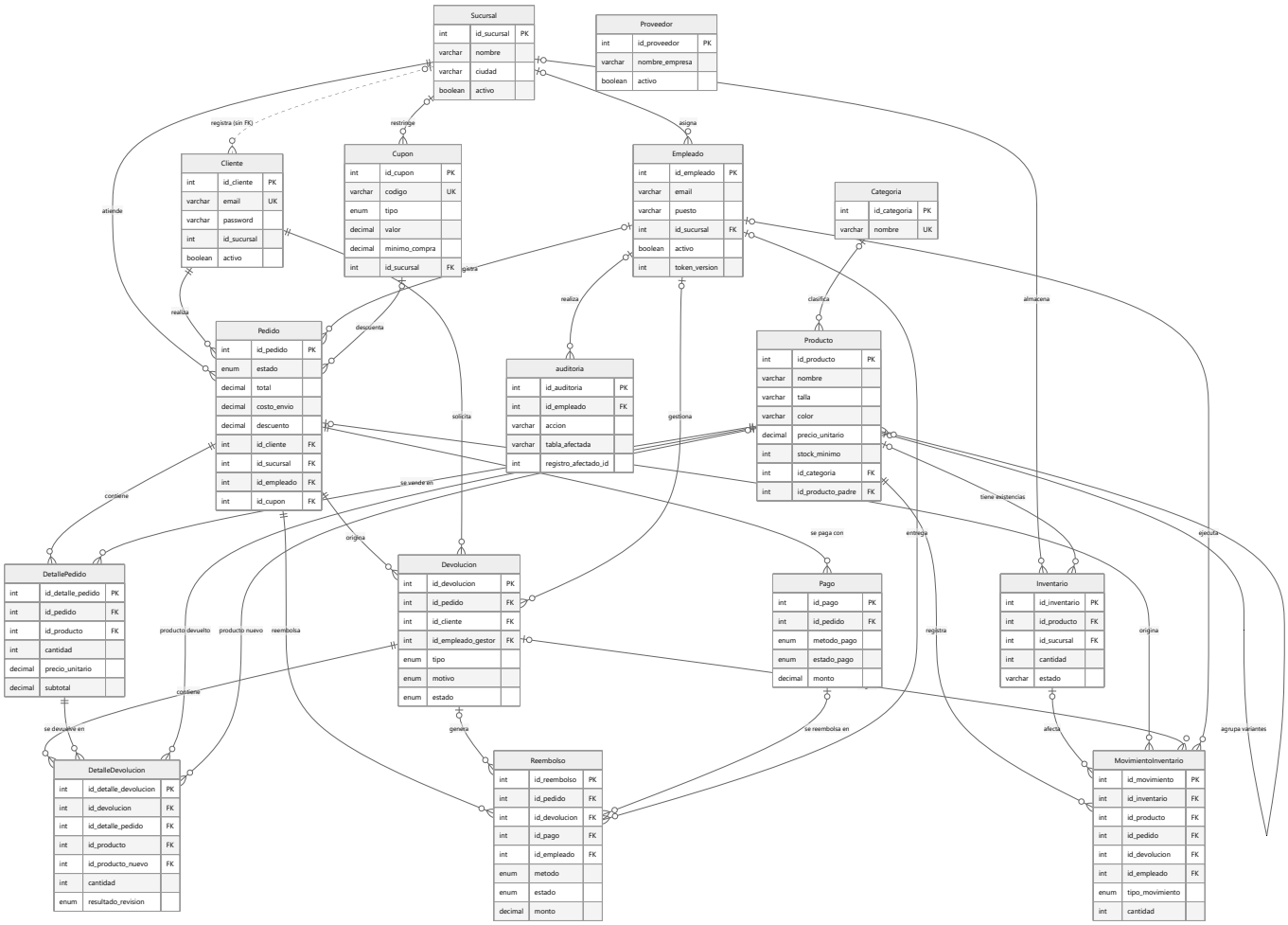
La base de datos se organiza en cinco grupos funcionales:

Grupo	Tablas	Propósito
Organización	`Sucursal`, `Empleado`, `Proveedor`	Tiendas físicas, personal del panel y catálogo de proveedores.
Catálogo e inventario	`Categoria`, `Producto`, `Inventario`	Tipos de prenda, prendas con sus variantes (talla/color) y existencias por sucursal.
Ventas	`Cliente`, `Pedido`, `DetallePedido`, `Pago`, `Cupon`	Clientas, pedidos de mostrador y de la tienda en línea, sus líneas, pagos y cupones de descuento.
Logística inversa	`Devolucion`, `DetalleDevolucion`, `Reembolso`	Devoluciones, cambios de talla o variante y devolución de dinero.
Trazabilidad y operación	`MovimientoInventario`, `auditoria`, `IntentoLimite`, `ErrorServidor`, `SequelizeMeta`	Historial de movimientos de stock, bitácora de acciones, límite de intentos, registro de errores y control de migraciones.

El esquema no se sincroniza con `sync({ alter: true })`. Al arrancar, `server.js` invoca `prepararBase` (`app/utils/migrador.js`), que aplica con Umzug las migraciones pendientes bajo un candado de PostgreSQL (`pg_advisory_xact_lock`) y luego ejecuta el mantenimiento de datos. Los detalles de operación se describen en Despliegue y operación.

2. Diagrama entidad-relación

El diagrama muestra todas las entidades con sus llaves primarias (PK), llaves foráneas (FK) y los atributos más relevantes. La lista completa de columnas de cada tabla se encuentra en la sección 4. La relación entre `Cliente` y `Sucursal` se dibuja con línea punteada porque la columna existe, pero no hay una asociación declarada en `index.js` y, por lo tanto, tampoco una llave foránea.



`Proveedor` no tiene relaciones declaradas con otras tablas: funciona como un catálogo independiente de empresas proveedoras.

3. Convenciones de tipos

Las tablas de columnas de la sección 4 usan los tipos de Sequelize y su equivalente en PostgreSQL:

Tipo en el modelo	Tipo en PostgreSQL	Observación
`INTEGER`	`integer`	Las llaves primarias usan `autoIncrement` (`serial`).
`STRING`	`varchar(255)`	Sin longitud explícita.
`STRING(n)`	`varchar(n)`	Longitud máxima `n`.
`TEXT`	`text`	Texto sin límite de longitud.
`DECIMAL(10, 2)`	`numeric(10,2)`	Montos en quetzales con dos decimales.
`BOOLEAN`	`boolean`	

Tipo en el modelo	Tipo en PostgreSQL	Observación
`DATE`	`timestamp with time zone`	Fecha y hora.
`DATEONLY`	`date`	Solo fecha.
`ENUM(...)`	Tipo enumerado `enum_<Tabla>_<campo>`	Ejemplo: `enum_Pedido_estado`. Las migraciones solo agregan valores, nunca los eliminan.

En las tablas, la columna "Nulo" indica si la columna admite `NULL`. Cuando el modelo no especifica `allowNull`, Sequelize la crea como anulable. El valor `NOW` como valor por defecto corresponde a la fecha y hora del momento de inserción.

La mayoría de los modelos declaran `timestamps: false`, por lo que no tienen las columnas `createdAt` y `updatedAt`. La única excepción es `auditoria`.

4. Tablas del modelo

4.1 Sucursal

Archivo: `app/models/sucursal.model.js`. Tabla: `Sucursal`. Representa cada tienda física de la distribuidora.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_sucursal`	INTEGER (PK, autoincremental)	No		Identificador de la sucursal.
`nombre`	STRING(100)	No		Nombre de la sucursal.
`direccion`	STRING(200)	No		Dirección física.
`ciudad`	STRING(80)	No		Ciudad.
`telefono`	STRING(20)	Sí		Teléfono de contacto.
`activo`	BOOLEAN	No	`true`	Indica si la sucursal está en operación.

4.2 Cliente

Archivo: `app/models/cliente.model.js`. Tabla: `Cliente`. Almacena tanto a las clientas registradas en mostrador como a las que tienen cuenta en la tienda en línea.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_cliente`	INTEGER (PK, autoincremental)	No		Identificador de la clienta.
`nombre`	STRING(80)	No		Nombre.
`apellido`	STRING(80)	No		Apellido.

Columna	Tipo	Nulo	Por defecto	Descripción
`telefono`	STRING(20)	Sí		Teléfono.
`email`	STRING(120), único	Sí		Correo electrónico. Único también sin distinguir mayúsculas (índice `cliente_email_lower_unico`).
`direccion`	STRING(200)	Sí		Dirección de la clienta.
`fecha_registro`	DATEONLY	Sí	`NOW`	Fecha de registro.
`activo`	BOOLEAN	No	`true`	Indica si la clienta está activa.
`id_sucursal`	INTEGER	Sí		Sucursal donde se registró en mostrador; `NULL` si se registró en la tienda en línea. Cada sucursal ve a las clientas que registró y a las que le han comprado. No tiene llave foránea declarada.
`password`	STRING	Sí		Hash bcrypt de la contraseña de la tienda en línea. `NULL` indica una clienta de mostrador sin cuenta web.
`codigo_activacion`	STRING(64)	Sí		Hash SHA-256 del código de un solo uso que entrega caja para que una clienta de mostrador active su cuenta web conservando su historial.
`codigo_activacion_expira`	DATE	Sí		Vencimiento del código de activación.
`reset_token_hash`	STRING(64)	Sí		Hash SHA-256 del enlace de un solo uso de "Olvidé mi contraseña".
`reset_expira`	DATE	Sí		Vencimiento del enlace de restablecimiento.
`token_version`	INTEGER	No	`0`	Versión de sesión incluida en el token de la tienda. Aumenta al cambiar o restablecer la contraseña, lo que invalida las sesiones abiertas en otros dispositivos.

Alcances (scopes). El `defaultScope` excluye `password`, `codigo_activacion`, `codigo_activacion_expira`, `reset_token_hash`, `reset_expira` y `token_version`, de modo que estos datos nunca aparecen en listados ni en consultas con `include`. El inicio de sesión usa el alcance `conPassword`, la activación de cuenta usa `conCodigo` y el restablecimiento de contraseña usa `conReset`. Cada uno incluye únicamente los campos que necesita.

4.3 Empleado

Archivo: `app/models/empleado.model.js`. Tabla: `Empleado` (`freezeTableName`). Personal que accede al panel.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_empleado`	INTEGER (PK, autoincremental)	No		Identificador del empleado.
`nombre`	STRING	Sí		Nombre.
`apellido`	STRING	Sí		Apellido.
`puesto`	STRING	Sí		Rol del empleado. Los valores válidos (`PUESTOS_VALIDOS` en `app/middlewares/authJwt.js`) son `DISTRITAL` , `ADMIN` , `CAJERO` y `BODEGUERO` .
`email`	STRING	Sí		Correo de acceso. Único sin distinguir mayúsculas (índice `empleado_email_lower_unico`).
`password`	STRING	Sí		Hash de la contraseña.
`id_sucursal`	INTEGER (FK a `Sucursal`)	Sí		Sucursal asignada. Puede ser `NULL` (por ejemplo, un DISTRITAL trabaja con todas).
`activo`	BOOLEAN	No	`true`	Un empleado que deja la empresa se desactiva en lugar de borrarse, porque su identificador queda en pedidos, movimientos y auditoría. Desactivado no puede iniciar sesión.
`token_version`	INTEGER	No	`0`	Versión de las sesiones incluida en el JWT. Se incrementa al cambiar o restablecer la contraseña o al desactivar la cuenta, de modo que los tokens emitidos antes dejan de ser válidos.

El puesto no es un ENUM en la base: se valida en la aplicación. Los permisos de cada puesto se describen en Roles y permisos.

4.4 Proveedor

Archivo: `app/models/proveedor.model.js` . Tabla: `Proveedor` .

Columna	Tipo	Nulo	Por defecto	Descripción
`id_proveedor`	INTEGER (PK, autoincremental)	No		Identificador del proveedor.
`nombre_empresa`	STRING(120)	No		Razón o nombre comercial.
`nombre_contacto`	STRING(100)	Sí		Persona de contacto.
`telefono`	STRING(20)	Sí		Teléfono.
`email`	STRING(120)	Sí		Correo.
`direccion`	STRING(200)	Sí		Dirección.
`activo`	BOOLEAN	No	`true`	Indica si el proveedor está vigente.

4.5 Categoria

Archivo: `app/models/categoria.model.js`. Tabla: `Categoria`. En el panel se presenta como "tipos de prenda".

Columna	Tipo	Nulo	Por defecto	Descripción
`id_categoria`	INTEGER (PK, autoincremental)	No		Identificador de la categoría.
`nombre`	STRING(60), único	No		Nombre del tipo de prenda. Único también sin distinguir mayúsculas (índice `categoria_nombre_lower_unico`).
`descripcion`	STRING(200)	Sí		Descripción.

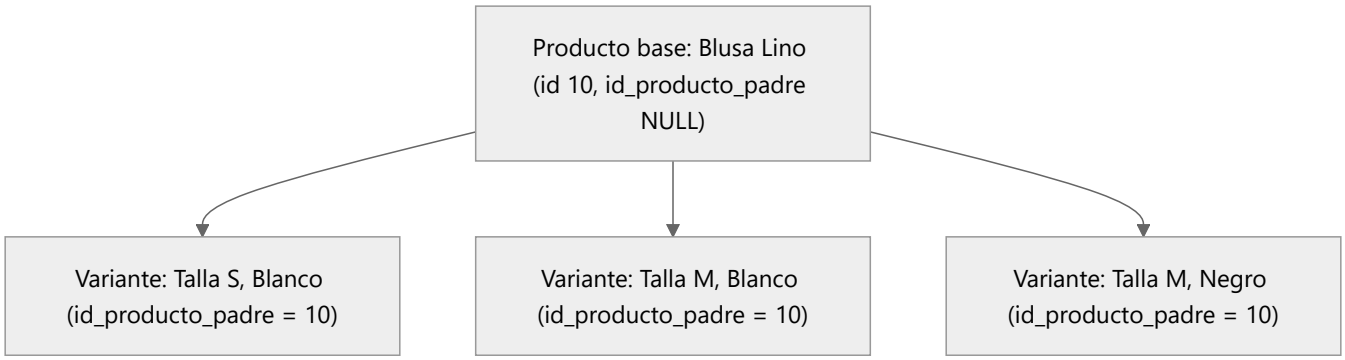
4.6 Producto

Archivo: `app/models/producto.model.js`. Tabla: `Producto`. Cada fila es una prenda concreta (una combinación de talla y color).

Columna	Tipo	Nulo	Por defecto	Descripción
`id_producto`	INTEGER (PK, autoincremental)	No		Identificador del producto.
`nombre`	STRING(120)	No		Nombre de la prenda.
`descripcion`	STRING(255)	Sí		Descripción.
`talla`	STRING(10)	Sí		Talla de la variante.
`color`	STRING(40)	Sí		Color de la variante.
`precio_unitario`	DECIMAL(10,2)	No		Precio de venta con IVA incluido.
`stock_minimo`	INTEGER	No	`5`	Umbral del reporte de stock crítico (`GET /api/reportes/stock-critico`): se alerta cuando la suma de inventario `DISPONIBLE` del producto en una sucursal es igual o menor a este valor.
`activo`	BOOLEAN	No	`true`	Indica si el producto está a la venta.
`id_categoria`	INTEGER (FK a `Categoria`)	Sí		Tipo de prenda.
`id_producto_padre`	INTEGER (FK a `Producto`)	Sí		Agrupa las variantes de una misma prenda. `NULL` indica un producto base o que no pertenece a un grupo.

Columna	Tipo	Nulo	Por defecto	Descripción
`imagen_url`	STRING(500)	Sí		URL (http/https) de la foto que muestra la tienda en línea.

Variantes. La relación reflexiva se declara con dos asociaciones: `Producto.belongsTo(Producto, { as: 'ProductoPadre' })` y `Producto.hasMany(Producto, { as: 'Variantes' })`, ambas sobre `id_producto_padre`. La raíz de un grupo es `id_producto_padre` si existe o, en caso contrario, el propio `id_producto` (función `raizDeGrupo` en `devolucion.controller.js`). Esta agrupación permite ofrecer un cambio de talla o de variante dentro de la misma prenda. El siguiente diagrama es un ejemplo ilustrativo con identificadores ficticios:



4.7 Inventario

Archivo: `app/models/inventario.model.js`. Tabla: `Inventario` (`freezeTableName`). Existencias de un producto en una sucursal, organizadas en filas (lotes) con un estado.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_inventario`	INTEGER (PK, autoincremental)	No		Identificador de la fila de inventario.
`cantidad`	INTEGER	Sí		Unidades en la fila.
`id_producto`	INTEGER (FK a `Producto`)	Sí		Producto.
`id_sucursal`	INTEGER (FK a `Sucursal`)	Sí		Sucursal donde están las unidades.
`lote`	STRING	Sí		Identificador del lote.
`estado`	STRING	Sí	`'DISPONIBLE'`	Estado de las unidades.

El campo `estado` es texto libre en la base, pero el código usa tres valores, que son también los únicos admitidos al ingresar stock desde el panel (`ESTADOS_INGRESO` en `inventario.controller.js`):

Valor	Uso
`DISPONIBLE`	Unidades a la venta. Es el único estado que descuentan las ventas y que cuentan los reportes de stock.

Valor	Uso
`DANADA`	Unidades dañadas: ingresadas con ese estado o devueltas con revisión `DANADO`.
`DEFECTUOSA`	Unidades defectuosas: ingresadas con ese estado o devueltas con revisión `DEFECTUOSO`.

Un mismo producto puede tener varias filas en una sucursal. Al reingresar unidades de una devolución, la función `reingresarAInventario` busca la fila con el mismo producto, sucursal y estado y le suma la cantidad; si no existe, la crea.

4.8 Pedido

Archivo: `app/models/pedido.model.js`. Tabla: `Pedido`. Representa tanto las ventas de mostrador como las compras de la tienda en línea.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_pedido`	INTEGER (PK, autoincremental)	No		Identificador del pedido.
`fecha_pedido`	DATE	Sí	`NOW`	Fecha y hora de creación.
`estado`	ENUM	No	`'PENDIENTE'`	Estado del pedido (valores abajo).
`total`	DECIMAL(10,2)	No	`0`	Total a pagar. Lo calcula el sistema (ver sección 8); nunca se recibe del cliente.
`id_cliente`	INTEGER (FK a `Cliente`)	No		Clienta del pedido.
`id_sucursal`	INTEGER (FK a `Sucursal`)	No		Sucursal que atiende el pedido.
`id_empleado`	INTEGER (FK a `Empleado`)	Sí		Empleado que registró la venta. `NULL` en los pedidos de la tienda en línea.
`direccion_entrega`	STRING(200)	Sí		Dirección de despacho de un pedido en línea a domicilio. `NULL` en ventas de mostrador y en retiro en sucursal.
`fecha_entrega`	DATE	Sí		Momento en que el pedido pasó a `ENTREGADO`. Desde allí corre el plazo para solicitar cambio o devolución (`HORAS_PLAZO_DEVOLUCION`, 24 horas por defecto).
`costo_envio`	DECIMAL(10,2)	No	`0`	Costo de envío a domicilio; `0` en retiro en sucursal y en mostrador.
`descuento`	DECIMAL(10,2)	No	`0`	Monto que descontó el cupón sobre las prendas.
`id_cupon`	INTEGER (FK a `Cupon`)	Sí		Cupón aplicado.

Columna	Tipo	Nulo	Por defecto	Descripción
`cupon_tipo`	STRING(12)	Sí		Copia del tipo del cupón al momento de usarlo.
`cupon_valor`	DECIMAL(10,2)	Sí		Copia del valor del cupón.
`cupon_minimo`	DECIMAL(10,2)	Sí		Copia de la compra mínima del cupón.

Valores del ENUM `estado`:

Valor	Significado
`PENDIENTE`	Pedido creado y pendiente de pago.
`PROCESADO`	Pedido pagado: el pedido pasa automáticamente a este estado cuando los pagos completados cubren el total.
`ENVIADO`	Pedido despachado.
`ENTREGADO`	Pedido recibido por la clienta; se registra `fecha_entrega`.
`CANCELADO`	Pedido anulado; las unidades vuelven al inventario y se libera el uso del cupón.

Las transiciones permitidas entre estados se describen en Reglas de negocio.

4.9 DetallePedido

Archivo: `app/models/detalle_pedido.model.js`. Tabla: `DetallePedido`. Cada fila es una línea del pedido.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_detalle_pedido`	INTEGER (PK, autoincremental)	No		Identificador de la línea.
`cantidad`	INTEGER	No		Unidades. Validación: mínimo 1.
`precio_unitario`	DECIMAL(10,2)	No		Precio de la prenda al momento de la venta. Validación: mínimo 0.
`subtotal`	DECIMAL(10,2)	No		`cantidad × precio_unitario`, redondeado a dos decimales.
`id_pedido`	INTEGER (FK a `Pedido`)	No		Pedido al que pertenece la línea.
`id_producto`	INTEGER (FK a `Producto`)	No		Producto vendido.

Hooks del modelo. El subtotal nunca se confía a quien envía la petición:

Hook	Acción
`beforeValidate`	Calcula `subtotal = round(cantidad × precio_unitario × 100) / 100`.

Hook	Acción
`beforeUpdate`	Recalcula el subtotal y, si la actualización usa `fields`, agrega `subtotal` para que se guarde.
`afterCreate`, `afterDestroy`	Recalculan el total del pedido con `recalcularPedido`.
`afterUpdate`	Recalcula el total del pedido y, si la línea se movió a otro pedido, también el del pedido anterior.
`beforeBulkCreate`, `afterBulkCreate`	Calculan el subtotal de cada línea y recalculan una sola vez cada pedido afectado.
`beforeBulkDestroy`, `beforeBulkUpdate`	Activan `individualHooks` para que las operaciones masivas también actualicen el total.

4.10 Pago

Archivo: `app/models/pago.model.js`. Tabla: `Pago`.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_pago`	INTEGER (PK, autoincremental)	No		Identificador del pago.
`metodo_pago`	ENUM	No		Medio de pago (valores abajo).
`monto`	DECIMAL(10,2)	No		Monto pagado.
`fecha_pago`	DATE	Sí	`NOW`	Fecha y hora del pago.
`estado_pago`	ENUM	No	`'PENDIENTE'`	Estado del pago (valores abajo).
`id_transaccion_externa`	STRING(150)	Sí		Identificador del pago en el proveedor externo (por ejemplo, Stripe).
`id_pedido`	INTEGER (FK a `Pedido`)	No		Pedido pagado.

ENUM `metodo_pago`	Significado
`STRIPE`	Tarjeta en la tienda en línea, procesada por Stripe.
`PAYPAL`	Queda pendiente hasta que caja lo confirma.
`EFECTIVO`	Pago en efectivo en la tienda física; el pago nace `COMPLETADO`.
`TARJETA`	Terminal POS de la tienda física; el pago nace `COMPLETADO`.

ENUM `estado_pago`	Significado
`PENDIENTE`	Pago iniciado sin confirmar.
`COMPLETADO`	Pago confirmado.

ENUM `estado_pago`	Significado
`FALLIDO`	Pago rechazado o fallido.
`REEMBOLSADO`	Pago devuelto.

4.11 Devolucion

Archivo: `app/models/devolucion.model.js`. Tabla: `Devolucion`. Solicitud de devolución o cambio sobre un pedido.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_devolucion`	INTEGER (PK, autoincremental)	No		Identificador de la solicitud.
`id_pedido`	INTEGER (FK a `Pedido`)	No		Pedido sobre el que se solicita.
`id_cliente`	INTEGER (FK a `Cliente`)	No		Clienta que solicita.
`tipo`	ENUM	No		Tipo de solicitud.
`motivo`	ENUM	No		Motivo declarado.
`estado`	ENUM	No	`'SOLICITADA'`	Estado del proceso.
`observacion`	STRING(255)	Sí		Comentario del gestor o de la clienta.
`id_empleado_gestor`	INTEGER (FK a `Empleado`)	Sí		Empleado que gestiona la solicitud.
`fecha_solicitud`	DATE	Sí	`NOW`	Fecha de la solicitud.
`fecha_actualizacion`	DATE	Sí		Última actualización de estado.

ENUM `tipo`	Significado
`DEVOLUCION`	Devolución de la prenda con reembolso.
`CAMBIO_TALLA`	Cambio por otra talla de la misma prenda.
`CAMBIO_VARIANTE`	Cambio por otra variante (por ejemplo, color) de la misma prenda.

ENUM `motivo`	Significado
`DEFECTUOSO`	La prenda presenta un defecto.
`TALLA_INCORRECTA`	La talla no corresponde.
`PRODUCTO_INCORRECTO`	Se entregó una prenda distinta.
`DANADO_ENVIO`	La prenda se dañó durante el envío.
`OTRO`	Otro motivo.

ENUM `estado`	Significado
`SOLICITADA`	Solicitud registrada.
`APROBADA`	Solicitud aceptada para recibir las prendas.

ENUM `estado`	Significado
`RECHAZADA`	Solicitud denegada antes de recibir las prendas.
`RECIBIDA`	Prendas recibidas en la sucursal.
`EN_REVISION`	Prendas en inspección.
`ACEPTADA`	Revisión favorable.
`RECHAZADA_EN_REVISION`	Revisión desfavorable.
`REINTEGRADA`	Prendas reintegradas al inventario (cierre de una devolución).
`CAMBIADA`	Cambio de talla o variante completado.

El diagrama de estados y las condiciones de cada transición están en Reglas de negocio.

4.12 DetalleDevolucion

Archivo: `app/models/detalle_devolucion.model.js`. Tabla: `DetalleDevolucion`. Cada fila indica qué línea del pedido se devuelve o se cambia, y en qué cantidad.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_detalle_devolucion`	INTEGER (PK, autoincremental)	No		Identificador.
`id_devolucion`	INTEGER (FK a `Devolucion`)	No		Solicitud a la que pertenece.
`id_detalle_pedido`	INTEGER (FK a `DetallePedido`)	No		Línea del pedido original.
`id_producto`	INTEGER (FK a `Producto`)	No		Producto devuelto.
`cantidad`	INTEGER	No		Unidades devueltas o cambiadas.
`id_producto_nuevo`	INTEGER (FK a `Producto`)	Sí		Variante que se entrega a cambio; solo en `CAMBIO_TALLA` y `CAMBIO_VARIANTE`.
`precio_unitario_nuevo`	DECIMAL(10,2)	Sí		Precio de la variante nueva cuando se pidió el cambio. La diferencia a reembolsar se calcula con este valor aunque después cambie el precio del catálogo.
`resultado_revision`	ENUM	No	`'PENDIENTE'`	Resultado de la inspección.

ENUM `resultado_revision`	Significado	Estado de inventario al reingresar
`PENDIENTE`	Sin revisar.	

ENUM `resultado_revision`	Significado	Estado de inventario al reingresar
`BUEN_ESTADO`	Prenda en buen estado.	`DISPONIBLE`
`DANADO`	Prenda dañada.	`DANADA`
`DEFECTUOSO`	Prenda con defecto de fábrica.	`DEFECTUOSA`

El modelo declara dos asociaciones con `Producto`: una por `id_producto` y otra por `id_producto_nuevo` (alias `ProductoNuevo` en el detalle y `DetallesComoProductoNuevo` en el producto).

4.13 MovimientoInventario

Archivo: `app/models/movimiento_inventario.model.js`. Tabla: `MovimientoInventario`. Historial inmutable de cada entrada y salida de stock: solo se crean registros y el controlador no expone operaciones de actualización ni eliminación.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_movimiento`	INTEGER (PK, autoincremental)	No		Identificador del movimiento.
`id_inventario`	INTEGER (FK a `Inventario`)	Sí		Fila de inventario afectada.
`id_producto`	INTEGER (FK a `Producto`)	No		Producto movido.
`id_pedido`	INTEGER (FK a `Pedido`)	Sí		Pedido que originó el movimiento.
`id_devolucion`	INTEGER (FK a `Devolucion`)	Sí		Devolución que originó el movimiento.
`id_empleado`	INTEGER (FK a `Empleado`)	Sí		Empleado responsable. `NULL` cuando lo origina la propia clienta (por ejemplo, al solicitar una devolución) o el sistema.
`tipo_movimiento`	ENUM	No		Tipo de movimiento (valores abajo).
`cantidad`	INTEGER	No		Unidades movidas.
`estado_anterior`	STRING(30)	Sí		Estado de las unidades antes del movimiento.
`estado_nuevo`	STRING(30)	Sí		Estado de las unidades después del movimiento.
`observacion`	STRING(255)	Sí		Comentario.
`fecha_movimiento`	DATE	Sí	`NOW`	Fecha y hora del movimiento.

ENUM `tipo_movimiento`	Significado
`VENTA`	Salida de stock por una venta.
`DEVOLUCION_SOLICITADA`	Registro de la solicitud de devolución.
`DEVOLUCION_RECIBIDA`	Recepción de las prendas devueltas.
`EN_REVISION`	Revisión de una prenda devuelta con resultado `BUEN_ESTADO`.
`REINTEGRACION`	Reingreso de unidades al inventario tras la revisión.
`CAMBIO_SALIDA`	Reingreso al inventario de la variante original que la clienta devuelve en un cambio, con el estado que corresponde a su revisión.
`CAMBIO_ENTRADA`	Descuento del stock `DISPONIBLE` de la variante nueva que se entrega a la clienta en un cambio.
`PRODUCTO_DANADO`	Revisión de una prenda devuelta con resultado `DANADO`.
`PRODUCTO_DEFECTUOSO`	Revisión de una prenda devuelta con resultado `DEFECTUOSO`.
`CANCELACION`	Unidades devueltas a stock al quitar un producto de un pedido `PENDIENTE` o al cancelarlo.
`AJUSTE_SALIDA`	Baja manual de stock desde el panel (Sucursales, ajuste): conteo, pérdida, daño u otra causa.

4.14 auditoria

Archivo: `app/models/auditoria.model.js`. Tabla: `auditoria` (en minúsculas). Bitácora genérica de acciones administrativas. Es el único modelo con las columnas automáticas `createdAt` y `updatedAt`.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_auditoria`	INTEGER (PK, autoincremental)	No		Identificador del registro.
`accion`	STRING	No		Código de la acción, por ejemplo `CLIENTE_REGISTRADO`, `PEDIDO_ESTADO_ACTUALIZADO`, `CLIENTE_CUENTA_WEB_ACTIVADA` o `EMPLEADO_PROMOVIDO_DISTRITAL`.
`tabla_afectada`	STRING	No		Tabla sobre la que se actuó (por ejemplo, `Pedido`, `Cliente`, `Empleado`).
`registro_afectado_id`	INTEGER	Sí		Identificador del registro afectado.
`detalles`	TEXT	Sí		Descripción legible de la acción.
`id_empleado`	INTEGER (FK a `Empleado`)	Sí		Empleado que realizó la acción; `NULL` si la realizó una clienta o el sistema. Columna agregada por la asociación `Empleado.hasMany(auditoria)`.

Columna	Tipo	Nulo	Por defecto	Descripción
`createdAt`	DATE	No	Automático	Fecha y hora de creación.
`updatedAt`	DATE	No	Automático	Fecha y hora de última modificación.

El mantenimiento de arranque (`app/utils/mantenimiento.js`) consulta esta tabla para completar datos antiguos: la sucursal de clientas registradas en mostrador (acción `CLIENTE_REGISTRADO`) y la `fecha_entrega` de pedidos entregados (acción `PEDIDO_ESTADO_ACTUALIZADO`).

4.15 Reembolso

Archivo: `app/models/reembolso.model.js` . Tabla: `Reembolso` . Dinero que se devuelve a la clienta por una devolución o por la diferencia de un cambio hacia una variante más barata. Con tarjeta en línea (`STRIPE`) el reembolso se procesa automáticamente en Stripe; con `EFECTIVO` , `TARJETA` (POS) o `PAYPAL` queda `PENDIENTE` hasta que caja lo entrega y lo confirma.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_reembolso`	INTEGER (PK, autoincremental)	No		Identificador del reembolso.
`id_devolucion`	INTEGER (FK a `Devolucion`)	Sí		Devolución que lo origina.
`id_pedido`	INTEGER (FK a `Pedido`)	No		Pedido reembolsado.
`id_pago`	INTEGER (FK a `Pago`)	Sí		Pago de Stripe sobre el que se hace el reembolso (solo método `STRIPE`).
`metodo`	ENUM	No		Medio de devolución del dinero.
`monto`	DECIMAL(10,2)	No		Monto a devolver.
`estado`	ENUM	No	`'PENDIENTE'`	Estado del reembolso.
`id_transaccion_externa`	STRING(150)	Sí		Identificador del reembolso en Stripe (`re_...`) o referencia del comprobante en caja.

Columna	Tipo	Nulo	Por defecto	Descripción
`id_empleado`	INTEGER (FK a `Empleado`)	Sí		Empleado que lo entregó en caja o que lo reintentó en Stripe.
`observacion`	STRING(255)	Sí		Comentario.
`procesando_desde`	DATE	Sí		Reserva mientras se comunica con Stripe: impide que dos reintentos simultáneos procesen el mismo reembolso. Una reserva de más de 2 minutos se considera abandonada.
`fecha_creacion`	DATE	No	`NOW`	Fecha de creación.
`fecha_completado`	DATE	Sí		Fecha en que se completó.

ENUM `metodo`	Significado
`STRIPE`	Reembolso automático a la tarjeta usada en la tienda en línea.
`EFECTIVO`	Entrega en efectivo en caja.
`TARJETA`	Devolución a través de la terminal POS.
`PAYPAL`	Devolución por PayPal, confirmada por caja.

ENUM `estado`	Significado
`PENDIENTE`	Por entregar o por procesar.
`COMPLETADO`	Dinero devuelto.
`FALLIDO`	El procesamiento falló; puede reintentarse.

4.16 Cupon

Archivo: `app/models/cupon.model.js`. Tabla: `Cupon` (la agrega la migración `0004` a las bases existentes; en una base nueva la crea la línea base `0001`). Cupón de descuento creado por el DISTRITAL; la clienta lo escribe al comprar en la tienda y caja lo aplica en una venta de mostrador. El código se guarda en mayúsculas y debe cumplir la expresión `^[A-Z0-9_-]{3,30}$` (`app/utils/cupones.js`).

Columna	Tipo	Nulo	Por defecto	Descripción
`id_cupon`	INTEGER (PK, autoincremental)	No		Identificador del cupón.
`codigo`	STRING(30), único	No		Código que escribe la clienta. Único también sin distinguir mayúsculas (índice `cupon_codigo_lower_unico`).

Columna	Tipo	Nulo	Por defecto	Descripción
`descripcion`	STRING(150)	Sí		Descripción visible.
`tipo`	ENUM	No		`PORCENTAJE` o `MONTO`.
`valor`	DECIMAL(10,2)	No		Con `PORCENTAJE`, porcentaje mayor que 0 y hasta 100; con `MONTO`, quetzales de descuento. Admite como máximo dos decimales (`cupon.controller.js`).
`minimo_compra`	DECIMAL(10,2)	No	`0`	Suma mínima de prendas para que el cupón aplique.
`desde`	DATEONLY	Sí		Primer día de vigencia (hora de Guatemala, incluido). `NULL` = sin límite.
`hasta`	DATEONLY	Sí		Último día de vigencia (incluido). `NULL` = sin límite.
`usos_maximos`	INTEGER	Sí		Usos permitidos en total. `NULL` = ilimitado.
`usos`	INTEGER	No	`0`	Usos realizados (pedidos no cancelados). Se incrementa al crear el pedido y se reduce al cancelarlo.
`id_sucursal`	INTEGER (FK a `Sucursal`)	Sí		Restringe el cupón a una sucursal. `NULL` = todas.
`activo`	BOOLEAN	No	`true`	Permite desactivar el cupón.
`fecha_creacion`	DATE	No	`NOW`	Fecha de creación.

ENUM `tipo`	Significado
`PORCENTAJE`	Descuento proporcional a la suma de prendas.
`MONTO`	Descuento fijo en quetzales, sin superar la suma de prendas.

5. Tablas creadas por migraciones sin modelo

Estas tablas no tienen un modelo en `app/models/` ; se crean y consultan con SQL directo.

5.1 SequelizeMeta

La crea Umzug (`SequelizeStorage` , configurado en `app/utils/migrador.js`) para registrar qué migraciones ya se aplicaron. Cada archivo de `app/migraciones/` se aplica una sola vez.

Columna	Tipo	Nulo	Por defecto	Descripción
`name`	varchar(255) (PK)	No		Nombre del archivo de migración aplicado, por ejemplo `0001-esquema-base.js` .

5.2 IntentoLimite (migración 0002)

Contadores del límite de intentos (inicio de sesión, registro, "Olvidé mi contraseña", entre otros). Se guardan en la base para que sobrevivan a un reinicio y se compartan entre varias instancias del servidor. La clave es un hash SHA-256 de la IP y la ruta, de modo que la IP no se almacena en claro.

Columna	Tipo	Nulo	Por defecto	Descripción
`clave`	varchar(64) (PK)	No		Hash SHA-256 de IP + ruta.
`n`	integer	No		Número de intentos en la ventana actual.
`desde`	timestamptz	No		Inicio de la ventana de conteo.

Índice: `intento_limite_desde` sobre `desde`.

5.3 ErrorServidor (migración 0003)

Registro de errores del servidor (respuestas 5xx y excepciones no controladas) que se muestra en el panel "Estado del sistema". Nunca se guarda el cuerpo de la petición, porque puede contener contraseñas.

Columna	Tipo	Nulo	Por defecto	Descripción
`id`	serial (PK)	No	Secuencia	Identificador del error.
`fecha`	timestamptz	No	`now()`	Fecha y hora del error.
`request_id`	varchar(36)	Sí		Identificador de la petición.
`metodo`	varchar(10)	Sí		Método HTTP.
`ruta`	varchar(300)	Sí		Ruta solicitada.
`estado`	integer	Sí		Código de estado HTTP.
`mensaje`	text	Sí		Mensaje del error.
`pila`	text	Sí		Pila de llamadas.
`usuario`	varchar(60)	Sí		Usuario asociado a la petición, si lo hay.

Índice: `error_servidor_fecha` sobre `fecha DESC`.

6. Llaves foráneas y acciones referenciales

Las llaves foráneas las genera Sequelize a partir de las asociaciones de `app/models/index.js` cuando la migración base crea cada tabla (`modelo.sync()`). Todas usan `ON UPDATE CASCADE`. La acción `ON DELETE` la asigna Sequelize según cómo declara el modelo la columna: `SET NULL` cuando el modelo indica `allowNull: true` (o cuando la columna la agrega la propia asociación, como `auditoria.id_empleado`) y `CASCADE` cuando indica `allowNull: false` o no especifica `allowNull`. Por esta razón `Producto.id_categoria`, `Inventario.id_producto`, `Inventario.id_sucursal` y `Pedido.id_empleado` usan `CASCADE` aunque admitan `NULL`:

Tabla.columna	Referencia	ON DELETE
`Empleado.id_sucursal`	`Sucursal`	SET NULL
`Producto.id_categoria`	`Categoria`	CASCADE
`Producto.id_producto_padre`	`Producto`	SET NULL
`Inventario.id_producto`	`Producto`	CASCADE
`Inventario.id_sucursal`	`Sucursal`	CASCADE
`Cupon.id_sucursal`	`Sucursal`	SET NULL
`Pedido.id_cliente`	`Cliente`	CASCADE
`Pedido.id_sucursal`	`Sucursal`	CASCADE
`Pedido.id_empleado`	`Empleado`	CASCADE
`Pedido.id_cupon`	`Cupon`	SET NULL
`DetallePedido.id_pedido`	`Pedido`	CASCADE
`DetallePedido.id_producto`	`Producto`	CASCADE
`Pago.id_pedido`	`Pedido`	CASCADE
`Devolucion.id_pedido`	`Pedido`	CASCADE
`Devolucion.id_cliente`	`Cliente`	CASCADE
`Devolucion.id_empleado_gestor`	`Empleado`	SET NULL
`DetalleDevolucion.id_devolucion`	`Devolucion`	CASCADE
`DetalleDevolucion.id_detalle_pedido`	`DetallePedido`	CASCADE
`DetalleDevolucion.id_producto`	`Producto`	CASCADE
`DetalleDevolucion.id_producto_nuevo`	`Producto`	SET NULL
`MovimientoInventario.id_inventario`	`Inventario`	SET NULL
`MovimientoInventario.id_producto`	`Producto`	CASCADE
`MovimientoInventario.id_pedido`	`Pedido`	SET NULL
`MovimientoInventario.id_devolucion`	`Devolucion`	SET NULL
`MovimientoInventario.id_empleado`	`Empleado`	SET NULL
`auditoria.id_empleado`	`Empleado`	SET NULL
`Reembolso.id_devolucion`	`Devolucion`	SET NULL
`Reembolso.id_pedido`	`Pedido`	CASCADE
`Reembolso.id_pago`	`Pago`	SET NULL
`Reembolso.id_empleado`	`Empleado`	SET NULL

En la práctica, la aplicación no elimina empleados, sucursales, productos ni clientas: los desactiva con su campo `activo`, de modo que el historial de pedidos, movimientos y auditoría se conserva. `Cliente.id_sucursal` no tiene llave foránea porque no existe una asociación declarada para esa columna.

7. Índices únicos e índices adicionales

Además de las restricciones `UNIQUE` declaradas en los modelos (`Categoria.nombre`, `Cliente.email` y `Cupon.codigo`), las migraciones crean índices únicos sobre `lower(...)` para que no existan dos valores que solo se diferencien en mayúsculas y minúsculas (por ejemplo, `Ana@correo.com` y `ana@correo.com`).

Índice	Tabla	Expresión	Migración	Propósito
`empleado_email_lower_unico`	`Empleado`	`lower(email)`	0001	Correos de empleados únicos sin distinguir mayúsculas.
`categoria_nombre_lower_unico`	`Categoria`	`lower(nombre)`	0001	Nombres de tipos de prenda únicos sin distinguir mayúsculas.
`cliente_email_lower_unico`	`Cliente`	`lower(email)`	0001	Correos de clientas únicos sin distinguir mayúsculas.
`cupon_codigo_lower_unico`	`Cupon`	`lower(codigo)`	0004	Códigos de cupón únicos sin distinguir mayúsculas.
`intento_limite_desde`	`IntentoLimite`	`desde`	0002	Índice no único para depurar contadores vencidos.
`error_servidor_fecha`	`ErrorServidor`	`fecha DESC`	0003	Índice no único para listar los errores más recientes.

Todos se crean con `CREATE ... INDEX IF NOT EXISTS`. Si la base ya contiene valores repetidos, la migración 0001 no puede crear el índice correspondiente: muestra una advertencia y el sistema continúa, ya que la aplicación también valida los duplicados sin distinguir mayúsculas antes de guardar.

8. Cálculo del total del pedido

El total de un pedido se calcula en un único lugar, la función `recalcularPedido` de `app/utils/totales.js`, que utilizan los hooks de `DetallePedido` y la creación de pedidos. Las reglas de precio auxiliares están en `app/utils/precios.js`.

Fórmula:

```
lineas     = suma de DetallePedido.subtotal del pedido
descuento  = descuentoDeCupon(copia del cupón del pedido, lineas)
envio      = 0                                  si lineas <= 0
           = costoEnvio(lineas - descuento)     si es pedido en línea a domicilio
           = Pedido.costo_envio (sin cambios)   en otro caso
total      = max(0, lineas - descuento) + envio
```

Todos los montos se redondean a dos decimales. La función guarda `total`, `descuento` y `costo_envio` en el pedido.

Descuento del cupón (`descuentoDeCupon`). Es 0 si no hay cupón, si la suma de líneas es 0 o si no alcanza `cupon_minimo`. Con `PORCENTAJE` vale `lineas × valor / 100`; con `MONTO` vale `valor`. En ambos casos se limita al rango entre 0 y la suma de líneas, por lo que el descuento nunca supera el valor de las prendas.

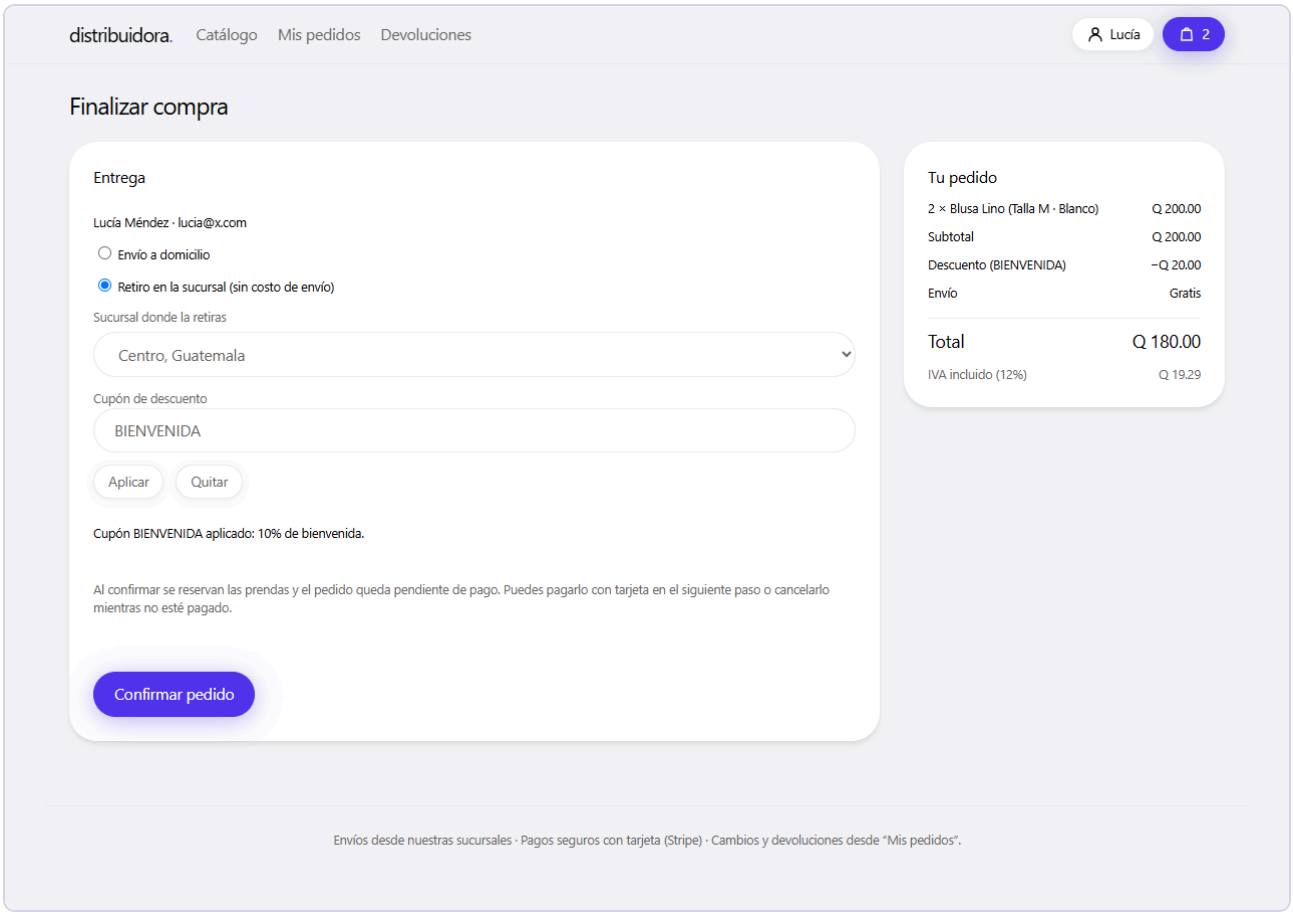
Envío (`costoEnvio`). Un pedido se considera "en línea a domicilio" cuando `id_empleado` es `NULL` y tiene `direccion_entrega`. En ese caso el envío se recalcula cada vez que cambian las líneas, porque agregar o quitar prendas puede alcanzar el umbral de envío gratis o dejar de alcanzarlo. La comparación con el umbral se hace en centavos para evitar errores de coma flotante. Las ventas de mostrador y los retiros en sucursal no pagan envío.

Variable de entorno	Valor por defecto	Uso
`ENVIO_COSTO`	`25`	Costo del envío a domicilio, en quetzales.
`ENVIO_GRATIS_DESDE`	`300`	Monto de prendas (después del descuento) desde el cual el envío es gratis; `0` significa que nunca es gratis.
`IVA_PORCENTAJE`	`12`	Porcentaje de IVA que se desglosa del total.

IVA. Los precios del catálogo ya incluyen IVA. El sistema no lo suma: lo desglosa a partir del total con `desgloseIva`, donde `base_imponible = total / (1 + IVA/100)` e `iva = total − base_imponible`.

Ejemplo. Una compra en línea con retiro en sucursal de 2 unidades de una blusa de Q100.00, con un cupón `PORCENTAJE` del 10 %:

Concepto	Cálculo	Monto
Suma de líneas	2 × 100.00	Q200.00
Descuento	200.00 × 10 / 100	Q20.00
Envío	Retiro en sucursal	Q0.00
Total	max(0, 200.00 − 20.00) + 0.00	Q180.00
IVA incluido (12 %)	180.00 − 180.00 / 1.12	Q19.29



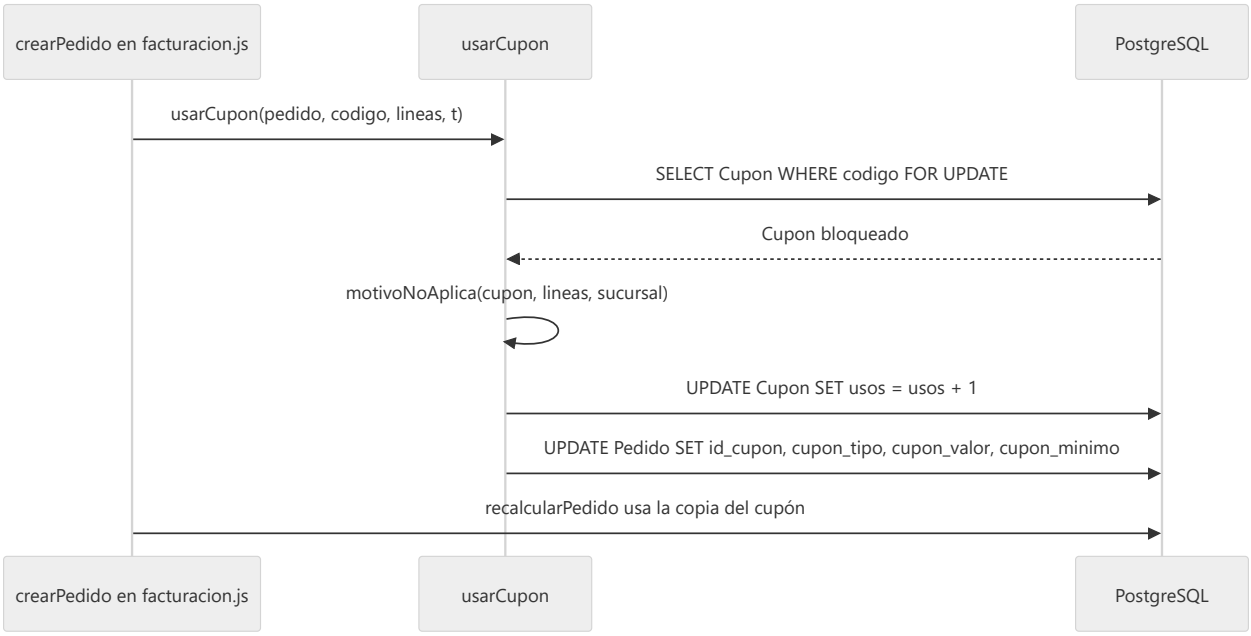
Resumen del pedido en la tienda en línea con cupón aplicado

Pantalla "Finalizar compra" de la tienda en línea: subtotal de Q200.00, descuento de Q20.00 del cupón BIENVENIDA (10 %), envío gratis por retiro en sucursal, total de Q180.00 e IVA incluido de Q19.29.

9. Copia del cupón en el pedido

Cuando un pedido usa un cupón, la función `crearPedido` (`app/utils/facturacion.js`) guarda primero las líneas y luego invoca `usarCupon` (`app/utils/cupones.js`), que realiza, dentro de la transacción de creación del pedido, los siguientes pasos:

1. Bloquea la fila del cupón (`SELECT ... FOR UPDATE`) para que dos compras simultáneas no superen el límite de usos.
2. Valida que el cupón exista, esté activo, esté vigente según la fecha de Guatemala, tenga usos disponibles, corresponda a la sucursal del pedido y que la suma de prendas alcance la compra mínima.
3. Incrementa `Cupon.usos` en 1.
4. Copia en el pedido `id_cupon` , `cupon_tipo` (desde `tipo`), `cupon_valor` (desde `valor`) y `cupon_minimo` (desde `minimo_compra`).



`recalcularPedido` utiliza exclusivamente la copia guardada en el pedido (`cupon_tipo`, `cupon_valor`, `cupon_minimo`) y no la fila actual de `Cupon`. Por ello, si el DISTRITAL edita el valor o la compra mínima de un cupón después de usado, los pedidos ya creados conservan su precio. La copia solo se usa si el pedido tiene `id_cupon` y `cupon_tipo`.

Al cancelar un pedido con cupón, `liberarCupon` devuelve el uso con `usos = GREATEST(usos - 1, 0)`. Para no revelar qué códigos existen, un cupón inexistente, inactivo, fuera de vigencia, agotado o de otra sucursal recibe el mismo mensaje ("Ese cupón no es válido para esta compra."); solo la falta de la compra mínima se explica con el monto requerido.

La migración `0005-cupon-en-pedido.js` agregó las tres columnas de copia y rellenó los pedidos que ya tenían cupón con los datos vigentes en ese momento.

10. Evolución del esquema (migraciones)

Las migraciones están en `app/migraciones/`, se ejecutan en orden por nombre y deben poder ejecutarse dos veces sin fallar. Las herramientas de `app/utils/esquema.js` solo agregan tablas, columnas y valores de ENUM; nunca eliminan ni cambian el tipo de una columna existente.

Migración	Cambios
`0001-esquema-base.js`	Línea base: crea las tablas, columnas y valores de ENUM que falten a partir de los modelos, y los índices únicos `lower()` de `Empleado.email`, `Categoria.nombre` y `Cliente.email`. No se puede deshacer.
`0002-intentos-limite.js`	Crea la tabla `IntentoLimite` y su índice.

Migración	Cambios
`0003-errores-servidor.js`	Crea la tabla `ErrorServidor` y su índice.
`0004-envio-cupones.js`	Crea la tabla `Cupon`, agrega a `Pedido` las columnas `costo_envio`, `descuento` e `id_cupon`, y crea el índice `cupon_codigo_lower_unico`.
`0005-cupon-en-pedido.js`	Agrega a `Pedido` las columnas `cupon_tipo`, `cupon_valor` y `cupon_minimo` y las rellena en los pedidos existentes con cupón.

Después de aplicar las migraciones, el servidor compara los modelos con la base (`diferencias`) y emite una advertencia si algún modelo tiene columnas o valores de ENUM que la base no tiene, lo que indica que falta escribir una migración. El procedimiento de ejecución, respaldo y restauración se describe en Despliegue y operación.

Roles y permisos

Este documento describe el modelo de autorización del sistema Distribuidora: los puestos de empleado que reconoce la API, la cuenta de la clienta de la tienda en línea, el alcance de datos por sucursal, las reglas para gestionar empleados, la matriz completa de permisos por endpoint, el menú del panel según el rol y el mecanismo de invalidación de sesiones. Todo lo expuesto se obtuvo de `app/middlewares/authJwt.js`, `app/utils/alcance.js`, `app/utils/empleados.js`, los archivos `app/routes/*.js`, los controladores que aplican reglas adicionales y `frontend/src/utils/permisos.js`.

Contenido

1. Principios generales
2. Roles del sistema
3. Autenticación y controles de rol
4. Alcance por sucursal
5. Gestión de empleados
6. Matriz de permisos
7. Menú del panel por rol
8. Invalidación de sesiones (token_version)
9. Códigos de respuesta de autorización

---

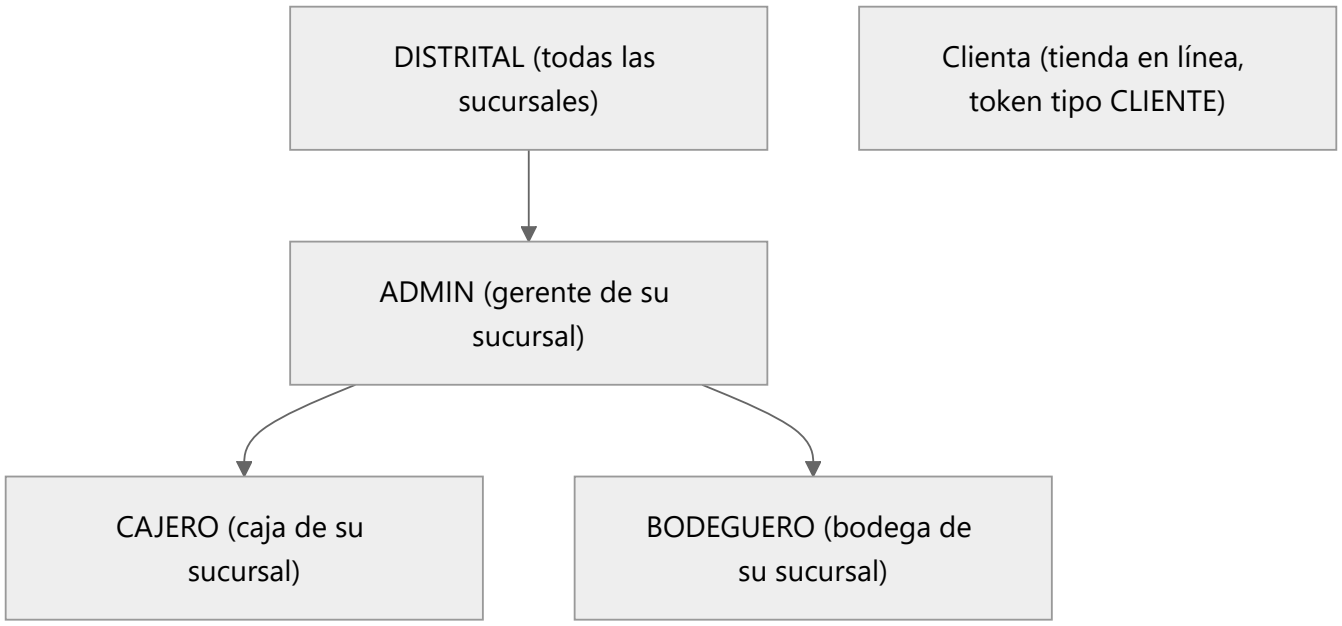
1. Principios generales

- **El servidor decide.** El panel oculta opciones según el rol (`frontend/src/utils/permisos.js` indica expresamente: "el servidor siempre valida"), pero cada endpoint aplica sus propios middlewares de autenticación, de rol y de alcance.
- **Dos tipos de token separados.** El token de empleado (panel) y el token de clienta (tienda) se firman con el mismo secreto `JWT_SECRET`, pero el contenido los distingue: un token de clienta lleva `tipo: "CLIENTE"` y es rechazado en las rutas de empleados, y un token de empleado es rechazado en las rutas de la tienda.
- **El puesto se lee de la base de datos en cada petición.** Los controles de rol no confían en el contenido del token: consultan el registro `Empleado` vigente, de modo que un cambio de puesto o una desactivación se aplican de inmediato.
- **Jerarquía de puestos.** Los controles se construyen de forma acumulativa: el DISTRITAL satisface cualquier control de rol; el ADMIN satisface los de ADMIN, CAJERO y BODEGUERO; CAJERO y BODEGUERO solo el suyo.
- **Fuera de alcance equivale a inexistente.** Un registro de otra sucursal responde `404`, igual que uno que no existe, para no revelar información de otras sucursales.

2. Roles del sistema

Los puestos válidos se definen en `PUESTOS_VALIDOS = ["DISTRITAL", "ADMIN", "CAJERO", "BODEGUERO"]` (`app/middlewares/authJwt.js`). El valor se normaliza a mayúsculas y sin espacios antes de compararlo.

Rol	Descripción	Sucursal	Responsabilidades principales
DISTRITAL	Gerente distrital. Está por encima de todos los demás puestos.	No tiene sucursal (`id_sucursal = null`). Ve y opera todas.	Catálogo único (prendas, tipos de prenda y fotos), sucursales, promociones (cupones), auditoría, estado del sistema y gestión de cualquier empleado. Puede realizar además todo lo que hacen los demás puestos, en cualquier sucursal.
ADMIN	Gerente de una sucursal.	Obligatoria. Solo opera la suya.	Aprobar o rechazar devoluciones, gestionar cajeros y bodegueros de su sucursal, reportes de ventas de su sucursal, activar o desactivar clientes, eliminar proveedores. Además puede realizar las tareas de caja y de bodega.
CAJERO	Personal de caja.	Obligatoria.	Ventas de mostrador, cobros, anulación y devolución de pagos, cancelación de pedidos pendientes, alta y edición de clientes, códigos de activación web, registro de solicitudes de devolución, entrega de reembolsos.
BODEGUERO	Personal de bodega.	Obligatoria.	Ingreso y bajas de stock, despacho y entrega de pedidos, recepción, revisión, reintegro o cambio en devoluciones, alta y edición de proveedores.
Clienta	Usuaria de la tienda en línea. No es un empleado.	Ninguna (su ficha puede registrar la sucursal donde se dio de alta en mostrador).	Gestionar su perfil y contraseña, comprar, pagar con tarjeta, cancelar sus pedidos pendientes y solicitar devoluciones o cambios de sus propios pedidos.



La flecha indica que el puesto superior satisface los controles de rol del inferior. La clienta queda fuera de esa jerarquía: su token solo es válido en `/api/tienda`.

3. Autenticación y controles de rol

3.1 Token de empleado (`verificarToken`)

El token se envía en la cabecera `Authorization: Bearer <token>` y se emite en `POST /api/auth/login` mediante `firmarToken` (`app/utils/empleados.js`). Su contenido es `{ id, id_sucursal, v }` y vence a las 24 horas (`expiresIn: 86400`).

El middleware `verificarToken` realiza, en orden:

1. Sin cabecera `Authorization`: responde `403`.
2. Firma inválida o token vencido: responde `401`.
3. Token con `tipo: "CLIENTE"` o sin `id`: responde `403` ("Este token no pertenece a un empleado").
4. Empleado inexistente o con `activo = false`: responde `401`.
5. Versión del token (`v`) distinta de `Empleado.token_version`: responde `401` ("La sesión ya no es válida").
6. Si todo es correcto, deja el empleado en `req.empleado` y su id en `req.empleadoId`.

3.2 Token de clienta (`verificarCliente`)

Se emite en `POST /api/tienda/auth/login` (y también al registrarse, al activar la cuenta, al restablecer la contraseña y al cambiarla desde el perfil). Su contenido es `{ id_cliente, tipo: "CLIENTE", v }`, con vigencia de 24 horas. El middleware rechaza con `403` cualquier token que no sea de tipo `CLIENTE`, con `401` una cuenta inexistente o desactivada y con `401` una versión de sesión desactualizada. Deja el id en `req.clienteId`.

Las rutas de la tienda solo devuelven datos de la propia clienta: un pedido o una devolución de otra clienta responde `404` ("No se encontró el pedido." o "No se encontró la devolución."), sin revelar su existencia.

3.3 Controles de rol

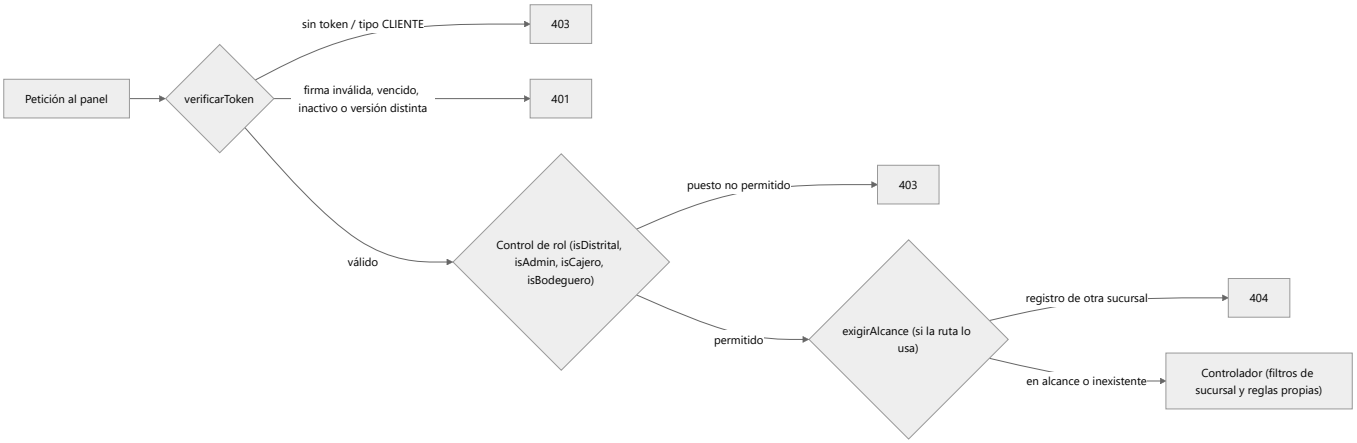
Los controles se generan con la función `exigirPuesto(puestos, mensaje)`, que lee el puesto del empleado ya cargado y responde `403` si no está en la lista.

Middleware	Puestos aceptados	Uso típico
`isDistrital`	DISTRITAL	Catálogo, sucursales, cupones, auditoría, estado del sistema, fotos.
`isAdmin`	ADMIN, DISTRITAL	Empleados, aprobación de devoluciones, tablero de ventas, eliminar proveedores.
`isCajero`	CAJERO, ADMIN, DISTRITAL	Ventas, pagos, clientes, solicitudes de devolución, reembolsos.

Middleware	Puestos aceptados	Uso típico
`isBodeguero`	BODEGUERO, ADMIN, DISTRITAL	Inventario, proveedores, etapas físicas de las devoluciones.
`permitirRegistro`	Sin token si la tabla `Empleado` está vacía; en otro caso `verificarToken` + `isAdmin`.	`POST /api/auth/registro`.

Algunos controladores aplican reglas adicionales de rol dentro de la lógica:

- `PUT /api/pedidos/:id/estado` usa `PUESTOS_POR_ESTADO` (`pedido.controller.js`): `PROCESADO` y `CANCELADO` los pueden asignar DISTRITAL, ADMIN y CAJERO; `ENVIADO` y `ENTREGADO`, DISTRITAL, ADMIN y BODEGUERO.
- `PUT /api/clientes/:id` con el campo `activo` exige ADMIN o DISTRITAL aunque la ruta acepte a CAJERO.
- Las reglas de gestión de empleados se describen en la sección 5.



4. Alcance por sucursal

El módulo `app/utils/alcance.js` define qué datos ve y opera cada empleado.

Puesto	Alcance (`sucursalDe(req)`)	Efecto
DISTRITAL	`null` (todas)	Sin filtro. En listados y reportes puede elegir una sucursal con `?id_sucursal=`.
ADMIN, CAJERO, BODEGUERO con sucursal	Su `id_sucursal`	Solo datos de esa sucursal, aunque se pida otra por filtro.
Cualquier puesto distinto de DISTRITAL sin sucursal	`-1` (`SIN_SUCURSAL`)	No ve datos de ninguna sucursal.

Las funciones del módulo se aplican de tres formas:

1. `whereSucursal(req)` agrega la condición `id_sucursal = <propia>` a las consultas de listados: pedidos, detalles, pagos, devoluciones, reembolsos, inventario, stock y empleados. Los reportes y la trazabilidad de una prenda aplican el mismo filtro leyendo directamente `sucursalDe(req)`.
2. `exigirAlcance(cargador)` es un middleware que, antes del controlador, obtiene la sucursal del registro afectado (pedido, pago, detalle o devolución, siempre a través del pedido) y responde `404` ("No se encontró el registro en tu sucursal") si pertenece a otra. Si el registro no existe, deja pasar la petición para que el controlador responda su propio `404`.
3. `enAlcance(req, idSucursal)` se usa cuando la sucursal llega en el cuerpo de la petición (registrar una venta, ingresar o ajustar stock). En ese caso la respuesta es `403` con un mensaje explícito ("Solo puedes registrar ventas en tu sucursal", "Solo puedes ingresar stock en tu sucursal", "Solo puedes ajustar el stock de tu sucursal").

4.1 Clientes visibles para una sucursal

Las clientas no pertenecen de forma exclusiva a una sucursal. La función `whereClientesDeSucursal(idSucursal)` considera que una clienta es visible para una sucursal cuando se cumple al menos una de estas condiciones:

- se registró en esa sucursal (`Cliente.id_sucursal`), o
- tiene algún pedido en esa sucursal cuyo estado no sea `CANCELADO`.

`clienteEnAlcance(req, idCliente)` aplica esta regla al editar una clienta o generar su código web (fuera de alcance: `404`). Existen dos excepciones controladas:

- `GET /api/clientes?email=<correo exacto>` busca en todas las sucursales y devuelve solo datos básicos (id, nombre, apellido, correo, estado y si tiene cuenta web), para atender a una clienta registrada en otra sucursal.
- Al registrar una venta para una clienta de otra sucursal, la API la acepta solo si la petición incluye `email_cliente` igual al correo de la clienta; así no es posible descubrir clientas ajenas probando identificadores. Tras la venta la clienta pasa a ser visible para la sucursal.

4.2 Datos globales

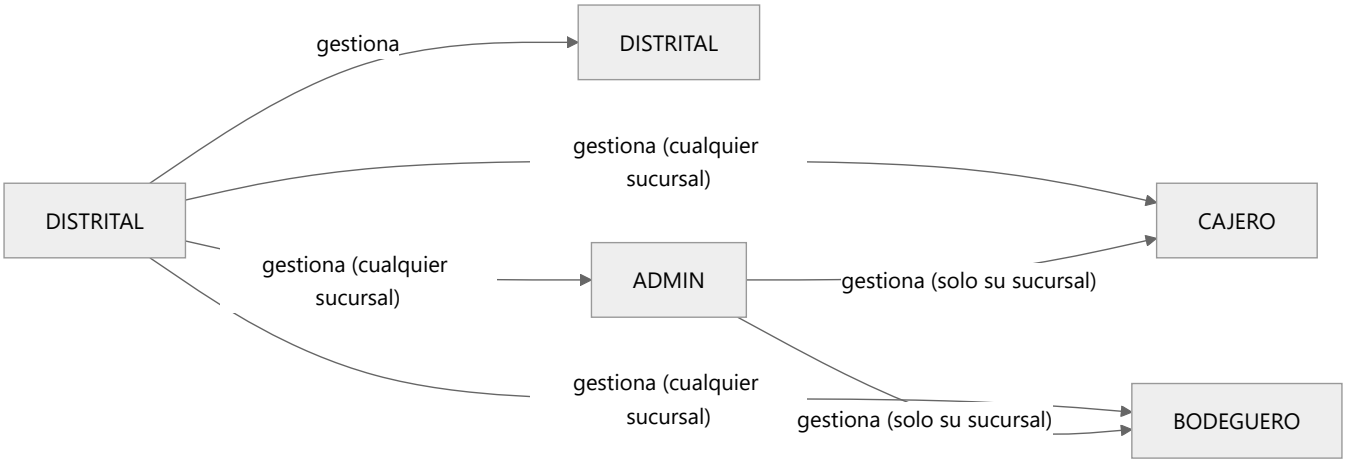
Algunos datos son comunes a todas las sucursales y no se filtran: el catálogo de prendas y tipos de prenda (consulta pública), la lista de sucursales (consulta pública), los proveedores (cualquier empleado autenticado los consulta) y los cupones (solo DISTRITAL).

5. Gestión de empleados

Las reglas se implementan en `aplicarReglasDeGestion` (`app/utils/empleados.js`) y en `empleado.controller.js` / `auth.controller.js` . Solo pueden gestionar empleados los puestos ADMIN y DISTRITAL (`isAdmin` en todas las rutas de `/api/empleados` y en el registro).

5.1 Quién gestiona a quién

Quien opera	Puede crear, editar, desactivar y restablecer la contraseña de	Restricciones
DISTRITAL	Cualquier empleado de cualquier sucursal y con cualquier puesto, incluido otro DISTRITAL.	Un DISTRITAL siempre queda sin sucursal (`id_sucursal` se fuerza a `null`). Cualquier otro puesto requiere sucursal; si falta, la API responde `400` ("Asigna una sucursal: solo el puesto DISTRITAL trabaja sin sucursal").
ADMIN	Solo CAJERO y BODEGUERO de su propia sucursal.	Solo puede asignar los puestos CAJERO o BODEGUERO y solo a su sucursal (`403` en otro caso). En un alta, la sucursal del nuevo empleado se fija en la del ADMIN. Si el ADMIN no tiene sucursal asignada, responde `403` . Un empleado de otra sucursal no existe para él (`404`).
CAJERO , BODEGUERO	Nadie.	Las rutas de empleados responden `403` .



5.2 Altas

- **`POST /api/auth/registro`** crea un empleado con contraseña, que puede iniciar sesión de inmediato. Valida la contraseña (mínimo 8 caracteres y máximo 72 bytes), el nombre, el correo (único, sin distinguir mayúsculas) y el puesto.
- **`POST /api/empleados`** crea una ficha **sin contraseña**: el empleado no puede iniciar sesión hasta que un superior le asigne una con `PUT /api/empleados/:id/password` .
- No existe eliminación de empleados: quien deja la empresa se desactiva (`activo = false`) para conservar su historial en pedidos, movimientos y auditoría.

5.3 Primer registro

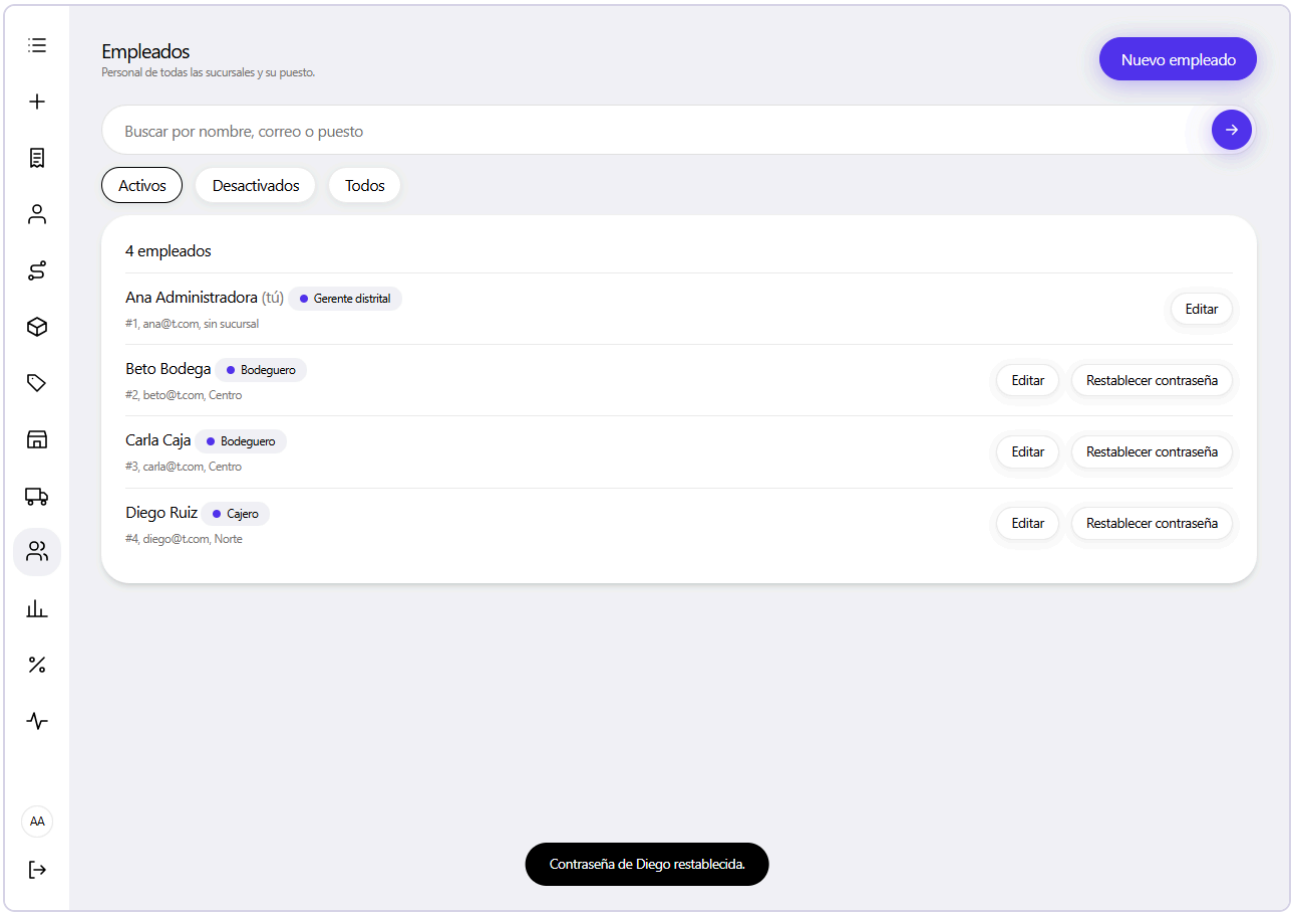
Con la tabla `Empleado` vacía, `permitirRegistro` deja pasar `POST /api/auth/registro` sin token. El controlador fuerza el puesto **DISTRITAL** y la sucursal `null`, independientemente de lo enviado, y registra en auditoría la acción `EMPLEADO_REGISTRO_INICIAL`. Para evitar que dos registros simultáneos creen dos cuentas iniciales, la transacción toma un candado consultivo de PostgreSQL (`pg_advisory_xact_lock`) y vuelve a contar los empleados; si otro registro ganó, responde `403`. A partir de ese momento solo un ADMIN o DISTRITAL autenticado puede registrar empleados.

5.4 Reglas al editar (`PUT /api/empleados/:id`)

- Ningún empleado puede desactivarse ni cambiarse el puesto a sí mismo (`400`); tampoco cambiarse de sucursal. Sí puede editar su nombre, apellido y correo.
- **Siempre debe existir al menos un DISTRITAL activo con contraseña.** Si el cambio desactivaría o quitaría el puesto al último DISTRITAL utilizable, la API responde `400` ("Debe quedar al menos un DISTRITAL activo").
- La operación se ejecuta en una transacción que bloquea (`SELECT ... FOR UPDATE`) a todos los empleados activos con acceso antes de leer al empleado editado. Esto impide que dos cambios simultáneos dejen el sistema sin DISTRITAL y verifica que quien edita siga siendo ADMIN o DISTRITAL activo en el momento de aplicar el cambio.
- Cada cambio efectivo queda en la auditoría (`EMPLEADO_ACTUALIZADO`) con el detalle de los valores anteriores y nuevos.

5.5 Restablecer contraseña (`PUT /api/empleados/:id/password`)

El DISTRITAL puede asignar una contraseña temporal a cualquier empleado; el ADMIN, solo a cajeros y bodegueros de su sucursal (otro ADMIN de su sucursal: `403`; un empleado de otra sucursal: `404`). Se rechaza si el empleado no tiene correo, porque no podría iniciar sesión. La operación cierra todas las sesiones abiertas de ese empleado (ver sección 8).



Pantalla Empleados del panel vista por un DISTRITAL

Figura 1. Pantalla Empleados vista por el gerente distrital: lista el personal de todas las sucursales con su puesto y sucursal; en su propia fila solo aparece "Editar", mientras que en las demás se ofrecen "Editar" y "Restablecer contraseña". La barra lateral muestra el menú completo del rol DISTRITAL.

6. Matriz de permisos

Convenciones:

- **Sí**: permitido sin restricción de sucursal.
- **Su sucursal**: permitido solo sobre datos de la sucursal del empleado (los registros de otras sucursales no aparecen en listados o responden `404` / `403`, según la sección 4).
- **—**: denegado (`403` , o `401` / `403` por tipo de token).
- **Público**: no requiere token.
- La columna DISTRITAL siempre abarca todas las sucursales.
- La columna Clienta se refiere al token de la tienda; en las rutas del panel ese token es rechazado.

6.1 Autenticación de empleados (`/api/auth`)

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Iniciar sesión en el panel	`POST /api/auth/login`	Público	Público	Público	Público	Público
Registrar empleado con contraseña	`POST /api/auth/registro`	Sí	Su sucursal (solo CAJERO/BODEGUERO)	—	—	—
Registrar el primer empleado (solo con la tabla `Empleado` vacía; queda como DISTRITAL)	`POST /api/auth/registro`	Público	Público	Público	Público	Público
Consultar la sesión actual	`GET /api/auth/yo`	Sí	Sí	Sí	Sí	—
Cambiar la propia contraseña	`PUT /api/auth/password`	Sí	Sí	Sí	Sí	—

6.2 Empleados (`/api/empleados`)

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar empleados	`GET /api/empleados`	Sí	Su sucursal	—	—	—
Crear ficha sin contraseña	`POST /api/empleados`	Sí	Su sucursal (solo CAJERO/BODEGUERO)	—	—	—
Editar o desactivar empleado	`PUT /api/empleados/:id`	Sí	Su sucursal (solo CAJERO/BODEGUERO)	—	—	—
Restablecer contraseña	`PUT /api/empleados/:id/password`	Sí	Su sucursal (solo CAJERO/BODEGUERO)	—	—	—

6.3 Sucursales (`/api/sucursales`)

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar sucursales	`GET /api/sucursales`	Público	Público	Público	Público	Público
Crear sucursal	`POST /api/sucursales`	Sí	—	—	—	—
Editar sucursal	`PUT /api/sucursales/:id`	Sí	—	—	—	—

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Eliminar sucursal (solo sin historial; con historial responde `409` y corresponde desactivarla)	`DELETE /api/sucursales/:id`	Sí	—	—	—	—

6.4 Catálogo: prendas, tipos de prenda y fotos

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar prendas	`GET /api/productos`	Público	Público	Público	Público	Público
Crear, editar o eliminar prenda	`POST`, `PUT`, `DELETE /api/productos[/:id]`	Sí	—	—	—	—
Trazabilidad de una prenda	`GET /api/productos/:id/historial`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Listar tipos de prenda	`GET /api/categorias`	Público	Público	Público	Público	Público
Crear, editar o eliminar tipo de prenda	`POST`, `PUT`, `DELETE /api/categorias[/:id]`	Sí	—	—	—	—
Configuración de fotos (Cloudinary)	`GET /api/imagenes/config`	Sí	Sí	Sí	Sí	—
Firmar subida de foto	`POST /api/imagenes/firma`	Sí	—	—	—	—
Descartar foto subida	`POST /api/imagenes/descartar`	Sí	—	—	—	—

6.5 Inventario y proveedores

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Consultar inventario	`GET /api/inventario`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Consultar stock disponible	`GET /api/inventario/stock`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Ingresar stock	`POST /api/inventario`	Sí	Su sucursal	—	Su sucursal	—
Ajuste (baja) de stock	`POST /api/inventario/ajustes`	Sí	Su sucursal	—	Su sucursal	—
Listar proveedores	`GET /api/proveedores`	Sí	Sí	Sí	Sí	—
Crear o editar proveedor	`POST /api/proveedores`, `PUT /api/proveedores/:id`	Sí	Sí	—	Sí	—
Eliminar proveedor	`DELETE /api/proveedores/:id`	Sí	Sí	—	—	—

6.6 Clientes (`/api/clientes`)

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar clientes	`GET /api/clientes`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Buscar por correo exacto (todas las sucursales, datos básicos)	`GET /api/clientes?email=`	Sí	Sí	Sí	Sí	—
Alta en mostrador	`POST /api/clientes`	Sí (puede indicar sucursal)	Su sucursal	Su sucursal	—	—
Editar datos de contacto	`PUT /api/clientes/:id`	Sí	Su sucursal	Su sucursal	—	—
Activar o desactivar cliente	`PUT /api/clientes/:id` con `activo`	Sí	Su sucursal	—	—	—
Generar código de activación web	`POST /api/clientes/:id/codigo-web`	Sí	Su sucursal	Su sucursal	—	—

6.7 Pedidos y detalle de pedido

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar pedidos	`GET /api/pedidos`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Ver pedido	`GET /api/pedidos/:id`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Historial del pedido	`GET /api/pedidos/:id/historial`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Cotizar venta de mostrador	`POST /api/pedidos/cotizar`	Sí	Su sucursal	Su sucursal	—	—
Registrar venta de mostrador	`POST /api/pedidos`	Sí	Su sucursal	Su sucursal	—	—
Cambiar estado a PROCESADO o CANCELADO	`PUT /api/pedidos/:id/estado`	Sí	Su sucursal	Su sucursal	—	—
Cambiar estado a ENVIADO o ENTREGADO	`PUT /api/pedidos/:id/estado`	Sí	Su sucursal	—	Su sucursal	—
Listar o ver líneas de pedido	`GET /api/detalles` , `GET /api/detalles/:id`	Sí	Su sucursal	Su sucursal	Su sucursal	—

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Agregar o quitar líneas	`POST /api/detalles`, `DELETE /api/detalles/:id`	Sí	Su sucursal	Su sucursal	—	—

6.8 Pagos y reembolsos

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar pagos	`GET /api/pagos`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Registrar cobro	`POST /api/pagos`	Sí	Su sucursal	Su sucursal	—	—
Confirmar pago	`PUT /api/pagos/:id/confirmar`	Sí	Su sucursal	Su sucursal	—	—
Anular pago	`PUT /api/pagos/:id/anular`	Sí	Su sucursal	Su sucursal	—	—
Devolver cobro presencial de un pedido pendiente	`PUT /api/pagos/:id/devolver`	Sí	Su sucursal	Su sucursal	—	—
Listar reembolsos	`GET /api/reembolsos`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Completar reembolso (entrega en caja)	`PUT /api/reembolsos/:id/completar`	Sí	Su sucursal	Su sucursal	—	—
Reintentar reembolso en Stripe	`POST /api/reembolsos/:id/reintentar`	Sí	Su sucursal	Su sucursal	—	—
Webhook de Stripe (sin token; lo invoca Stripe y se valida con su firma)	`POST /api/webhook`	No aplica	No aplica	No aplica	No aplica	No aplica

6.9 Devoluciones y cambios (`/api/devoluciones`)

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Listar devoluciones	`GET /api/devoluciones`	Sí	Su sucursal	Su sucursal	Su sucursal	—

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Ver devolución y su historial	`GET /api/devoluciones/:id`, `GET /api/devoluciones/:id/historial`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Cantidades devolvibles de un pedido	`GET /api/devoluciones/pedido/:id_pedido/devolvible`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Registrar solicitud en mostrador	`POST /api/devoluciones`	Sí	Su sucursal	Su sucursal	—	—
Aprobar o rechazar	`PUT /api/devoluciones/:id/aprobar`, `/rechazar`	Sí	Su sucursal	—	—	—
Recibir y revisar mercadería	`PUT /api/devoluciones/:id/recibir`, `/revisar`	Sí	Su sucursal	—	Su sucursal	—
Reintegrar al stock o entregar cambio	`PUT /api/devoluciones/:id/reintegrar`, `/cambiar`	Sí	Su sucursal	—	Su sucursal	—

6.10 Reportes, promociones, auditoría y sistema

Acción	Endpoint	DISTRITAL	ADMIN	CAJERO	BODEGUERO	Clienta
Historial de ventas	`GET /api/reportes/ventas`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Stock crítico	`GET /api/reportes/stock-critico`	Sí	Su sucursal	Su sucursal	Su sucursal	—
Tablero de ventas para gerentes	`GET /api/reportes/resumen`	Sí	Su sucursal	—	—	—
Listar, crear, editar o eliminar cupones	`GET`, `POST`, `PUT`, `DELETE /api/cupones[/:id]`	Sí	—	—	—	—
Consultar o registrar auditoría	`GET`, `POST /api/auditoria`	Sí	—	—	—	—
Salud básica de la API	`GET /api/salud`	Público	Público	Público	Público	Público
Detalle del sistema y errores recientes	`GET /api/salud/detalle`, `GET /api/salud/errores`	Sí	—	—	—	—

6.11 Tienda en línea (`/api/tienda`)

Estas rutas no aceptan tokens de empleado. Las operaciones con sesión actúan únicamente sobre los datos de la clienta autenticada.

Acción	Endpoint	Sin sesión	Clienta	Empleados
Registrarse	`POST /api/tienda/auth/registro`	Sí	Sí	No aplica
Iniciar sesión	`POST /api/tienda/auth/login`	Sí	Sí	No aplica
Activar cuenta con código de caja	`POST /api/tienda/auth/activar`	Sí	Sí	No aplica
Solicitar enlace de restablecimiento	`POST /api/tienda/auth/olvide`	Sí	Sí	No aplica
Restablecer contraseña con el enlace	`POST /api/tienda/auth/restablecer`	Sí	Sí	No aplica
Ver o editar su perfil	`GET` , `PUT /api/tienda/perfil`	—	Sí (propio)	—
Cambiar su contraseña	`PUT /api/tienda/perfil/password`	—	Sí (propio)	—
Ver catálogo y detalle de prenda	`GET /api/tienda/catalogo` , `GET /api/tienda/catalogo/:id`	Sí	Sí	Sí
Cotizar carrito	`POST /api/tienda/cotizar`	—	Sí	—
Crear pedido	`POST /api/tienda/pedidos`	—	Sí	—
Ver sus pedidos	`GET /api/tienda/pedidos` , `GET /api/tienda/pedidos/:id`	—	Sí (propios)	—
Cancelar su pedido	`POST /api/tienda/pedidos/:id/cancelar`	—	Sí (propios)	—
Iniciar y confirmar pago con tarjeta	`POST /api/tienda/pedidos/:id/pago` , `/pago/confirmar`	—	Sí (propios)	—
Cantidades devolvibles	`GET /api/tienda/pedidos/:id/devolvible`	—	Sí (propios)	—
Solicitar devolución o cambio	`POST /api/tienda/devoluciones`	—	Sí (propios)	—
Ver sus devoluciones	`GET /api/tienda/devoluciones` , `GET /api/tienda/devoluciones/:id`	—	Sí (propias)	—

Las rutas de cuenta de la tienda tienen límite de intentos por IP (`limitarIntentos`): registro, activación y restablecimiento 10, inicio de sesión 20, solicitud de enlace 5 y cambio de contraseña desde el perfil 10; la cotización del carrito admite 300. En el panel, el inicio de sesión admite 20 intentos y el cambio de contraseña 10.

7. Menú del panel por rol

La barra lateral del panel (`frontend/src/components/Rail.jsx`) construye sus enlaces a partir de `PERMISOS` (`frontend/src/utils/permisos.js`). Las rutas protegidas se envuelven además en `RoleRoute` (`frontend/src/App.jsx`), que muestra un mensaje de acceso denegado si el rol no corresponde, aunque se escriba la dirección a mano.

Opción del menú	Ruta	Permiso en `PERMISOS`	DISTRITAL	ADMIN	CAJERO	BODEGUERO
Devoluciones	`/`	Siempre visible	Sí	Sí	Sí	Sí
Nueva solicitud	`/nueva`	Siempre visible; el envío exige `crear`	Sí	Sí	Sí	Visible, sin poder registrar
Pedidos	`/pedidos`	Siempre visible	Sí	Sí	Sí	Sí
Clientes	`/clientes`	`clientes`	Sí	Sí	Sí	—
Trazabilidad	`/trazabilidad`	Siempre visible	Sí	Sí	Sí	Sí
Inventario	`/inventario`	Siempre visible	Sí	Sí	Sí	Sí
Catálogo y stock	`/catalogo`	`catalogo`	Sí	Sí	—	Sí
Sucursales	`/sucursales`	`sucursales`	Sí	Sí	—	Sí
Proveedores	`/proveedores`	`proveedores`	Sí	Sí	—	Sí
Empleados	`/empleados`	`empleados`	Sí	Sí	—	—
Reportes de ventas	`/reportes`	`reportes`	Sí	Sí	—	—
Promociones	`/promociones`	`promociones`	Sí	—	—	—
Estado del sistema	`/sistema`	`sistema`	Sí	—	—	—

Dentro de cada pantalla, los botones también dependen del rol:

Permiso	Roles	Efecto en el panel
`editarCatalogo`	DISTRITAL	Crear y editar prendas y tipos de prenda en "Catálogo y stock".
`gestionarSucursales`	DISTRITAL	Abrir, editar o cerrar sucursales.
`ajustarStock`	DISTRITAL, ADMIN, BODEGUERO	Registrar bajas de stock desde Sucursales.
`vender` , `cobrar`	DISTRITAL, ADMIN, CAJERO	Botón "Nueva venta", cobros en el detalle del pedido y entrega de reembolsos.

Permiso	Roles	Efecto en el panel
`CANCELADO`	DISTRITAL, ADMIN, CAJERO	Cancelar un pedido pendiente.
`ENVIADO`, `ENTREGADO`	DISTRITAL, ADMIN, BODEGUERO	Avanzar el pedido en el despacho; a los demás se les indica "Este paso lo hace bodega (ADMIN o BODEGUERO)."
`aprobar`, `rechazar`	DISTRITAL, ADMIN	Decisión sobre una devolución en estado SOLICITADA.
`recibir`, `revisar`, `reintegrar`, `cambiar`	DISTRITAL, ADMIN, BODEGUERO	Etapas físicas de la devolución, según su estado.
`activarCliente`	DISTRITAL, ADMIN	Casilla "Cliente activo" al editar una clienta.
`eliminarProveedor`	DISTRITAL, ADMIN	Botón "Eliminar" en Proveedores.
`todasLasSucursales`	DISTRITAL	Filtros y selectores de sucursal en Pedidos, Nueva venta, Reportes, Sucursales, Catálogo y Clientes; en Empleados, gestión del personal de todas las sucursales.

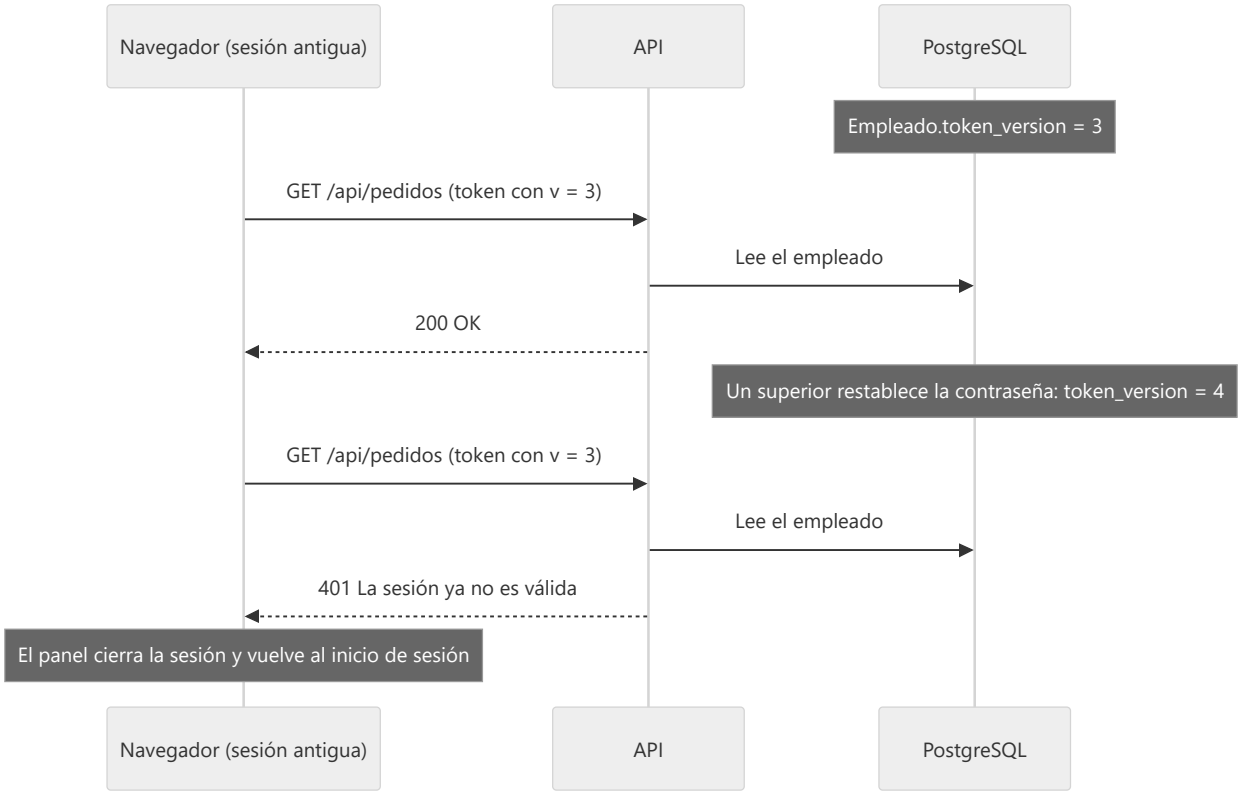
El ícono de la cuenta, al pie de la barra, muestra el nombre, el puesto y la sucursal del empleado (o "todas las sucursales" para el DISTRITAL) y abre el diálogo para cambiar la contraseña. Al cargar, el panel consulta `GET /api/auth/yo` para refrescar el puesto y la sucursal en caso de que hayan cambiado después del inicio de sesión.

8. Invalidación de sesiones (token_version)

Tanto `Empleado` como `Cliente` tienen la columna `token_version` (entero, valor por defecto `0`). Su valor se copia en el token como `v` al emitirlo, y los middlewares `verificarToken` y `verificarCliente` rechazan con `401` cualquier token cuyo `v` no coincida con el valor actual en la base de datos. Incrementar `token_version` invalida, por lo tanto, todos los tokens emitidos antes, sin necesidad de una lista de revocación.

Evento	Cuenta	Efecto
Cambio de la propia contraseña (`PUT /api/auth/password`)	Empleado	Cierra las demás sesiones; la respuesta incluye un token nuevo para la sesión actual.
Restablecimiento por un superior (`PUT /api/empleados/:id/password`)	Empleado	Cierra todas sus sesiones.
Desactivación (`activo = false`)	Empleado	Cierra todas sus sesiones (además, `verificarToken` rechaza cuentas inactivas).
Cambio de puesto o de sucursal	Empleado	Cierra sus sesiones, para que al volver a entrar el panel muestre lo que corresponde a su nuevo puesto y sucursal.

Evento	Cuenta	Efecto
Cambio de contraseña desde el perfil (`PUT /api/tienda/perfil/password`)	Cliente	Cierra sus sesiones en otros dispositivos; devuelve un token nuevo.
Restablecimiento con enlace de correo (`POST /api/tienda/auth/restablecer`)	Cliente	Cierra todas sus sesiones.
Activación de la cuenta web con código de caja (`POST /api/tienda/auth/activar`)	Cliente	Incrementa la versión al asignar la contraseña.
Desactivación desde el panel (`PUT /api/clientes/:id` con `activo: false`)	Cliente	Cierra sus sesiones; si luego se reactiva, los tokens anteriores siguen sin valer.



En ambos clientes web (`frontend/src/services/api.js` y `tienda/src/services/api.js`), una respuesta `401` , o un `403` cuyo mensaje se refiera al token, ejecuta el cierre de sesión local y devuelve al usuario a la pantalla de ingreso.

9. Códigos de respuesta de autorización

Código	Situación
`401`	Token con firma inválida o vencido; cuenta inexistente o desactivada; versión de sesión desactualizada.

Código	Situación
`403`	No se envió token; token del tipo equivocado (clienta en el panel o empleado en la tienda); el puesto no está autorizado para la acción; la sucursal indicada en el cuerpo de la petición no es la del empleado; reglas de gestión de empleados no cumplidas.
`404`	El registro pertenece a otra sucursal (se trata como inexistente); el pedido o la devolución pertenece a otra clienta.
`400`	Reglas de negocio de la gestión de empleados: desactivarse o cambiarse el puesto a sí mismo, dejar el sistema sin DISTRITAL activo, puesto distinto de DISTRITAL sin sucursal.

Para el detalle de cada endpoint y sus parámetros, consulte API; para las medidas de seguridad asociadas (contraseñas, límite de intentos), consulte Seguridad; y para las tablas `Empleado` y `Cliente`, consulte Modelo de datos.

Reglas de negocio y flujos

Este documento describe las reglas que el sistema aplica a las ventas, el inventario, los pagos, las devoluciones, los reembolsos y las promociones, junto con las máquinas de estados que gobiernan cada proceso. Todas las reglas descritas se obtuvieron del código del backend (principalmente `app/controllers/` y `app/utils/`); cuando se cita un valor por defecto, corresponde al que el código usa si la variable de entorno no está definida.

Contenido

- 1. Parámetros configurables
- 2. Pedidos
  - 2.1 Venta de mostrador y venta en la tienda en línea
  - 2.2 Estados del pedido
  - 2.3 Modificación de un pedido pendiente
- 3. Stock por sucursal
- 4. Pagos
  - 4.1 Métodos de pago
  - 4.2 Saldo, por confirmar y por cobrar
  - 4.3 Estados del pago
  - 4.4 Pago con tarjeta en la tienda en línea
  - 4.5 Cobros de Stripe que no corresponden
- 5. Vencimiento de pedidos pendientes
- 6. Devoluciones y cambios
  - 6.1 Política de solicitud
  - 6.2 Estados de la devolución
  - 6.3 Inspección y destino en inventario
  - 6.4 Cambios de talla o variante
- 7. Reembolsos
  - 7.1 Cálculo del monto
  - 7.2 Reparto entre Stripe y caja
  - 7.3 Estados del reembolso
- 8. Cupones de descuento
- 9. Envío
- 10. IVA incluido
- 11. Trazabilidad
- 12. Correos a la clienta

---

1. Parámetros configurables

Las reglas que dependen de un valor numérico se configuran con variables de entorno (ver `.env.example`). La siguiente tabla resume las que afectan a este documento.

Variable	Valor por defecto	Archivo que la lee	Efecto
`PLAZO_PEDIDO_PENDIENTE_HORAS`	24	`app/utils/vencimientos.js`	Horas para pagar un pedido pendiente antes de que se cancele automáticamente. `0` desactiva el vencimiento.
`VENCIMIENTO_INTERVALO_MIN`	10	`app/utils/vencimientos.js`	Cada cuántos minutos se revisan pedidos vencidos y reembolsos a tarjeta pendientes.
`HORAS_PLAZO_DEVOLUCION`	24	`app/utils/plazos.js`	Horas desde la entrega durante las cuales se admite una solicitud de cambio o devolución.
`ENVIO_COSTO`	25	`app/utils/precios.js`	Costo del envío a domicilio de la tienda en línea, en quetzales.
`ENVIO_GRATIS_DESDE`	300	`app/utils/precios.js`	Monto de prendas (después del descuento) desde el cual el envío es gratis. `0` significa que nunca es gratis.
`IVA_PORCENTAJE`	12	`app/utils/precios.js`	Porcentaje de IVA incluido en los precios; se usa para desglosarlo.
`ZONA_HORARIA_NOMBRE`	`America/Guatemala`	`app/utils/cupones.js`	Zona usada para evaluar la vigencia de los cupones por día.
`ZONA_HORARIA_TIENDA`	`-06:00`	`app/controllers/pedido.controller.js`	Desfase usado en los filtros de fecha de pedidos.
`STRIPE_SECRET_KEY`	(sin valor)	`app/config/stripe.config.js`	Con el valor de pruebas `sk_test_clave_de_prueba_universidad` el backend simula los cobros con tarjeta (se completan al instante sin llamar a Stripe).

Además, el código define constantes fijas: 30 minutos de gracia para cobros con tarjeta en curso (`MINUTOS_GRACIA`), un máximo de 3 pedidos pendientes por clienta en la tienda (`MAX_PEDIDOS_PENDIENTES`) y un máximo de 20 unidades por prenda en un pedido de la tienda (`MAX_UNIDADES_POR_LINEA`).

---

2. Pedidos

2.1 Venta de mostrador y venta en la tienda en línea

Ambos tipos de venta usan la misma función de creación (`crearPedido` en `app/utils/facturacion.js`), por lo que comparten las reglas de precio, stock y cupones. Se distinguen por el campo `id_empleado`: un pedido con empleado es de **mostrador** y uno sin empleado es de la **tienda en línea** (el panel lo muestra como `origen`).

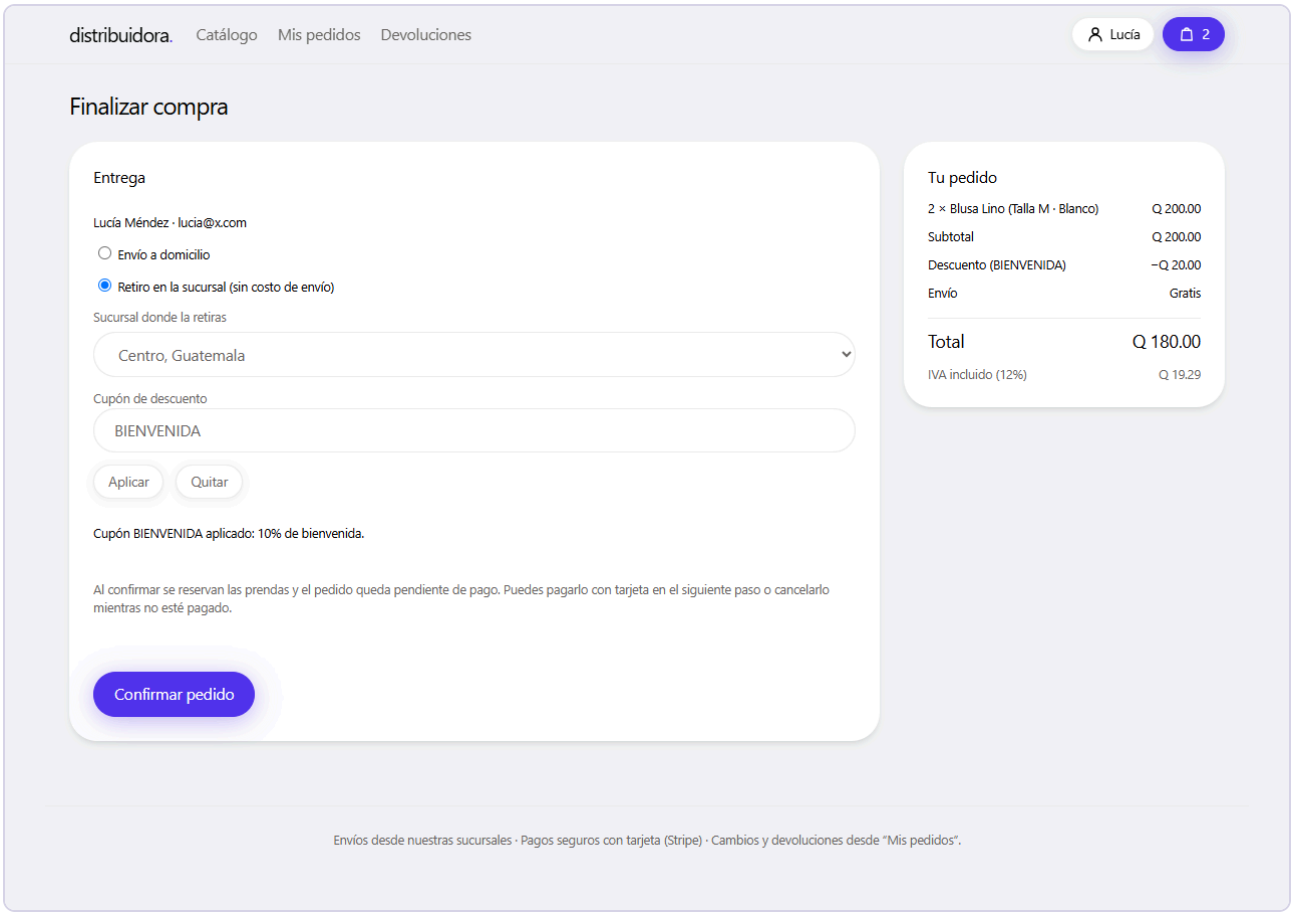
Aspecto	Venta de mostrador	Venta en la tienda en línea
Endpoint	`POST /api/pedidos`	`POST /api/tienda/pedidos`
Quién la registra	CAJERO, ADMIN o DISTRITAL	La clienta autenticada
Sucursal	La del empleado (el DISTRITAL elige cualquiera)	La elegida por la clienta
Clienta	De la propia sucursal; una clienta de otra sucursal solo si se la buscó por su correo exacto	La de la sesión
Entrega	Sin costo de envío	Envío a domicilio o retiro en sucursal (ver Envío)
Cupón	Opcional	Opcional
Límites adicionales	Ninguno	Máximo 3 pedidos pendientes por clienta, 50 líneas de productos distintos y 20 unidades por prenda

Reglas comunes de creación:

- Todo pedido nace en estado `PENDIENTE` de pago.
- El precio de cada línea se toma siempre del catálogo (`Producto.precio_unitario`); los valores de precio, total o estado enviados en la petición se ignoran.
- La clienta y la sucursal deben existir y estar activas; cada producto debe existir y estar activo.
- Las líneas repetidas del mismo producto se suman y se procesan en orden de `id_producto` , de modo que todas las ventas bloquean el inventario en el mismo orden y se evitan bloqueos mutuos entre transacciones.
- El stock se descuenta al crear el pedido (las prendas quedan reservadas aunque el pedido aún no esté pagado).
- El subtotal de cada línea es `cantidad × precio_unitario` y el total del pedido lo recalculan los *hooks* del modelo `DetallePedido` con la fórmula:

```
total = max(0, suma de líneas − descuento del cupón) + costo de envío
```

- Si un cupón cubre toda la compra y el total queda en 0, el pedido pasa directamente a `PROCESADO` , porque no hay nada que cobrar.
- Antes de confirmar, la cotización (`POST /api/pedidos/cotizar` y `POST /api/tienda/cotizar`) calcula subtotal, descuento, envío, total e IVA sin reservar stock ni consumir el cupón.

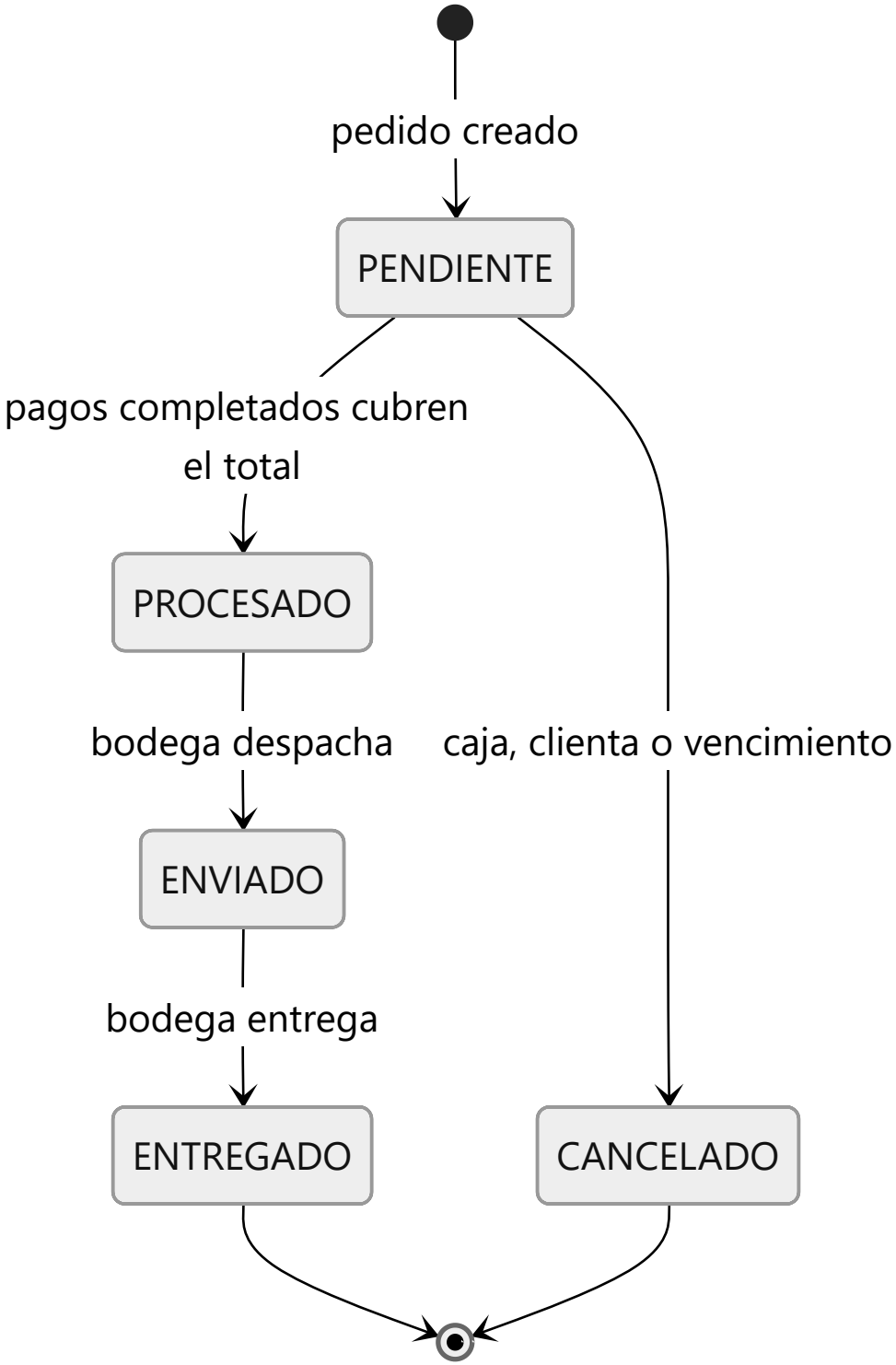


Pantalla de finalización de compra de la tienda con retiro en sucursal y el cupón BIENVENIDA aplicado: subtotal Q200.00, descuento Q20.00, envío gratis, total Q180.00 e IVA incluido Q19.29

Figura 1. Finalización de compra en la tienda con retiro en sucursal y un cupón de 10 % aplicado.

2.2 Estados del pedido

La máquina de estados está definida en `validarTransicionPedido` (`app/controllers/pedido.controller.js`).



Estado	Significado
`PENDIENTE`	Pedido creado con stock reservado, pendiente de pago. Es el único estado en que se pueden registrar pagos, modificar líneas o cancelar.
`PROCESADO`	Pagado por completo.

Estado	Significado
`ENVIADO`	Despachado a la dirección de entrega o, en ventas sin dirección, listo para recoger en la sucursal.
`ENTREGADO`	Entregado a la clienta. Se registra `fecha_entrega`, desde la cual corre el plazo de devolución. Estado final.
`CANCELADO`	Cancelado antes del pago; el stock vuelve a estar disponible. Estado final.

Reglas de transición:

- Un pedido no puede regresar a `PENDIENTE` ni cambiar desde `ENTREGADO` o `CANCELADO`. Lo que ocurre después de la entrega se gestiona como devolución.
- Un pedido `PENDIENTE` no puede marcarse como `ENVIADO` ni `ENTREGADO`.
- Un pedido `PROCESADO` debe pasar por `ENVIADO` antes de `ENTREGADO` y no puede cancelarse (ya fue pagado).
- `PENDIENTE → PROCESADO` solo se acepta si el total es mayor que 0 y los pagos `COMPLETADO` cubren el total. Normalmente ocurre de forma automática al completarse el último pago (`procesarPedidoSiEstaPagado` en `app/utils/facturacion.js`).
- `PENDIENTE → CANCELADO` exige que el pedido no tenga pagos completados ni pagos pendientes; un pago pendiente debe anularse antes (`PUT /api/pagos/:id/anular`). Al cancelar, todas las líneas se reintegran al stock y se libera el uso del cupón.
- La fila del pedido se bloquea durante la transición para que dos peticiones simultáneas no apliquen cambios a la vez.

Puestos autorizados para cada transición manual (`PUT /api/pedidos/:id/estado`), siempre dentro de la sucursal del empleado salvo el DISTRITAL:

Estado destino	Puestos autorizados
`PROCESADO`	DISTRITAL, ADMIN, CAJERO
`CANCELADO`	DISTRITAL, ADMIN, CAJERO
`ENVIADO`	DISTRITAL, ADMIN, BODEGUERO
`ENTREGADO`	DISTRITAL, ADMIN, BODEGUERO

La clienta también puede cancelar su propio pedido desde la tienda (`POST /api/tienda/pedidos/:id/cancelar`) mientras siga pendiente, no tenga pagos completados ni pagos de caja por confirmar. Si había iniciado un cobro con tarjeta, este se cancela en Stripe; si Stripe ya lo había cobrado, el pago se registra, el pedido pasa a `PROCESADO` y la cancelación se rechaza con código 409.

2.3 Modificación de un pedido pendiente

Caja (CAJERO, ADMIN o DISTRITAL) puede agregar (`POST /api/detalles`) o quitar (`DELETE /api/detalles/:id`) prendas de un pedido mientras esté `PENDIENTE`:

- Al agregar, el stock se descuenta de la sucursal del pedido (no se permite tomarlo de otra) y el precio sale del catálogo.
- Al quitar, las unidades vuelven a stock disponible con un movimiento `CANCELACION`.
- El total se recalcula, incluidos el descuento del cupón (que puede dejar de aplicar si no se alcanza la compra mínima) y el envío a domicilio (que puede volverse gratis o dejar de serlo).
- La operación se rechaza si lo ya pagado o en proceso de pago quedaría por encima del nuevo total.
- Si con el nuevo total lo pagado ya lo cubre, el pedido pasa a `PROCESADO`.

---

3. Stock por sucursal

El inventario se guarda por producto, sucursal, lote y estado en la tabla `Inventario`. Solo las filas con estado `DISPONIBLE` se pueden vender; las filas `DANADA` y `DEFECTUOSA` existen para registrar prendas no aptas para la venta. El catálogo de la tienda muestra únicamente el stock disponible de sucursales activas.

Descuento por venta (FIFO por lote). La función `descontarStockVenta` (`app/utils/facturacion.js`):

1. Bloquea las filas `DISPONIBLE` con cantidad mayor que 0 del producto en la sucursal del pedido, ordenadas por `id_inventario` ascendente (primero el lote que ingresó primero).
2. Si la suma no alcanza la cantidad solicitada, rechaza la venta con el mensaje de stock insuficiente.
3. Descuenta lote por lote hasta completar la cantidad.
4. Registra un movimiento `VENTA` por cada lote afectado; si el lote queda en 0, el movimiento indica `AGOTADO` como estado nuevo.

Reintegro por cancelación o por quitar una línea. La función `reintegrarStockVenta` suma las unidades a la primera fila `DISPONIBLE` del producto en la sucursal (o crea una si no existe) y registra un movimiento `CANCELACION`.

Otras entradas y salidas:

Operación	Endpoint	Regla
Ingreso de stock	`POST /api/inventario`	BODEGUERO, ADMIN o DISTRITAL de la sucursal. Cantidad entera mayor que 0 y estado `DISPONIBLE`, `DANADA` o `DEFECTUOSA`.
Ajuste de salida	`POST /api/inventario/ajustes`	Solo reduce stock. Motivos: `CORRECCION_CONTEO`, `PERDIDA`, `ROBO`, `DANO`, `DESCARTE`, `OTRO` (este último exige observación). Si la cantidad cambió mientras se editaba, responde 409 sin aplicar nada. Registra un movimiento `AJUSTE_SALIDA` por fila y una auditoría `INVENTARIO_AJUSTADO`.
Devoluciones y cambios	Ver sección 6	Las prendas devueltas entran a `DISPONIBLE`, `DANADA` o `DEFECTUOSA` según la inspección.

---

4. Pagos

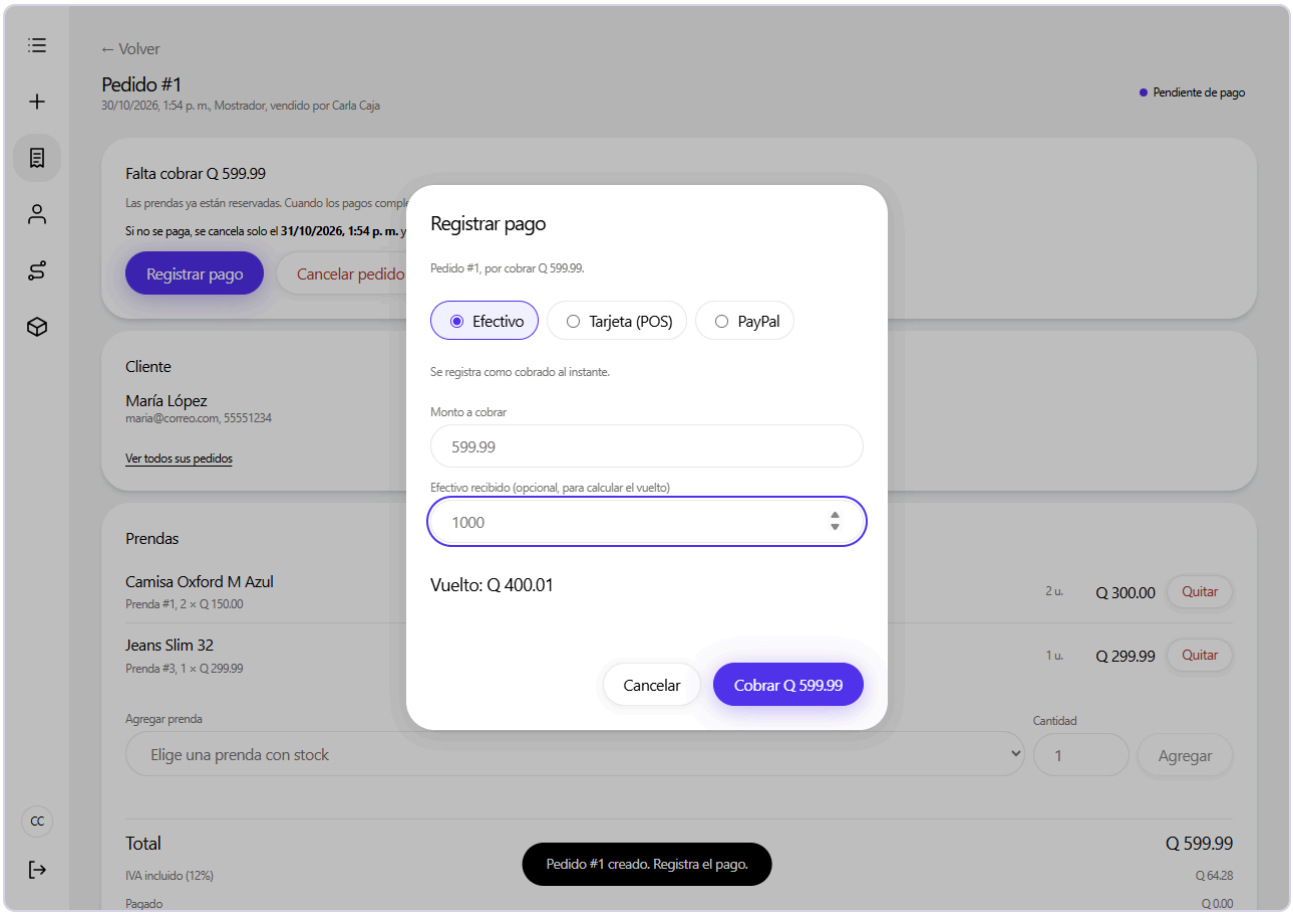
4.1 Métodos de pago

Método	Uso	Estado inicial del pago
`EFECTIVO`	Cobro en caja	`COMPLETADO` (el dinero se recibe en el momento)
`TARJETA`	Terminal POS de la tienda física; admite una referencia opcional (autorización o voucher, máximo 150 caracteres)	`COMPLETADO`
`PAYPAL`	Pago que caja verifica después	`PENDIENTE` hasta que caja lo confirma (`PUT /api/pagos/:id/confirmar`)
`STRIPE`	Tarjeta en línea; se crea un PaymentIntent en Stripe en moneda `gtq`	`PENDIENTE` hasta que Stripe confirma el cobro. En modo simulado nace `COMPLETADO` .

Reglas para registrar un pago (`POST /api/pagos` , CAJERO, ADMIN o DISTRITAL de la sucursal del pedido):

- El pedido debe estar `PENDIENTE` y tener total mayor que 0.
- El monto debe ser mayor que 0 y tener como máximo 2 decimales.
- El monto no puede superar el saldo pendiente, descontando tanto los pagos completados como los pendientes de confirmar, para que al confirmarse todos no se supere el total.
- Si con el pago el pedido queda cubierto, pasa a `PROCESADO` en la misma transacción.

Un pago `STRIPE` solo puede confirmarse manualmente si Stripe reporta el PaymentIntent como `succeeded` .



Ventana de registro de pago del panel con las opciones Efectivo, Tarjeta (POS) y PayPal, monto a cobrar Q599.99, efectivo recibido Q1000 y vuelto calculado de Q400.01

Figura 2. Registro de un cobro en efectivo desde el detalle del pedido en el panel. El recuadro superior del pedido indica la fecha en que se cancelará si no se paga.

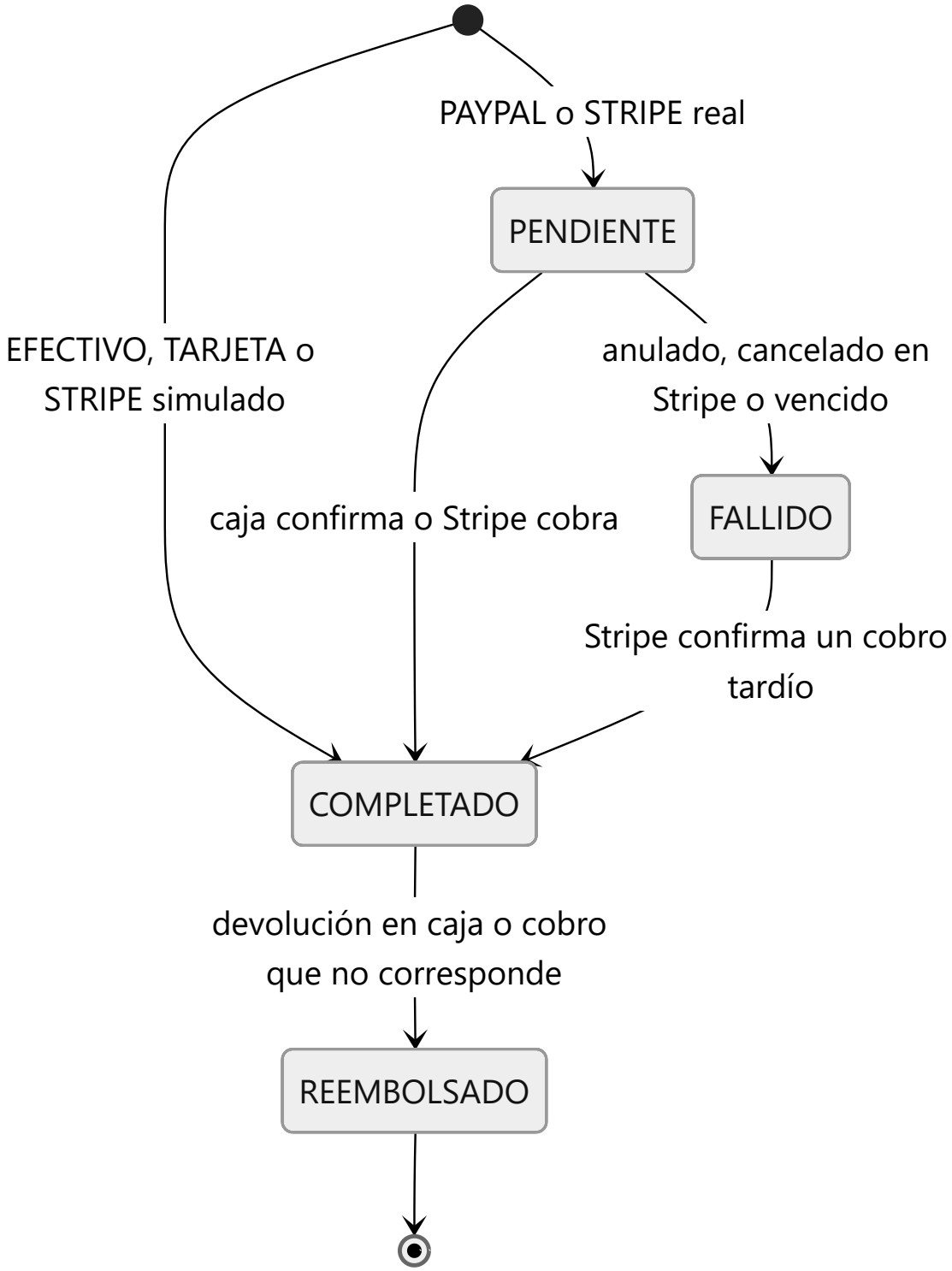
4.2 Saldo, por confirmar y por cobrar

El detalle del pedido en el panel (`GET /api/pedidos/:id`) calcula los siguientes valores:

Campo	Cálculo
`pagado`	Suma de los pagos `COMPLETADO` .
`por_confirmar`	Suma de los pagos `PENDIENTE` .
`saldo`	`max(0, total - pagado)` .
`por_cobrar`	`max(0, saldo - por_confirmar)` : lo que todavía se puede cobrar sin pasarse del total.
`pago_vence`	Fecha límite de pago si el pedido sigue pendiente (ver sección 5).

En la tienda, el detalle del pedido muestra además `a_pagar_con_tarjeta` , que es el saldo menos los pagos de caja pendientes de confirmar.

4.3 Estados del pago



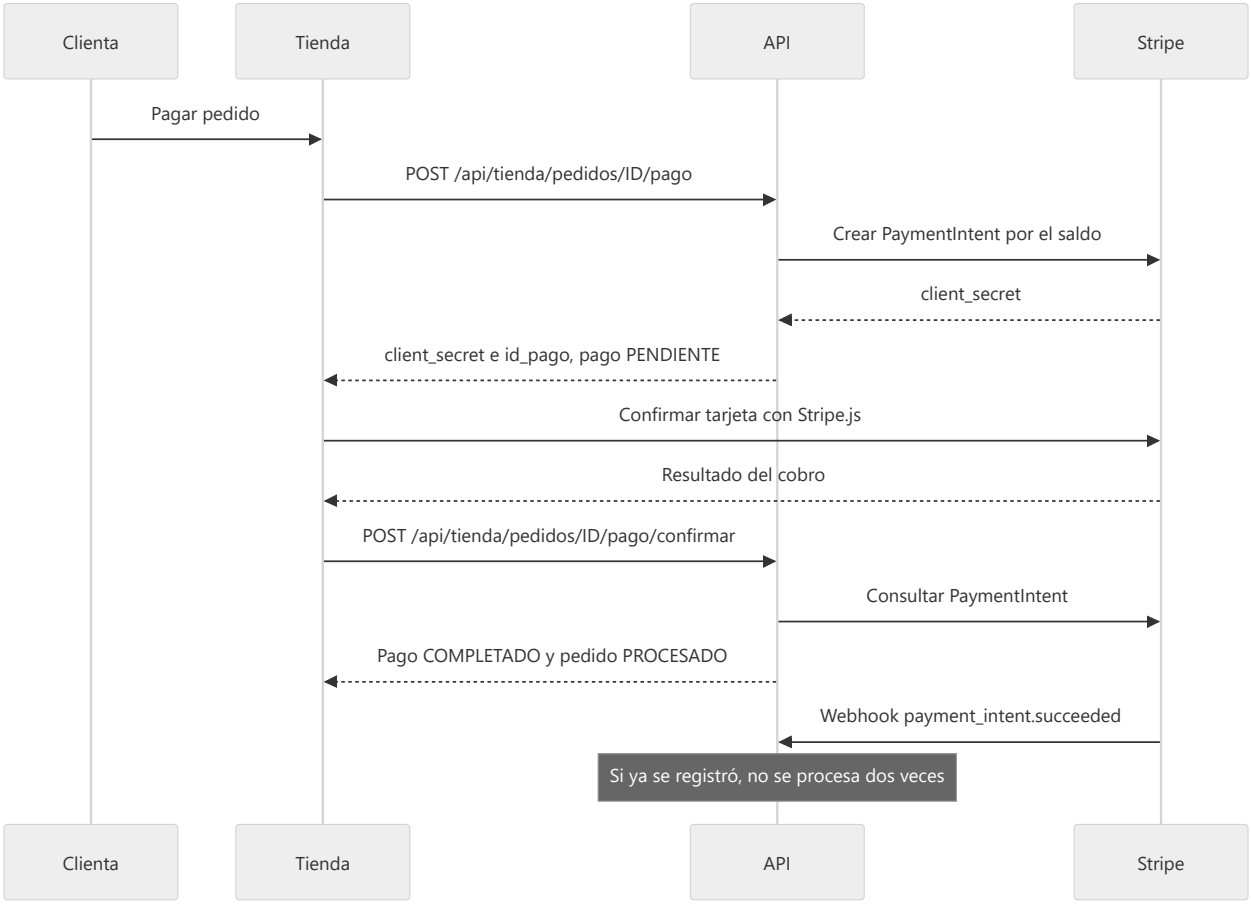
Transición	Origen en el código
`PENDIENTE → COMPLETADO`	`PUT /api/pagos/:id/confirmar` ; webhook `payment_intent.succeeded` ; confirmación desde la tienda (`POST /api/tienda/pedidos/:id/pago/confirmar`).

Transición	Origen en el código
`PENDIENTE → FALLIDO`	`PUT /api/pagos/:id/anular`; webhook `payment_intent.canceled`; cancelación del pedido por la clienta; vencimiento del pedido; reemplazo de un intento de pago por otro nuevo.
`FALLIDO → COMPLETADO`	Webhook `payment_intent.succeeded` o confirmación desde la tienda sobre un pago que había quedado fallido.
`COMPLETADO → REEMBOLSADO`	`PUT /api/pagos/:id/devolver`: caja devuelve un cobro presencial (`EFECTIVO` o `TARJETA`) de un pedido que sigue `PENDIENTE`; o un cobro de Stripe que no corresponde (ver 4.5).

El evento `payment_intent.payment_failed` no cambia el estado: la clienta puede reintentar con otra tarjeta sobre el mismo PaymentIntent, por lo que el pago sigue `PENDIENTE` y solo se registra la auditoría `PAGO_INTENTO_FALLIDO`.

Un pago `REEMBOLSADO` deja de contar como pagado. Si un pedido `PROCESADO` necesita devolver dinero, se gestiona mediante una devolución, no desde el pago.

4.4 Pago con tarjeta en la tienda en línea



Reglas del inicio de pago (`POST /api/tienda/pedidos/:id/pago`):

- El pedido debe estar `PENDIENTE`. Si su plazo de pago ya venció y no hay un cobro en curso, se rechaza y se indica a la clienta que haga un pedido nuevo.
- Si ya existe un cobro con tarjeta en curso, se reutiliza en lugar de crear otro (doble clic o recarga de la página), siempre que cobre exactamente lo que falta pagar. Si Stripe ya lo está procesando o lo cobró, se informa que está en proceso.
- Un intento que ya no corresponde (cancelado o con monto distinto) se cancela en Stripe y el pago se anula antes de crear uno nuevo.
- El monto del nuevo PaymentIntent es el total menos lo ya pagado y lo pendiente de confirmar.
- En modo simulado se crea directamente un pago `STRIPE` `COMPLETADO` por el saldo.

4.5 Cobros de Stripe que no corresponden

Cuando Stripe confirma un cobro (`registrarCobroExitoso` en `app/utils/facturacion.js`), el pago se marca `COMPLETADO` porque el dinero ya ingresó. Si en ese momento el pedido ya no está `PENDIENTE` (fue cancelado, venció o ya estaba pagado) o el monto supera el saldo:

1. Se registra la auditoría `PAGO_REQUIERE_REEMBOLSO`.
2. El pago pasa a `REEMBOLSADO`, por lo que no cuenta como pagado.
3. Se crea un reembolso `STRIPE` por el monto completo del pago (`reembolsarPagoStripe` en `app/utils/reembolsos.js`), con su auditoría `REEMBOLSO_CREADO`.
4. Al confirmarse la transacción, el reembolso se solicita a Stripe y se avisa a la clienta del resultado.

Solo se crea un reembolso de este tipo por pago, aunque el cobro se notifique varias veces.

---

5. Vencimiento de pedidos pendientes

El módulo `app/utils/vencimientos.js` cancela automáticamente los pedidos que no se pagan a tiempo para liberar el stock reservado.

- **Plazo:** un pedido `PENDIENTE` vence `PLAZO_PEDIDO_PENDIENTE_HORAS` horas (24 por defecto) después de `fecha_pedido`. Con valor `0` no vence nunca.
- **Frecuencia:** la revisión corre cada `VENCIMIENTO_INTERVALO_MIN` minutos (10 por defecto); la primera se ejecuta pocos segundos después de arrancar el servidor. Cada vuelta procesa hasta 200 pedidos, cada uno en su propia transacción.
- **Instancia única:** la revisión usa un candado de PostgreSQL, de modo que si hay varias instancias del servidor solo una revisa a la vez.

No se cancela un pedido vencido en los siguientes casos:

Situación	Motivo
Tiene algún pago `COMPLETADO` (pago parcial)	Lo resuelve caja.
Tiene un pago de caja (`PAYPAL`, por ejemplo) pendiente de confirmar	Lo resuelve caja.

Situación	Motivo
Tiene un cobro `STRIPE` iniciado hace menos de 30 minutos	Periodo de gracia: la clienta podría estar escribiendo los datos de la tarjeta.
Stripe reporta el cobro como en proceso	Se reintenta en la siguiente revisión.

Cuando el pedido sí vence, cada cobro con tarjeta pendiente se cancela en Stripe y el pago queda `FALLIDO`. Si Stripe ya había cobrado, en vez de cancelar se registra el cobro y el pedido pasa a `PROCESADO`. En caso contrario, el pedido pasa a `CANCELADO`, el stock vuelve a `DISPONIBLE`, se libera el cupón, se registra la auditoría y se envía a la clienta el correo de pedido cancelado indicando que no se recibió el pago dentro del plazo.

En la misma vuelta se reintentan los reembolsos a tarjeta que siguen `PENDIENTE` con más de 5 minutos de antigüedad (hasta 50 por vuelta).

---

6. Devoluciones y cambios

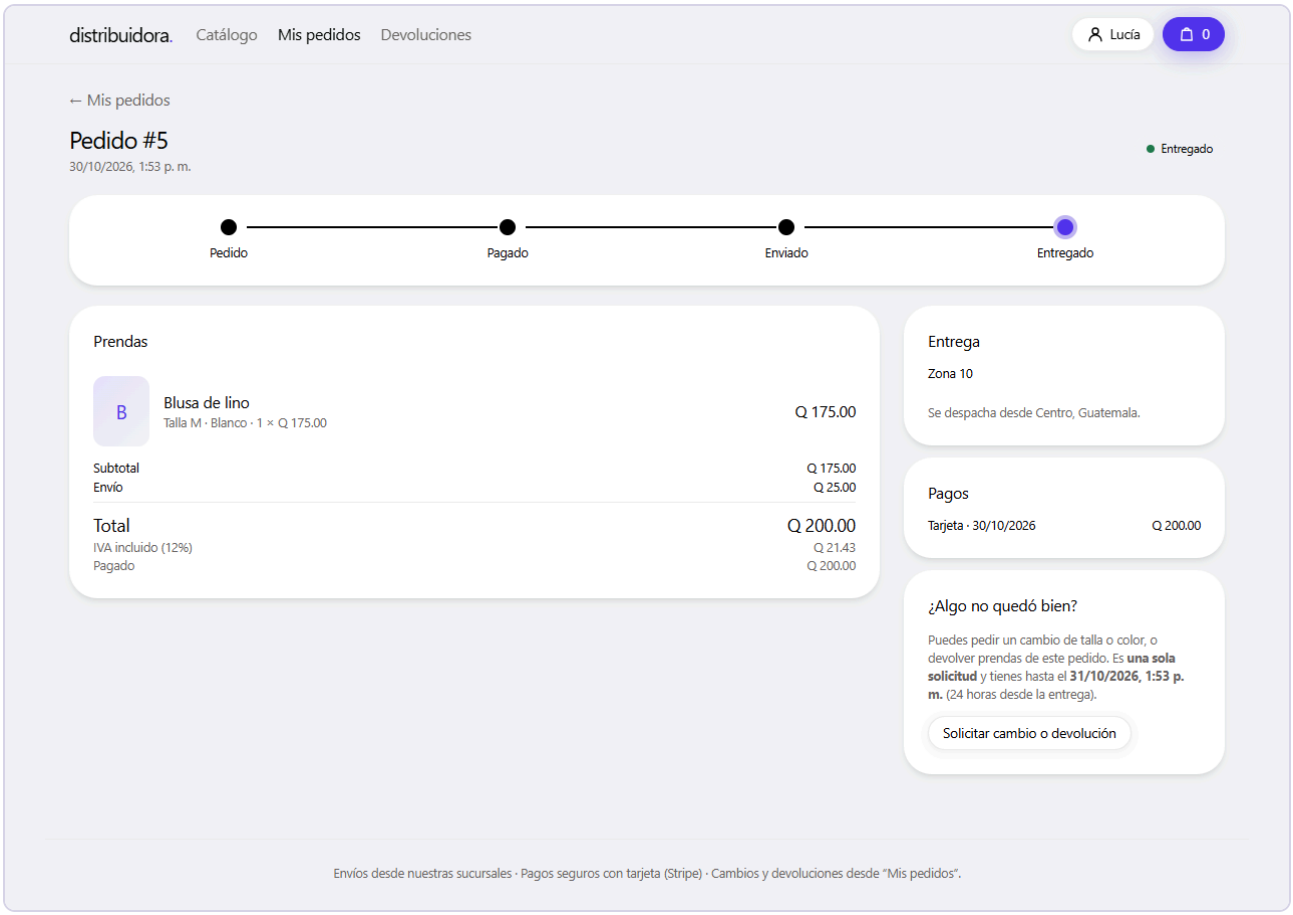
6.1 Política de solicitud

La función `politicaDevolucion` (`app/controllers/devolucion.controller.js`) decide si un pedido admite una solicitud:

Regla	Código de motivo si no se cumple
El pedido debe estar `ENTREGADO`.	`NO_ENTREGADO`
Solo se admite **una solicitud por pedido**, aunque la anterior haya sido rechazada.	`YA_SOLICITADO`
La solicitud debe hacerse dentro de `HORAS_PLAZO_DEVOLUCION` horas (24 por defecto) desde `fecha_entrega`.	`PLAZO_VENCIDO`

Otras reglas de la solicitud:

- **Tipos:** `DEVOLUCION`, `CAMBIO_TALLA` y `CAMBIO_VARIANTE`.
- **Motivos:** `DEFECTUOSO`, `TALLA_INCORRECTA`, `PRODUCTO_INCORRECTO`, `DANADO_ENVIO` y `OTRO`.
- El pedido debe pertenecer a la clienta indicada y cada línea debe pertenecer al pedido.
- La cantidad solicitada de cada línea no puede superar lo comprado menos lo ya solicitado en solicitudes no rechazadas.
- La solicitud la registra la clienta desde la tienda (`POST /api/tienda/devoluciones`) o caja a su nombre desde el panel (`POST /api/devoluciones`, CAJERO, ADMIN o DISTRITAL).
- Cada línea solicitada genera un movimiento `DEVOLUCION_SOLICITADA`.

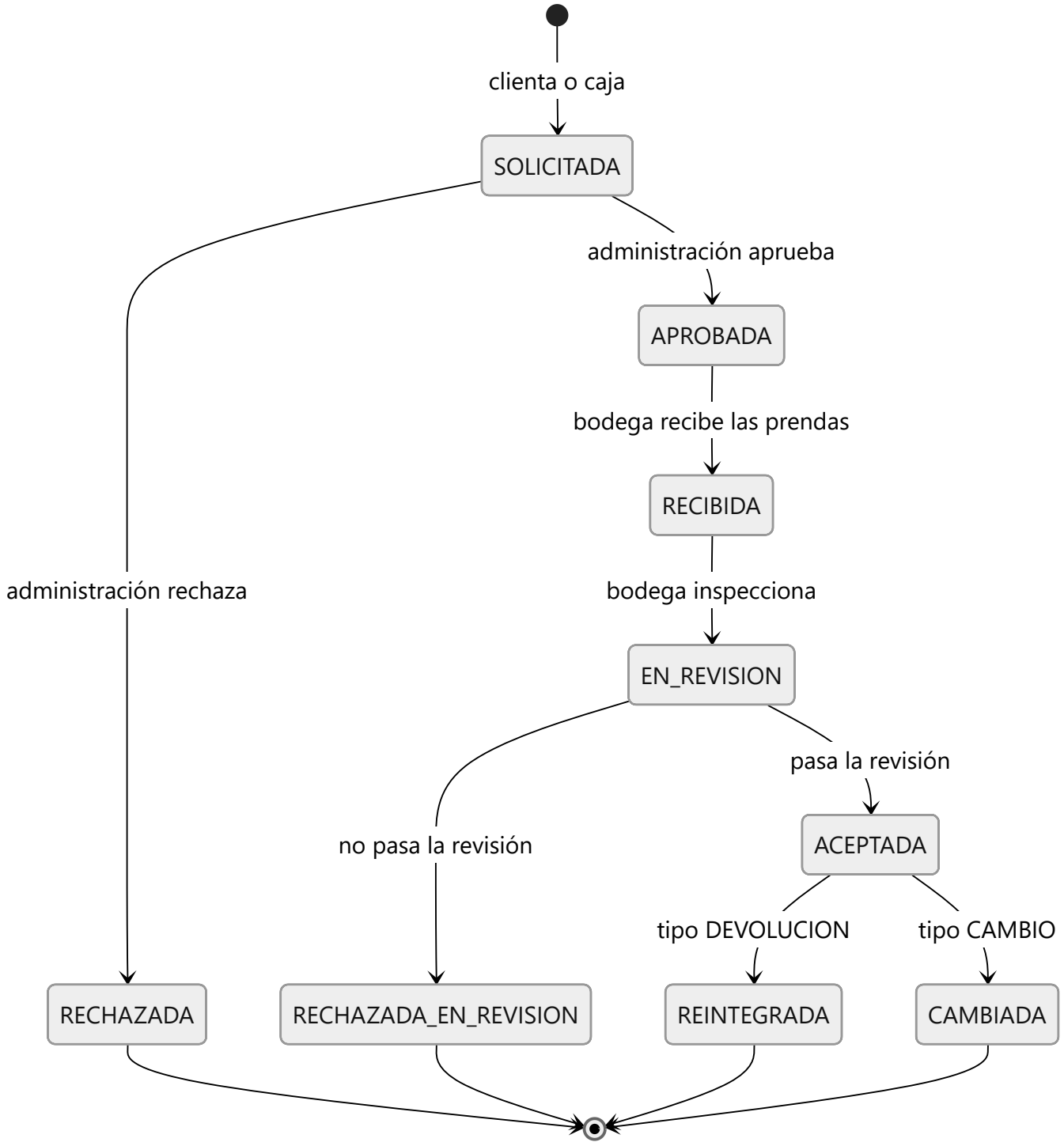


Detalle de un pedido entregado en la tienda: Q200.00 pagados con tarjeta y la tarjeta "¿Algo no quedó bien?" que indica que la solicitud es única y vence el 31/10/2026 a la 1:53 p. m., 24 horas después de la entrega

Figura 3. Pedido entregado en la tienda: la clienta ve el plazo exacto para su única solicitud de cambio o devolución.

6.2 Estados de la devolución

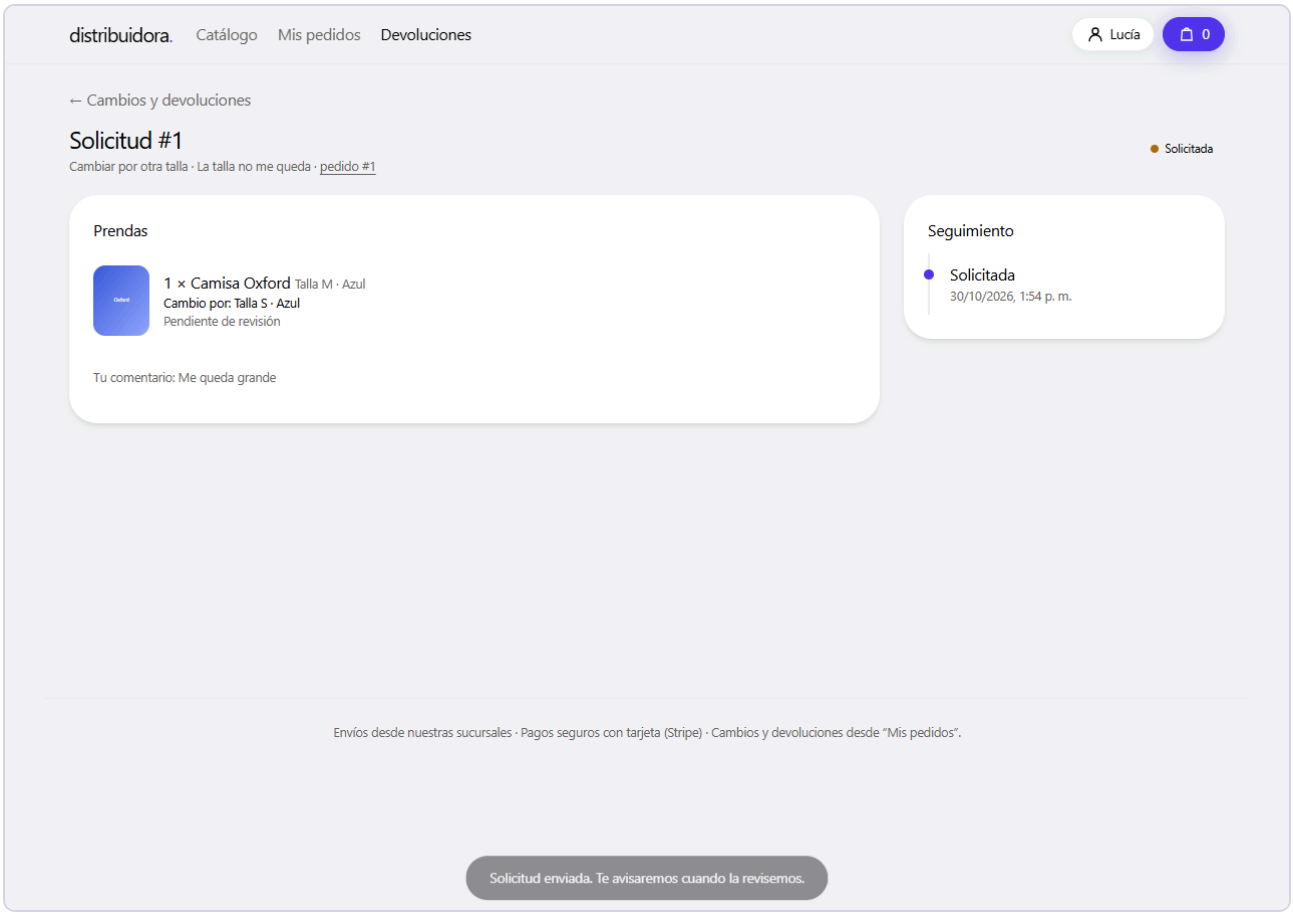
Las transiciones válidas están centralizadas en `app/utils/estadosDevolucion.js`, de modo que ningún controlador puede omitir pasos del proceso.



Acción	Endpoint	Puestos	Resultado
Aprobar	`PUT /api/devoluciones/:id/aprobar`	ADMIN, DISTRITAL	`APROBADA`

Acción	Endpoint	Puestos	Resultado
Rechazar	`PUT /api/devoluciones/:id/rechazar`	ADMIN, DISTRITAL	`RECHAZADA` (final)
Recibir	`PUT /api/devoluciones/:id/recibir`	BODEGUERO, ADMIN, DISTRITAL	`RECIBIDA`; movimiento `DEVOLUCION_RECIBIDA` por línea
Revisar	`PUT /api/devoluciones/:id/revisar`	BODEGUERO, ADMIN, DISTRITAL	`ACEPTADA` o `RECHAZADA_EN_REVISION`
Reintegrar	`PUT /api/devoluciones/:id/reintegrar`	BODEGUERO, ADMIN, DISTRITAL	`REINTEGRADA` y reembolso (solo tipo `DEVOLUCION`)
Cambiar	`PUT /api/devoluciones/:id/cambiar`	BODEGUERO, ADMIN, DISTRITAL	`CAMBIADA` y, si aplica, reembolso de la diferencia (solo tipos de cambio)

`EN_REVISION` es un paso que la operación de revisión valida (`RECIBIDA → EN_REVISION → resultado`) en una sola transacción: la devolución queda guardada directamente en `ACEPTADA` o `RECHAZADA_EN_REVISION`. Los estados `RECHAZADA`, `RECHAZADA_EN_REVISION`, `REINTEGRADA` y `CAMBIADA` son finales. Cada empleado solo gestiona devoluciones de pedidos de su sucursal, salvo el DISTRITAL. Las acciones aprobar, rechazar, recibir y revisar aceptan una observación opcional de hasta 255 caracteres que queda en la auditoría (y, al recibir o revisar, también en el movimiento de inventario).



Detalle de la solicitud número 1 en la tienda: cambio de una Camisa Oxford talla M azul por la talla S azul, pendiente de revisión, con el seguimiento en estado Solicitada

Figura 4. Seguimiento de una solicitud de cambio de talla desde la tienda.

6.3 Inspección y destino en inventario

La revisión debe incluir el resultado de **todas** las líneas de la devolución, sin repetidas, e indicar si se acepta (`aceptar: true`) o no. Cada línea recibe uno de tres resultados, que determinan el movimiento registrado y el estado de inventario al que vuelve la prenda al reintegrarla o cambiarla:

Resultado de revisión	Movimiento al revisar	Estado de inventario al reintegrar	¿Vendible?
`BUEN_ESTADO`	`EN_REVISION`	`DISPONIBLE`	Sí
`DANADO`	`PRODUCTO_DANADO`	`DANADA`	No
`DEFECTUOSO`	`PRODUCTO_DEFECTUOSO`	`DEFECTUOSA`	No

Al reintegrar una devolución de tipo `DEVOLUCION`, cada prenda se suma a la fila de inventario de la sucursal del pedido con el estado correspondiente (o se crea una) y se registra un movimiento `REINTEGRACION`. Una devolución `RECHAZADA_EN_REVISION` no mueve inventario ni genera reembolso.

6.4 Cambios de talla o variante

- Cada línea de un cambio debe indicar `id_producto_nuevo`.
- El producto nuevo debe existir, estar activo, ser distinto del original y pertenecer al mismo grupo de variantes (mismo producto padre).
- **Solo se admiten variantes de igual o menor precio** que lo pagado por la prenda original. El sistema nunca cobra una diferencia; si la variante cuesta menos, la diferencia se reembolsa. En la tienda, una variante más cara se rechaza indicando que la clienta debe acudir a una sucursal.
- El precio de la variante nueva se guarda en la solicitud (`precio_unitario_nuevo`), de modo que el reembolso se calcula con ese precio aunque después cambie el catálogo.
- Al ejecutar el cambio, primero se verifica que haya stock disponible de **todas** las variantes nuevas en la sucursal del pedido; si alguna no alcanza, no se mueve nada.
- Después, por cada línea, la prenda original entra al inventario según su resultado de revisión (movimiento `CAMBIO_SALIDA`) y la variante nueva se descuenta del stock disponible por orden de ingreso (movimiento `CAMBIO_ENTRADA`).

---

7. Reembolsos

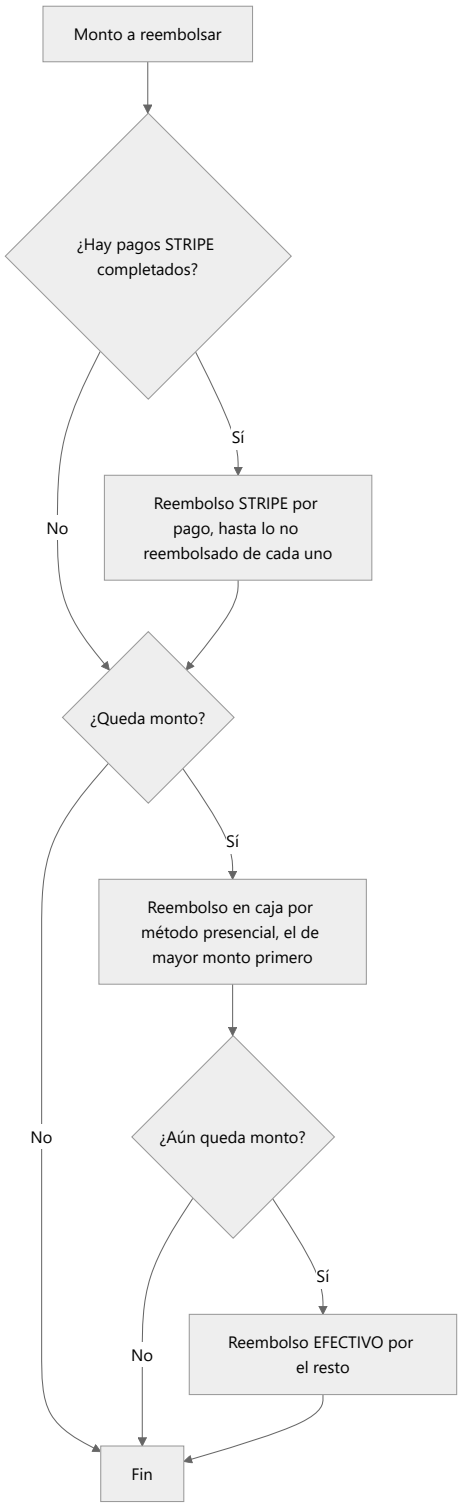
Los reembolsos se crean automáticamente al reintegrar una devolución o al completar un cambio por una variante más barata (`crearReembolsosDeDevolucion` en `app/utils/reembolsos.js`), y también para los cobros de Stripe que no corresponden (4.5).

7.1 Cálculo del monto

1. **Monto bruto por línea:**
   - `DEVOLUCION`: precio unitario pagado en la línea × cantidad devuelta.
   - `CAMBIO_TALLA` y `CAMBIO_VARIANTE`: `max(0, precio pagado − precio de la variante nueva)` × cantidad.
2. **Prorrateo del descuento:** el bruto se multiplica por el factor `(suma de líneas − descuento del pedido) / suma de líneas`, limitado entre 0 y 1. Así la clienta recibe lo que realmente pagó por cada prenda después del cupón.
3. **El envío no se devuelve:** el cálculo solo considera las líneas de prendas.
4. **Tope:** el monto nunca supera lo cobrado al pedido (pagos `COMPLETADO`) menos lo ya reembolsado o pendiente de reembolsar.

Ejemplo: un pedido con prendas por Q200.00, un cupón de Q20.00 y envío de Q25.00 tiene un factor de 0.9. Si se devuelve una prenda de Q100.00, el reembolso es Q90.00; los Q25.00 del envío no se devuelven.

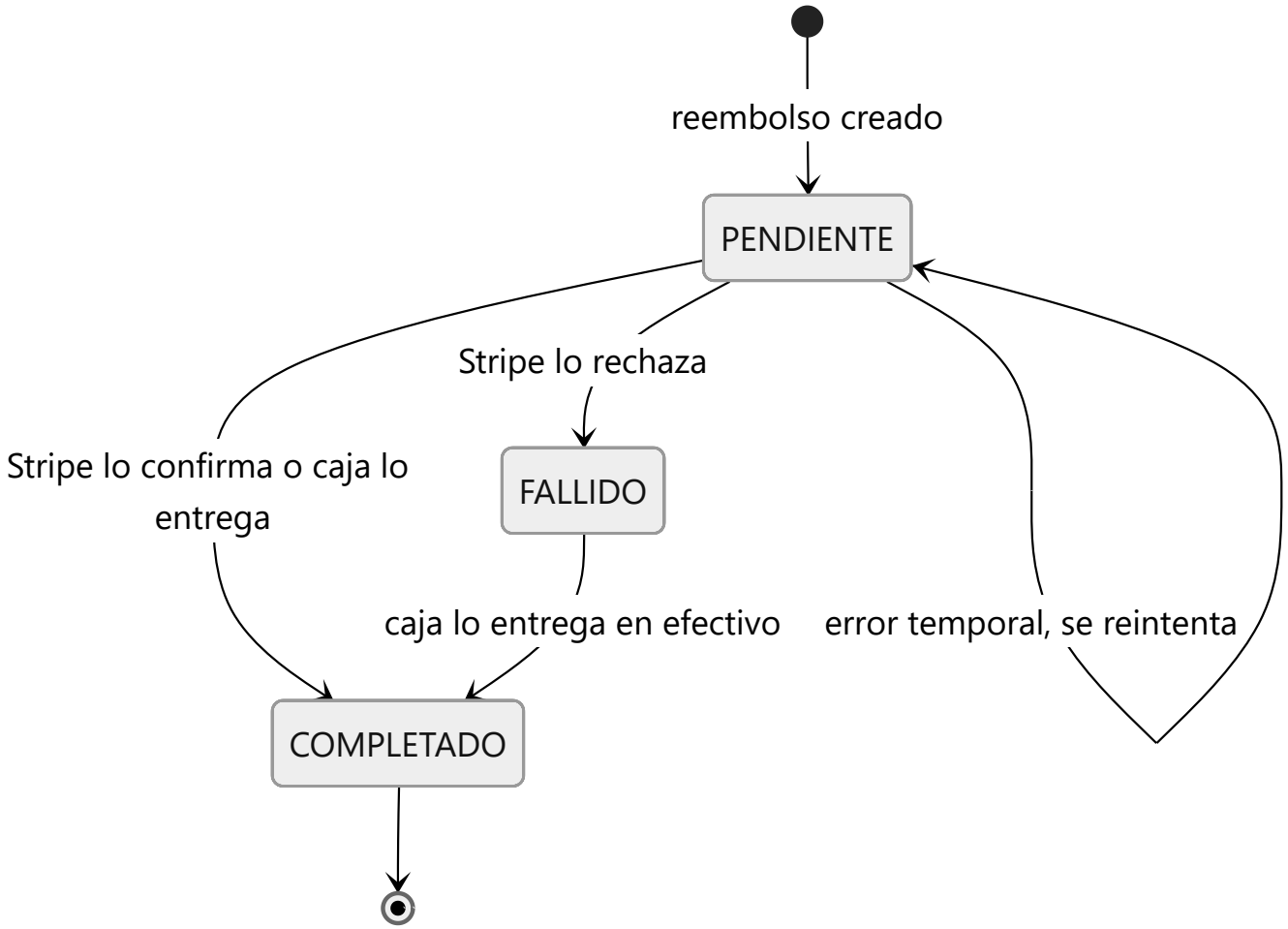
7.2 Reparto entre Stripe y caja



- Primero se reembolsa a la tarjeta con la que se pagó en línea, pago por pago.
- El resto se asigna a los métodos presenciales del pedido (`EFECTIVO` , `TARJETA` , `PAYPAL`), empezando por el de mayor monto disponible, sin superar lo cobrado con cada método (un terminal POS no puede revertir más de lo que cobró).

- Todos los reembolsos nacen `PENDIENTE`. Los de Stripe se solicitan inmediatamente después de confirmar la transacción; los de caja quedan pendientes hasta que caja entrega el dinero (`PUT /api/reembolsos/:id/completar` , con referencia opcional de hasta 150 caracteres).

7.3 Estados del reembolso



Situación	Comportamiento
Stripe responde `succeeded`	`COMPLETADO` , se guarda el identificador del reembolso de Stripe y se avisa a la clienta.
Stripe responde `failed` o `canceled` , o devuelve un error definitivo (HTTP 400, 402 o 404)	`FALLIDO` ; la observación indica que debe entregarse en efectivo en caja y se avisa a la clienta.
Error de red, 409, 429, 5xx o configuración	Sigue `PENDIENTE` ; se reintenta en la revisión periódica, con el botón «Reintentar en Stripe» del panel (`POST /api/reembolsos/:id/reintentar`) o al recibir el webhook de reembolso.
`FALLIDO` entregado en caja	`PUT /api/reembolsos/:id/completar` lo marca `COMPLETADO` y cambia su método a `EFECTIVO` , conservando la referencia al pago de Stripe.

Protecciones contra reembolsos dobles: la fila se reserva mientras se habla con Stripe (una reserva de más de 2 minutos se considera abandonada), se usa una clave de idempotencia por reembolso, antes de crear uno nuevo se busca en Stripe un reembolso ya hecho con el mismo identificador interno y el resultado solo se escribe si la fila sigue `PENDIENTE`. Los reembolsos `STRIPE` pendientes no pueden completarse manualmente en caja; solo pueden reintentarse.

---

8. Cupones de descuento

Los cupones los crea, edita y elimina únicamente el DISTRITAL (`/api/cupones`). La clienta los escribe al comprar en la tienda y caja los aplica en ventas de mostrador.

Tipos:

Tipo	Valor	Descuento sobre la suma de prendas
`PORCENTAJE`	Entre 0 y 100 (mayor que 0)	`suma × valor / 100`
`MONTO`	Quetzales, mayor que 0	`valor`

El descuento nunca supera la suma de prendas y es 0 si no se alcanza la compra mínima. El código del cupón tiene de 3 a 30 caracteres (letras, números, guion o guion bajo) y se guarda en mayúsculas.

Validaciones al aplicar un cupón (`motivoNoAplica` en `app/utils/cupones.js`):

- Debe existir y estar activo.
- La fecha actual de Guatemala debe estar dentro de la vigencia `desde` – `hasta` (ambos días incluidos; un valor nulo significa sin límite).
- No debe haber agotado sus `usos_maximos` (nulo significa ilimitado).
- Si está restringido a una sucursal, el pedido debe ser de esa sucursal.
- La suma de prendas debe alcanzar `minimo_compra` .

Para no revelar qué códigos existen, todas las causas de rechazo devuelven el mismo mensaje («Ese cupón no es válido para esta compra»), excepto la compra mínima, que se explica porque la clienta puede cumplirla. En la cotización, un cupón que no aplica no es un error: se informa en `cupon_error` y se cotiza sin descuento.

Usos:

- Al crear el pedido, el cupón se bloquea, se valida y se incrementa `usos` en la misma transacción, de modo que dos compras simultáneas no superan el límite.
- Al cancelar el pedido (por caja, por la clienta o por vencimiento), el uso se libera (`usos - 1` , sin bajar de 0).

Copia en el pedido: el pedido guarda `cupon_tipo` , `cupon_valor` y `cupon_minimo` al momento de la compra. Si después el DISTRITAL edita el cupón, los pedidos ya creados no cambian de precio; los recálculos por agregar o quitar prendas usan esa copia.

Total 0: si el descuento cubre toda la compra y no hay envío, el pedido pasa directamente a `PROCESADO`.

Edición y eliminación:

- Un cupón ya usado no puede cambiar su código ni su tipo.
- `usos_maximos` no puede fijarse por debajo de los usos ya realizados.
- Solo puede eliminarse un cupón que no se haya usado en pedidos no cancelados; en caso contrario debe desactivarse.

---

9. Envío

Las reglas están en `costoEnvio` (`app/utils/precios.js`) y `recalcularPedido` (`app/utils/totales.js`):

Tipo de entrega	Costo de envío
Venta de mostrador	0
Tienda en línea con retiro en sucursal	0 (sin dirección de entrega)
Tienda en línea a domicilio, prendas después del descuento menores que `ENVIO_GRATIS_DESDE`	`ENVIO_COSTO` (Q25.00 por defecto)
Tienda en línea a domicilio, prendas después del descuento iguales o mayores que `ENVIO_GRATIS_DESDE` (Q300.00 por defecto)	0

- El umbral de envío gratis se compara con el monto **después del descuento del cupón**, en centavos para evitar errores de redondeo.
- Con `ENVIO_GRATIS_DESDE = 0` el envío nunca es gratis.
- En un pedido a domicilio de la tienda, el envío se recalcula cada vez que cambian las líneas; un pedido sin prendas no paga envío.
- Para envío a domicilio se usa la dirección indicada o, si no se indica, la registrada en el perfil de la clienta (máximo 200 caracteres). Si no hay dirección y no se eligió retiro, el pedido se rechaza.
- El costo de envío no se devuelve en los reembolsos.

---

10. IVA incluido

Los precios del catálogo ya incluyen IVA. El sistema no lo suma: lo desglosa a partir del total (`desgloseIva` en `app/utils/precios.js`) con `IVA_PORCENTAJE` (12 por defecto):

```
base_imponible = redondear(total / (1 + IVA_PORCENTAJE / 100), 2)
iva            = total - base_imponible
```

Por ejemplo, un total de Q200.00 tiene una base imponible de Q178.57 y un IVA incluido de Q21.43. El desglose (`subtotal`, `descuento`, `costo_envio`, `total`, `base_imponible`, `iva`, `porcentaje`) se muestra en el detalle del pedido del panel, en el detalle del pedido de la tienda y en las cotizaciones. Los reportes desglosan

con el mismo porcentaje el IVA incluido en el total vendido.

---

11. Trazabilidad

El sistema mantiene dos registros complementarios.

Movimientos de inventario (`MovimientoInventario`). Historial de solo inserción: se crea y nunca se modifica ni se elimina desde los controladores. Cada fila guarda producto, lote (`id_inventario`), pedido, devolución, empleado responsable (nulo si lo originó la clienta o el sistema), tipo, cantidad, estado anterior, estado nuevo y observación.

Tipo de movimiento	Cuándo se registra
`VENTA`	Por cada lote descontado al vender o agregar una prenda a un pedido.
`CANCELACION`	Al quitar una prenda de un pedido pendiente o al cancelarlo.
`DEVOLUCION_SOLICITADA`	Por cada línea de una solicitud de devolución o cambio.
`DEVOLUCION_RECIBIDA`	Al recibir físicamente las prendas en la sucursal.
`EN_REVISION`, `PRODUCTO_DANADO`, `PRODUCTO_DEFECTUOSO`	Al revisar cada línea, según su resultado.
`REINTEGRACION`	Al reintegrar una devolución al inventario.
`CAMBIO_SALIDA`, `CAMBIO_ENTRADA`	Al completar un cambio: entra la prenda original y sale la variante nueva.
`AJUSTE_SALIDA`	Baja manual de stock desde el panel.

Las consultas `GET /api/pedidos/:id/historial` y `GET /api/devoluciones/:id/historial` devuelven estos movimientos en orden cronológico; la segunda incluye también la auditoría de la devolución.

Auditoría (`auditoria`). Registra cada acción con el empleado (cuando lo hay), la acción, la tabla afectada, el identificador del registro y un detalle legible, con su fecha de creación. Se escribe dentro de la misma transacción que la operación, por lo que no queda una auditoría de algo que falló ni una operación sin auditoría. La excepción es el resultado de un reembolso procesado en Stripe (`REEMBOLSO_COMPLETADO` o `REEMBOLSO_FALLIDO`), que se registra después de actualizar el reembolso, fuera de una transacción. Algunas acciones registradas son:

Ámbito	Acciones
Pedidos	`PEDIDO_CREADO`, `PEDIDO_ESTADO_ACTUALIZADO`, `DETALLE_PEDIDO_AGREGADO`, `DETALLE_PEDIDO_ELIMINADO`
Pagos	`PAGO_REGISTRADO`, `PAGO_CONFIRMADO`, `PAGO_ANULADO`, `PAGO_DEVUELTO`, `PAGO_INTENTO_FALLIDO`, `PAGO_REQUIERE_REEMBOLSO`

Ámbito	Acciones
Devoluciones	`DEVOLUCION_APROBADA`, `DEVOLUCION_RECHAZADA`, `DEVOLUCION_RECIBIDA`, `DEVOLUCION_ACEPTADA`, `DEVOLUCION_RECHAZADA_EN_REVISION`, `DEVOLUCION_REINTEGRADA`, `DEVOLUCION_CAMBIADA`
Reembolsos	`REEMBOLSO_CREADO`, `REEMBOLSO_COMPLETADO`, `REEMBOLSO_FALLIDO`
Cupones	`CUPON_CREADO`, `CUPON_ACTUALIZADO`, `CUPON_ELIMINADO`
Inventario	`INVENTARIO_AJUSTADO`

La línea de tiempo que la clienta ve de su solicitud en la tienda se construye con la fecha de solicitud y las acciones de auditoría de esa devolución, sin mostrar notas internas.

---

12. Correos a la clienta

Los avisos están en `app/utils/notificaciones.js`. Se envían **después** de confirmar la operación y nunca la hacen fallar: si un envío falla, solo se registra en consola. Además, cada aviso de pedido verifica que el pedido esté realmente en el estado que anuncia, y solo se envía si la clienta tiene correo registrado.

Aviso	Cuándo se envía	Contenido
Pago recibido	El pedido pasa a `PROCESADO` (manualmente o al completarse el último pago)	Monto pagado y si se prepara para envío o en la sucursal.
Pedido enviado	El pedido pasa a `ENVIADO`	«Va en camino» con la dirección, o «listo para recoger» si no tiene dirección de entrega.
Pedido entregado	El pedido pasa a `ENTREGADO`	Fecha y hora límite para pedir un cambio o una devolución (una sola solicitud).
Pedido cancelado	Caja cancela el pedido o este vence sin pago	Motivo de la cancelación e indicación de que las prendas volvieron a estar disponibles.
Solicitud de devolución o cambio	Al registrarse y en cada avance: aprobada, rechazada, recibida, aceptada, rechazada en revisión, reintegrada o cambiada	Estado de la solicitud y enlace a su seguimiento.
Reembolso en proceso	Al crearse un reembolso de caja, o uno a tarjeta que Stripe no completó al momento	Qué debe hacer la clienta según el método (recoger efectivo, reversión en POS, PayPal o esperar a la tarjeta).
Reembolso completado	Stripe confirma el reembolso o caja lo entrega	Monto devuelto y forma en que se devolvió.
Reembolso fallido	Stripe rechaza el reembolso	Indicación de pasar a la sucursal a recogerlo en efectivo.
Restablecer contraseña	La clienta usa «¿Olvidaste tu contraseña?»	Enlace de un solo uso que vence en 1 hora.

Aviso	Cuándo se envía	Contenido
Contraseña cambiada	Se cambia la contraseña de la cuenta	Aviso de que se cerraron las sesiones abiertas.

La cancelación hecha por la propia clienta desde la tienda no genera correo. El modo de envío se define con `CORREO_MODO`: `smtp` (por defecto si existe `SMTP_HOST`), `consola` (por defecto sin SMTP; solo muestra el aviso en la consola) o `archivo` (guarda cada correo como JSON en `CORREO_CARPETA`, usado por las pruebas). Los enlaces de los correos apuntan a `TIENDA_URL`.

Documentos relacionados: Modelo de datos · Roles y permisos · API · Manual de usuario

Referencia de la API REST

Este documento describe la API REST del sistema Distribuidora: las URL base, el mecanismo de autenticación de empleados y de clientas, el formato de los errores, la paginación, el límite de intentos y la lista completa de endpoints agrupados por módulo, con el rol que exige cada uno. Incluye ejemplos de petición y respuesta para las operaciones más representativas y explica el uso de la colección de Postman incluida en el repositorio. Todo lo descrito corresponde a `server.js`, a los archivos de `app/routes/` y a los controladores de `app/controllers/`.

Contenido

- 1. URL base
- 2. Recorrido de una petición
- 3. Autenticación
- 4. Formato de errores
- 5. Paginación y búsqueda
- 6. Límite de intentos
- 7. Convenciones de las tablas de endpoints
- 8. Endpoints por módulo
- 9. Ejemplos de petición y respuesta
- 10. Colección de Postman
- Documentos relacionados

---

1. URL base

Entorno	URL base de la API	Observaciones
Local (desarrollo)	`http://localhost:8080`	Puerto definido por la variable `PORT` (8080 por defecto).
Pública (Render)	`https://distribuidora-api-r6rw.onrender.com`	Comprobación de salud en `https://distribuidora-api-r6rw.onrender.com/api/salud`.

Todas las rutas de negocio cuelgan del prefijo `/api`. La raíz `GET /` responde un JSON informativo con el nombre del proyecto y el ambiente (`development` o `production`).

Los clientes web autorizados por CORS se configuran en la variable `CORS_ORIGIN` (lista separada por comas). Por defecto se permiten `http://localhost:8081` (panel de empleados) y `http://localhost:8082` (tienda en línea). En producción se configuran los orígenes del panel (`https://distribuidora-panel.onrender.com`) y de la tienda (`https://distribuidora-tienda.onrender.com`). La cabecera `X-Request-Id` está expuesta a los navegadores mediante CORS.

Los cuerpos de las peticiones se envían en JSON (`Content-Type: application/json`). El servidor no acepta cuerpos comprimidos.

2. Recorrido de una petición

El siguiente diagrama resume el orden en que `server.js` registra los componentes que atienden una petición.



No todas las rutas aplican todos los pasos: las rutas públicas omiten la verificación del token, y el alcance por sucursal solo se aplica a las rutas que operan sobre un registro concreto (pedidos, pagos, detalles y devoluciones). En el resto de listados el filtro por sucursal se aplica dentro del propio controlador.

3. Autenticación

La API utiliza JSON Web Tokens (JWT) firmados con la variable `JWT_SECRET`. Existen dos tipos de token que no son intercambiables.

Tipo	Cómo se obtiene	Contenido del token	Middleware que lo valida	Rutas donde sirve
Empleado	`POST /api/auth/login` con `{ email, password }`	`id`, `id_sucursal`, `v` (versión de sesión)	`verificarToken`	Todas las rutas protegidas del panel (`/api/...` excepto `/api/tienda/...`)
Clienta	`POST /api/tienda/auth/login`, `POST /api/tienda/auth/registro`, `POST /api/tienda/auth/activar` o `POST /api/tienda/auth/restablecer`	`id_cliente`, `tipo: "CLIENTE"`, `v`	`verificarClienta`	Rutas protegidas de `/api/tienda/...`

El token se envía en la cabecera:

```
Authorization: Bearer <token>
```

Reglas de validación aplicadas por el código:

- Los tokens tienen una vigencia de 24 horas (86400 segundos).
- Un token de clienta presentado en una ruta de empleado se rechaza con `403` ("Este token no pertenece a un empleado"), y viceversa.
- En cada petición se consulta la base de datos: si la cuenta del empleado o de la clienta ya no existe o está desactivada, se responde `401`.
- Cada cuenta guarda un contador `token_version`. Cambiar o restablecer la contraseña, activar la cuenta web de una clienta, desactivar a un empleado o a una clienta, o cambiar el puesto o la sucursal de un empleado incrementa ese contador; los tokens emitidos antes dejan de ser válidos (`401`). Por esa razón los endpoints de cambio de contraseña devuelven un token nuevo.
- El puesto del empleado se lee de la base en cada petición, no del token: un cambio de puesto se aplica de inmediato.
- Si falta la cabecera `Authorization`, la respuesta es `403` ("No se proporcionó un token de seguridad"). Si el token es inválido o expiró, la respuesta es `401`.

Alcance por sucursal: el puesto DISTRITAL opera sobre todas las sucursales; ADMIN, CAJERO y BODEGUERO solo sobre la sucursal asignada en su ficha (`Empleado.id_sucursal`). Cuando un empleado intenta acceder a un pedido, pago, detalle o devolución de otra sucursal, la API responde `404` ("No se encontró el registro en tu sucursal."), de modo que el registro ajeno no se revela. El detalle de los permisos se encuentra en Roles y permisos.

4. Formato de errores

Todas las respuestas de error de la API se devuelven en JSON con un campo `message` en español (la única excepción es `POST /api/webhook`, que ante una firma inválida responde texto plano y ante un error de base de datos responde `500` sin cuerpo):

```
{ "message": "El email y la contraseña son obligatorios." }
```

Cuando el error lo produce el manejador final de Express (JSON mal formado, cuerpo demasiado grande, excepción no controlada), la respuesta incluye además el campo `referencia`, que coincide con el identificador de la petición:

```
{
  "message": "Error interno del servidor (referencia 3f1c9a52-7b0e-4d6a-9a43-2c1e8f0b6d21).",
  "referencia": "3f1c9a52-7b0e-4d6a-9a43-2c1e8f0b6d21"
}
```

Cabecera `X-Request-Id`: toda respuesta, exitosa o no, incluye esta cabecera con un UUID generado para la petición. Toda respuesta con código 5xx se registra en la tabla `ErrorServidor` junto con ese identificador, el método, la ruta (sin parámetros de consulta) y el usuario (`empleado:N` o `cliente:N`). El DISTRITAL puede consultar ese registro en `GET /api/salud/errores`, y el identificador permite relacionar el error que vio un usuario con su entrada en el registro. Al cliente nunca se le envía la pila de la excepción.

Códigos de estado utilizados:

Código	Significado en esta API	Ejemplos
200	Operación exitosa.	Consultas, cambios de estado.
201	Registro creado.	Venta, pago, devolución, cupón, inicio de pago con tarjeta.
400	Datos inválidos o regla de negocio incumplida.	Campo obligatorio ausente, stock insuficiente, transición de estado no permitida, `:id` no numérico ("El identificador no es válido."), JSON mal formado.
401	Credenciales o sesión inválidas.	Contraseña incorrecta, token expirado, cuenta desactivada, sesión cerrada por cambio de contraseña.
403	Sin permiso para la acción.	Falta el token, puesto insuficiente, token de clienta en ruta de empleado, venta en otra sucursal.
404	Registro o ruta inexistente.	Pedido de otra sucursal o de otra clienta, ruta `/api` desconocida ("No existe la ruta GET /api/...").
409	Conflicto con el estado actual de los datos.	Eliminar una sucursal, prenda, tipo de prenda o proveedor con registros asociados; ajuste de inventario sobre stock que cambió; pago con tarjeta ya cobrado por Stripe.
413	Cuerpo de la petición demasiado grande.	Manejador final de errores.

Código	Significado en esta API	Ejemplos
415	Cuerpo comprimido no admitido.	Manejador final de errores.
429	Demasiados intentos.	Inicio de sesión, registro, recuperación de contraseña (ver sección 6).
500	Error interno.	Falla inesperada; queda registrada en `ErrorServidor`.
502	Error al comunicarse con Stripe.	La pasarela no respondió al crear o consultar un cobro.
503	Servicio no disponible.	La base de datos no responde (`/api/salud`) o Cloudinary no está configurado (`/api/imagenes/firma`).

Particularidad del módulo de devoluciones: los endpoints de `/api/devoluciones` que modifican datos (registro, aprobación, rechazo, recepción, revisión, reintegro y cambio) responden `400` ante cualquier error, incluido el caso de una devolución inexistente.

5. Paginación y búsqueda

La paginación es opcional y la implementa `app/utils/paginacion.js`. Sin el parámetro `pagina`, el endpoint responde el arreglo completo como siempre; con `pagina`, responde un objeto con metadatos.

Parámetro	Descripción	Valores
`pagina`	Número de página.	Entero desde 1 (máximo 100000).
`por_pagina`	Registros por página.	Entero de 1 a 100; 25 por defecto.
`q`	Texto de búsqueda (sin distinguir mayúsculas).	Hasta 80 caracteres. Un valor como `123` o `#123` también busca por número de registro.

Respuesta paginada:

```
{
  "datos": [ { "id_pedido": 58, "estado": "PENDIENTE" } ],
  "total": 134,
  "pagina": 1,
  "por_pagina": 25,
  "paginas": 6
}
```

Endpoints que admiten paginación:

Endpoint	Campos sobre los que busca `q`	Filtros adicionales
`GET /api/pedidos`	Número de pedido, nombre y apellido de la clienta.	`estado`, `id_cliente`, `id_sucursal`, `origen` (`TIENDA` o `MOSTRADOR`), `desde`, `hasta` (AAAA-MM-DD, hora de Guatemala).

Endpoint	Campos sobre los que busca `q`	Filtros adicionales
`GET /api/clientes`	Número de cliente, nombre, apellido, correo y teléfono.	`filtro` (`web`, `mostrador` o `inactivos`; por defecto, activas).
`GET /api/devoluciones`	Número de devolución o de pedido, nombre y apellido de la clienta.	`estado`, `tipo`, `motivo`, `id_pedido`, `id_cliente`, `desde`, `hasta`.
`GET /api/salud/errores`	No aplica.	Siempre paginado (página 1 por defecto).

Un valor inválido de `pagina` o `por_pagina` produce `400` con el mensaje correspondiente.

6. Límite de intentos

El middleware `limitarIntentos` (`app/middlewares/limitador.js`) protege los endpoints de cuentas frente a ataques de adivinación de contraseñas y abuso del envío de correos. Características:

- El contador se lleva por dirección IP y por ruta, en una ventana de 15 minutos. Las direcciones IPv6 se agrupan por su red /64. Las variantes de una misma ruta (mayúsculas o barra final) comparten el contador.
- Los contadores se guardan en la tabla `IntentoLimite`, de modo que sobreviven a reinicios y son comunes a varias instancias del servidor. Si la base no responde, se usa un contador en memoria.
- Al superar el máximo se responde `429` con la cabecera `Retry-After` (segundos) y el mensaje "Demasiados intentos. Vuelve a intentarlo en N minuto(s)."

Endpoint	Máximo de peticiones por IP cada 15 minutos
`POST /api/auth/login`	20
`PUT /api/auth/password`	10
`POST /api/imagenes/firma`	60
`POST /api/tienda/auth/registro`	10
`POST /api/tienda/auth/login`	20
`POST /api/tienda/auth/activar`	10
`POST /api/tienda/auth/olvide`	5
`POST /api/tienda/auth/restablecer`	10
`PUT /api/tienda/perfil/password`	10
`POST /api/tienda/cotizar`	300

Además, `POST /api/tienda/auth/olvide` envía como máximo 3 enlaces por correo electrónico por hora; a partir del cuarto la respuesta es la misma, pero no se envía otro correo. Ver también Seguridad.

7. Convenciones de las tablas de endpoints

La columna "Rol requerido" usa la siguiente notación, que corresponde a los middlewares de `app/middlewares/authJwt.js`:

Notación	Quién puede invocar el endpoint
Pública	Cualquiera, sin token.
Empleado	Cualquier empleado autenticado y activo (DISTRITAL, ADMIN, CAJERO o BODEGUERO).
CAJERO+	CAJERO, ADMIN o DISTRITAL (`isCajero`).
BODEGUERO+	BODEGUERO, ADMIN o DISTRITAL (`isBodeguero`).
ADMIN+	ADMIN o DISTRITAL (`isAdmin`).
DISTRITAL	Solo el gerente distrital (`isDistrital`).
Clienta	Clienta de la tienda con token de tipo CLIENTE (`verificarCliente`).
Stripe	Invocado por Stripe; se valida la firma `stripe-signature`.

La marca "(sucursal)" indica que, además del rol, se exige que el registro pertenezca a la sucursal del empleado (salvo DISTRITAL), ya sea mediante `exigirAlcance` o mediante un filtro dentro del controlador. Los parámetros `:id` se validan antes de consultar la base (middleware `validarId`): un valor no numérico o fuera del rango de un entero de PostgreSQL produce `400`. Esta validación se aplica en todos los módulos con `:id` excepto `/api/devoluciones`, cuyo archivo de rutas no registra `validarId`.

8. Endpoints por módulo

8.1 Autenticación de empleados (`/api/auth`)

Método	Ruta	Rol requerido	Descripción
POST	`/api/auth/registro`	Pública solo con la base vacía; después ADMIN+	Registra un empleado con contraseña. Con la base sin empleados, el primero queda siempre como DISTRITAL sin sucursal. Después, el ADMIN solo registra cajeros y bodegueros de su sucursal; el DISTRITAL, cualquier puesto. Responde `201` con `{ message, id, puesto }`.
POST	`/api/auth/login`	Pública (límite 20)	Inicia sesión con `{ email, password }`. Devuelve los datos del empleado y el `token`. Mismo mensaje para correo inexistente y contraseña incorrecta.
GET	`/api/auth/yo`	Empleado	Datos actuales del empleado del token, incluido el nombre de su sucursal.
PUT	`/api/auth/password`	Empleado (límite 10)	Cambia la propia contraseña con `{ actual, nueva }`. Cierra las demás sesiones y devuelve un token nuevo.

8.2 Sucursales (`/api/sucursales`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/sucursales`	Pública	Lista todas las sucursales (la usan el panel y la tienda).
POST	`/api/sucursales`	DISTRITAL	Crea una sucursal.
PUT	`/api/sucursales/:id`	DISTRITAL	Edita los campos enviados (incluido `activo`); cada cambio queda en auditoría.
DELETE	`/api/sucursales/:id`	DISTRITAL	Elimina una sucursal sin actividad. Con empleados, clientes registrados, inventario o pedidos asociados responde `409`; en ese caso debe desactivarse con `PUT` (`activo: false`).

8.3 Clientes (`/api/clientes`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/clientes`	Empleado (sucursal)	Lista las clientas registradas en la sucursal o que le han comprado (el DISTRITAL, todas). Admite paginación, `q` y `filtro`. Con `?email=` busca por correo exacto en todas las sucursales y devuelve solo datos básicos.
POST	`/api/clientes`	CAJERO+	Alta de una clienta en mostrador, sin contraseña.
PUT	`/api/clientes/:id`	CAJERO+ (sucursal)	Edita nombre, apellido, teléfono y dirección; el correo solo si aún no tiene cuenta web. Activar o desactivar (`activo`) es exclusivo de ADMIN o DISTRITAL.
POST	`/api/clientes/:id/codigo-web`	CAJERO+ (sucursal)	Genera el código de activación de la cuenta web (8 caracteres, válido 72 horas). Se muestra una sola vez; responde `{ codigo, expira, email }`.

8.4 Proveedores (`/api/proveedores`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/proveedores`	Empleado	Lista los proveedores.
POST	`/api/proveedores`	BODEGUERO+	Crea un proveedor (`nombre_empresa` obligatorio).
PUT	`/api/proveedores/:id`	BODEGUERO+	Edita los campos enviados; cada cambio queda en auditoría.
DELETE	`/api/proveedores/:id`	ADMIN+	Elimina un proveedor; si tiene registros asociados responde `409`.

8.5 Categorías o tipos de prenda (`/api/categorias`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/categorias`	Pública	Lista los tipos de prenda.
POST	`/api/categorias`	DISTRITAL	Crea un tipo (nombre único sin distinguir mayúsculas, hasta 60 caracteres).
PUT	`/api/categorias/:id`	DISTRITAL	Edita nombre y descripción.
DELETE	`/api/categorias/:id`	DISTRITAL	Elimina un tipo; si alguna prenda lo usa responde `409` .

8.6 Empleados (`/api/empleados`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/empleados`	ADMIN + (sucursal)	Lista empleados (el DISTRITAL, todos; el ADMIN, los de su sucursal). Nunca expone contraseñas; indica `tiene_acceso` .
POST	`/api/empleados`	ADMIN +	Crea la ficha de un empleado sin contraseña (no puede iniciar sesión hasta que se le asigne una).
PUT	`/api/empleados/:id`	ADMIN +	Edita nombre, apellido, correo, puesto, sucursal y `activo` . Nadie puede desactivarse ni cambiarse el puesto a sí mismo, y siempre debe quedar un DISTRITAL activo. Desactivar al empleado o cambiarle el puesto o la sucursal cierra sus sesiones.
PUT	`/api/empleados/:id/password`	ADMIN +	Asigna una contraseña temporal con `{ password }` y cierra las sesiones de ese empleado.

No existe `DELETE` : un empleado que deja la empresa se desactiva para conservar su historial.

8.7 Productos o prendas (`/api/productos`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/productos`	Pública	Lista todos los productos del catálogo.
POST	`/api/productos`	DISTRITAL	Crea un producto o variante (`nombre` , `precio_unitario` e `id_categoria` obligatorios; `id_producto_padre` para variantes; `imagen_url` opcional).
PUT	`/api/productos/:id`	DISTRITAL	Edita los campos enviados; cada cambio queda en auditoría.
DELETE	`/api/productos/:id`	DISTRITAL	Elimina una prenda sin actividad; con historial responde `409` y debe desactivarse.

Método	Ruta	Rol requerido	Descripción
GET	`/api/productos/:id/historial`	Empleado (sucursal)	Trazabilidad de la prenda: ventas, devoluciones, revisiones, reintegros y ajustes de la sucursal del empleado.

8.8 Inventario (`/api/inventario`)

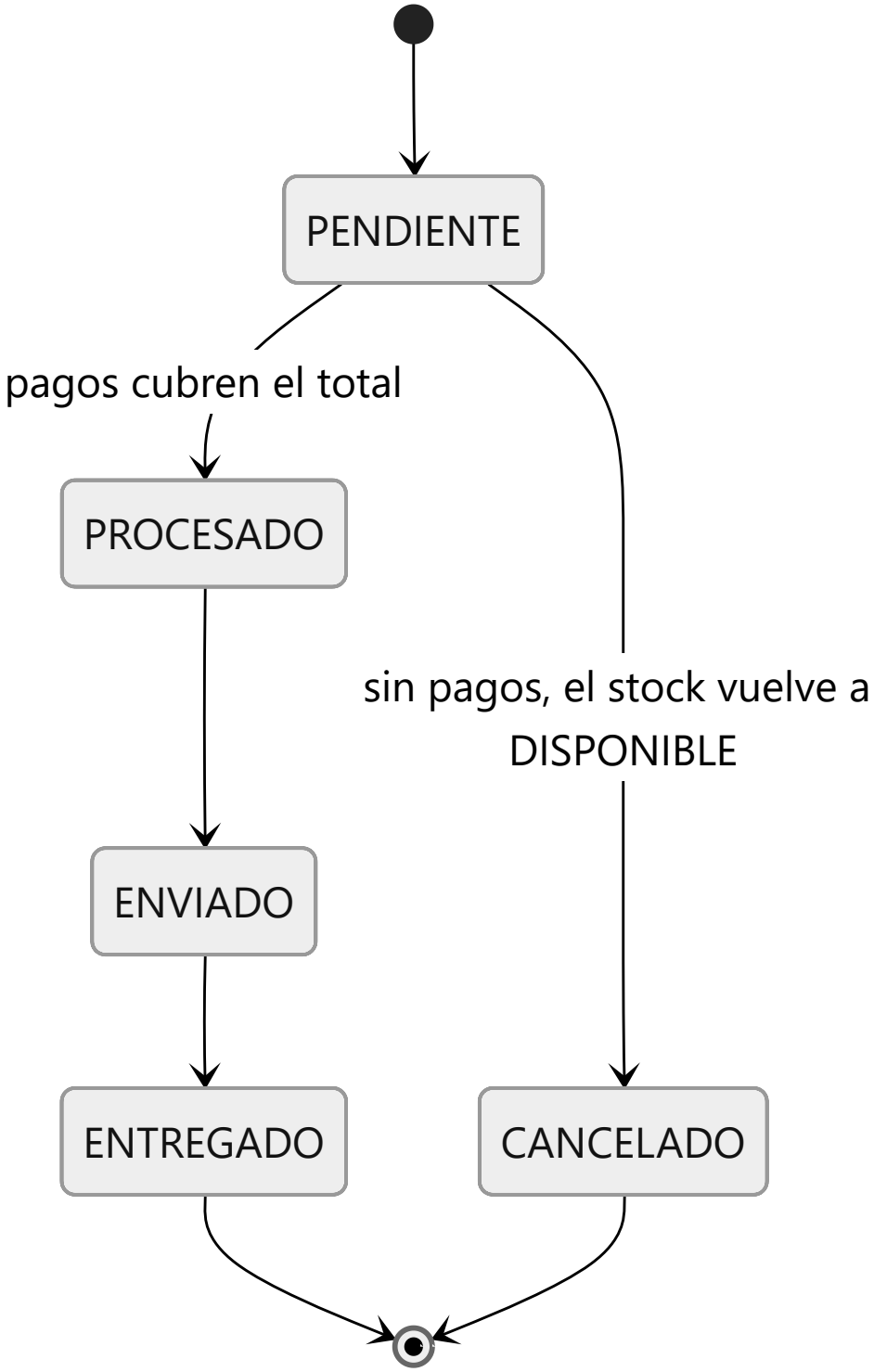
Método	Ruta	Rol requerido	Descripción
GET	`/api/inventario`	Empleado (sucursal)	Filas de inventario; filtro opcional `?id_sucursal=` (fuera del DISTRITAL siempre se limita a la propia sucursal).
GET	`/api/inventario/stock`	Empleado (sucursal)	Stock DISPONIBLE agregado por producto y sucursal (`stock_total`).
POST	`/api/inventario`	BODEGUERO+ (sucursal)	Ingreso de stock: `{ id_producto, id_sucursal, cantidad, estado?, lote? }`. `estado` admite `DISPONIBLE` (por defecto), `DANADA` o `DEFECTUOSA`.
POST	`/api/inventario/ajustes`	BODEGUERO+ (sucursal)	Baja de stock: `{ id_sucursal, motivo, observacion?, ajustes: [{ id_inventario, cantidad_actual, cantidad_nueva }] }`. Motivos: `CORRECCION_CONTEO`, `PERDIDA`, `ROBO`, `DANO`, `DESCARTE`, `OTRO` (este exige observación). Si el stock cambió desde que se leyó, responde `409`.

8.9 Pedidos (`/api/pedidos`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/pedidos`	Empleado (sucursal)	Lista pedidos con cliente, sucursal, vendedor y `origen` (`TIENDA` o `MOSTRADOR`). Admite paginación, `q` y filtros (ver sección 5).
GET	`/api/pedidos/:id`	Empleado (sucursal)	Detalle completo: líneas, pagos, devoluciones, cupón y saldos calculados (`pagado`, `por_confirmar`, `saldo`, `por_cobrar`, `pago_vence`, `desglose`).
POST	`/api/pedidos/cotizar`	CAJERO+	Calcula el total de una venta de mostrador antes de crearla: `{ id_sucursal, detalles, cupon? }`. Sin costo de envío.
POST	`/api/pedidos`	CAJERO+ (sucursal)	Registra una venta de mostrador: `{ id_cliente, id_sucursal, detalles: [{ id_producto, cantidad }], cupon?, direccion_entrega?, email_cliente? }`. El pedido nace `PENDIENTE`; los precios salen del catálogo. Una clienta de otra sucursal solo puede atenderse si se envía su correo exacto en `email_cliente`.

Método	Ruta	Rol requerido	Descripción
PUT	`/api/pedidos/:id/estado`	Empleado (sucursal), según el estado destino	Avanza el estado con `{ estado }`. `PROCESADO` y `CANCELADO`: CAJERO, ADMIN o DISTRITAL. `ENVIADO` y `ENTREGADO`: BODEGUERO, ADMIN o DISTRITAL. `PROCESADO` exige que los pagos completados cubran el total.
GET	`/api/pedidos/:id/historial`	Empleado (sucursal)	Movimientos de inventario del pedido y de sus devoluciones.

Transiciones admitidas por `PUT /api/pedidos/:id/estado` (cualquier otra responde `400`):



Las reglas completas se describen en Reglas de negocio.

8.10 Detalles de pedido (`/api/detalles`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/detalles`	Empleado (sucursal)	Líneas de pedido; filtro opcional `?id_pedido=` .
GET	`/api/detalles/:id`	Empleado (sucursal)	Una línea de pedido.
POST	`/api/detalles`	CAJERO+ (sucursal)	Agrega un producto a un pedido `PENDIENTE` : `{ id_pedido, id_producto, cantidad }` . Descuenta stock y recalcula el total.
DELETE	`/api/detalles/:id`	CAJERO+ (sucursal)	Quita una línea de un pedido `PENDIENTE` ; las unidades vuelven a stock DISPONIBLE.

8.11 Pagos (`/api/pagos`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/pagos`	Empleado (sucursal)	Lista los pagos de pedidos de la sucursal.
POST	`/api/pagos`	CAJERO+ (sucursal)	Registra un cobro: `{ id_pedido, metodo_pago, monto, referencia? }` . Métodos: `EFECTIVO` y `TARJETA` (POS) nacen `COMPLETADO` ; `PAYPAL` nace `PENDIENTE` ; `STRIPE` crea un PaymentIntent y nace `PENDIENTE` (en modo simulado nace `COMPLETADO`). El monto no puede superar el saldo. Si el pedido queda cubierto pasa a `PROCESADO` .
PUT	`/api/pagos/:id/confirmar`	CAJERO+ (sucursal)	Confirma un pago `PENDIENTE` (un pago STRIPE solo si Stripe reporta el cobro).
PUT	`/api/pagos/:id/anular`	CAJERO+ (sucursal)	Anula un pago `PENDIENTE` que no se completará; en STRIPE cancela el PaymentIntent. Si Stripe ya cobró responde `409` y registra el pago como completado.
PUT	`/api/pagos/:id/devolver`	CAJERO+ (sucursal)	Devuelve a la clienta un cobro presencial (EFECTIVO o TARJETA) de un pedido que sigue `PENDIENTE` ; el pago queda `REEMBOLSADO` .

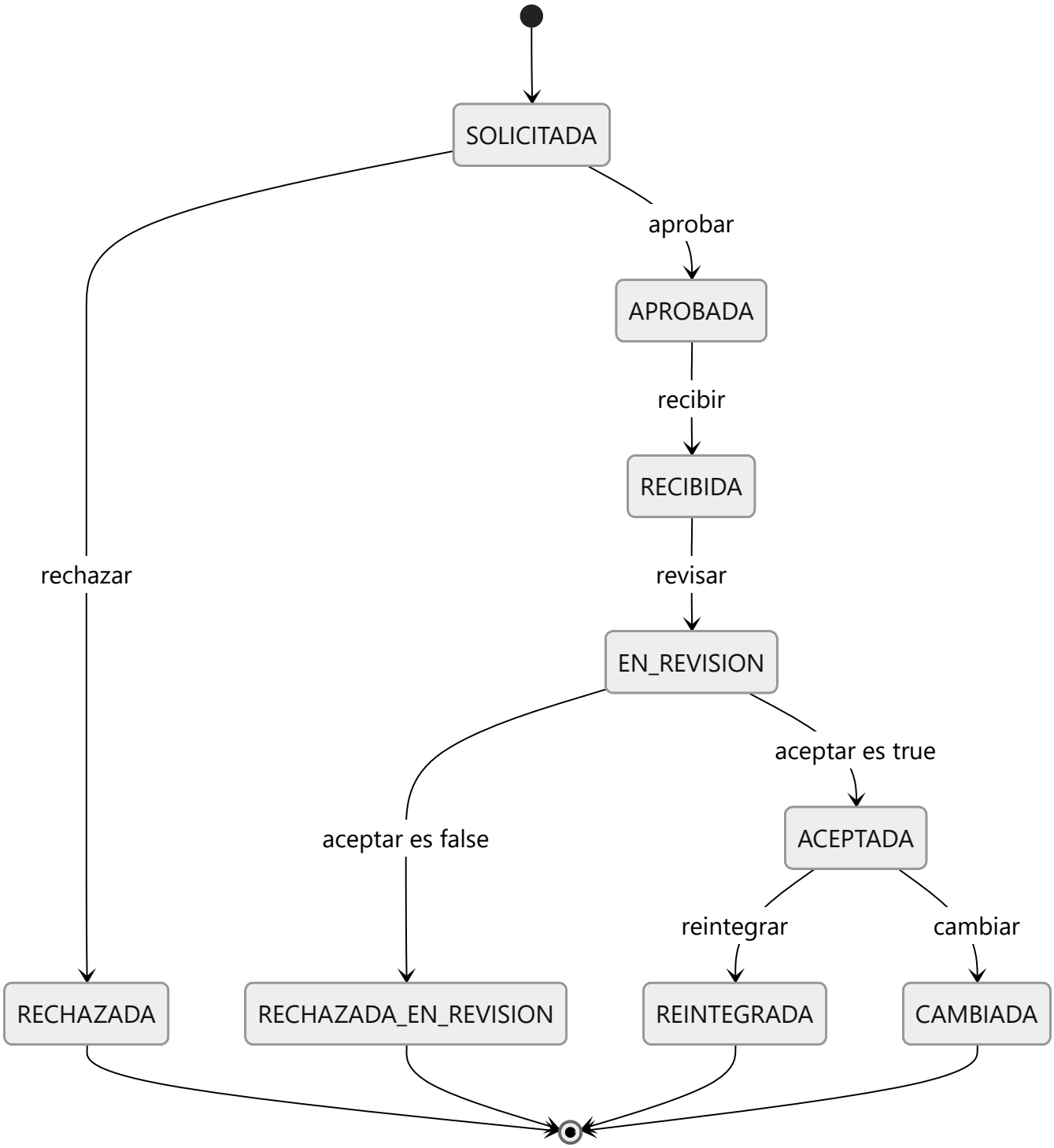
8.12 Reportes (`/api/reportes`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/reportes/ventas`	Empleado (sucursal)	Historial de pedidos con sucursal, clienta, total y estado. El DISTRITAL puede filtrar con `?id_sucursal=` .
GET	`/api/reportes/stock-critico`	Empleado (sucursal)	Productos cuyo stock DISPONIBLE por sucursal es igual o menor que su `stock_minimo` .
GET	`/api/reportes/resumen`	ADMIN+ (sucursal)	Tablero de ventas del periodo `?desde=&hasta=` (AAAA-MM-DD, máximo un año; por defecto, el día actual en Guatemala). Cuenta como venta un pedido `PROCESADO` , `ENVIADO` o `ENTREGADO` .

8.13 Devoluciones y cambios (`/api/devoluciones`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/devoluciones`	Empleado (sucursal)	Lista devoluciones; admite paginación, `q` y filtros.
GET	`/api/devoluciones/pedido/:id_pedido/devolvible`	Empleado (sucursal)	Cantidades aún devolvibles por línea y política vigente (`permite_devolucion` , `motivo_no_permite` , `plazo_vence` , `horas_plazo`).
GET	`/api/devoluciones/:id`	Empleado (sucursal)	Detalle de una devolución.
GET	`/api/devoluciones/:id/historial`	Empleado (sucursal)	Línea de tiempo: movimientos de inventario y auditoría.
POST	`/api/devoluciones`	CAJERO+ (sucursal)	Registra una solicitud: `{ id_pedido, id_cliente, tipo, motivo, observacion?, detalles: [{ id_detalle_pedido, cantidad, id_producto_nuevo? }] }` . Tipos: `DEVOLUCION` , `CAMBIO_TALLA` , `CAMBIO_VARIANTE` . Motivos: `DEFECTUOSO` , `TALLA_INCORRECTA` , `PRODUCTO_INCORRECTO` , `DANADO_ENVIO` , `OTRO` . Solo pedidos `ENTREGADO` , una solicitud por pedido y dentro del plazo.
PUT	`/api/devoluciones/:id/aprobar`	ADMIN+ (sucursal)	`SOLICITADA` a `APROBADA` . Admite `{ observacion? }` .

Método	Ruta	Rol requerido	Descripción
PUT	`/api/devoluciones/:id/rechazar`	ADMIN+ (sucursal)	`SOLICITADA` a `RECHAZADA`. Admite `{ observacion? }`.
PUT	`/api/devoluciones/:id/recibir`	BODEGUERO+ (sucursal)	`APROBADA` a `RECIBIDA`: las prendas llegaron a la sucursal.
PUT	`/api/devoluciones/:id/revisar`	BODEGUERO+ (sucursal)	Revisión física: `{ aceptar, detalles: [{ id_detalle_devolucion, resultado_revision }] }` con `BUEN_ESTADO`, `DANADO` o `DEFECTUOSO` para cada prenda. Resultado `ACEPTADA` o `RECHAZADA_EN_REVISION`.
PUT	`/api/devoluciones/:id/reintegrar`	BODEGUERO+ (sucursal)	Solo `DEVOLUCION` en estado `ACEPTADA`: reingresa las prendas al inventario según su condición y crea los reembolsos. Estado final `REINTEGRADA`.
PUT	`/api/devoluciones/:id/cambiar`	BODEGUERO+ (sucursal)	Solo `CAMBIO_TALLA` o `CAMBIO_VARIANTE` en estado `ACEPTADA`: entrega la variante nueva y, si cuesta menos, reembolsa la diferencia. Estado final `CAMBIADA`.



El paso por `EN_REVISION` ocurre dentro de la misma petición `revisar`, que deja la devolución directamente en `ACEPTADA` o `RECHAZADA_EN_REVISION`.

8.14 Reembolsos (`/api/reembolsos`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/reembolsos`	Empleado (sucursal)	Reembolsos de la sucursal. Filtros: `estado` (uno o varios separados por coma entre `PENDIENTE` , `COMPLETADO` , `FALLIDO`), `id_pedido` , `id_devolucion` .
PUT	`/api/reembolsos/:id/completar`	CAJERO+ (sucursal)	Caja confirma que entregó el dinero (efectivo, reversión en POS o PayPal), con `{ referencia? }` . Un reembolso STRIPE solo se completa aquí si Stripe lo rechazó (`FALLIDO`).
POST	`/api/reembolsos/:id/reintentar`	CAJERO+ (sucursal)	Vuelve a solicitar o consultar en Stripe un reembolso a tarjeta que quedó `PENDIENTE` . Es idempotente.

8.15 Cupones o promociones (`/api/cupones`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/cupones`	DISTRITAL	Lista los cupones con su sucursal.
POST	`/api/cupones`	DISTRITAL	Crea un cupón: `codigo` (3 a 30 letras, números, guion o guion bajo), `tipo` (`PORCENTAJE` o `MONTO`), `valor` , y opcionales `descripcion` , `minimo_compra` , `desde` , `hasta` , `usos_maximos` , `id_sucursal` , `activo` .
PUT	`/api/cupones/:id`	DISTRITAL	Edita un cupón; el código y el tipo no cambian si ya se usó.
DELETE	`/api/cupones/:id`	DISTRITAL	Elimina un cupón que nunca se usó; si ya se usó, debe desactivarse.

8.16 Imágenes de prendas (`/api/imagenes`)

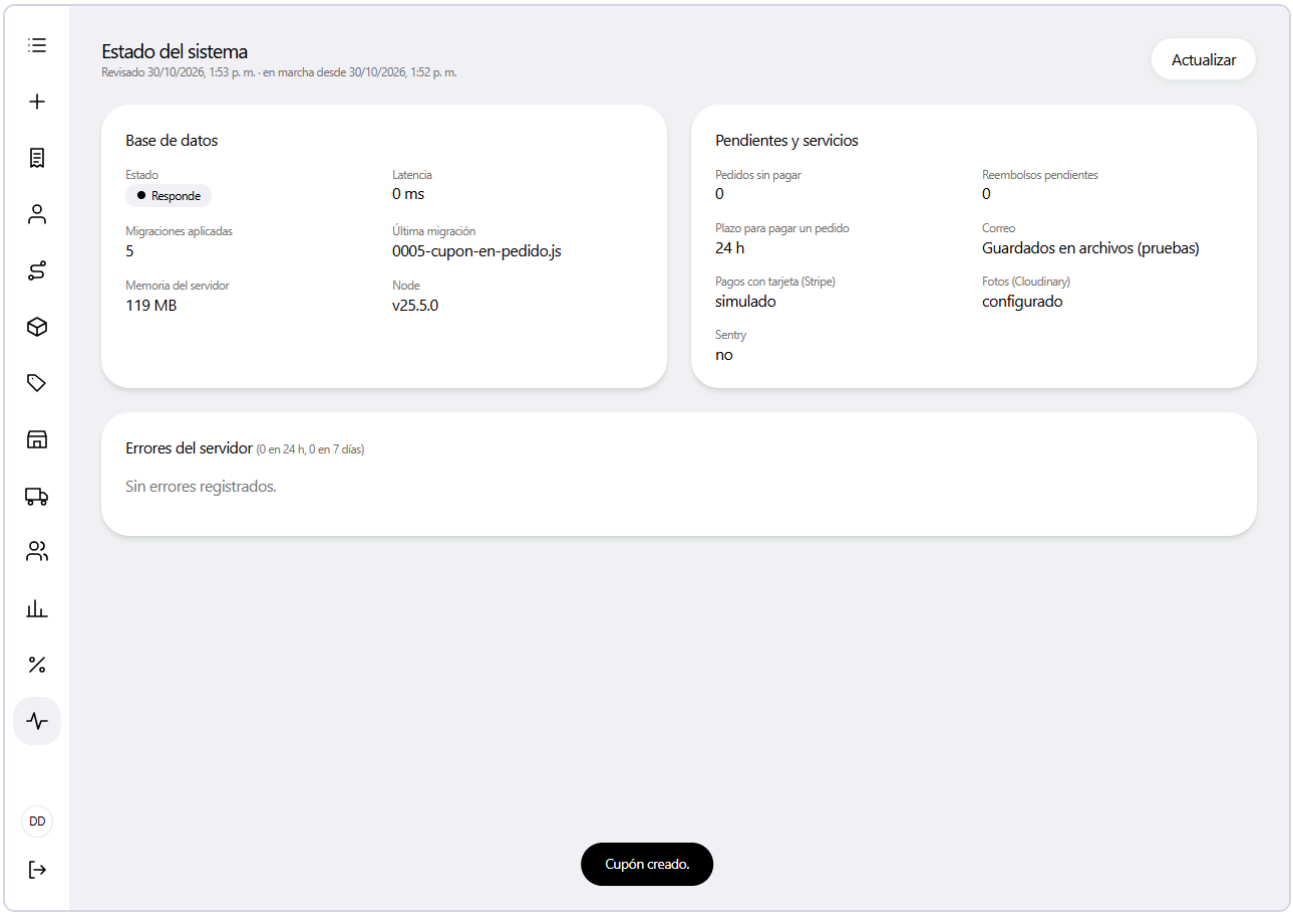
Método	Ruta	Rol requerido	Descripción
GET	`/api/imagenes/config`	Empleado	Indica si la subida a Cloudinary está habilitada y el tamaño máximo (`{ habilitado, max_mb }`).
POST	`/api/imagenes/firma`	DISTRITAL (límite 60)	Devuelve una firma para subir una foto directamente a Cloudinary. Sin las variables `CLOUDINARY_*` responde `503` .
POST	`/api/imagenes/descartar`	DISTRITAL	Borra de Cloudinary fotos subidas que no se guardaron (`{ urls: [...] }` , hasta 20), solo si ninguna prenda las usa. Responde `{ borradas }` .

8.17 Auditoría (`/api/auditoria`)

Método	Ruta	Rol requerido	Descripción
GET	`/api/auditoria`	DISTRITAL	Registros de auditoría, del más reciente al más antiguo. Filtros: `tabla_afectada`, `registro_afectado_id`, `id_empleado`.
POST	`/api/auditoria`	DISTRITAL	Registro manual de auditoría: `{ accion, tabla_afectada, registro_afectado_id?, detalles? }`. Normalmente los controladores registran la auditoría de forma interna.

8.18 Salud del sistema (`/api/salud`)

Método	Ruta	Rol requerido	Descripción
GET	`/`	Pública	Mensaje informativo y ambiente.
GET	`/api/salud`	Pública	`200` con `{ estado: "ok", base: "ok", latencia_base_ms, hora }` si la API y la base responden; `503` con `estado: "degradado"` si la base no responde. Lo usa el monitor del hosting.
GET	`/api/salud/detalle`	DISTRITAL	Tiempo en marcha, versión de Node, memoria, migraciones aplicadas y pendientes, diferencias de esquema, conteo de errores (24 horas y 7 días), pendientes (pedidos sin pagar, reembolsos pendientes y fallidos) y estado de los servicios (correo, Stripe, Cloudinary, Sentry).
GET	`/api/salud/errores`	DISTRITAL	Errores registrados en `ErrorServidor`, paginados.



Pantalla Estado del sistema del panel, alimentada por GET /api/salud/detalle: estado de la base de datos, migraciones, pendientes, servicios externos y errores del servidor.

Figura 1. La pantalla "Estado del sistema" del panel consume `GET /api/salud/detalle` y `GET /api/salud/errores`.

8.19 Webhook de Stripe (`/api/webhook`)

Método	Ruta	Rol requerido	Descripción
POST	`/api/webhook`	Stripe	Recibe eventos de Stripe con el cuerpo sin procesar y valida la cabecera `stripe-signature` con `STRIPE_WEBHOOK_SECRET` (firma inválida: `400`). Atiende `payment_intent.succeeded` (registra el cobro y procesa el pedido), `payment_intent.payment_failed` (deja constancia en auditoría; el pago sigue `PENDIENTE`), `payment_intent.canceled` (el pago pasa a `FALLIDO`) y los eventos de reembolso `refund.created`, `refund.updated`, `refund.failed` y `charge.refund.updated`. Responde `500` ante un error de base de datos para que Stripe reintente.

Esta ruta se registra antes del lector de JSON, porque la firma se valida sobre el cuerpo original.

8.20 Tienda en línea (`/api/tienda`)

Cuenta de la clienta:

Método	Ruta	Rol requerido	Descripción
POST	`/api/tienda/auth/registro`	Pública (límite 10)	Crea una cuenta: `{ nombre, apellido, email, password, telefono?, direccion? }` . Contraseña de 8 caracteres como mínimo. Si el correo ya existe como clienta de mostrador, indica usar "Activar cuenta". Responde `201` con `{ token, cliente }` .
POST	`/api/tienda/auth/login`	Pública (límite 20)	Inicia sesión con `{ email, password }` ; responde `{ token, cliente }` .
POST	`/api/tienda/auth/activar`	Pública (límite 10)	Una clienta de mostrador crea su contraseña con el código de caja: `{ email, codigo, password }` .
POST	`/api/tienda/auth/olvide`	Pública (límite 5)	Solicita un enlace de restablecimiento con `{ email }` (válido una hora). La respuesta es siempre la misma, exista o no la cuenta.
POST	`/api/tienda/auth/restablecer`	Pública (límite 10)	Define una contraseña nueva con el enlace: `{ email, token, password }` . Cierra las demás sesiones.
GET	`/api/tienda/perfil`	Clienta	Datos del perfil (nunca contraseñas ni códigos).
PUT	`/api/tienda/perfil`	Clienta	Edita `nombre` , `apellido` , `telefono` , `direccion` (el correo no se cambia).
PUT	`/api/tienda/perfil/password`	Clienta (límite 10)	Cambia la contraseña con `{ actual, nueva }` ; devuelve un token nuevo.

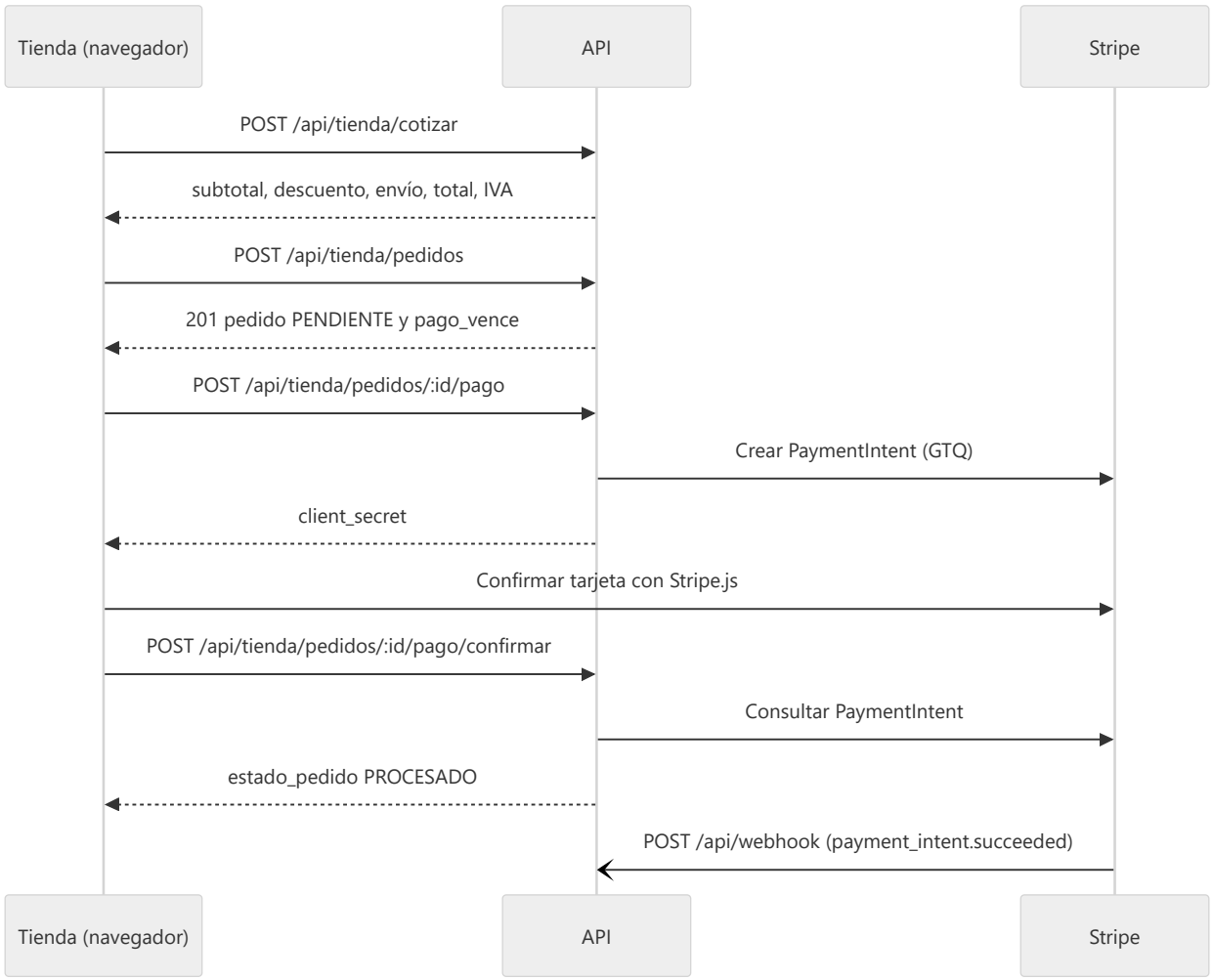
Catálogo:

Método	Ruta	Rol requerido	Descripción
GET	`/api/tienda/catalogo`	Pública	Prendas activas agrupadas con sus variantes, precio mínimo y máximo, y stock DISPONIBLE en sucursales activas. Filtros: `q` , `id_categoria` , `id_sucursal` .
GET	`/api/tienda/catalogo/:id`	Pública	Una prenda (cualquier id de su grupo) con sus variantes, stock por sucursal y lista de sucursales activas.

Pedidos, pagos y devoluciones de la clienta:

Método	Ruta	Rol requerido	Descripción
POST	`/api/tienda/cotizar`	Clienta (límite 300)	Calcula `subtotal`, `descuento`, `costo_envio`, `total`, `base_imponible`, `iva` y `porcentaje` de un carrito: `{ id_sucursal, detalles, cupon?, retiro_en_sucursal? }`. No reserva stock ni consume el cupón.
POST	`/api/tienda/pedidos`	Clienta	Crea un pedido `PENDIENTE`: `{ id_sucursal, detalles, direccion_entrega?, retiro_en_sucursal?, cupon? }`. Máximo 50 productos distintos, 20 unidades por prenda y 3 pedidos pendientes por cuenta. Incluye `pago_vence`.
GET	`/api/tienda/pedidos`	Clienta	Pedidos de la clienta, del más reciente al más antiguo.
GET	`/api/tienda/pedidos/:id`	Clienta	Detalle con líneas, pagos, devoluciones, reembolsos, cupón, `desglose`, saldo, `a_pagar_con_tarjeta` y si admite devolución. Un pedido ajeno responde `404`.
POST	`/api/tienda/pedidos/:id/cancelar`	Clienta	Cancela un pedido aún `PENDIENTE` y sin pagos. Si Stripe ya cobró, responde `409`.
POST	`/api/tienda/pedidos/:id/pago`	Clienta	Inicia o retoma el pago con tarjeta del saldo. Responde `{ modo: "stripe", client_secret, id_pago, monto }`, `{ modo: "stripe", procesando: true, id_pago }` o, en modo simulado, `{ modo: "simulado", id_pago, monto, estado_pedido }`.
POST	`/api/tienda/pedidos/:id/pago/confirmar`	Clienta	Tras confirmar la tarjeta con Stripe.js, consulta el PaymentIntent y registra el cobro sin esperar al webhook: `{ id_pago? }`.
GET	`/api/tienda/pedidos/:id/devolvible`	Clienta	Cantidades devolvibles por línea y política vigente.
POST	`/api/tienda/devoluciones`	Clienta	Solicita devolución o cambio: `{ id_pedido, tipo, motivo, observacion?, detalles }`. El `id_cliente` se toma del token. Un cambio no puede ser por una variante más cara que la pagada.
GET	`/api/tienda/devoluciones`	Clienta	Devoluciones de la clienta con sus prendas.
GET	`/api/tienda/devoluciones/:id`	Clienta	Detalle con reembolsos y `historial` de estados (sin notas internas).

Flujo de compra en la tienda:



El cobro se registra una sola vez: si el webhook llega antes que la confirmación desde la tienda, la segunda operación no repite el registro.

9. Ejemplos de petición y respuesta

Los valores de los ejemplos son ilustrativos. Los campos de tipo DECIMAL de los modelos (por ejemplo `total`, `descuento` o `monto`) se devuelven como texto, tal como los entrega PostgreSQL; los montos calculados por la cotización se devuelven como números.

9.1 Inicio de sesión de un empleado

Petición:

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "cajera.centro@distribuidora.com",
  "password": "********"
}
```

Respuesta `200`:

```
{
  "id": 5,
  "email": "cajera.centro@distribuidora.com",
  "nombre": "Ana",
  "apellido": "López",
  "puesto": "CAJERO",
  "id_sucursal": 2,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Con credenciales incorrectas la respuesta es `401` con `{ "token": null, "message": "Correo o contraseña incorrectos." }`; con la cuenta desactivada, `403`. El valor de `token` se envía en las peticiones siguientes como `Authorization: Bearer <token>`.

9.2 Venta de mostrador con cupón

Se recomienda cotizar primero con `POST /api/pedidos/cotizar` (mismo cuerpo, sin `id_cliente`) para mostrar el total con el descuento. Luego se crea la venta:

```
POST /api/pedidos
Authorization: Bearer <token de empleado>
Content-Type: application/json

{
  "id_cliente": 14,
  "id_sucursal": 2,
  "detalles": [
    { "id_producto": 31, "cantidad": 2 }
  ],
  "cupon": "OTONO10"
}
```

Respuesta `201` (dos unidades de Q175.00 con un cupón de 10 %; en mostrador no se cobra envío):

```
{
  "message": "Venta procesada exitosamente",
  "pedido": {
    "id_pedido": 58,
    "fecha_pedido": "2026-10-30T16:20:11.000Z",
    "estado": "PENDIENTE",
    "total": "315.00",
    "id_cliente": 14,
    "id_sucursal": 2,
    "id_empleado": 5,
    "direccion_entrega": null,
    "fecha_entrega": null,
    "costo_envio": "0.00",
    "descuento": "35.00",
    "id_cupon": 3,
    "cupon_tipo": "PORCENTAJE",
    "cupon_valor": "10.00",
    "cupon_minimo": "0.00"
  }
}
```


El pedido nace `PENDIENTE`. Para cobrarlo se registra el pago con `POST /api/pagos` y `{ "id_pedido": 58, "metodo_pago": "EFECTIVO", "monto": 315 }`; si el pago cubre el total, la respuesta indica `"estado_pedido": "PROCESADO"`. Si el cupón no aplica, la venta se rechaza con `400` y el mensaje "Ese cupón no es válido para esta compra." (o el de compra mínima).

9.3 Cotización en la tienda en línea

```
POST /api/tienda/cotizar
Authorization: Bearer <token de clienta>
Content-Type: application/json

{
  "id_sucursal": 1,
  "detalles": [
    { "id_producto": 12, "cantidad": 1 }
  ],
  "cupon": "bienvenida",
  "retiro_en_sucursal": false
}
```

Respuesta `200` (prenda de Q189.00, cupón de monto fijo Q20.00 y envío a domicilio de Q25.00, porque la compra después del descuento no alcanza el monto de envío gratis):

```
{
  "subtotal": 189,
  "descuento": 20,
  "costo_envio": 25,
  "total": 194,
  "base_imponible": 173.21,
  "iva": 20.79,
  "porcentaje": 12,
  "cupon": {
    "codigo": "BIENVENIDA",
    "descripcion": "Descuento de bienvenida"
  },
  "cupon_error": null,
  "codigo": "BIENVENIDA"
}
```

Un cupón que no aplica no produce error: la cotización se calcula sin descuento, `cupon` es `null` y `cupon_error` contiene el motivo. El costo de envío y el monto mínimo para envío gratis dependen de `ENVIO_COSTO` y `ENVIO_GRATIS_DESDE` (25 y 300 por defecto); el IVA incluido, de `IVA_PORCENTAJE` (12 por defecto). Con `"retiro_en_sucursal": true` el envío es 0.

9.4 Reintegro de una devolución aceptada

La devolución debe ser de tipo `DEVOLUCION` y estar en estado `ACEPTADA` (ya revisada). La petición no requiere cuerpo:

```
PUT /api/devoluciones/7/reintegrar
Authorization: Bearer <token de bodeguero, admin o distrital>
```

Respuesta `200` (respuesta abreviada: se omiten las relaciones `DetalleDevolucions` y `Pedido` que acompañan al objeto `devolucion`, y algunos campos de cada reembolso, como `id_empleado`, `fecha_creacion` y `fecha_completado`):

```
{
  "message": "Devolución reintegrada al inventario. Se reembolsaron Q189.00 a la tarjeta.",
  "devolucion": {
    "id_devolucion": 7,
    "id_pedido": 41,
    "id_cliente": 9,
    "tipo": "DEVOLUCION",
    "motivo": "DEFECTUOSO",
    "estado": "REINTEGRADA",
    "observacion": null,
    "id_empleado_gestor": 6,
    "fecha_solicitud": "2026-10-29T15:02:40.000Z",
    "fecha_actualizacion": "2026-10-30T17:45:12.000Z"
  },
  "reembolsos": [
    {
      "id_reembolso": 4,
      "id_devolucion": 7,
      "id_pedido": 41,
      "id_pago": 37,
      "metodo": "STRIPE",
      "monto": "189.00",
      "estado": "COMPLETADO",
      "id_transaccion_externa": "re_simulado_4",
      "observacion": "Reembolso a la tarjeta con la que pagó en línea."
    }
  ]
}
```


Cada prenda vuelve al inventario de la sucursal del pedido según el resultado de la revisión: `BUEN_ESTADO` a `DISPONIBLE`, `DANADO` a `DANADA` y `DEFECTUOSO` a `DEFECTUOSA`. El monto reembolsado es lo pagado por las prendas devueltas con el descuento del pedido prorrateado; el envío no se devuelve. Se reembolsa primero a la tarjeta con la que se pagó en línea y el resto queda `PENDIENTE` en caja por el método presencial usado, para completarse con `PUT /api/reembolsos/:id/completar`. Si la devolución no está en `ACEPTADA`, la respuesta es `400` con el mensaje "Transición inválida: no se puede pasar de ... a REINTEGRADA."

10. Colección de Postman

El directorio `postman/` contiene una colección lista para importar con los endpoints reales del backend.

Archivo	Función
`postman/distribuidora-backend.postman_collection.json`	Colección en formato Postman v2.1 con 118 peticiones agrupadas en 18 carpetas (autenticación, sucursales, clientes, proveedores, categorías, empleados, productos, inventario, pedidos, detalle de pedido, pagos, estados del pedido, reportes, devoluciones, auditoría, webhook, tienda en línea, y promociones, salud, reembolsos y cuentas). Todas las peticiones incluyen pruebas automáticas que verifican el código de estado.

Archivo	Función
`postman/generate-collection.js`	Script que genera el archivo JSON anterior a partir de una definición declarativa de cada endpoint. El JSON no se edita a mano: cuando se agrega o cambia un endpoint, se modifica el script y se regenera con `node postman/generate-collection.js`.

Variables de colección principales:

Variable	Uso
`baseUrl`	URL base de la API. Por defecto `http://localhost:8080`; para probar el servidor publicado se cambia a `https://distribuidora-api-r6rw.onrender.com`.
`token`	Token del empleado. La petición "Login" lo guarda automáticamente y las peticiones protegidas lo envían en `Authorization: Bearer {{token}}`.
`token_cliente`	Token de la clienta, usado por las peticiones de la carpeta de tienda en línea.
`token_cajero`	Token de un cajero, usado en las pruebas de permisos por puesto.
`admin_email`, `admin_password`	Credenciales con las que se ejecutan "Registro inicial (primer ADMIN)" y "Login". Deben reemplazarse por las de la cuenta DISTRITAL propia.
`id_sucursal`, `id_cliente`, `id_producto`, `id_pedido`, `id_pago`, `id_devolucion`, `id_cupon` y similares	Identificadores que las propias peticiones capturan de las respuestas de creación y reutilizan en las siguientes.

Procedimiento para importarla y usarla:

1. Iniciar el backend (`npm start`) o decidir usar el servidor publicado.
2. En Postman, seleccionar **Import**, elegir el archivo `postman/distribuidora-backend.postman_collection.json` y confirmar.
3. Abrir la colección, pestaña **Variables**, y ajustar `baseUrl`, `admin_email` y `admin_password`.
4. Ejecutar la carpeta "0. Autenticación": "Registro inicial (primer ADMIN)" crea el primer empleado solo si la base está vacía (si ya hay empleados responde `403`, lo cual es esperado) y "Login" guarda el `token`.
5. Ejecutar el resto de carpetas en orden, de forma individual o con **Run collection**, ya que cada carpeta reutiliza los identificadores creados por las anteriores.

La petición de la carpeta "15. Webhook (Stripe)" se incluye por completitud con una firma de ejemplo inválida, por lo que la respuesta esperada es `400`; los eventos reales los envía Stripe.

Documentos relacionados

- Arquitectura
- Modelo de datos
- Roles y permisos
- Reglas de negocio

- Seguridad
- Pruebas
- Despliegue y operación

Manual de usuario

Este manual explica, paso a paso y sin términos técnicos, cómo se usa el sistema Distribuidora. La **Parte A** está dirigida a las clientas que compran en la tienda en línea. La **Parte B** está dirigida al personal de las sucursales que trabaja en el panel de empleados y se organiza por tarea, indicando en cada caso qué puesto puede realizarla. Los nombres de botones, pestañas y mensajes que aparecen entre comillas son los mismos que muestra la pantalla.

Aplicación	Dirección	Quién la usa
Tienda en línea	https://distribuidora-tienda.onrender.com	Clientas
Panel de empleados	https://distribuidora-panel.onrender.com	Gerente distrital, administradores de sucursal, cajeros y bodegueros

Contenido

- Conceptos básicos
- Parte A. Tienda en línea (clientas)
  - A1. Crear una cuenta
  - A2. Activar la cuenta con el código de caja
  - A3. Olvidé mi contraseña
  - A4. Catálogo y ficha de la prenda
  - A5. Carrito
  - A6. Finalizar compra: entrega y cupón
  - A7. Pagar con tarjeta
  - A8. Seguimiento del pedido y plazo de pago
  - A9. Solicitar un cambio o una devolución
  - A10. Seguimiento de la solicitud y reembolsos
  - A11. Perfil y cambio de contraseña
- Parte B. Panel de empleados
  - B1. Puestos y secciones visibles
  - B2. Iniciar sesión y cambiar la contraseña
  - B3. Registrar una venta de mostrador
  - B4. Cobrar y confirmar pagos
  - B5. Avanzar el estado de un pedido
  - B6. Procesar una devolución o un cambio
  - B7. Clientes y código web
  - B8. Catálogo, tipos de prenda y fotos
  - B9. Inventario y trazabilidad
  - B10. Sucursales y ajustes de stock
  - B11. Proveedores

    - B12. Empleados
    - B13. Promociones y cupones
    - B14. Reportes de ventas y PDF
    - B15. Estado del sistema
- Preguntas frecuentes

Conceptos básicos

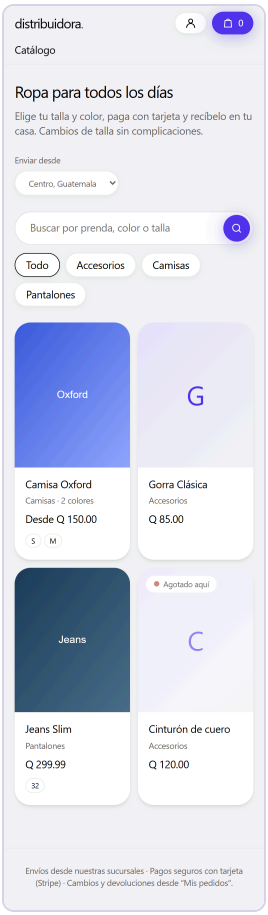
Término	Significado
Sucursal	Tienda física. Cada pedido se despacha desde una sola sucursal y descuenta el stock de esa sucursal.
Prenda base y variante	Una prenda (por ejemplo, "Camisa Oxford") puede tener variantes por talla y color. Cada variante tiene su propio precio y stock.
Pedido	Compra hecha en la tienda en línea (origen "Tienda en línea") o en caja (origen "Mostrador").
Cupón	Código de descuento creado por el gerente distrital; se aplica en la tienda o en caja.
Solicitud de cambio o devolución	Petición de la clienta para cambiar la talla o el color de una prenda, o devolverla. Se permite una sola solicitud por pedido.
Reembolso	Dinero que se devuelve a la clienta como resultado de una devolución o de un cambio a una variante más barata.

Los precios incluyen IVA (12 % con la configuración por defecto). Los montos se expresan en quetzales (Q).

Parte A. Tienda en línea (clientas)

La tienda permite ver el catálogo sin iniciar sesión. Para comprar, pagar y dar seguimiento a los pedidos se necesita una cuenta. En la parte superior se encuentran los enlaces "Catálogo", "Mis pedidos" y "Devoluciones" (los dos últimos solo con sesión iniciada), el botón de la cuenta y el botón del carrito con el número de prendas agregadas.

La tienda se adapta a la pantalla del teléfono: los menús, el selector de sucursal y la búsqueda se reorganizan en una sola columna.



Catálogo de la tienda visto desde un teléfono, con el selector de sucursal, el buscador, los filtros por tipo de prenda y las tarjetas de prendas en dos columnas

Figura A-1. Tienda en línea en un teléfono.

A1. Crear una cuenta

1. Seleccionar "Iniciar sesión" en la esquina superior derecha y luego el enlace "Crea una".
2. Completar el formulario "Crea tu cuenta": nombre, apellido, correo, contraseña (mínimo 8 caracteres) y su repetición. El teléfono y la dirección de entrega son opcionales; la dirección se propondrá después al finalizar la compra.
3. Presionar "Crear cuenta". La sesión queda iniciada y la tienda regresa a la página desde la que se llegó.

Si el correo ya fue registrado en una sucursal (porque la clienta compró en caja), el sistema muestra: "Ese correo ya está registrado en una de nuestras tiendas. Pide en caja tu código de activación y usa «Activar cuenta» para conservar tu historial", junto con el enlace "Activar mi cuenta". En ese caso debe seguirse la sección A2.

A2. Activar la cuenta con el código de caja

Una clienta que ya compró en una tienda física y dejó su correo en caja puede activar su cuenta en línea y ver también sus compras anteriores.

1. Pedir en caja el código de activación. El personal lo genera desde el panel (ver B7); tiene el formato `xxxx-xxxx`, sirve una sola vez y vence a las 72 horas.
2. En la tienda, abrir "Iniciar sesión" y seleccionar "Activa tu cuenta con tu código".
3. Completar "Correo registrado en caja", "Código de activación", "Crea tu contraseña" y "Repite la contraseña".
4. Presionar "Activar cuenta". La sesión queda iniciada y en "Mis pedidos" aparecen las compras hechas en mostrador.

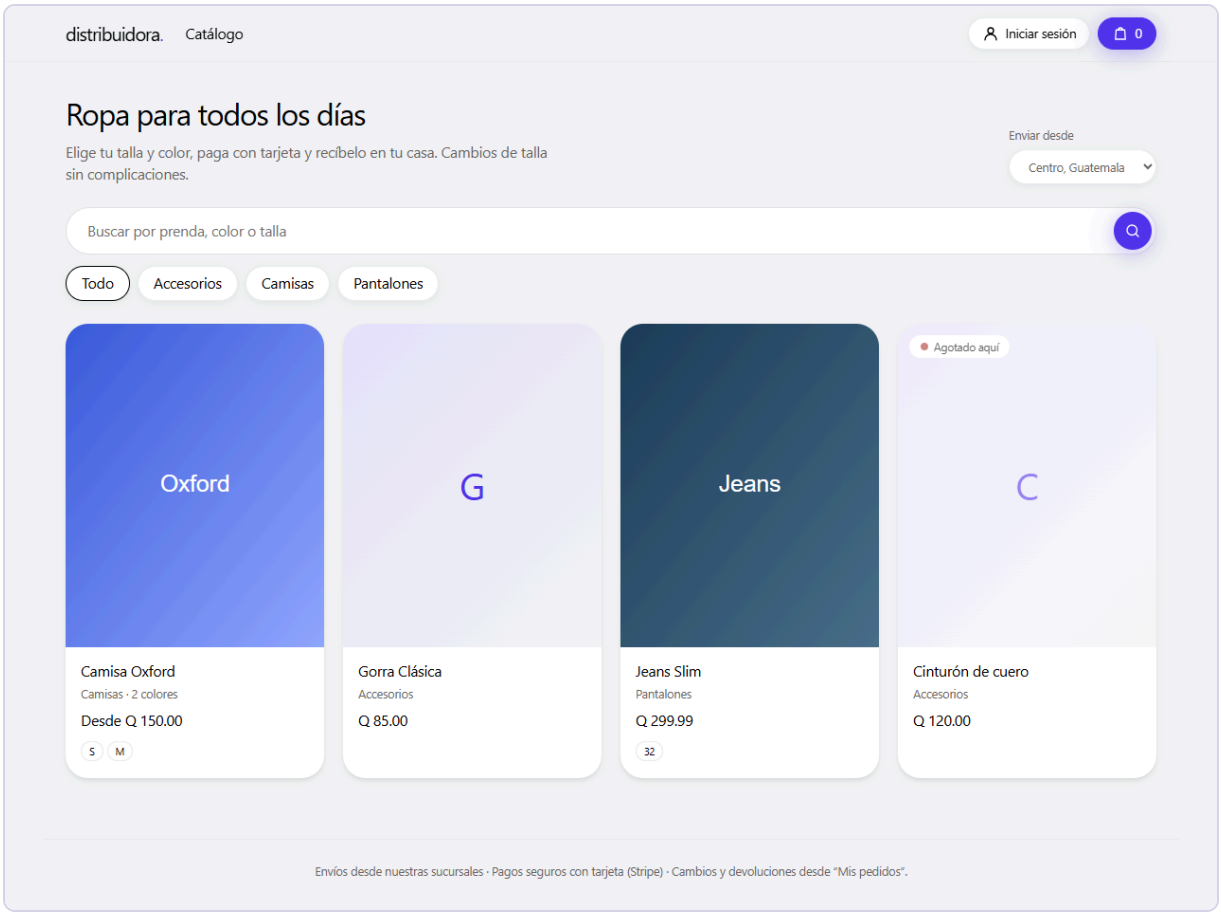
A3. Olvidé mi contraseña

1. En "Iniciar sesión", seleccionar "¿Olvidaste tu contraseña?".
2. Escribir el correo de la cuenta y presionar "Enviarme el enlace". Por seguridad, el mensaje es el mismo exista o no la cuenta: "Si ese correo tiene una cuenta, te enviamos un enlace para crear una contraseña nueva... El enlace vence en 1 hora."
3. Abrir el enlace recibido por correo. Se muestra la página "Crea tu contraseña nueva" con el correo de la cuenta.
4. Escribir la contraseña nueva dos veces y presionar "Guardar contraseña". Al guardarla se cierran las sesiones abiertas en otros dispositivos.

Si el enlace está incompleto o ya venció, la tienda ofrece "Pedir un enlace nuevo". Esta opción también sirve a una clienta registrada en caja que nunca creó contraseña.

A4. Catálogo y ficha de la prenda

La página principal muestra el catálogo con el título "Ropa para todos los días".

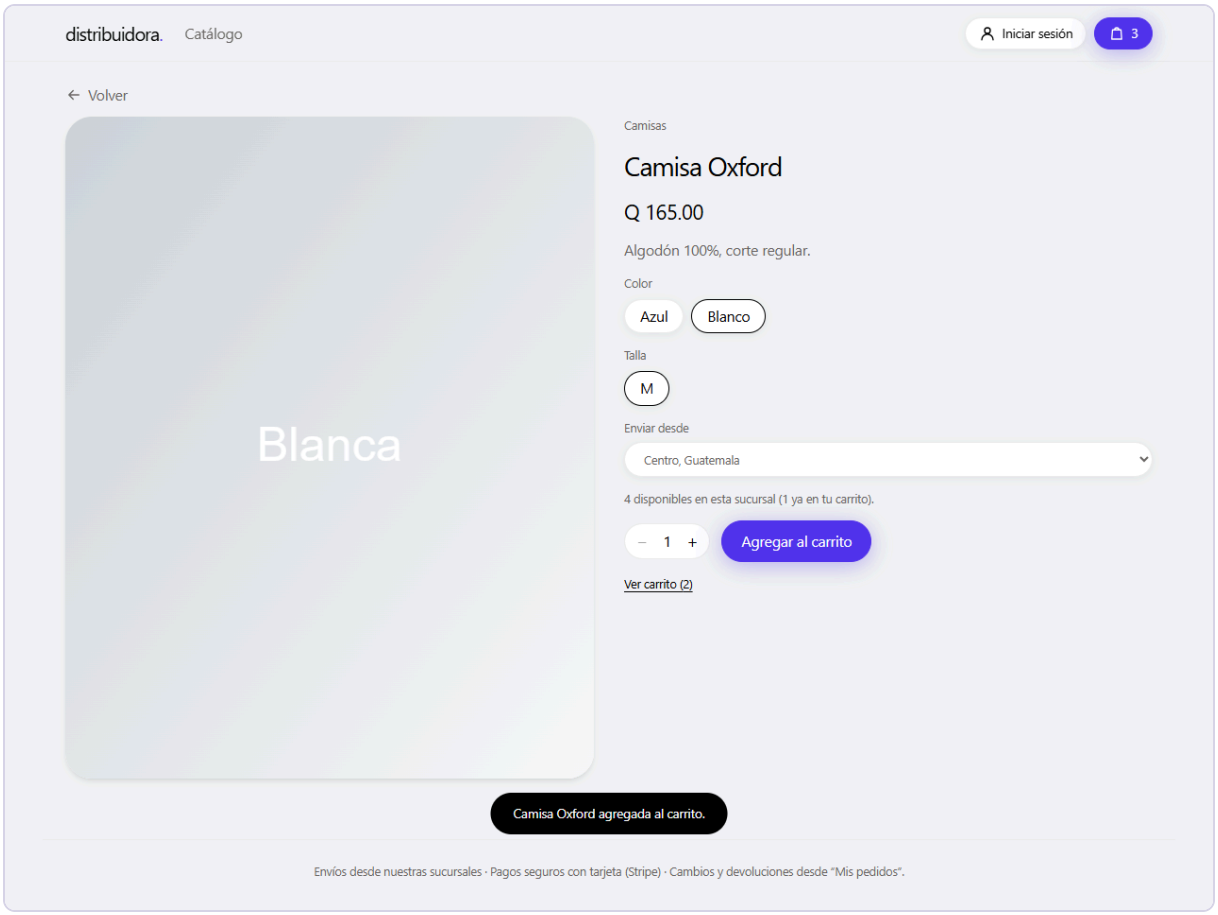


Página principal de la tienda con el selector "Enviar desde", el buscador, los filtros Todo, Accesorios, Camisas y Pantalones, y cuatro tarjetas de prendas; una de ellas marcada como "Agotado aquí"

Figura A-2. Catálogo de la tienda.

- **Enviar desde:** selector de la sucursal desde la que se despachará el pedido. El stock que se muestra es el de esa sucursal. Si no se elige ninguna, la tienda toma la primera sucursal activa.
- **Buscador:** acepta nombre de prenda, color o talla ("Buscar por prenda, color o talla").
- **Filtros por tipo de prenda:** botones "Todo" y uno por cada tipo registrado.
- **Tarjetas:** cada prenda muestra nombre, tipo, número de colores, precio (o "Desde Q..." si las variantes tienen precios distintos) y las tallas con existencias. La etiqueta "Agotado aquí" indica que la prenda no tiene stock en la sucursal elegida.

Al seleccionar una tarjeta se abre la ficha de la prenda.



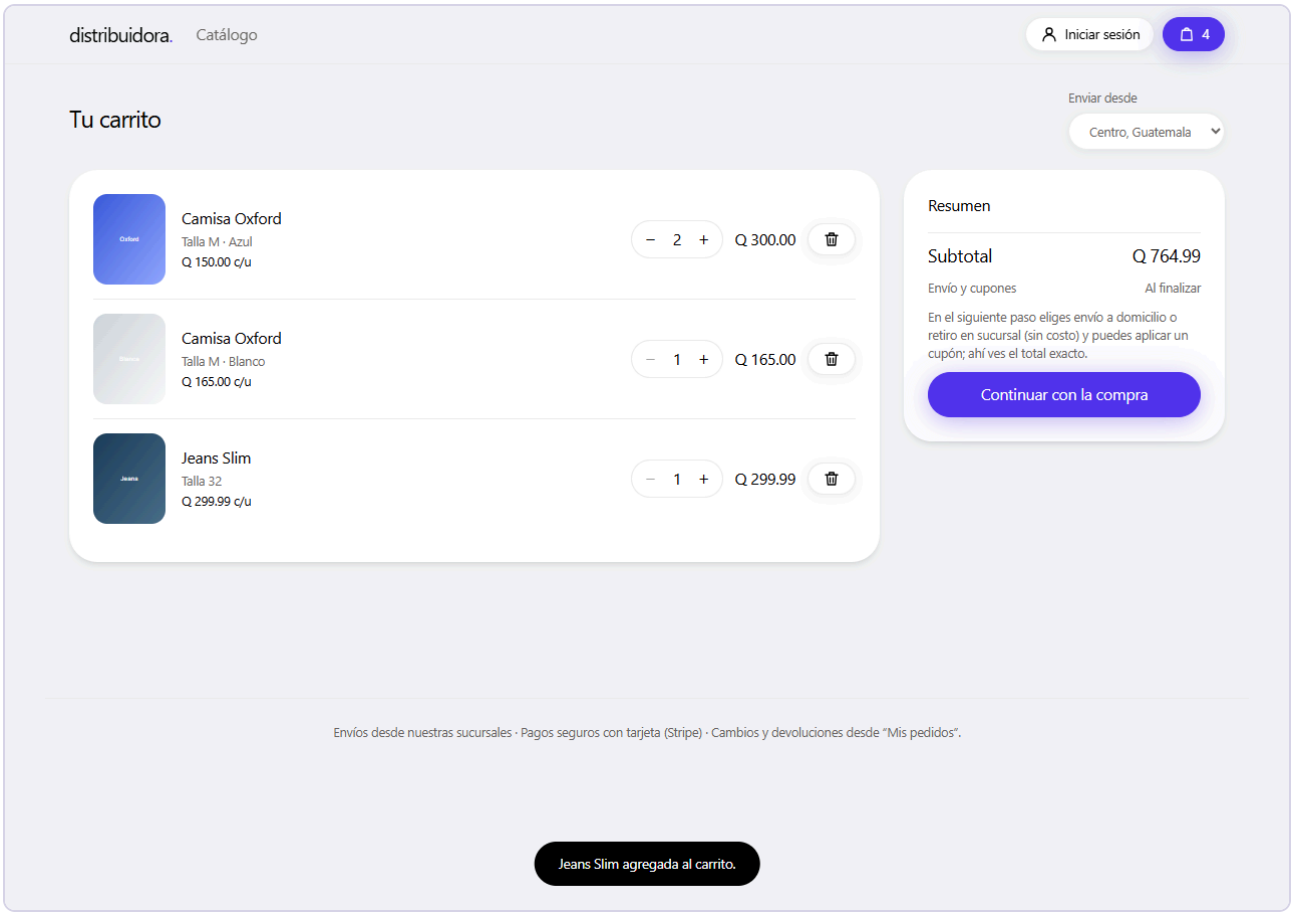
Ficha de la Camisa Oxford con precio Q 165.00, descripción, botones de color Azul y Blanco, talla M, selector de sucursal, el texto "4 disponibles en esta sucursal (1 ya en tu carrito)" y el botón "Agregar al carrito"

Figura A-3. Ficha de una prenda con variantes de color y talla.

1. Elegir el **Color** y luego la **Talla**. El precio y la foto cambian según la variante. Las tallas sin existencias en la sucursal elegida aparecen atenuadas.
2. Revisar el texto de disponibilidad, por ejemplo "4 disponibles en esta sucursal". Si no hay existencias, la ficha indica "Sin existencias de esta talla en la sucursal elegida" y, cuando corresponde, "Disponible en:" con las otras sucursales que sí la tienen.
3. Ajustar la cantidad con los botones "−" y "+" y presionar "Agregar al carrito". Aparece el aviso "[prenda] agregada al carrito".

La cantidad máxima por variante es 20 unidades y nunca supera el stock disponible. Si ya se agregó todo el stock, el botón cambia a "Ya tienes todo el stock en el carrito".

A5. Carrito

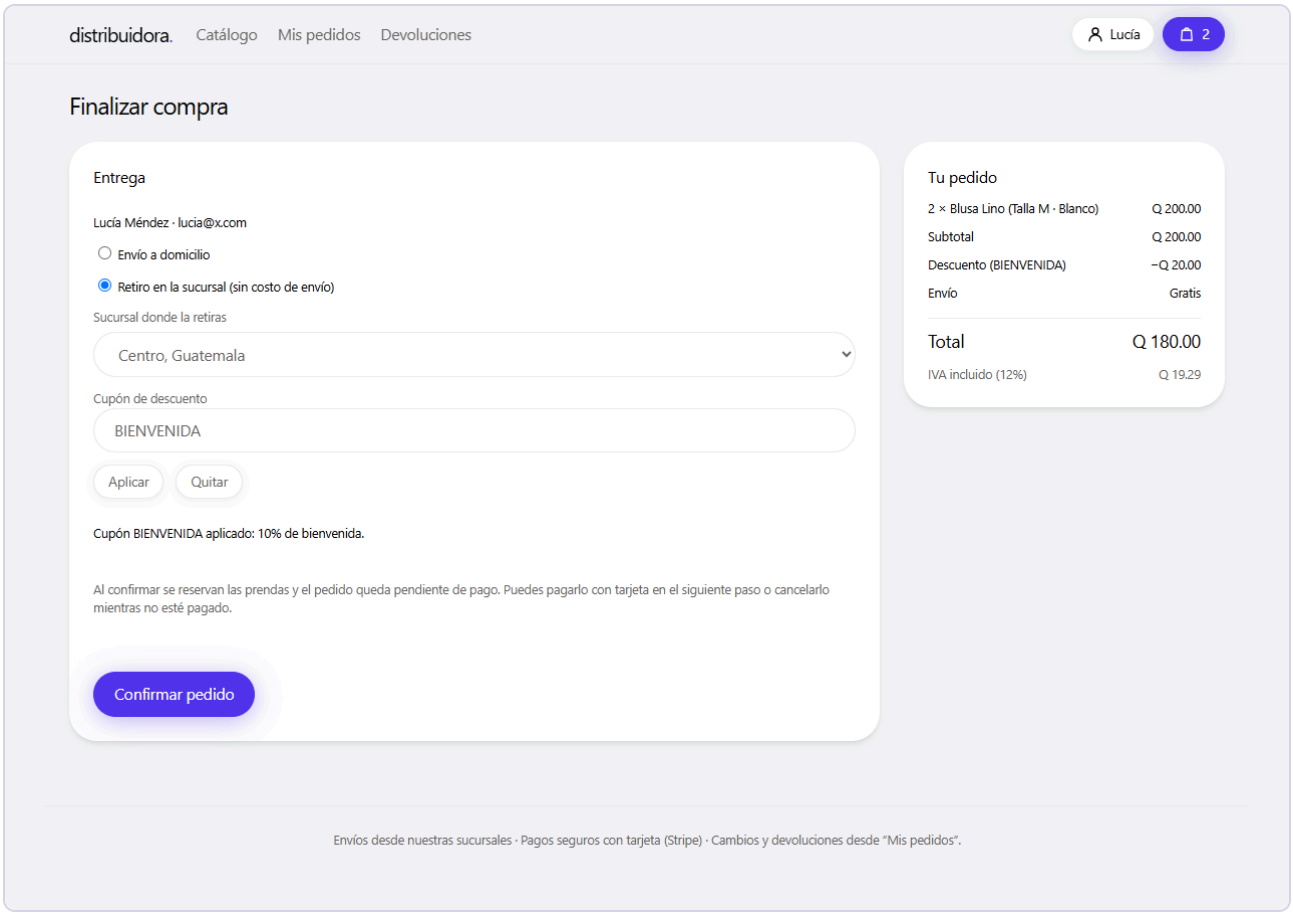


Carrito con tres líneas (dos variantes de Camisa Oxford y unos Jeans Slim), controles de cantidad, botón de eliminar por línea y el resumen con subtotal Q 764.99 y el botón "Continuar con la compra"

Figura A-4. Carrito de compras.

- Cada línea muestra la prenda, la variante, el precio unitario, los controles de cantidad y el botón de papelera para quitarla.
- El carrito se guarda en el navegador, por lo que se conserva al cerrar la página.
- Al abrirlo, la tienda revisa las existencias de la sucursal elegida: actualiza precios que hayan cambiado y marca las líneas con problemas ("Solo quedan N en la sucursal elegida", "Sin existencias en la sucursal elegida" o "Esta prenda ya no está a la venta. Quítala para continuar").
- El resumen indica el subtotal; el envío y los cupones se calculan en el paso siguiente.
- Presionar "Continuar con la compra". Si no hay sesión iniciada, la tienda pide iniciar sesión y luego regresa a este punto.

A6. Finalizar compra: entrega y cupón



Pantalla "Finalizar compra" con la opción "Retiro en la sucursal (sin costo de envío)" elegida, el cupón BIENVENIDA aplicado y el resumen con subtotal Q 200.00, descuento −Q 20.00, envío Gratis, total Q 180.00 e IVA incluido

Figura A-5. Finalizar compra con retiro en sucursal y cupón aplicado.

1. En **Entrega**, elegir una opción:
   - "Envío a domicilio": se debe escribir la "Dirección de entrega" (se propone la del perfil). El envío cuesta Q 25.00 y es gratis cuando el monto de las prendas, después del descuento, llega a Q 300.00 (valores por defecto del sistema).
   - "Retiro en la sucursal (sin costo de envío)": el selector cambia a "Sucursal donde la retiras".
2. En **Cupón de descuento**, escribir el código y presionar "Aplicar". Si es válido aparece "Cupón [código] aplicado"; si no aplica, se muestra el motivo en rojo. "Quitar" elimina el cupón.
3. Revisar el resumen "Tu pedido": subtotal, descuento, envío, total e IVA incluido. Este cálculo lo hace el servidor con los precios vigentes.
4. Presionar "Confirmar pedido". Las prendas quedan reservadas y el pedido queda "Pendiente de pago". La tienda abre el pedido con el aviso "Pedido #N creado. Ahora completa el pago."

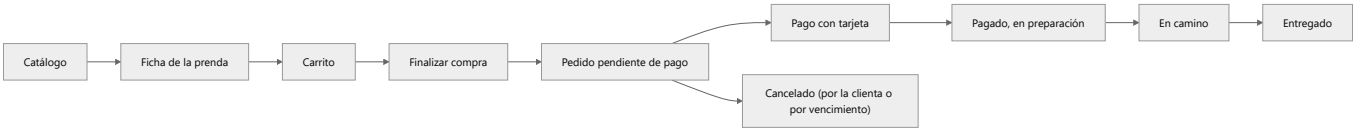
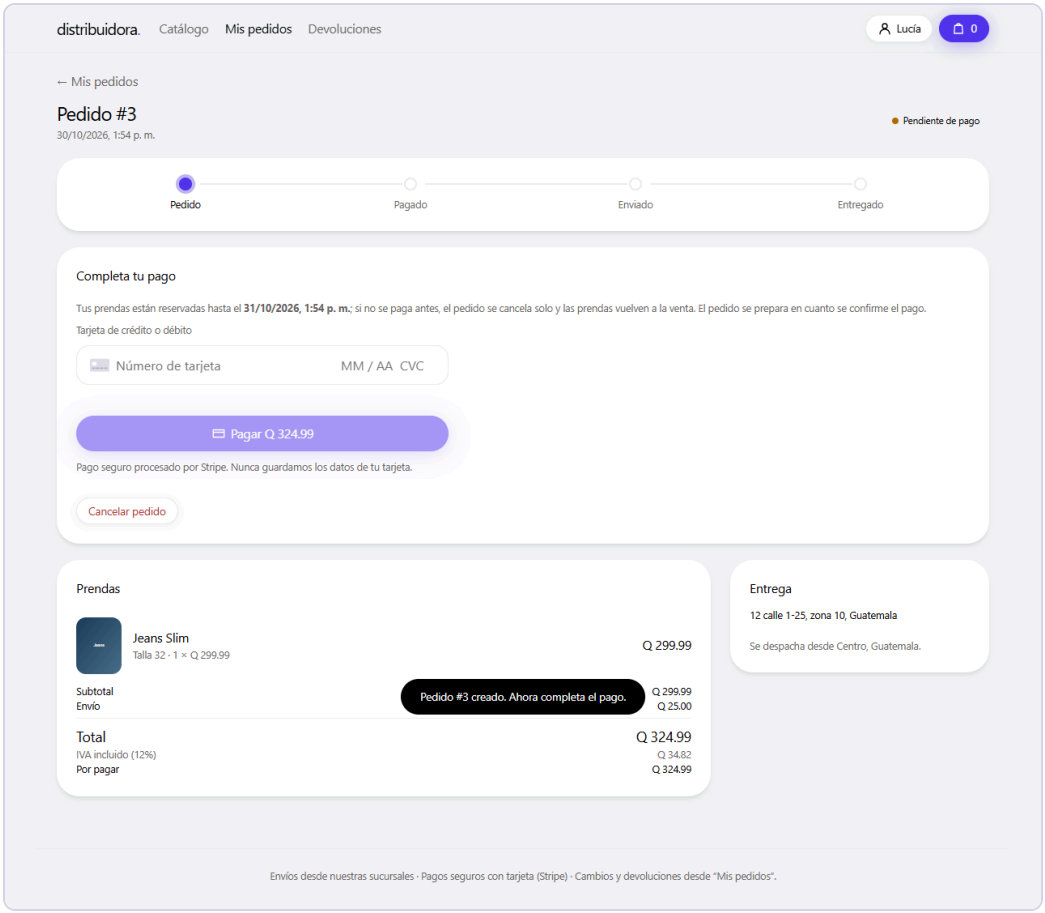


Figura A-6. Recorrido de una compra en la tienda.

A7. Pagar con tarjeta



Pedido #3 pendiente de pago con la sección "Completa tu pago", la fecha límite de reserva, el campo de tarjeta, el botón "Pagar Q 324.99" y el botón "Cancelar pedido"

Figura A-7. Sección de pago del pedido.

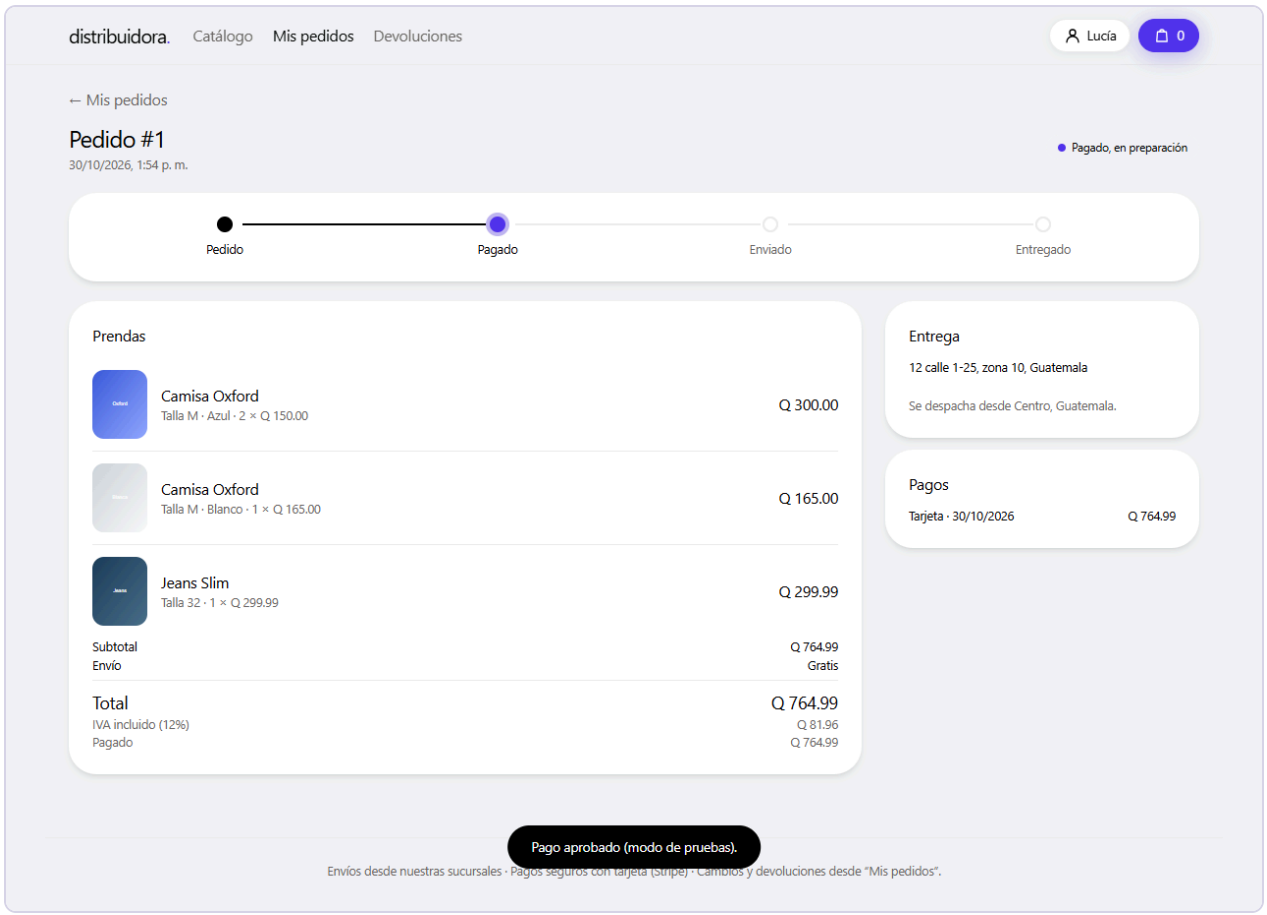
1. En la sección "Completa tu pago", escribir los datos en "Tarjeta de crédito o débito" (número, vencimiento y código de seguridad).
2. Presionar "Pagar Q...". El botón se habilita cuando los datos están completos.
3. Si el pago se aprueba, aparece "¡Pago aprobado! Tu pedido ya está en preparación." y el estado cambia a "Pagado, en preparación". Si la tarjeta es rechazada, se muestra el motivo y el pago puede reintentarse.

El cobro lo procesa Stripe; la tienda no guarda los datos de la tarjeta. En el entorno de pruebas se utiliza la tarjeta de prueba **4242 4242 4242 4242**, con cualquier fecha de vencimiento futura y cualquier código de seguridad. Cuando el servidor funciona en modo de pago simulado, el pago se aprueba al instante con el mensaje "Pago aprobado." (o "Pago aprobado (modo de pruebas)." si la tienda no tiene configurada la clave pública de Stripe).

Si parte del pedido ya se registró en una tienda física y está pendiente de confirmar, la sección lo indica y la tarjeta cobra solo el resto.

A8. Seguimiento del pedido y plazo de pago

"Mis pedidos" lista todos los pedidos con número, fecha, cantidad de prendas, sucursal, estado y total. Los pedidos pendientes muestran "Págalo antes del [fecha]".



Pedido #1 en estado "Pagado, en preparación" con la línea de pasos Pedido, Pagado, Enviado y Entregado, las prendas, el total, la entrega y el pago con tarjeta

Figura A-8. Detalle de un pedido pagado.

Al abrir un pedido se ven la línea de pasos (Pedido, Pagado, Enviado, Entregado), las prendas, el desglose de montos, la dirección de entrega o "Retiro en sucursal", la sucursal de despacho y los pagos realizados.

Estado que ve la clienta	Significado
Pendiente de pago	Las prendas están reservadas; falta pagar.
Pagado, en preparación	El pago se confirmó y la sucursal prepara el pedido.
En camino	El pedido salió de la sucursal.

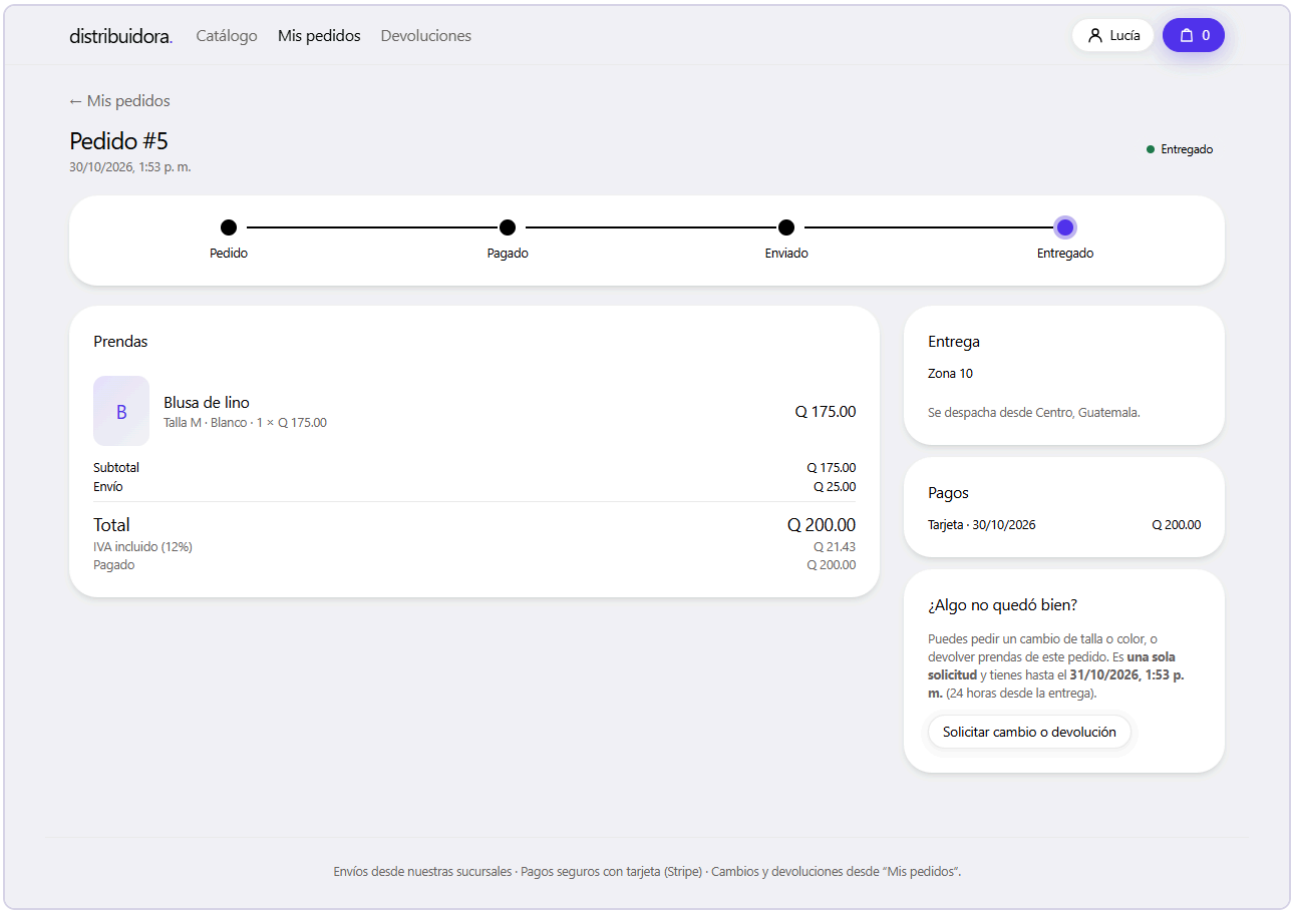
Estado que ve la clienta	Significado
Entregado	La clienta recibió el pedido. Desde aquí puede solicitar un cambio o una devolución.
Cancelado	El pedido se canceló y las prendas volvieron a la venta.

Plazo de pago. Un pedido pendiente debe pagarse dentro de 24 horas (valor por defecto). La fecha límite se muestra en "Completa tu pago". Si vence sin pago, la tienda muestra "El plazo para pagar venció" y el pedido se cancela automáticamente; las prendas vuelven a la venta.

Cancelar un pedido. Mientras el pedido esté pendiente, el botón "Cancelar pedido" abre una confirmación; al presionar "Sí, cancelar" las prendas dejan de estar reservadas y, si se había iniciado un pago con tarjeta, también se cancela.

A9. Solicitar un cambio o una devolución

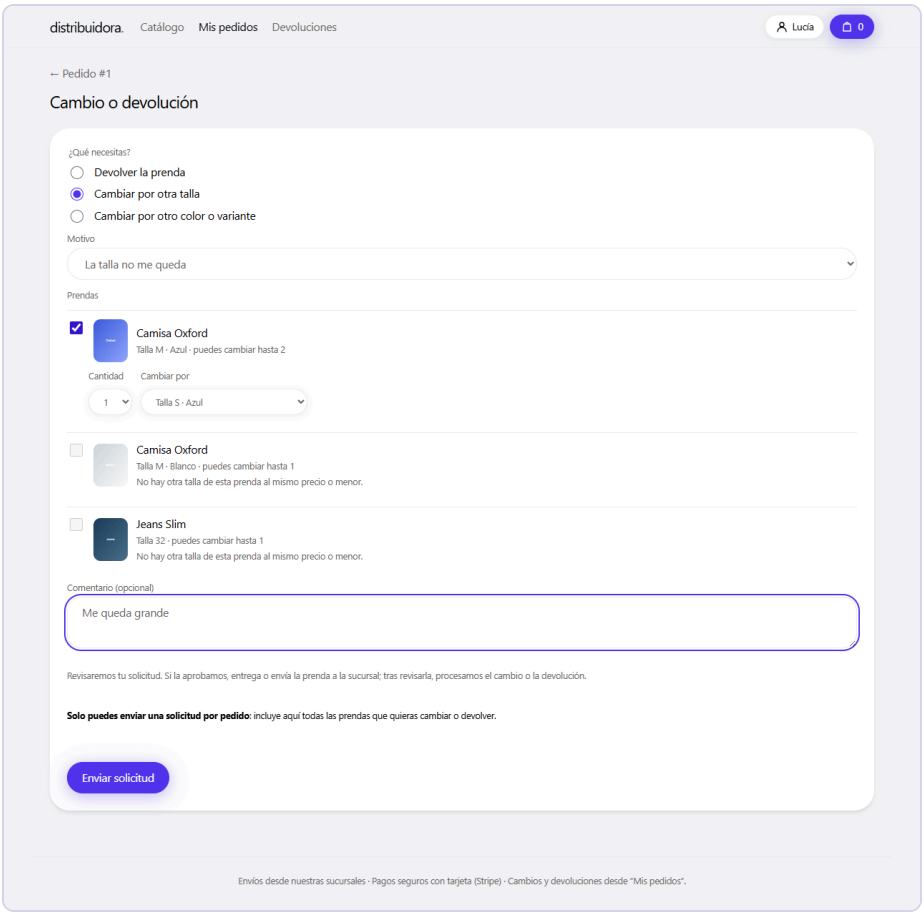
Cuando el pedido está "Entregado", su detalle muestra la sección "¿Algo no quedó bien?" con la fecha límite. El plazo es de **24 horas desde la entrega** y se permite **una sola solicitud por pedido**.



Pedido #5 entregado con la sección "¿Algo no quedó bien?", que indica que es una sola solicitud, la fecha límite (24 horas desde la entrega) y el botón "Solicitar cambio o devolución"

Figura A-9. Plazo para pedir un cambio o una devolución.

1. Presionar "Solicitar cambio o devolución".
2. En "¿Qué necesitas?", elegir "Devolver la prenda", "Cambiar por otra talla" o "Cambiar por otro color o variante".
3. Elegir el **Motivo**: "La talla no me queda", "Llegó con un defecto", "Se dañó en el envío", "No es lo que pedí" u "Otro motivo".
4. En **Prendas**, marcar cada prenda incluida, elegir la **Cantidad** y, si es un cambio, la opción de "Cambiar por". Solo se ofrecen variantes de igual o menor precio que lo pagado; si no hay ninguna, la prenda indica "No hay otra talla de esta prenda al mismo precio o menor". Las variantes sin existencias aparecen como "(sin stock por ahora)".
5. Escribir un "Comentario (opcional)" y presionar "Enviar solicitud". Aparece el aviso "Solicitud enviada. Te avisaremos cuando la revisemos."



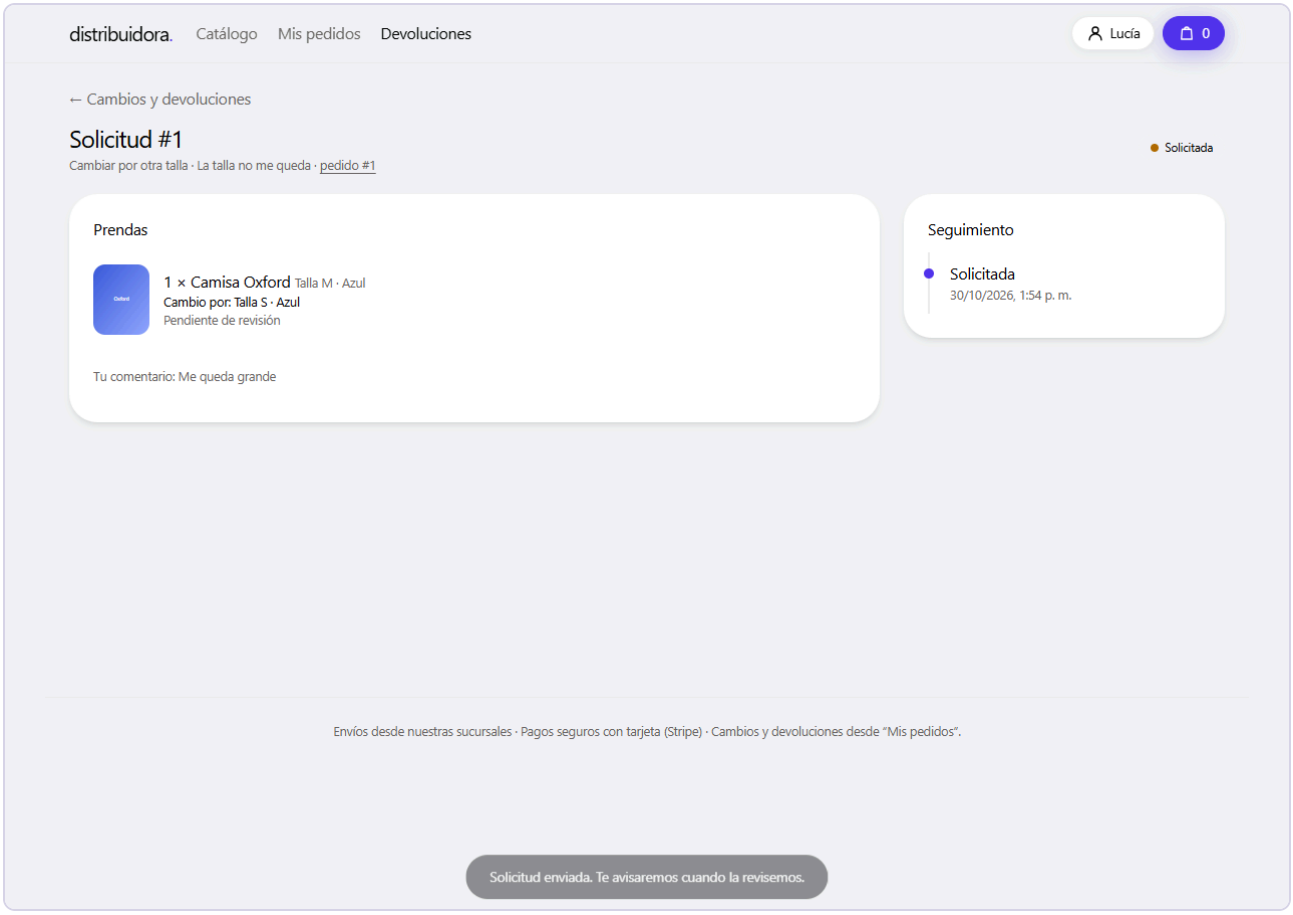
Formulario "Cambio o devolución" con la opción "Cambiar por otra talla", el motivo "La talla no me queda", una Camisa Oxford marcada para cambiar por la talla S, el comentario y el botón "Enviar solicitud"

Figura A-10. Formulario de solicitud de cambio.

Como la solicitud es única, deben incluirse en ella todas las prendas que se desean cambiar o devolver.

A10. Seguimiento de la solicitud y reembolsos

La sección "Devoluciones" (título "Cambios y devoluciones") lista las solicitudes. Cada una muestra las prendas, el cambio pedido, el resultado de la revisión, el comentario y la sección "Seguimiento" con cada estado y su fecha.



Solicitud #1 en estado "Solicitada", con la prenda a cambiar, "Cambio por: Talla S · Azul", "Pendiente de revisión" y el seguimiento

Figura A-11. Detalle de una solicitud de cambio.

Estado	Qué debe hacer la clienta
Solicitada	Esperar la revisión de la tienda.
Aprobada: envía o lleva la prenda	Entregar o enviar la prenda a la sucursal.
Rechazada	La solicitud se cerró; no se admite otra para el mismo pedido.
Recibida en tienda / En revisión / Aceptada	La sucursal revisa la prenda.
Rechazada tras revisión	La prenda no cumplió las condiciones tras la inspección.
Devolución completada / Cambio completado	El proceso terminó.

Reembolsos. Cuando corresponde devolver dinero, el pedido y la solicitud muestran la sección "Tu reembolso" con el monto y lo que debe hacerse:

Forma de pago original	Mensaje mostrado
Tarjeta en línea	"Procesando el reembolso a tu tarjeta" y luego "Devuelto a tu tarjeta. Según tu banco puede tardar de 5 a 10 días hábiles en verse." Si falla: pasar por la sucursal para recibirlo en efectivo.
Efectivo en tienda	"Listo para recogerlo en efectivo en la sucursal: acércate a caja."
Tarjeta en tienda (POS)	"Se revierte en la tarjeta con la que pagaste en la sucursal."
PayPal	"Te lo devolveremos a tu cuenta de PayPal."

El monto devuelto corresponde a lo pagado por las prendas, con el descuento del cupón repartido proporcionalmente; el costo de envío no se devuelve. En un cambio a una variante más barata se devuelve la diferencia.

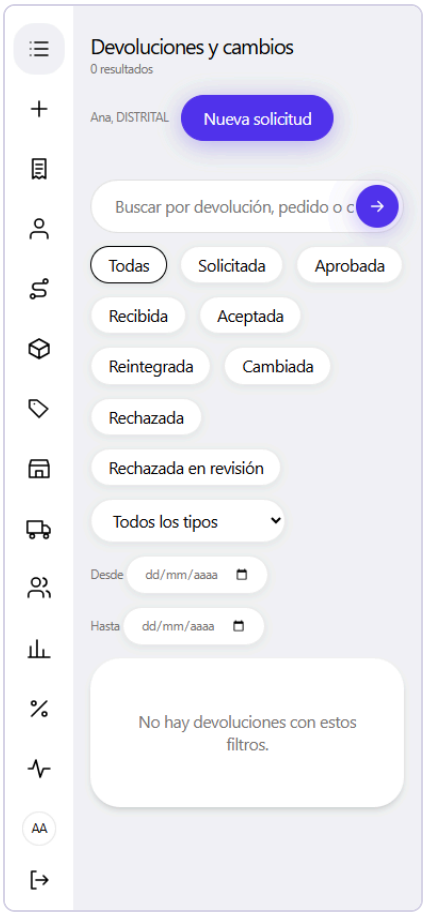
A11. Perfil y cambio de contraseña

En el menú de la cuenta (botón con el nombre de la clienta) se encuentran "Mi perfil", "Mis pedidos", "Devoluciones" y "Cerrar sesión".

- **Mi perfil:** permite modificar nombre, apellido, teléfono y "Dirección de entrega habitual" y guardar con "Guardar cambios". El correo no se modifica desde aquí.
- **Cambiar contraseña:** escribir "Contraseña actual", "Nueva" y "Repite la nueva" y presionar "Cambiar contraseña". La nueva debe ser distinta de la actual; al cambiarla se cierran las sesiones en otros dispositivos.

Parte B. Panel de empleados

El panel es la herramienta interna de las sucursales. A la izquierda hay una barra de íconos (cada ícono muestra su nombre al pasar el cursor); abajo están el avatar con las iniciales del empleado ("Mi cuenta") y el botón "Cerrar sesión". La pantalla inicial es la lista de "Devoluciones y cambios".



Panel en un teléfono: barra lateral de íconos y la lista "Devoluciones y cambios" con el botón "Nueva solicitud", el buscador, los filtros por estado, el tipo y las fechas

Figura B-1. Panel de empleados en un teléfono.

B1. Puestos y secciones visibles

El gerente distrital trabaja con todas las sucursales; los demás puestos solo con la sucursal que tienen asignada. El servidor valida siempre los permisos, aunque el menú ya oculta lo que no corresponde.

Sección del menú	Distrital	Administrador	Cajero	Bodeguero
Devoluciones (consultar)	Sí	Sí	Sí	Sí
Nueva solicitud (registrar)	Sí	Sí	Sí	No (el ícono aparece, pero no puede registrar)
Pedidos (consultar)	Sí	Sí	Sí	Sí
Nueva venta y cobros	Sí	Sí	Sí	No
Clientes	Sí	Sí	Sí	No

Sección del menú	Distrital	Administrador	Cajero	Bodeguero
Trazabilidad e Inventario	Sí	Sí	Sí	Sí
Catálogo y stock	Sí (edita)	Sí (consulta e ingresa stock)	No	Sí (consulta e ingresa stock)
Sucursales	Sí (todas, crea y edita)	Su sucursal	No	Su sucursal
Proveedores	Sí	Sí	No	Sí (sin eliminar)
Empleados	Sí (todos)	Cajeros y bodegueros de su sucursal	No	No
Reportes de ventas	Sí (todas)	Su sucursal	No	No
Promociones	Sí	No	No	No
Estado del sistema	Sí	No	No	No

En las tablas siguientes, "Administrador" corresponde al puesto ADMIN (gerente de sucursal).

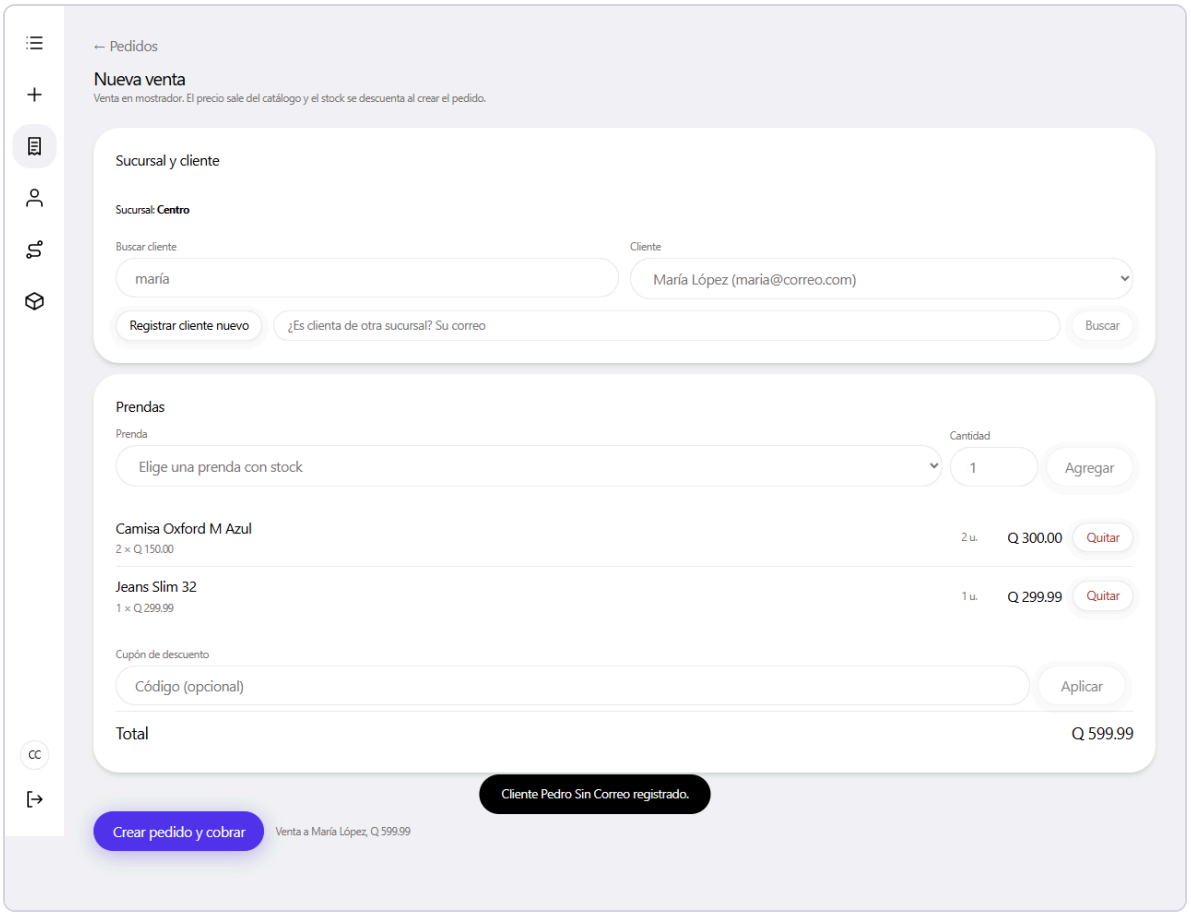
B2. Iniciar sesión y cambiar la contraseña

1. Abrir el panel, escribir "Correo" y "Contraseña" y presionar "Iniciar sesión".
2. Si no se tiene cuenta o se olvidó la contraseña, un administrador debe registrar al empleado o asignarle una contraseña temporal (ver B12).
3. Para cambiar la contraseña, presionar el avatar con las iniciales, abajo a la izquierda. En "Mi cuenta" escribir "Contraseña actual", "Nueva contraseña" y "Repite la nueva contraseña", y presionar "Cambiar contraseña". Las demás sesiones abiertas se cierran.

Si la sesión vence o deja de ser válida, el panel muestra "Tu sesión expiró o ya no es válida. Inicia sesión de nuevo."

B3. Registrar una venta de mostrador

Puestos: Distrital, Administrador, Cajero.



Pantalla "Nueva venta" con la sucursal Centro, el buscador de clientes, la clienta María López elegida, dos prendas agregadas, el campo de cupón y el total Q 599.99 con el botón "Crear pedido y cobrar"

Figura B-2. Nueva venta de mostrador.

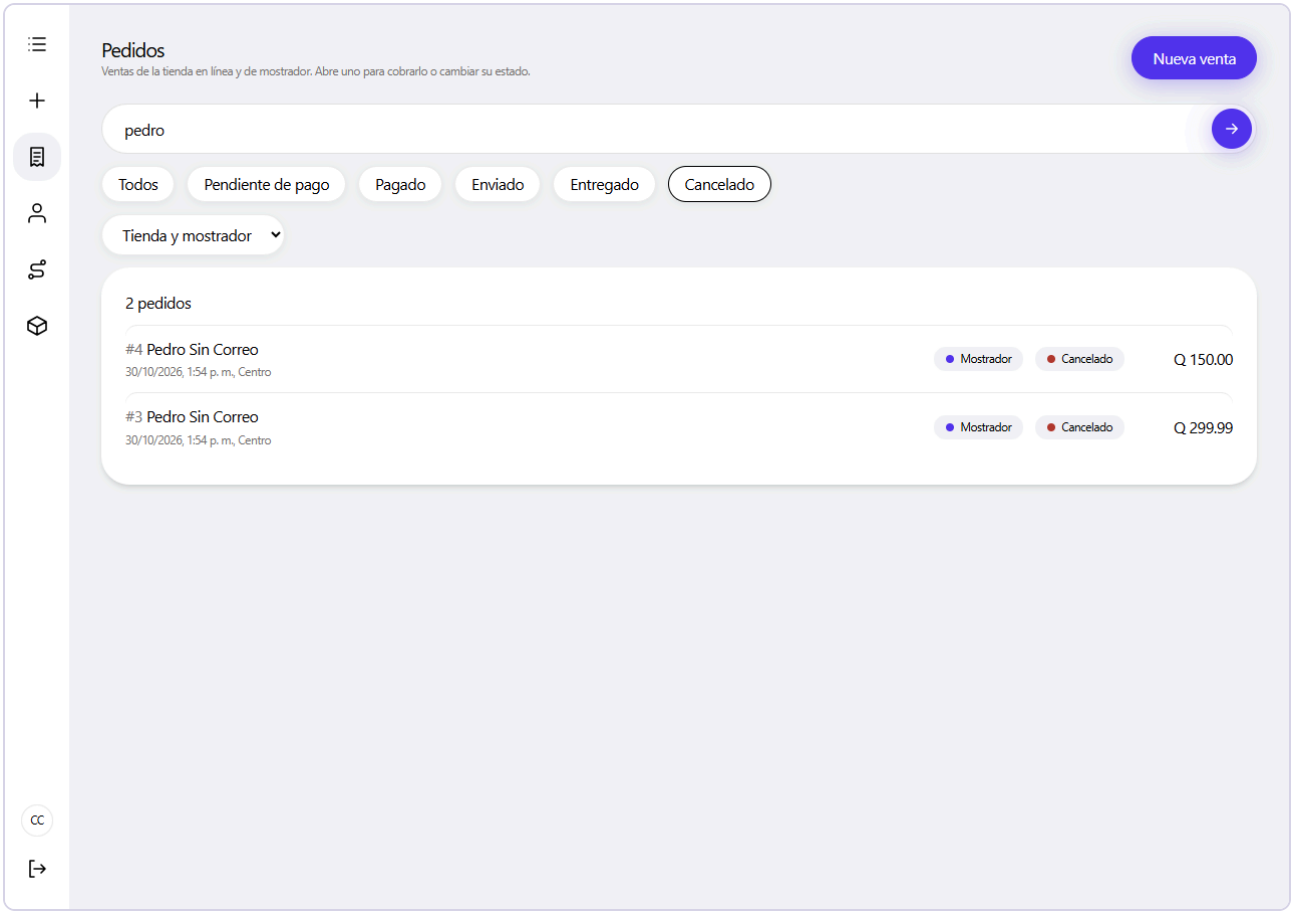
1. En "Pedidos", presionar "Nueva venta".
2. **Sucursal y cliente.** El cajero y el administrador venden en su sucursal (se muestra "Sucursal: [nombre]"); el distrital elige la sucursal. Escribir en "Buscar cliente" (nombre, correo o teléfono) y elegirlo en "Cliente".
    - Si el cliente no existe, presionar "Registrar cliente nuevo".
    - Si es clienta de otra sucursal, escribir su correo en "¿Es clienta de otra sucursal? Su correo" y presionar "Buscar".
3. **Prendas.** Elegir la prenda en "Prenda" (solo aparecen las que tienen stock en la sucursal, con su precio y unidades disponibles), escribir la "Cantidad" y presionar "Agregar". "Quitar" elimina una línea.
4. **Cupón (opcional).** Escribir el código en "Cupón de descuento" y presionar "Aplicar". Se muestran el subtotal y el descuento; si el cupón no aplica, aparece el motivo.
5. Presionar "Crear pedido y cobrar". El stock se descuenta al crear el pedido y se abre directamente el cobro (ver B4).

Si otra caja vendió las últimas unidades mientras se armaba la venta, la línea afectada indica "Ahora solo hay N disponibles" y se debe ajustar la cantidad.

B4. Cobrar y confirmar pagos

Puestos: Distrital, Administrador, Cajero.

La lista "Pedidos" permite buscar por número o cliente y filtrar por estado ("Pendiente de pago", "Pagado", "Enviado", "Entregado", "Cancelado"), por origen ("Tienda en línea" o "Mostrador") y, para el distrital, por sucursal.



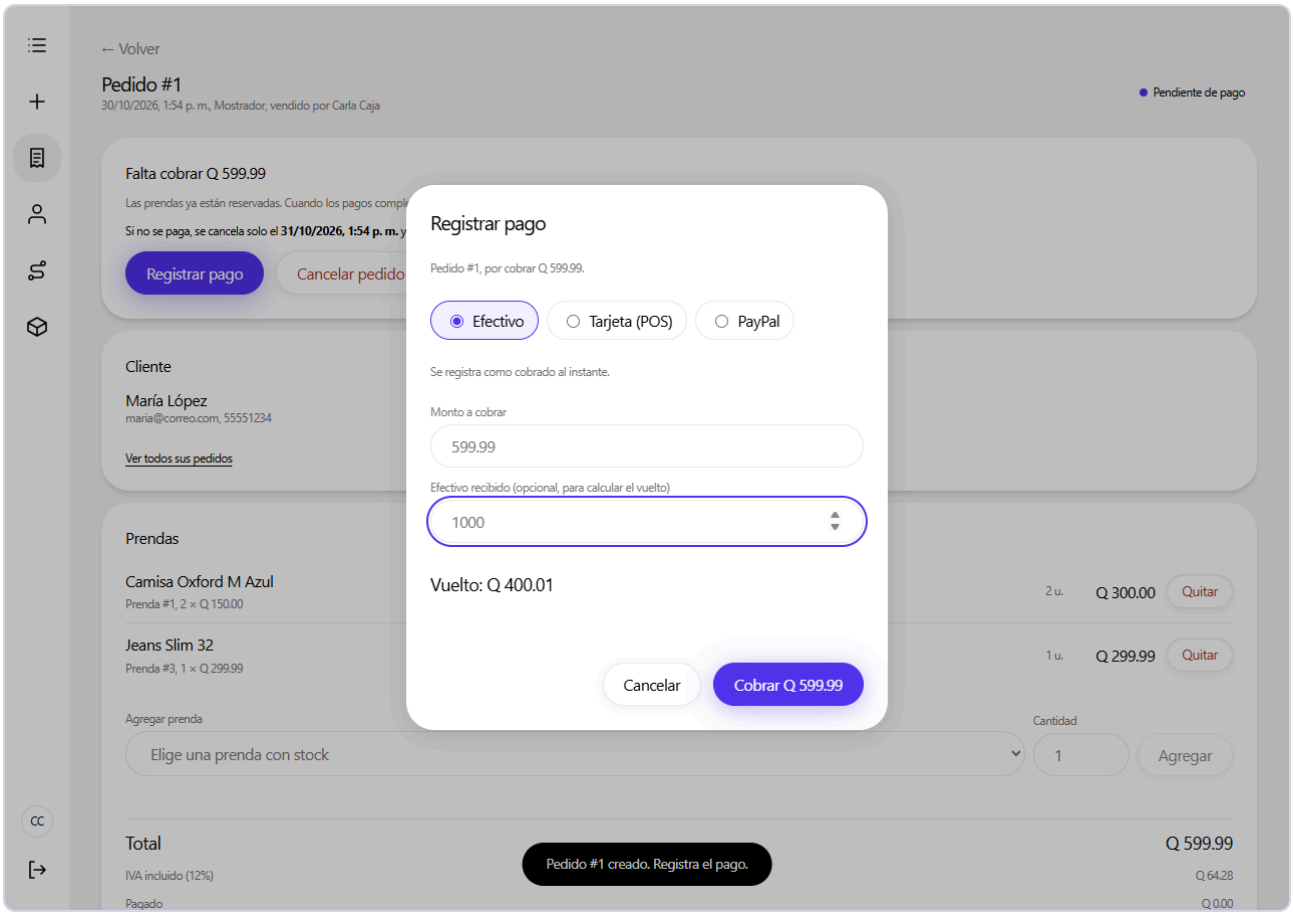
Lista de pedidos filtrada por la búsqueda "pedro" y el estado Cancelado, con dos pedidos de mostrador

Figura B-3. Lista de pedidos con filtros.

Para cobrar un pedido pendiente:

1. Abrir el pedido. La tarjeta superior indica "Falta cobrar Q..." y la fecha en que se cancelaría si no se paga.
2. Presionar "Registrar pago" y elegir el método:
    - **Efectivo:** se registra como cobrado al instante. El campo "Efectivo recibido" calcula el vuelto.
    - **Tarjeta (POS):** cobro en el terminal de la tienda; se registra como cobrado al instante. Se puede anotar el "Número de autorización del voucher".
    - **PayPal:** queda "Por confirmar" hasta verificar que el dinero llegó.

3. Revisar el "Monto a cobrar" (por defecto, el saldo completo; se admiten pagos parciales) y presionar "Cobrar Q...".



Diálogo "Registrar pago" con el método Efectivo, monto Q 599.99, efectivo recibido 1000 y el vuelto Q 400.01

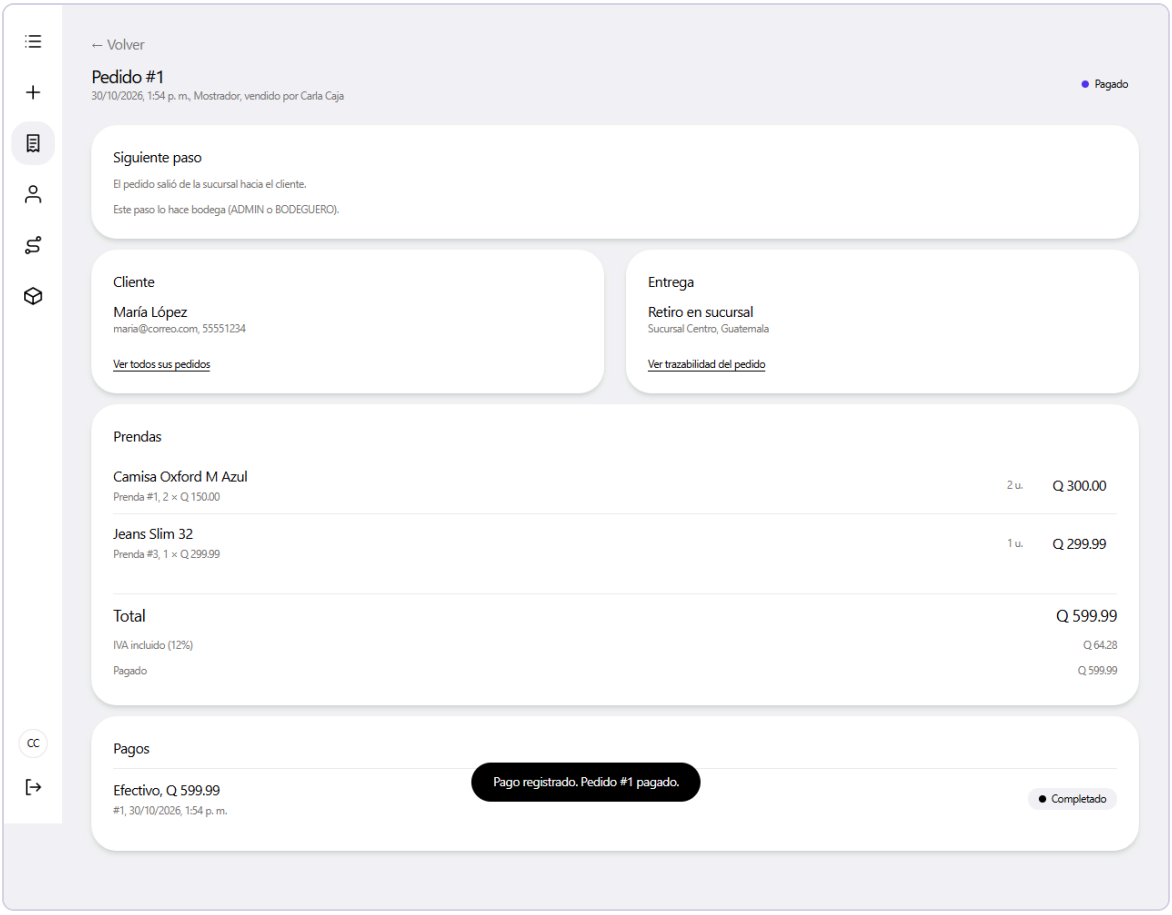
Figura B-4. Registro de un pago en efectivo con cálculo del vuelto.

Cuando los pagos completados cubren el total, el pedido pasa solo a "Pagado". En la sección "Pagos" del pedido:

Botón	Cuándo aparece	Efecto
"Confirmar"	Pago por confirmar (por ejemplo, PayPal)	Registra que el dinero se recibió.
"Anular"	Pago por confirmar	El monto vuelve a quedar por cobrar.
"Devolver"	Pago completado en efectivo o POS de un pedido aún pendiente	Registra que se devolvió el dinero ("Ya lo devolví").

Mientras el pedido esté pendiente, también pueden quitarse prendas ("Quitar") o agregarse ("Agregar prenda"); el total se recalcula solo. Para cancelar un pedido con pagos, primero deben devolverse o anularse esos pagos.

B5. Avanzar el estado de un pedido



Pedido #1 pagado con la tarjeta "Siguiente paso", los datos del cliente, la entrega "Retiro en sucursal", las prendas, el total y el pago en efectivo completado

Figura B-5. Detalle de un pedido pagado.

La tarjeta "Siguiente paso" muestra la acción que corresponde al estado actual:

Estado actual	Botón	Quién puede	Resultado
Pendiente de pago	"Registrar pago"	Distrital, Administrador, Cajero	Pasa solo a "Pagado" al cubrirse el total.
Pendiente de pago (sin pagos)	"Cancelar pedido"	Distrital, Administrador, Cajero	Las prendas vuelven al stock. No se puede deshacer.
Pagado	"Marcar como enviado"	Distrital, Administrador, Bodeguero	El pedido salió hacia el cliente.
Enviado	"Marcar como entregado"	Distrital, Administrador, Bodeguero	El cliente lo recibió; desde aquí puede pedir cambios o devoluciones.

Si el puesto no puede realizar el paso, el panel indica "Este paso lo hace bodega (ADMIN o BODEGUERO)". Cada acción pide confirmación antes de ejecutarse. El enlace "Ver trazabilidad del pedido" muestra los movimientos de inventario asociados.

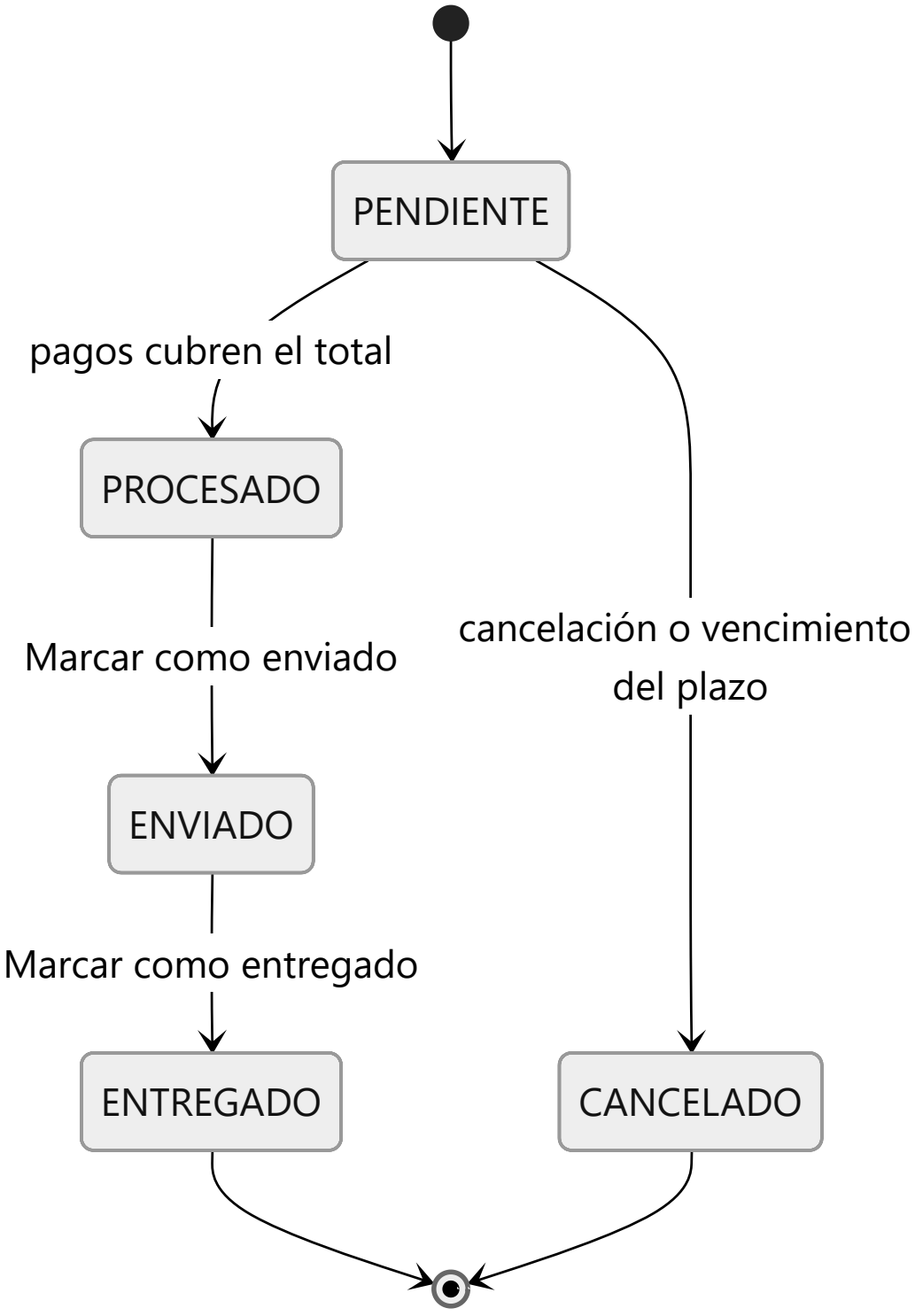


Figura B-6. Estados de un pedido (en pantalla: Pendiente de pago, Pagado, Enviado, Entregado y Cancelado).

B6. Procesar una devolución o un cambio

Las solicitudes llegan desde la tienda o se registran en el panel con "Nueva solicitud". La lista "Devoluciones y cambios" permite buscar por devolución, pedido o cliente y filtrar por estado, tipo y rango de fechas. Para cajeros y gerentes, la parte superior muestra además "Reembolsos por entregar" con el total pendiente.

Registrar una solicitud en mostrador (Distrital, Administrador, Cajero): en "Nueva solicitud" elegir el "Cliente" y el "Pedido entregado"; el panel indica el plazo restante. Marcar las prendas (se muestra lo comprado, lo ya solicitado y lo que queda por devolver), la cantidad y, si es cambio, la prenda nueva; elegir "Tipo" y "Motivo" y presionar "Registrar solicitud".

Procesar la solicitud paso a paso. Abrir la devolución desde la lista. La ficha muestra cliente, pedido, tipo, motivo, fechas y observación, y los botones del paso que corresponde al puesto. Si el paso lo realiza otro puesto, el panel lo indica ("Siguiente paso: ... Lo realiza ...").

Paso	Estado de partida	Botón	Quién puede	Qué hacer
1. Aprobar o rechazar	Solicitada	"Aprobar" / "Rechazar"	Distrital, Administrador	Aprobar permite recibir la prenda. Rechazar cierra la solicitud de forma definitiva.
2. Recibir	Aprobada	"Marcar como recibida"	Distrital, Administrador, Bodeguero	Confirmar que la prenda llegó a la sucursal.
3. Revisar	Recibida	"Registrar inspección"	Distrital, Administrador, Bodeguero	Indicar por prenda "Buen estado", "Dañado" o "Defectuoso" y elegir "Aceptar devolución" o "Rechazar".
4a. Reintegrar	Aceptada (devolución)	"Reintegrar a inventario"	Distrital, Administrador, Bodeguero	Cada prenda vuelve al inventario como disponible, dañada o defectuosa según la inspección.
4b. Cambiar	Aceptada (cambio)	"Procesar cambio"	Distrital, Administrador, Bodeguero	Se descuenta la prenda nueva y se reingresa la devuelta.
5. Entregar reembolso	Reintegrada o Cambiada	"Ya lo entregué" / "Entregar en efectivo"	Distrital, Administrador, Cajero	Entregar el dinero a la clienta y confirmarlo.

Los pasos de aprobación, recepción e inspección admiten una observación opcional. Las secciones "Decisiones" y "Movimientos de inventario" registran quién hizo cada paso y cuándo.

Entregar el reembolso en caja. Al completarse una devolución (o un cambio a una variante más barata), la ficha muestra "Reembolso a la clienta: Q...":

- Los reembolsos a la tarjeta en línea los procesa Stripe automáticamente. Si quedan pendientes, el botón "Reintentar en Stripe" consulta de nuevo; si Stripe los rechaza, aparece "Entregar en efectivo".

- Los reembolsos en efectivo, reversión en POS o PayPal quedan "Pendiente". Después de entregar el dinero, presionar "Ya lo entregué", anotar opcionalmente una referencia y confirmar con "Sí, ya lo entregué". Queda registrado con el nombre del empleado.

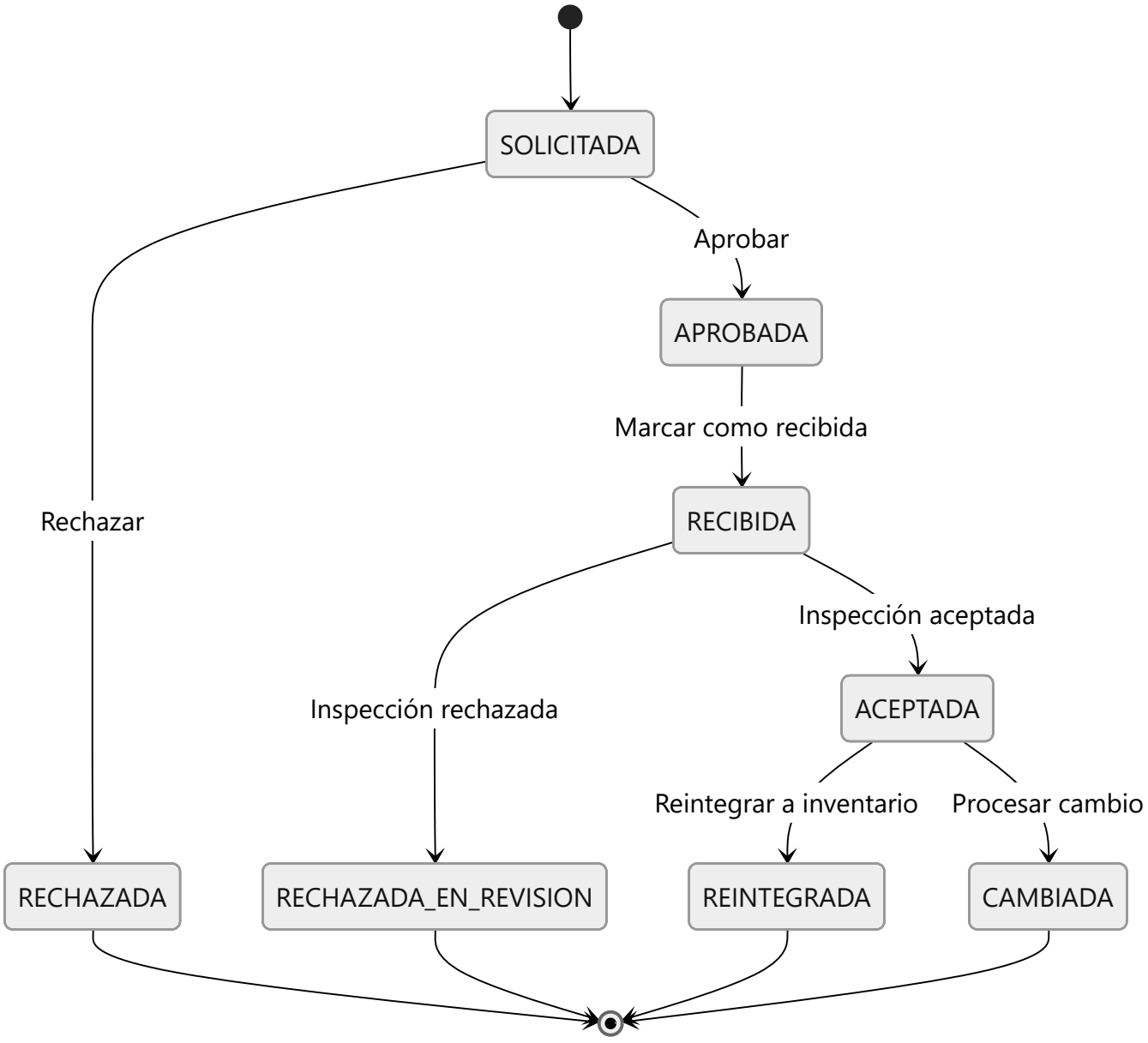


Figura B-7. Recorrido de una solicitud de cambio o devolución en el panel.

B7. Clientes y código web

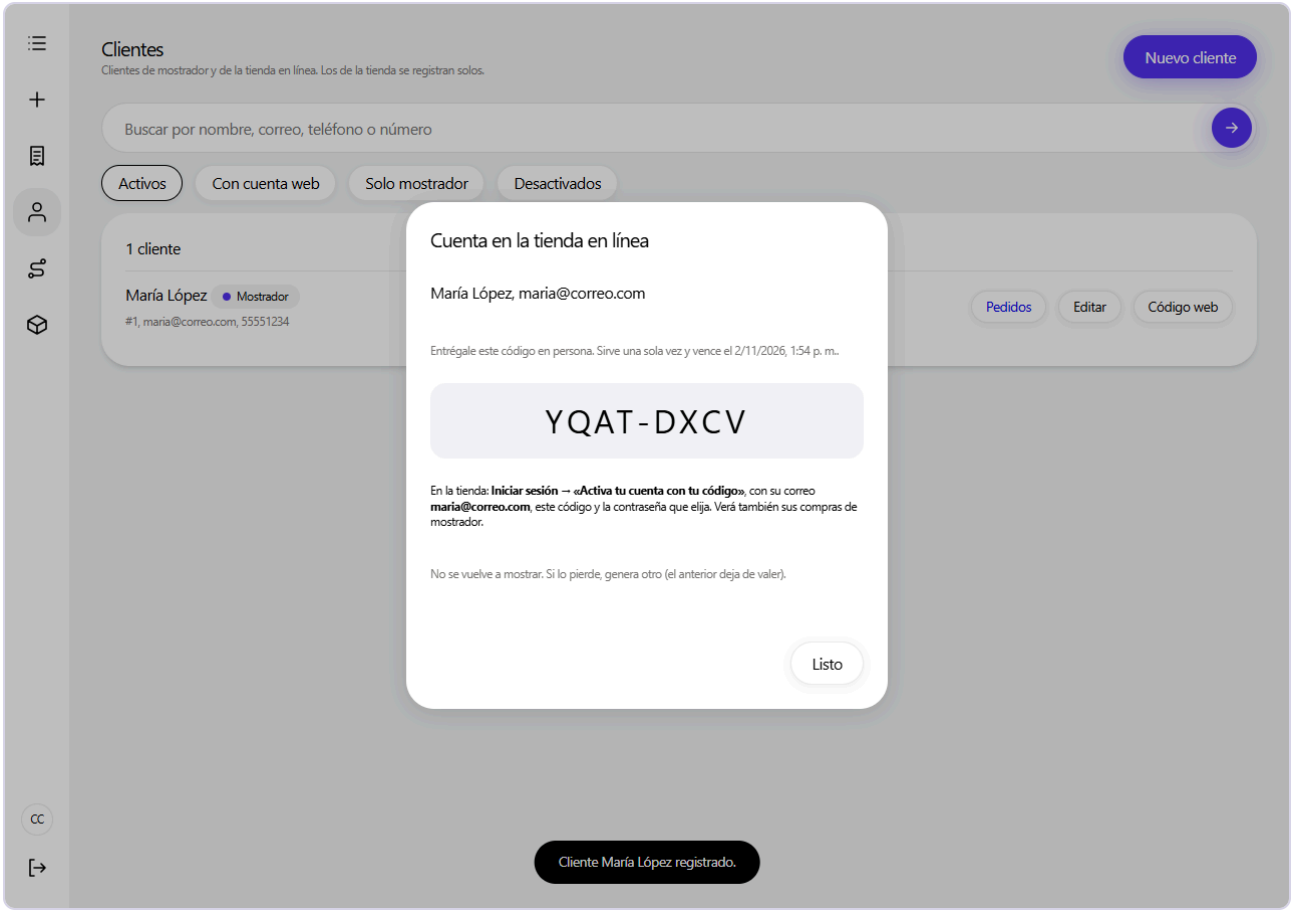
Puestos: Distrital, Administrador, Cajero.

La sección "Clientes" lista los clientes de mostrador y de la tienda en línea, con los filtros "Activos", "Con cuenta web", "Solo mostrador" y "Desactivados", y búsqueda por nombre, correo, teléfono o número. Cada cliente tiene la etiqueta "Cuenta web" o "Mostrador" y los botones "Pedidos", "Editar" y, si aún no tiene cuenta web, "Código web".

- **Nuevo cliente:** nombre y apellido son obligatorios; correo, teléfono y dirección son opcionales. El distrital elige además la "Sucursal donde se registra". Con correo, el cliente podrá activar después su cuenta en línea.
- **Editar:** si el cliente ya tiene cuenta web, su correo no se puede cambiar desde el panel. Solo el distrital y el administrador pueden desactivar o reactivar un cliente ("Cliente activo"); un cliente desactivado no puede comprar ni entrar a la tienda.

Generar el código web:

1. Presionar "Código web" en la fila del cliente. El cliente debe tener correo.
2. Presionar "Generar código". Se muestra un código de un solo uso, válido 72 horas.
3. Entregarlo en persona e indicar el camino en la tienda: "Iniciar sesión", luego "Activa tu cuenta con tu código" (ver A2).



Diálogo "Cuenta en la tienda en línea" con el código de activación YQAT-DXCV, su fecha de vencimiento y las instrucciones para la clienta

Figura B-8. Código de activación generado para una clienta de mostrador.

El código no se vuelve a mostrar. Si se pierde, se genera otro y el anterior deja de valer.

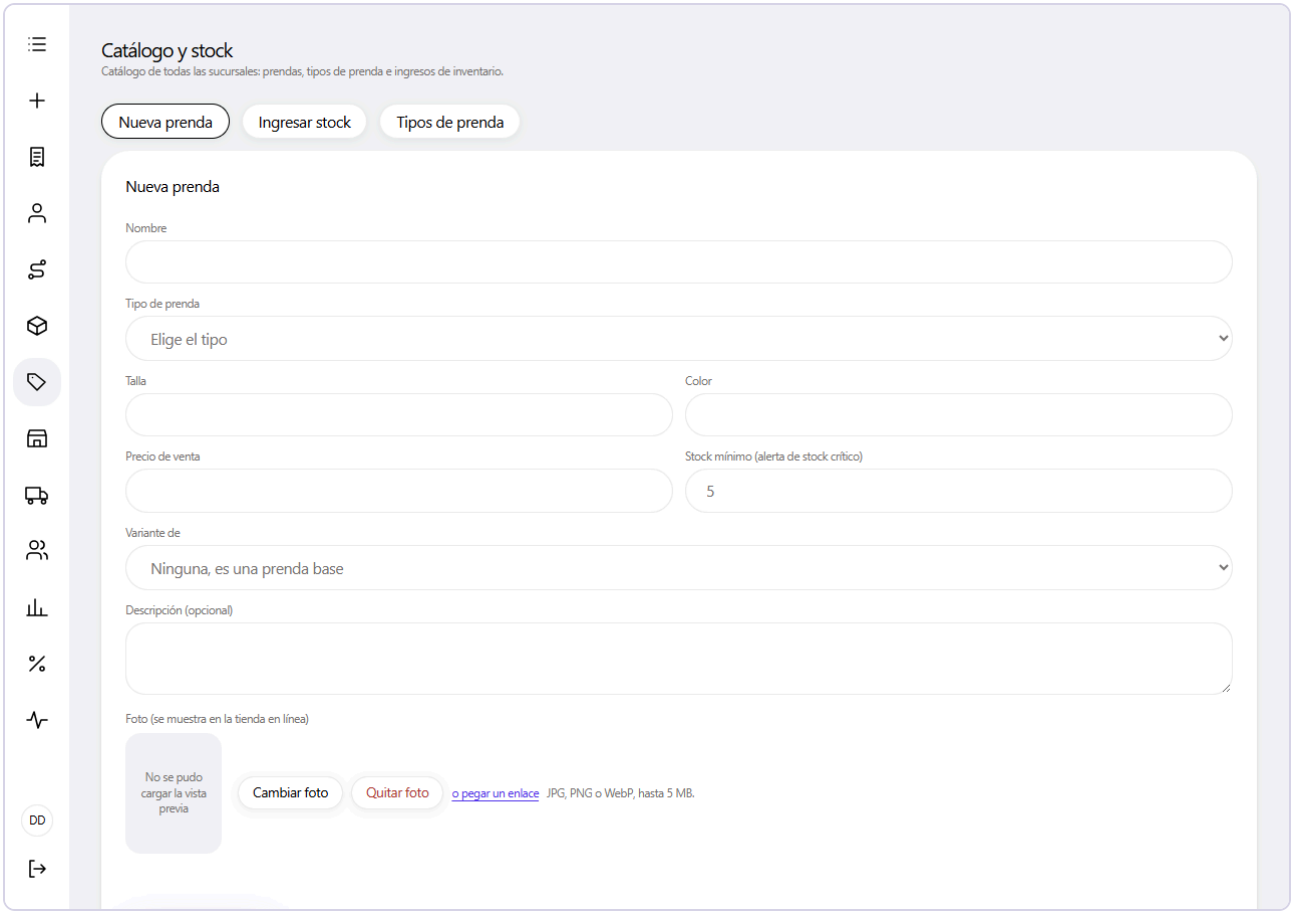
B8. Catálogo, tipos de prenda y fotos

Puestos: el Distrital crea y edita; Administrador y Bodeguero consultan e ingresan stock.

La sección "Catálogo y stock" tiene tres pestañas. El catálogo es único para todas las sucursales.

Nueva prenda (Distrital):

1. Completar "Nombre", "Tipo de prenda", "Talla", "Color", "Precio de venta" y "Stock mínimo (alerta de stock crítico)" (5 por defecto).
2. En "Variante de", elegir la prenda base si se registra una variante; dejar "Ninguna, es una prenda base" en caso contrario.
3. Agregar una "Descripción (opcional)" y la foto.
4. Presionar "Registrar prenda".



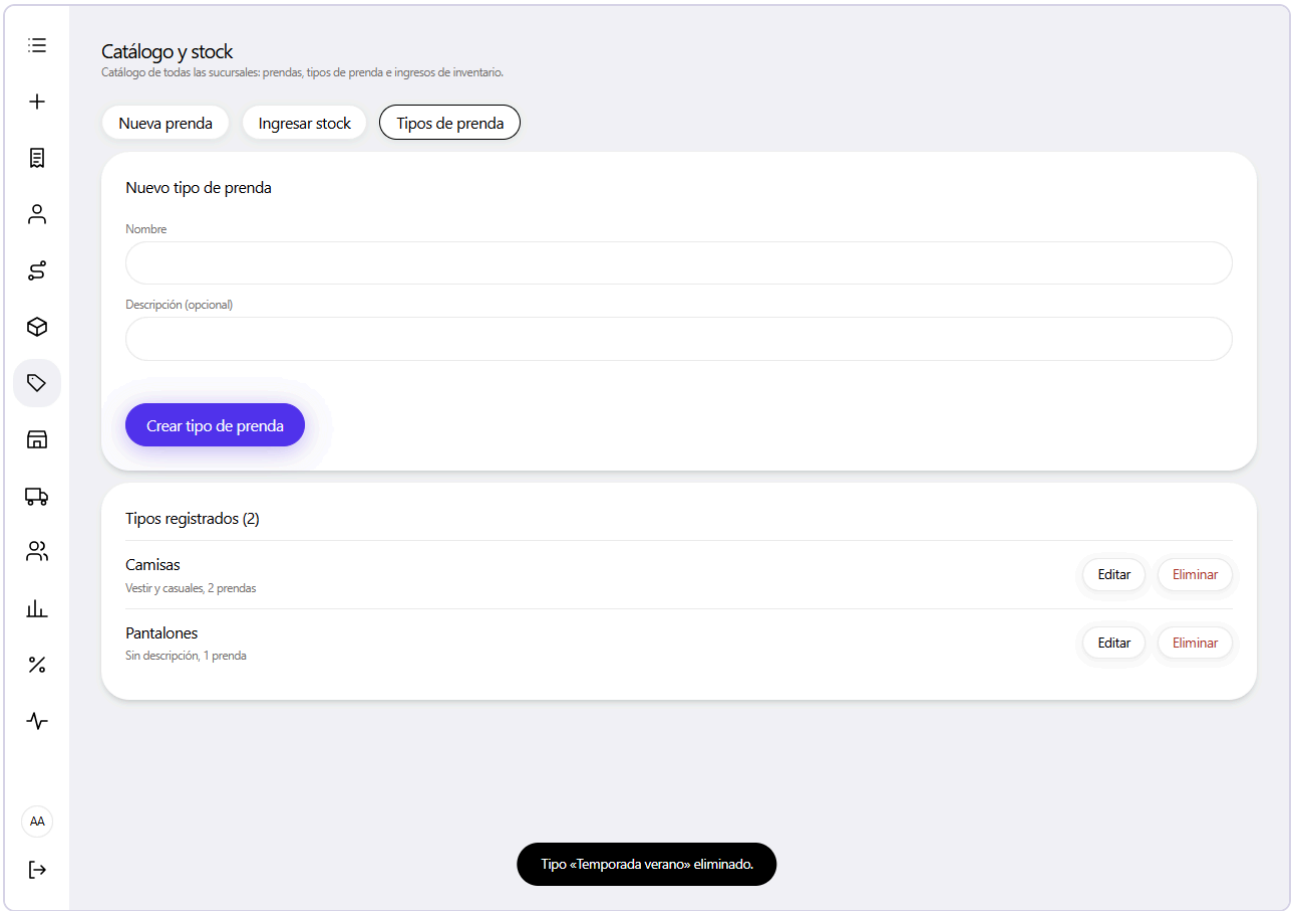
Pestaña "Nueva prenda" con los campos del formulario y la sección de foto con los botones "Cambiar foto", "Quitar foto" y "o pegar un enlace"

Figura B-9. Formulario de nueva prenda con la sección de foto.

Foto de la prenda. En "Foto (se muestra en la tienda en línea)", presionar "Subir foto" y elegir un archivo JPG, PNG o WebP de hasta 5 MB (límite indicado en pantalla). También puede usarse "o pegar un enlace". Los botones "Cambiar foto" y "Quitar foto" reemplazan o eliminan la imagen. Una variante sin foto propia muestra en la tienda la foto de su prenda base.

Prendas registradas. La lista muestra número, tipo, si es variante y si está inactiva. Con "Editar" se modifican los datos y se puede desactivar la prenda ("Prenda activa"). "Eliminar" solo funciona si la prenda nunca tuvo variantes, inventario, ventas ni devoluciones; si tiene historial, debe desactivarse.

Tipos de prenda (Distrital): escribir "Nombre" y "Descripción (opcional)" y presionar "Crear tipo de prenda". Un tipo solo se puede eliminar si ninguna prenda lo usa.



Pestaña "Tipos de prenda" con el formulario "Nuevo tipo de prenda" y la lista de tipos registrados (Camisas y Pantalones) con sus botones Editar y Eliminar

Figura B-10. Tipos de prenda.

Ingresar stock (Distrital, Administrador, Bodeguero): elegir la "Prenda", la "Sucursal" (solo el distrital; los demás ingresan en la suya), la "Cantidad", el "Lote (opcional)" y el "Estado" (Disponible, Dañada o Defectuosa), y presionar "Ingresar al inventario". Solo las unidades disponibles se venden.

B9. Inventario y trazabilidad

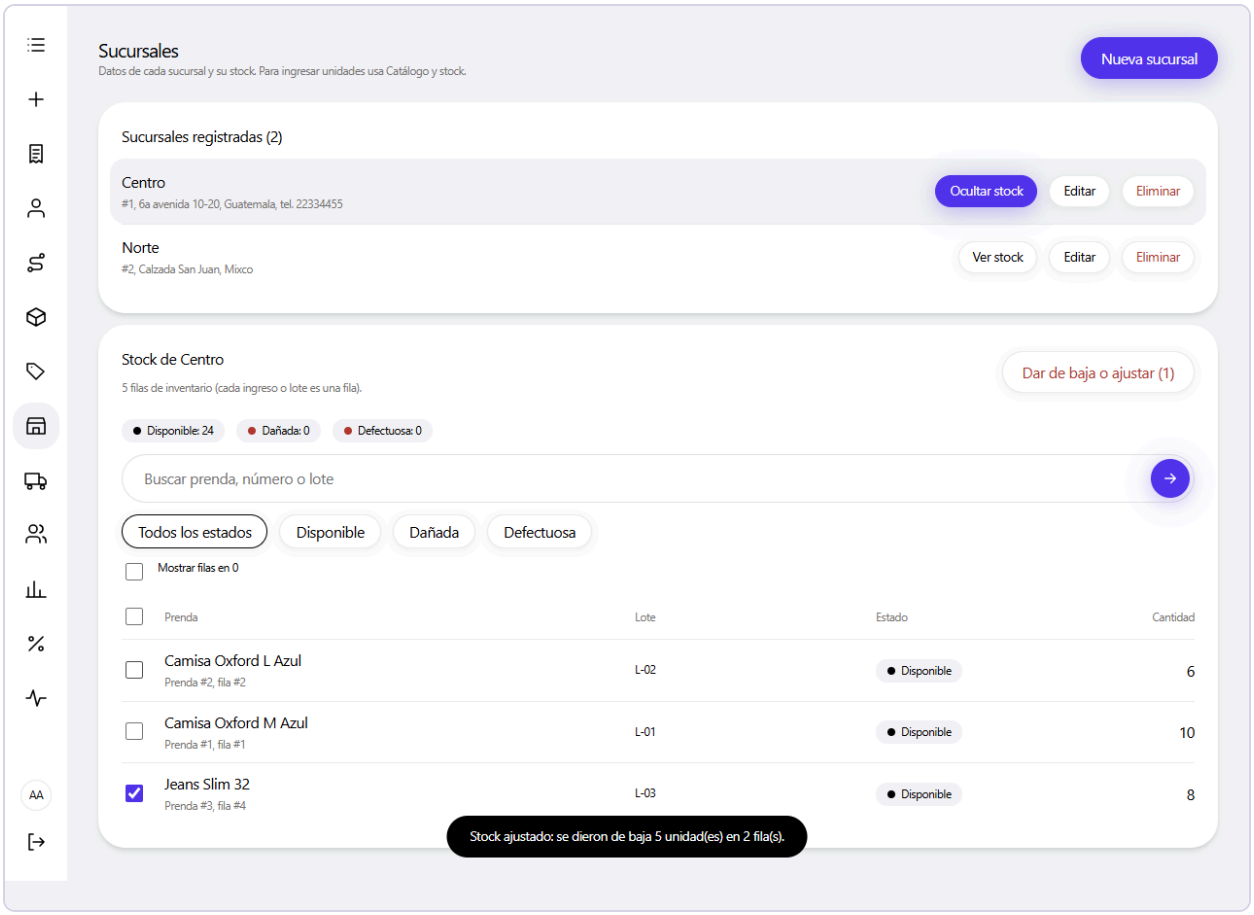
Puestos: todos.

- **Inventario** ("Inventario por estado"): muestra, por prenda y sucursal, las unidades disponibles, dañadas y defectuosas, con totales arriba. Permite buscar por nombre, talla, color o número y filtrar por estado y sucursal.
- **Trazabilidad:** historial de movimientos de inventario. Con "Por pedido" se escribe el número de pedido; con "Por producto" se elige la prenda. Cada movimiento indica el tipo, la prenda, la cantidad, el empleado, la fecha y la observación. Los ajustes de stock, ventas, devoluciones y cambios quedan registrados aquí.

B10. Sucursales y ajustes de stock

Puestos: el Distrital ve todas y puede crearlas, editarlas y eliminarlas; Administrador y Bodeguero ven "Mi sucursal" y ajustan su stock.

- **Nueva sucursal / Editar:** nombre, dirección, ciudad y teléfono opcional. Al desmarcar "Sucursal activa", deja de vender en la tienda en línea y no aparece para ingresar stock, pero conserva su historial.
- **Eliminar:** solo si la sucursal nunca tuvo empleados, inventario ni pedidos.
- **Ver stock:** muestra cada fila de inventario (cada ingreso o lote es una fila) con prenda, lote, estado y cantidad. Se puede buscar, filtrar por estado y activar "Mostrar filas en 0".



Sucursales Centro y Norte; el stock de Centro con filas por lote, una fila seleccionada y el botón "Dar de baja o ajustar (1)"

Figura B-11. Stock de una sucursal y selección de filas para ajustar.

Dar de baja o ajustar stock:

1. Marcar una o varias filas y presionar "Dar de baja o ajustar (N)".
2. En cada fila, el campo "Quedan" propone 0 (dar de baja todo); puede escribirse otra cantidad menor a la actual.
3. Elegir el "Motivo": "Corrección por conteo físico", "Pérdida o extravío", "Robo", "Daño en bodega", "Descarte de prenda dañada o defectuosa" u "Otro motivo (explícalo)". Con "Otro motivo" la observación es obligatoria.
4. Presionar "Confirmar baja". La baja queda registrada en Trazabilidad con el nombre del empleado.

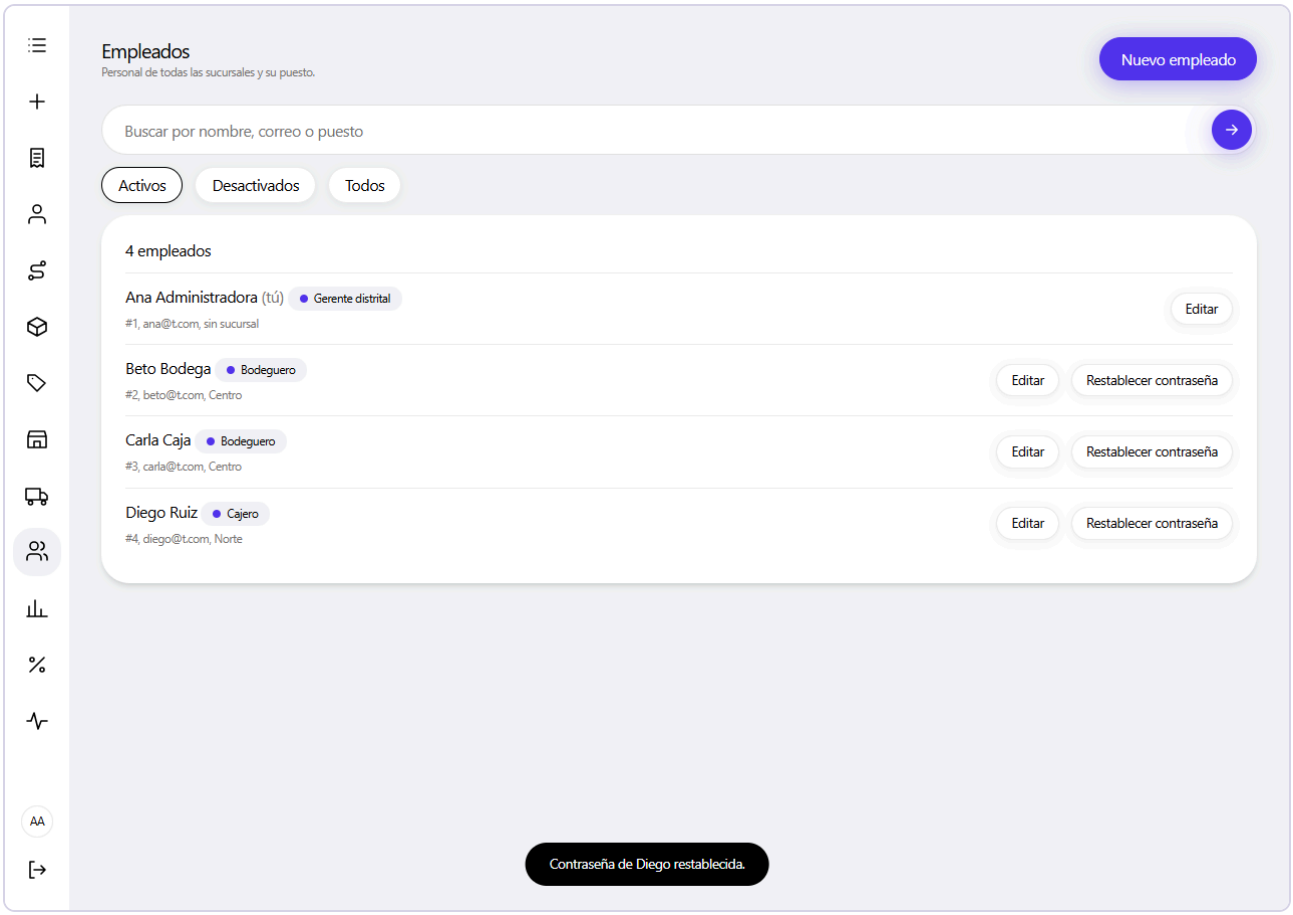
B11. Proveedores

Puestos: Distrital, Administrador, Bodeguero (eliminar: solo Distrital y Administrador).

La sección lista las empresas que surten las prendas, con filtros "Activos", "Desactivados" y "Todos" y búsqueda por empresa, contacto, correo o teléfono. "Nuevo proveedor" pide "Empresa" y, opcionalmente, contacto, teléfono, correo y dirección. Con "Editar" se modifican los datos o se desmarca "Proveedor activo". "Eliminar" borra el proveedor definitivamente; si solo se dejará de trabajar con él, se recomienda desactivarlo.

B12. Empleados

Puestos: Distrital (todo el personal) y Administrador (cajeros y bodegueros de su sucursal).



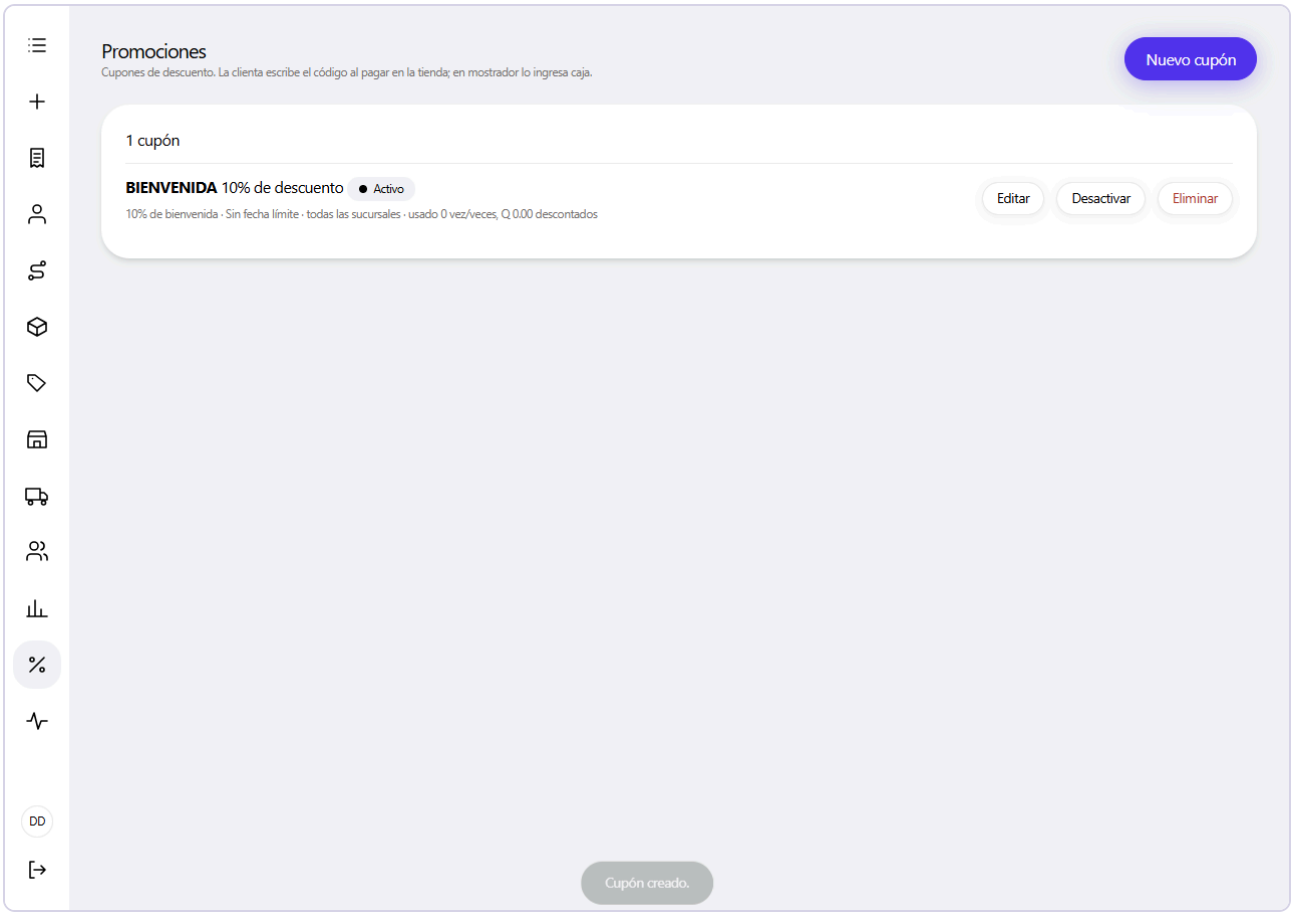
Lista de empleados activos con su puesto, correo y sucursal, y los botones "Editar" y "Restablecer contraseña"

Figura B-12. Gestión de empleados.

- **Nuevo empleado:** nombre, apellido, "Correo (con él inicia sesión)", "Puesto", sucursal (el distrital la elige; el puesto Distrital no tiene sucursal) y "Contraseña inicial" de 8 caracteres como mínimo. La contraseña se entrega en persona.
- **Editar:** permite cambiar los datos, el puesto y la sucursal, y desmarcar "Cuenta activa". Una cuenta desactivada no puede iniciar sesión y su sesión abierta deja de funcionar. Nadie puede cambiar su propio puesto ni desactivarse.
- **Restablecer contraseña / Dar acceso:** asigna una "Contraseña temporal". El empleado debe cambiarla al entrar desde su avatar. Si tenía una sesión abierta, se cierra.

B13. Promociones y cupones

Puesto: Distrital.



Sección "Promociones" con el cupón BIENVENIDA de 10 % activo, su vigencia, alcance y uso, y los botones Editar, Desactivar y Eliminar

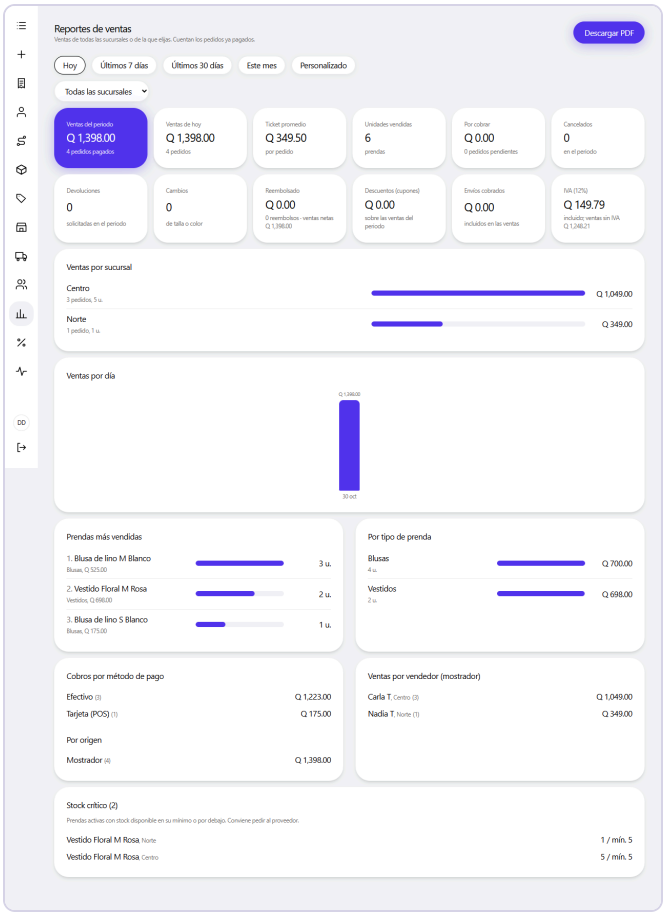
Figura B-13. Lista de cupones.

1. Presionar "Nuevo cupón".
2. Completar "Código (lo que escribe la clienta)" (de 3 a 30 letras, números, guiones o guion bajo), "Descripción", "Tipo" ("Porcentaje (%)" o "Monto fijo (Q)") y su valor.
3. Opcionalmente, definir "Compra mínima (Q)", "Usos máximos (vacío = ilimitado)", "Válido desde", "Válido hasta" y "Sucursal" (o "Todas las sucursales").
4. Presionar "Guardar".

Cada cupón muestra su vigencia, alcance, número de usos y el total descontado. "Desactivar" y "Activar" lo suspenden o reanudan. Un cupón ya usado no permite cambiar su código ni su tipo, y solo se puede eliminar si nunca se usó.

B14. Reportes de ventas y PDF

Puestos: Distrital (todas las sucursales o una) y Administrador (su sucursal).



Reporte de ventas con indicadores (ventas del periodo, ticket promedio, unidades, por cobrar, devoluciones, IVA), ventas por sucursal, ventas por día, prendas más vendidas, ventas por tipo, cobros por método, ventas por vendedor y stock crítico

Figura B-14. Reportes de ventas.

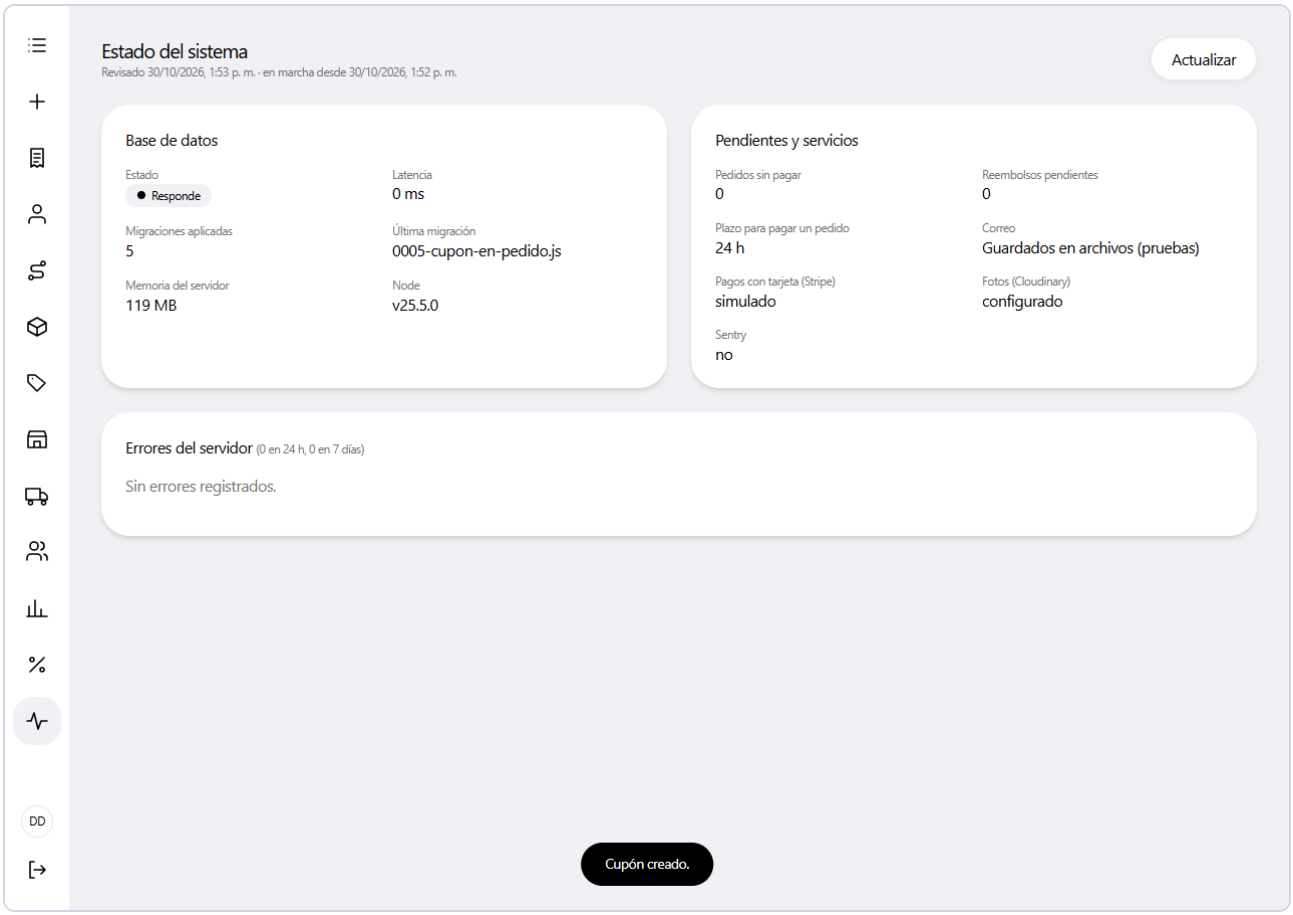
1. Elegir el periodo: "Hoy", "Últimos 7 días", "Últimos 30 días", "Este mes" o "Personalizado" (con fechas "Desde" y "Hasta").
2. El distrital puede elegir una sucursal o dejar "Todas las sucursales".
3. Revisar los indicadores y las tablas. Las ventas cuentan solo los pedidos ya pagados; los indicadores "Por cobrar" y "Cancelados" muestran aparte los pedidos pendientes y cancelados.
4. Presionar "Descargar PDF" para obtener el reporte con encabezado (sucursal, periodo y quién lo generó), indicadores y todas las tablas, incluidas todas las prendas vendidas.

Bloque	Contenido
Indicadores	Ventas del periodo, ventas de hoy, ticket promedio, unidades vendidas, por cobrar, cancelados, devoluciones, cambios, reembolsado y ventas netas, descuentos por cupones, envíos cobrados e IVA.
Ventas por sucursal	Solo para el distrital sin sucursal elegida.
Ventas por día	Gráfica de columnas del periodo.
Prendas más vendidas y por tipo de prenda	Las diez prendas con más unidades y las ventas por tipo.

Bloque	Contenido
Cobros por método de pago, por origen y con cupón	Efectivo, tarjeta POS, PayPal, tarjeta en línea; tienda en línea frente a mostrador.
Ventas por vendedor (mostrador)	Ventas registradas por cada empleado.
Stock crítico	Prendas activas con stock disponible en su mínimo o por debajo.

B15. Estado del sistema

Puesto: Distrital.



Pantalla "Estado del sistema" con la base de datos respondiendo, migraciones aplicadas, memoria, pendientes, plazo de pago de 24 h, estado del correo, Stripe, Cloudinary y Sentry, y la sección de errores del servidor sin errores

Figura B-15. Estado del sistema.

La pantalla permite verificar que el sistema funciona correctamente. El botón "Actualizar" vuelve a consultar la información.

- **Base de datos:** si responde, la latencia, las migraciones aplicadas y la última, la memoria del servidor y la versión de Node.
- **Pendientes y servicios:** pedidos sin pagar, reembolsos pendientes, plazo para pagar un pedido y estado del correo, de los pagos con tarjeta (Stripe), de las fotos (Cloudinary) y de Sentry.
- **Avisos:** en rojo se muestran problemas como "La base de datos no responde", migraciones pendientes, ausencia de SMTP ("las clientas no reciben correos") o reembolsos que Stripe rechazó y esperan entrega en efectivo.
- **Errores del servidor:** lista paginada con código, ruta, mensaje, fecha, usuario y referencia, y el conteo de las últimas 24 horas y 7 días.

Preguntas frecuentes

¿Por qué una prenda aparece como "Agotado aquí" si sé que hay en otra tienda? La tienda muestra el stock de la sucursal elegida en "Enviar desde". En la ficha de la prenda, la línea "Disponible en:" indica las otras sucursales con existencias; basta con cambiar la sucursal en el selector.

¿Puedo comprar prendas de dos sucursales en el mismo pedido? No. Cada pedido se despacha desde una sola sucursal. Al cambiar la sucursal, el carrito se revisa contra las existencias de la nueva.

¿Cuánto cuesta el envío? Con la configuración por defecto, Q 25.00, y es gratis cuando las prendas suman Q 300.00 o más después del descuento. El retiro en sucursal no tiene costo.

¿Qué pasa si no pago mi pedido? Las prendas quedan reservadas durante 24 horas. Si no se paga en ese plazo, el pedido se cancela solo y las prendas vuelven a la venta.

El cupón no se aplica. ¿Por qué? La pantalla muestra el motivo: puede estar inactivo, fuera de vigencia, limitado a otra sucursal, haber alcanzado sus usos máximos o requerir una compra mínima.

¿Puedo pedir un segundo cambio del mismo pedido? No. Se permite una sola solicitud por pedido, dentro de las 24 horas siguientes a la entrega, y una solicitud rechazada también cuenta. Por eso deben incluirse todas las prendas en la misma solicitud.

¿Por qué no puedo cambiar una prenda por una talla más cara? Los cambios solo admiten variantes de igual o menor precio, porque la diferencia no puede cobrarse en línea. Si la variante nueva es más barata, se reembolsa la diferencia.

¿Cuándo veo el reembolso en mi tarjeta? Una vez completado en el sistema, el banco puede tardar de 5 a 10 días hábiles en reflejarlo. Si el reembolso a la tarjeta falla, se entrega en efectivo en la sucursal.

Compré en tienda y no puedo crear la cuenta con mi correo. Ese correo ya existe como cliente de mostrador. Debe pedirse en caja el código de activación y usar "Activa tu cuenta con tu código" (sección A2).

El sistema dice "Demasiados intentos". Para proteger las cuentas, el sistema limita los intentos de inicio de sesión, registro y recuperación de contraseña. Debe esperarse el tiempo indicado en el mensaje antes de volver a intentar.

Soy empleado y olvidé mi contraseña. El panel no envía enlaces de recuperación. Un administrador de la sucursal o el gerente distrital debe asignar una contraseña temporal desde "Empleados", con "Restablecer contraseña".

No veo una sección del panel que usa un compañero. El menú depende del puesto (ver B1). Si el puesto es incorrecto, debe pedirse el cambio a un superior; el menú se actualiza al volver a abrir el panel.

¿Por qué no puedo eliminar una prenda, una sucursal o un cupón? El sistema conserva el historial. Las prendas y sucursales con movimientos, y los cupones ya usados, no se eliminan: se desactivan.

¿Cómo pruebo un pago sin usar una tarjeta real? En el entorno de pruebas se usa la tarjeta 4242 4242 4242 4242 con cualquier fecha futura y cualquier código de seguridad. El panel "Estado del sistema" indica si los pagos con tarjeta funcionan en modo simulado.

Documentos relacionados: Roles y permisos · Reglas de negocio · Seguridad

Seguridad

Este documento describe las medidas de seguridad implementadas en el sistema Distribuidora (API REST, panel de empleados y tienda en línea) e indica, para cada una, el archivo del código en el que se aplica. Su propósito es dejar constancia de qué riesgos se consideraron, cómo se mitigan y qué limitaciones conocidas permanecen. Todo lo expuesto se obtuvo de la lectura de `server.js`, `app/middlewares/`, `app/config/`, `app/controllers/`, `app/utils/`, `app/migraciones/` y de los clientes `frontend/` y `tienda/`. El modelo de roles se detalla en Roles y permisos y la arquitectura general en Arquitectura.

Contenido

1. Principios generales
2. Contraseñas
3. Sesiones con JWT
4. Autorización y alcance por sucursal
5. Validación de entradas y consultas a la base de datos
6. Límite de intentos
7. Cuentas de clientas: inicio de sesión, activación y restablecimiento
8. CORS y cuerpo de las peticiones
9. Pagos con Stripe
10. Fotos en Cloudinary
11. Concurrencia: transacciones y bloqueos
12. Idempotencia de reembolsos
13. Correos electrónicos
14. Manejo y registro de errores
15. Auditoría
16. Secretos y configuración
17. Dependencias con avisos de seguridad pendientes
18. Limitaciones conocidas
19. Resumen: riesgo, medida y archivo
20. Verificación en las pruebas automatizadas

---

1. Principios generales

- **El servidor es la autoridad.** El panel y la tienda ocultan opciones según el rol o el estado, pero cada endpoint aplica sus propios controles de autenticación, rol, alcance y reglas de negocio. Los montos a cobrar, los precios y los totales se calculan en el servidor a partir del catálogo; nunca se toman del cliente.

- **Estado leído de la base en cada petición.** La existencia de la cuenta, si está activa, su puesto y su versión de sesión se consultan en la base de datos en cada petición autenticada; el contenido del token no se considera suficiente.
- **Respuestas que no revelan información.** Los registros fuera de alcance responden igual que los inexistentes (`404`), y los flujos de inicio de sesión y restablecimiento de contraseña responden de forma uniforme para no revelar qué correos están registrados.
- **Secretos solo en el servidor.** Las claves de Stripe (secreta y de webhook), de Cloudinary, de JWT, de SMTP y de la base de datos solo existen en las variables de entorno del backend.
- **Operaciones atómicas.** Las operaciones que mueven inventario o dinero se ejecutan en transacciones con bloqueo de filas, y la auditoría se registra dentro de la misma transacción.

2. Contraseñas

Las contraseñas se almacenan únicamente como hash bcrypt, mediante la biblioteca `bcryptjs` .

Aspecto	Empleados	Clientas de la tienda
Validación	`validarPassword` en `app/utils/empleados.js`	`errorPassword` en `app/controllers/tienda_auth.controller.js`
Longitud mínima	8 caracteres	8 caracteres
Longitud máxima	72 bytes en UTF-8	72 bytes en UTF-8
Costo de bcrypt	10 (`COSTO_BCRYPT`)	8
Campo	`Empleado.password`	`Cliente.password`

Límite de 72 bytes. bcrypt solo considera los primeros 72 bytes de la contraseña. Si se admitieran contraseñas más largas, cualquier texto con el mismo prefijo de 72 bytes sería aceptado. Por ese motivo ambas validaciones miden la longitud con `Buffer.byteLength(..., "utf8")` y rechazan con `400` lo que excede ese límite.

Contraseña distinta de la actual. Al cambiar la propia contraseña (`PUT /api/auth/password` y `PUT /api/tienda/perfil/password`) se exige la contraseña actual y que la nueva sea distinta.

Exclusión del hash en las respuestas. Las funciones `publico` (`app/utils/empleados.js`) y `perfilDe` (`app/controllers/tienda_auth.controller.js`) eliminan `password` y `token_version` antes de responder. En el modelo `Cliente` (`app/models/cliente.model.js`) el ámbito por defecto (`defaultScope`) excluye `password` , `codigo_activacion` , `codigo_activacion_expira` , `reset_token_hash` , `reset_expira` y `token_version` ; los controladores que los necesitan usan ámbitos explícitos (`conPassword` , `conCodigo` , `conReset`). El middleware `verificarToken` carga al empleado excluyendo el atributo `password` .

Empleados sin contraseña. Un empleado dado de alta como ficha (sin contraseña) no puede iniciar sesión; el login lo trata igual que una contraseña incorrecta.

3. Sesiones con JWT

3.1 Emisión de tokens

Ambos tipos de token se firman con `JWT_SECRET` (biblioteca `jsonwebtoken`) y vencen a las 24 horas (`expiresIn: 86400`).

Token	Función	Contenido
Empleado (panel)	`firmarToken` en `app/utils/empleados.js`	`id`, `id_sucursal`, `v` (versión de sesión)
Clienta (tienda)	`firmarToken` en `app/controllers/tienda_auth.controller.js`	`id_cliente`, `tipo: "CLIENTE"`, `v` (versión de sesión)

3.2 Separación de tokens de empleado y de clienta

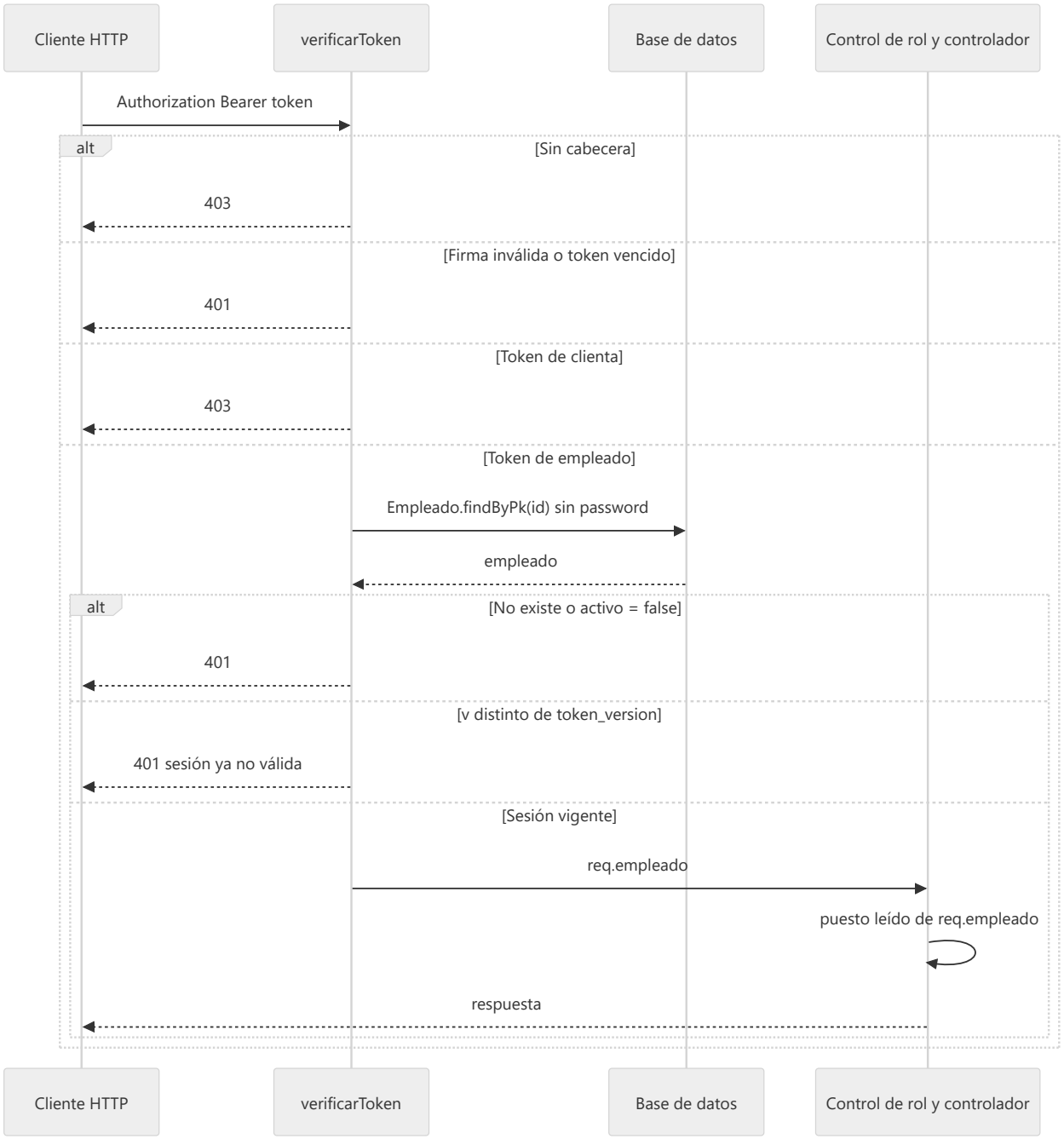
Los dos middlewares de `app/middlewares/authJwt.js` rechazan el token del otro tipo:

- `verificarToken` (rutas del panel) responde `403` si el token tiene `tipo === "CLIENTE"` o no contiene `id`.
- `verificarCliente` (rutas de la tienda) responde `403` si el token no tiene `tipo === "CLIENTE"` o no contiene `id_cliente`.

De esta forma, una sesión de la tienda en línea nunca da acceso al panel de empleados, y viceversa.

3.3 Verificación en cada petición

En cada petición autenticada, además de validar la firma y la expiración, el middleware consulta la cuenta en la base de datos:



Los controles de rol (`isDistrital` , `isAdmin` , `isCajero` , `isBodeguero` , construidos con `exigirPuesto`) leen el puesto del registro `Empleado` cargado en esa misma petición, no del token. Un cambio de puesto o una desactivación se aplica, por tanto, de forma inmediata. `verificarCliente` hace la comprobación equivalente sobre `Cliente` (existencia, `activo` y `token_version`).

3.4 Cierre de sesiones con `token_version`

`Empleado.token_version` y `Cliente.token_version` (entero, valor por defecto 0) se copian en el campo `v` del token. Cuando el valor de la base aumenta, todos los tokens emitidos antes dejan de ser válidos. Los eventos que lo incrementan son:

Evento	Archivo	Efecto adicional
El empleado cambia su propia contraseña	`app/controllers/auth.controller.js` (`cambiarPassword`)	Se devuelve un token nuevo para la sesión actual.
Un superior restablece la contraseña de un empleado	`app/controllers/empleado.controller.js` (`restablecerPassword`)	Se cierran todas las sesiones de ese empleado.
Un empleado es desactivado	`app/controllers/empleado.controller.js` (`update`)	Los tokens anteriores no vuelven a valer aunque se reactive.
Cambia el puesto o la sucursal de un empleado	`app/controllers/empleado.controller.js` (`update`, variable `cambiaAlcance`)	Al volver a entrar, el panel muestra lo que corresponde al nuevo puesto y sucursal.
La clienta cambia su contraseña	`app/controllers/tienda_auth.controller.js` (`cambiarPassword`)	Token nuevo para el dispositivo actual; se anula cualquier enlace de restablecimiento pendiente.
La clienta restablece su contraseña con el enlace del correo	`app/controllers/tienda_auth.controller.js` (`restablecer`)	Token nuevo; se anulan el enlace y el código de activación.
La clienta activa su cuenta web con el código de caja	`app/controllers/tienda_auth.controller.js` (`activar`)	Token nuevo.
Una clienta es desactivada desde el panel	`app/controllers/cliente.controller.js` (`update`)	También se anulan el código de activación y el enlace de restablecimiento.

4. Autorización y alcance por sucursal

El modelo completo de roles se describe en Roles y permisos. Desde el punto de vista de la seguridad, las medidas relevantes son:

- **Alcance por sucursal** (`app/utils/alcance.js`). El DISTRITAL ve todas las sucursales; ADMIN, CAJERO y BODEGUERO solo la propia (`Empleado.id_sucursal`). Un empleado sin sucursal que no sea DISTRITAL recibe el alcance `-1`, que no coincide con ninguna sucursal.
- **Fuera de alcance equivale a inexistente.** El middleware `exigirAlcance` determina la sucursal del registro solicitado (pedido, pago, detalle, devolución) y, si no pertenece a la sucursal del empleado, responde `404` con el mensaje "No se encontró el registro en tu sucursal.". Se usa en `app/routes/pedido.routes.js`, `pago.routes.js`, `detalle_pedido.routes.js` y `devolucion.routes.js`. Los listados aplican `whereSucursal` y, para clientes, `whereClientesDeSucursal` / `clienteEnAlcance`.
- **Gestión de empleados.** `aplicarReglasDeGestion` (`app/utils/empleados.js`) limita al ADMIN a cajeros y bodegueros de su sucursal; un empleado de otra sucursal responde `404` en `empleado.controller.js`. Nadie puede desactivarse, cambiarse el puesto ni cambiarse de sucursal a sí mismo, y siempre debe quedar al menos un DISTRITAL activo.

- **Pedidos propios en la tienda.** `pedidoPropio` (`app/controllers/tienda.controller.js`) responde `404` si el pedido no pertenece a la clienta del token.
- **Registro de empleados.** `permitirRegistro` (`authJwt.js`) solo admite registro sin token cuando la tabla `Empleado` está vacía; a partir de ahí exige un token de ADMIN o DISTRITAL.

5. Validación de entradas y consultas a la base de datos

5.1 Identificadores (`validarId`)

`app/middlewares/validarId.js` valida el parámetro `:id` antes de llegar a la base de datos. Solo acepta enteros positivos sin ceros a la izquierda, de hasta 10 dígitos y no mayores que 2 147 483 647 (máximo de `INTEGER` en PostgreSQL), mediante la expresión `^[1-9]\d{0,9}$`. Valores como `abc`, `1e3` o `99999999999` responden `400` ("El identificador no es válido.") en lugar de un `500` con el error interno de la base. Se registra con `router.param("id", ...)` en las rutas de categorías, clientes, cupones, detalles de pedido, empleados, pagos, pedidos, productos, proveedores, reembolsos, sucursales y tienda. La función `esIdValido` se reutiliza para identificadores recibidos en la query o el cuerpo (`devolucion.controller.js`, `reembolso.controller.js`).

5.2 Validación de campos

Los controladores validan tipo, obligatoriedad y longitud de los campos antes de guardar: por ejemplo, nombre y apellido hasta 80 caracteres, correo hasta 120 caracteres con formato válido, dirección hasta 200, teléfono hasta 20 y referencia de pago hasta 150. Los puestos se comparan contra la lista cerrada `PUESTOS_VALIDOS` y el campo `activo` debe ser booleano.

5.3 Consultas parametrizadas

Las consultas se construyen con Sequelize, que parametriza los valores. Las consultas SQL escritas a mano usan `replacements` (por ejemplo, en `app/middlewares/limitador.js` y `app/utils/monitoreo.js`). Las únicas interpolaciones directas en SQL son constantes del código o valores convertidos con `Number(...)` (por ejemplo, `whereClientesDeSucursal` en `app/utils/alcance.js` y los candados de `app/utils/migrador.js`), de modo que no admiten texto del usuario.

5.4 Unicidad de correos

La migración `app/migraciones/0001-esquema-base.js` crea índices únicos sobre `lower(email)` en `Empleado` y `Cliente`, y la aplicación busca los correos sin distinguir mayúsculas. Así no pueden coexistir dos cuentas cuyo correo difiera solo en mayúsculas.

6. Límite de intentos

El middleware `limitarIntentos` (`app/middlewares/limitador.js`) limita las peticiones por IP y ruta en los endpoints expuestos a adivinación de contraseñas, enumeración de correos o abuso del envío de correos. Por defecto permite 20 peticiones en una ventana de 15 minutos; al superarse responde `429` con la cabecera `Retry-After` y el mensaje "Demasiados intentos. Vuelve a intentarlo en N minuto(s).".

Endpoint	Máximo por ventana de 15 min
`POST /api/auth/login`	20
`PUT /api/auth/password`	10
`POST /api/imagenes/firma`	60
`POST /api/tienda/auth/registro`	10
`POST /api/tienda/auth/login`	20
`POST /api/tienda/auth/activar`	10
`POST /api/tienda/auth/olvide`	5 (además, 3 enlaces por correo por hora)
`POST /api/tienda/auth/restablecer`	10
`PUT /api/tienda/perfil/password`	10
`POST /api/tienda/cotizar`	300

6.1 Contadores en la base de datos (`IntentoLimite`)

Los contadores se guardan en la tabla `IntentoLimite` (migración `app/migraciones/0002-intentos-limite.js`, columnas `clave`, `n`, `desde`), por lo que sobreviven a un reinicio y se comparten entre varias instancias del servidor. Cada intento se registra con una sola sentencia atómica `INSERT ... ON CONFLICT DO UPDATE` que suma el intento o reinicia la ventana si ya venció. La clave es un hash SHA-256 de la IP y la ruta, de modo que la IP no se almacena en claro. Una tarea horaria elimina las filas de más de un día. Si la base de datos no responde, se usa un contador en memoria (con un tope de 10 000 claves) para no dejar la ruta sin límite.

6.2 Normalización de la ruta y de la IP

- **Ruta.** Express atiende igual `/api/auth/LOGIN`, `/api/auth/login` y `/api/auth/login/`. La función `rutaNormalizada` convierte la ruta a minúsculas y elimina la barra final, de modo que todas las variantes comparten el mismo contador.
- **IPv4.** Se usa la dirección tal cual; una IPv4 mapeada (`::ffff:a.b.c.d`) se reduce a su parte IPv4, de modo que comparte contador con la dirección sin prefijo.
- **IPv6.** Un proveedor asigna a cada conexión una red `/64` completa; cambiar de dirección dentro de ella no debe generar un contador nuevo. La función `ipClave` expande la dirección y conserva solo los primeros cuatro grupos (prefijo `/64`).
- **IP real detrás del proxy.** `server.js` configura `app.set("trust proxy", 1)` para que, detrás del proxy de Render (un salto), `req.ip` sea la IP del cliente y no la del proxy.

6.3 Límite por correo en «olvidé mi contraseña»

Además del límite por IP, `olvide` (`tienda_auth.controller.js`) cuenta los intentos por correo mediante `contarIntento("olvide:<correo>", 3600)`. A partir del cuarto pedido en una hora para el mismo correo (`MAX_ENLACES_POR_HORA = 3`) se devuelve la misma respuesta, pero no se envía otro enlace ni se invalida el que ya existe. Así se evita que un tercero llene el buzón de otra persona.

7. Cuentas de clientas: inicio de sesión, activación y restablecimiento

7.1 Inicio de sesión en tiempo constante

Tanto el login de empleados (`auth.controller.js`) como el de clientas (`tienda_auth.controller.js`) comparan siempre un hash bcrypt, aunque el correo no exista: en ese caso se compara contra un hash ficticio (`HASH_FICTICIO`) generado al arrancar con el mismo costo que los hashes reales (10 para empleados y 8 para clientas). La respuesta tarda lo mismo y el mensaje es idéntico ("Correo o contraseña incorrectos." en el panel; "Email o contraseña incorrectos." en la tienda), por lo que no puede deducirse qué correos están registrados. La comprobación de cuenta desactivada se hace después de validar la contraseña.

7.2 Registro en la tienda

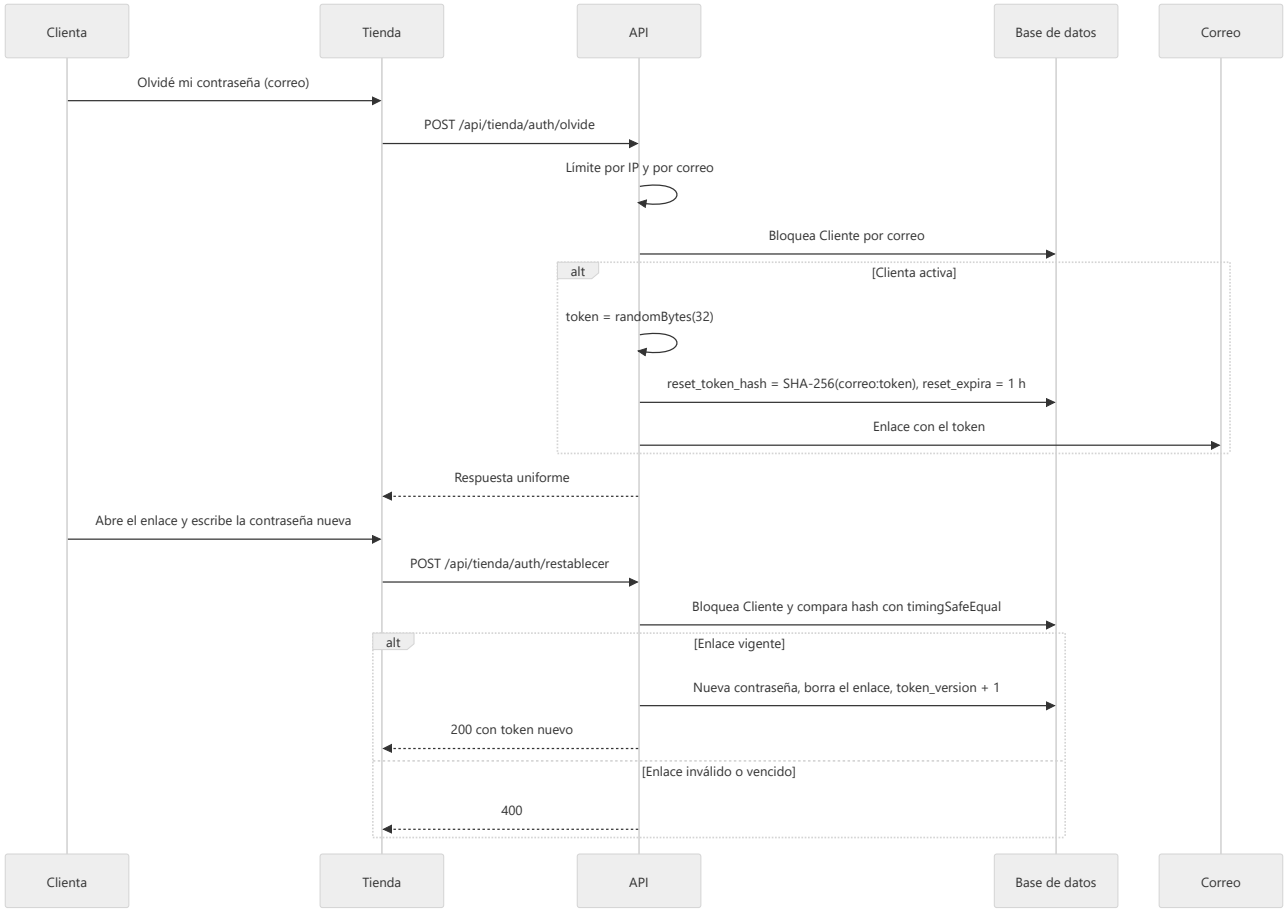
El registro (`POST /api/tienda/auth/registro`) no permite tomar una ficha existente: si el correo ya pertenece a una clienta registrada en mostrador, se le indica que solicite en caja su código de activación, porque el registro no verifica que el correo sea de quien se registra.

7.3 Activación con código de caja

Una clienta de mostrador crea su contraseña con un código de un solo uso generado por caja (`POST /api/clientes/:id/codigo-web`, `cliente.controller.js`):

- El código tiene 8 caracteres tomados de `crypto.randomBytes` sobre un alfabeto sin caracteres confusos, y vence a las 72 horas.
- Solo se guarda su hash SHA-256 (`codigo_activacion`); el código se muestra una sola vez al cajero.
- La comparación usa `crypto.timingSafeEqual`, dentro de una transacción con bloqueo de la fila del cliente.
- Correo inexistente, código incorrecto o vencido producen el mismo mensaje.
- Al activarse, el código se borra y se incrementa `token_version`.

7.4 Restablecimiento de contraseña



Características implementadas en `tienda_auth.controller.js`:

Característica	Implementación
Token aleatorio	`crypto.randomBytes(32)` en hexadecimal (64 caracteres).
Solo se guarda el hash	`reset_token_hash = SHA-256("<correo en minúsculas>:<token>")`.
Hash ligado al correo	Si caja corrige el correo de la ficha, el enlace enviado a la dirección anterior deja de coincidir; además `cliente.controller.js` borra el enlace pendiente al cambiar el correo o desactivar la cuenta.
Vigencia	60 minutos (`MINUTOS_RESET`), en `reset_expira`.
Un solo uso	Al usarse se borran `reset_token_hash` y `reset_expira`; pedir un enlace nuevo reemplaza al anterior.
Comparación segura	Formato validado (`^[0-9a-f]{64}$`) y `crypto.timingSafeEqual`, con la fila bloqueada.
Respuesta uniforme	`olvide` siempre responde el mismo texto, exista o no la cuenta, esté o no desactivada.

Característica	Implementación
Cierre de sesiones	Se incrementa `token_version` y se envía un aviso de "contraseña cambiada".
Enlace fuera de los logs	Sin SMTP en producción, `app/utils/correo.js` imprime el enlace sin la query, para que el token no quede en los registros.

8. CORS y cuerpo de las peticiones

8.1 CORS restringido

`server.js` configura el paquete `cors` con una lista explícita de orígenes tomada de `CORS_ORIGIN` (separados por coma; por defecto `http://localhost:8081` y `http://localhost:8082`). En producción deben indicarse los dominios del panel y de la tienda. Solo se expone la cabecera `X-Request-Id`. Las sesiones viajan en la cabecera `Authorization` y no en cookies, por lo que el navegador no las adjunta automáticamente en peticiones de otros sitios. CORS limita a los navegadores; los controles de autenticación descritos se aplican igual a cualquier cliente HTTP.

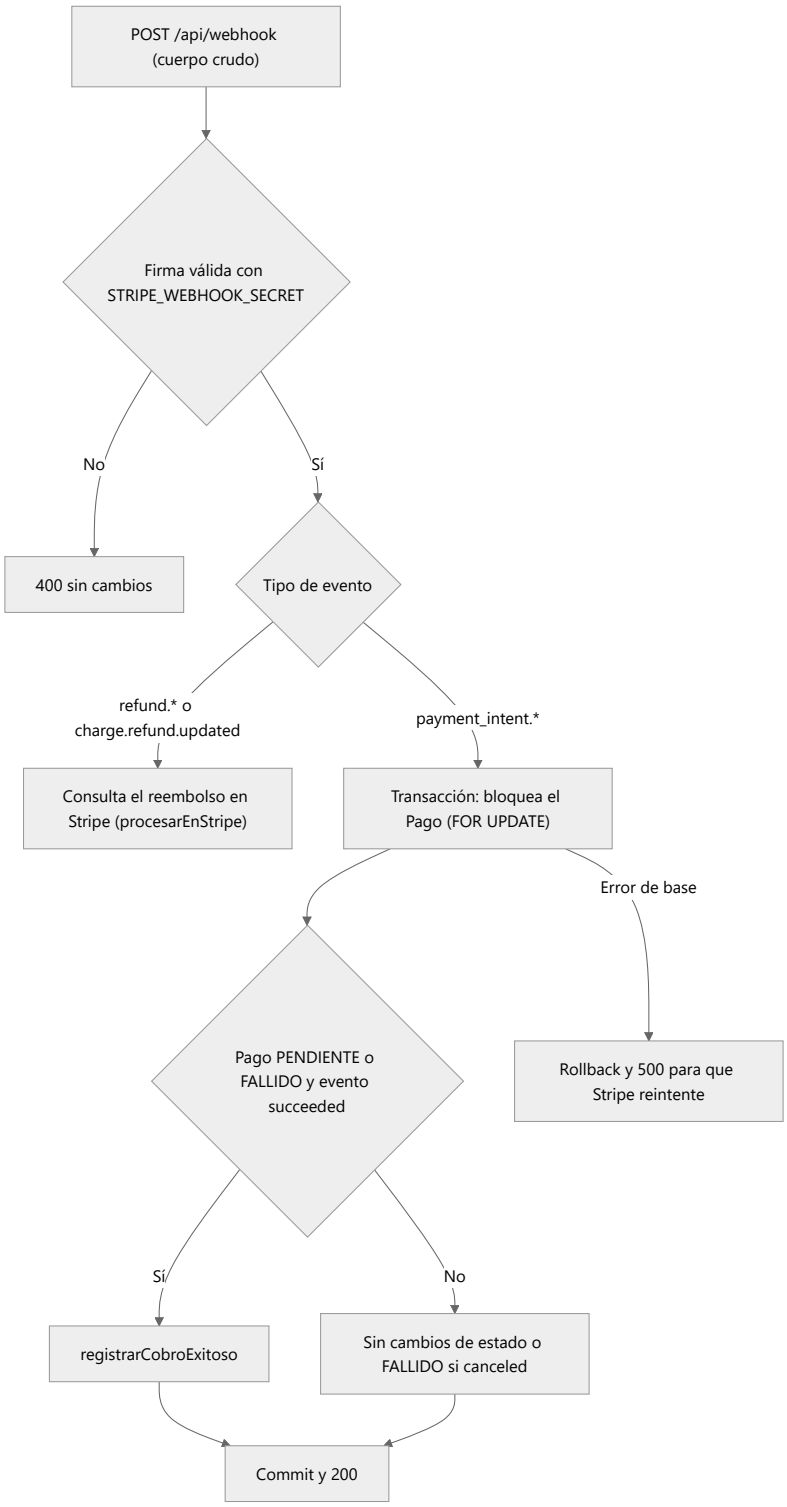
8.2 Cuerpos comprimidos rechazados

`bodyParser.json`, `bodyParser.urlencoded` y el `express.raw` del webhook se configuran con `inflate: false`. Una petición con `Content-Encoding` comprimido se rechaza con `415` ("No se aceptan cuerpos comprimidos."), mediante el manejador de `app/utils/monitoreo.js`. Ningún cliente del sistema envía cuerpos comprimidos, y esta configuración evita la denegación de servicio descrita en la sección 17. Los cuerpos que exceden el límite por defecto de body-parser responden `413` y los JSON mal formados `400`.

9. Pagos con Stripe

9.1 Webhook con firma sobre el cuerpo crudo

La firma de Stripe se calcula sobre los bytes exactos del cuerpo. Por eso `server.js` registra `app/routes/webhook.routes.js` **antes** de `bodyParser.json`, y la ruta `POST /api/webhook` recibe el cuerpo crudo con `express.raw({ type: "application/json", inflate: false })`. El controlador (`app/controllers/webhook.controller.js`) valida el evento con `stripe.webhooks.constructEvent(req.body, firma, STRIPE_WEBHOOK_SECRET)` y responde `400` si la firma no es válida.



Dentro del webhook, el pago se busca por `id_transaccion_externa` y se bloquea; solo un pago en `PENDIENTE` o `FALLIDO` pasa a `COMPLETADO`, de modo que un evento repetido no registra un cobro dos veces. Un error de base de datos devuelve `500` para que Stripe reintente la notificación.

9.2 La confirmación no confía en el navegador

`POST /api/tienda/pedidos/:id/pago/confirmar` (`tienda.controller.js`) no acepta del cliente el resultado del pago: consulta el `PaymentIntent` en Stripe (`stripe.paymentIntents.retrieve`) y solo registra el cobro si su estado es `succeeded`. El monto del `PaymentIntent` lo calcula el servidor a partir del saldo del pedido.

9.3 Claves secretas solo en el servidor

Variable	Dónde se usa
`STRIPE_SECRET_KEY`	`app/config/stripe.config.js` (backend)
`STRIPE_WEBHOOK_SECRET`	`app/controllers/webhook.controller.js` (backend)
`VITE_STRIPE_PUBLISHABLE_KEY`	`tienda/src/config/env.js` (clave publicable, apta para el navegador)

Vite incorpora en el código entregado al navegador todas las variables con prefijo `VITE_`. Por eso los clientes solo definen `VITE_API_URL` y, en la tienda, `VITE_STRIPE_PUBLISHABLE_KEY`; ninguna clave secreta lleva ese prefijo. La plantilla `.env.example` lo advierte expresamente: "La clave secreta NUNCA va en el frontend".

9.4 Tiempo de espera hacia Stripe

`stripe.config.js` fija `timeout: 15000` y `maxNetworkRetries: 1`. Algunas llamadas a Stripe se realizan con filas bloqueadas y una conexión de la base ocupada; con los valores por defecto de la biblioteca, una caída de Stripe podía retener conexiones durante minutos y agotar el pool.

10. Fotos en Cloudinary

Las fotos de las prendas se suben desde el navegador directamente a Cloudinary, con una **subida firmada** generada por el backend (`app/config/cloudinary.config.js`, `app/controllers/imagen.controller.js`):

- `POST /api/imagenes/firma` exige token de empleado, puesto DISTRITAL y está sujeto al límite de 60 intentos por ventana.
- La firma (SHA-1 de los parámetros ordenados más `CLOUDINARY_API_SECRET`) fija la carpeta `distribuidora/prendas`, los formatos permitidos `jpg, jpeg, png, webp` y la marca de tiempo. El navegador recibe la firma, nunca el secreto.
- `POST /api/imagenes/descartar` (solo DISTRITAL) borra como máximo 20 URL por petición y solo si son de este cloud y de la carpeta de prendas (`publicIdDe`), y ninguna prenda las usa. Los enlaces externos pegados a mano nunca se borran.
- En la base de datos solo se guarda la URL (`Producto.imagen_url`).

11. Concurrencia: transacciones y bloqueos

Las operaciones que mueven inventario o dinero se ejecutan en transacciones de Sequelize con bloqueo de filas (`lock: t.LOCK.UPDATE`, es decir, `SELECT ... FOR UPDATE`).

Riesgo	Medida	Archivo
Vender dos veces la misma unidad	`descontarStockVenta` bloquea los lotes `DISPONIBLE` y comprueba el stock dentro de la transacción. Las líneas se agrupan por producto y se procesan en orden de `id_producto` para que todas las ventas bloqueen en el mismo orden y no se produzcan interbloqueos.	`app/utils/facturacion.js`
Cobrar más que el total del pedido	El pedido se bloquea y el saldo se calcula descontando pagos completados y pendientes (`totalComprometido`); un monto mayor se rechaza.	`app/controllers/pago.controller.js`
Dos intentos de pago con tarjeta simultáneos	Orden de bloqueo fijo Pago → Pedido en `iniciarPago` , `confirmarPago` , `cancelarPedido` y el webhook. Un `PaymentIntent` en curso se reutiliza (doble clic o recarga); si otra petición creó uno que no está bloqueado, se responde `409` .	`app/controllers/tienda.controller.js`
Cobro confirmado sobre un pedido cancelado o ya pagado	`registrarCobroExitoso` bloquea el pedido; si ya no está `PENDIENTE` o el monto excede el saldo, el pago se registra y se crea automáticamente un reembolso a la tarjeta.	`app/utils/facturacion.js` , `app/utils/reembolsos.js`
Acaparar stock con pedidos sin pagar	Máximo 3 pedidos pendientes por clienta (`MAX_PEDIDOS_PENDIENTES`), contados con la fila del cliente bloqueada, y máximo 20 unidades por prenda.	`app/controllers/tienda.controller.js`
Superar el límite de usos de un cupón	`usarCupon` bloquea el cupón antes de validarlo y sumar el uso.	`app/utils/cupones.js`
Dos solicitudes de devolución simultáneas	El pedido se bloquea; la segunda solicitud ve a la primera.	`app/controllers/devolucion.controller.js`
Dos registros iniciales con la base vacía	Candado `pg_advisory_xact_lock(727001)` ; solo el primero queda como DISTRITAL.	`app/controllers/auth.controller.js`
Quedar sin ningún DISTRITAL utilizable	`activosConAcceso` bloquea a los empleados activos antes de aplicar cambios.	`app/utils/empleados.js` , `app/controllers/empleado.controller.js`
Validar un enlace o código dos veces	La fila del cliente se bloquea al activar, restablecer o cambiar la contraseña.	`app/controllers/tienda_auth.controller.js`
Dos instancias aplicando migraciones	Candado de transacción en una conexión dedicada.	`app/utils/migrador.js`

Antes de cancelar un `PaymentIntent` (operación irreversible que no se deshace con un rollback), `cancelarPedido` valida todas las condiciones que podrían impedir la cancelación.

12. Idempotencia de reembolsos

`procesarEnStripe` (`app/utils/reembolsos.js`) ejecuta en Stripe los reembolsos a tarjeta pendientes y está diseñado para que ningún reintento produzca un doble reembolso:

1. **Reserva de la fila.** Un `UPDATE` condicional marca `procesando_desde`; solo lo logra una petición a la vez (la reserva caduca a los 2 minutos).
2. **Consulta antes de crear.** Si ya se guardó el id de Stripe, solo se consulta. Si no se guardó (respuesta perdida o caída), se buscan en Stripe reembolsos previos del mismo `PaymentIntent` con `metadata.id_reembolso` antes de crear uno nuevo, porque la clave de idempotencia de Stripe dura solo 24 horas.
3. **Clave de idempotencia.** La creación usa `idempotencyKey: "reembolso-<id_reembolso>"`.
4. **Escritura condicional.** El resultado solo se escribe si la fila sigue en `PENDIENTE`, sin sobrescribir lo que haya resuelto otro proceso.
5. **Errores definitivos.** Respuestas 400, 402 o 404 de Stripe marcan el reembolso como `FALLIDO` para entregarlo en efectivo en caja; los errores de red, 409, 429 y 5xx se reintentan en la revisión periódica.

`reembolsarPagoStripe` no crea un segundo reembolso si ya existe uno `PENDIENTE` o `COMPLETADO` para el mismo pago, y la llamada a Stripe se hace después del commit (`t.afterCommit`). Los avisos por correo de reembolso se envían una sola vez, desde el punto en que la escritura condicional tiene éxito.

13. Correos electrónicos

El envío se concentra en `app/utils/correo.js`:

- **Escape de HTML.** La plantilla aplica `escapar` (sustituye `& < > " '`) al título, a cada párrafo, al texto del botón y a la URL del enlace. Un nombre o una dirección de entrega con etiquetas HTML se muestra como texto y no se interpreta.
- **Direcciones restringidas.** Solo se envía a direcciones simples que cumplen `DIRECCION_SIMPLE` (sin comillas, comentarios, `;`, `<` ni `>`) y de hasta 120 caracteres. El destinatario se pasa a nodemailer como objeto `{ name: "", address }`. La misma expresión (`EMAIL_VALIDO`) valida los correos en el registro de la tienda, de modo que nunca se acepta una dirección a la que después no se podría escribir.
- **Envío que no rompe operaciones.** `enviar` nunca lanza excepciones, y los avisos (`app/utils/notificaciones.js`) se programan después del commit y verifican que el pedido esté realmente en el estado anunciado.
- **Avisos de configuración.** En producción, si falta SMTP o `TIENDA_URL`, el servidor lo advierte en la consola al arrancar.

14. Manejo y registro de errores

14.1 Respuestas sin detalles internos

`app/utils/monitoreo.js` define el manejador final de errores de Express:

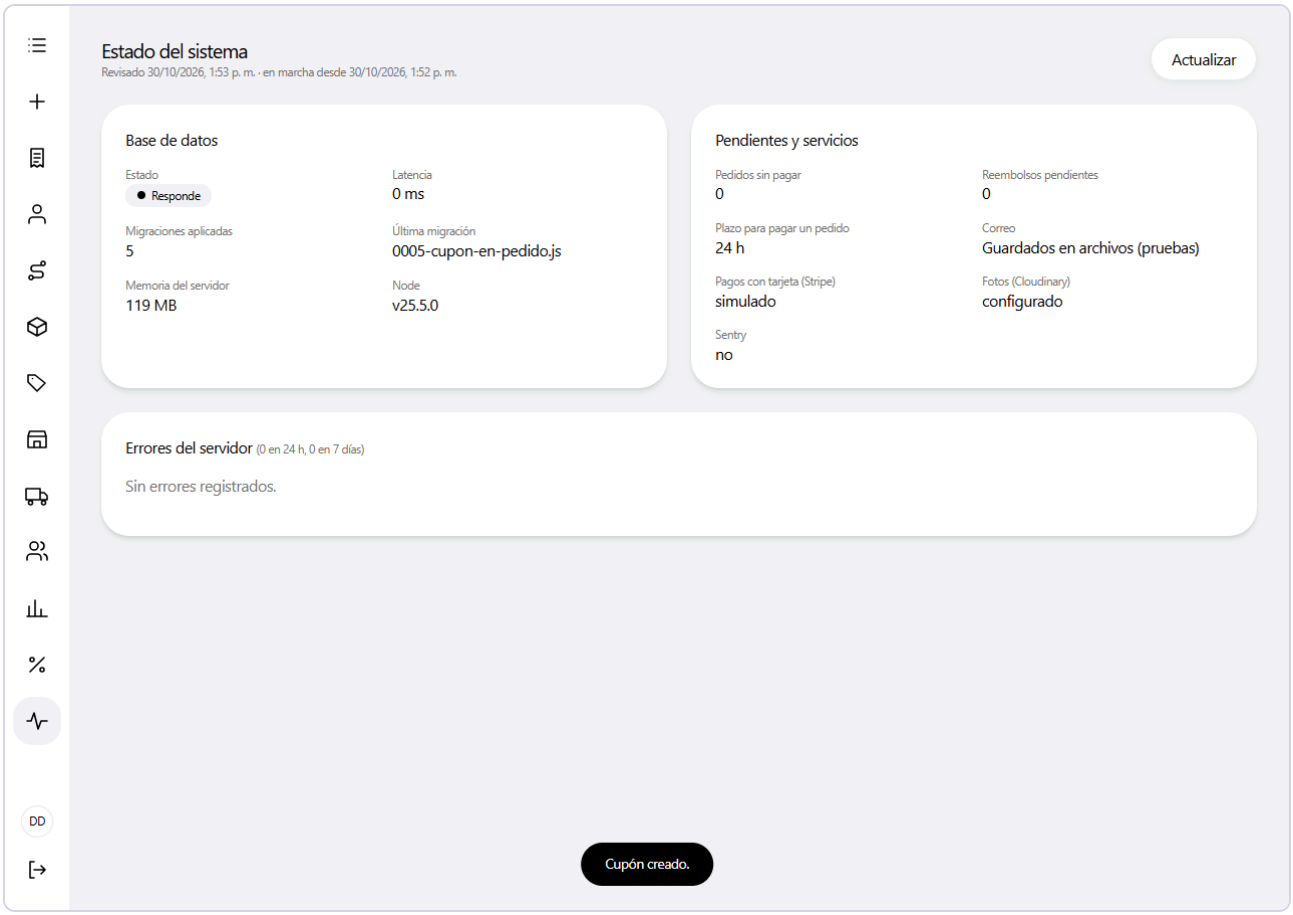
- Los errores 5xx responden "Error interno del servidor (referencia `<id>`)." junto con el identificador de la petición; la pila nunca se envía al cliente.
- JSON mal formado (`400`), cuerpo comprimido (`415`) y cuerpo demasiado grande (`413`) tienen mensajes fijos.
- Las rutas `/api` inexistentes responden `404` en JSON en lugar de la página HTML de Express.

En la tienda, `responderError` (`tienda.controller.js`) devuelve el mensaje de los errores previstos (400, 404, 409) y un mensaje genérico para los inesperados, dejando el detalle real en el registro de errores. Los flujos de cuentas de clientas (`olvide`, `restablecer`, `cambiarPassword`) también responden mensajes genéricos ante un error interno.

14.2 Registro de errores sin cuerpos ni query

Cada petición recibe un identificador (`crypto.randomUUID`) devuelto en la cabecera `X-Request-Id`. Toda respuesta 5xx y toda excepción no controlada se guarda en la tabla `ErrorServidor` (migración `app/migraciones/0003-errores-servidor.js`) con: `request_id`, `metodo`, `ruta`, `estado`, `mensaje`, `pila` y `usuario` (`empleado:<id>` o `cliente:<id>`).

- **Nunca se guarda el cuerpo de la petición**, que puede contener contraseñas.
- **La ruta se guarda sin la query** (`rutaDe` corta en `?`), porque podría contener correos o tokens.
- Los textos se recortan a longitudes máximas y los registros se eliminan a los 30 días.
- Los errores solo los consulta el DISTRITAL (`GET /api/salud/errores`), y esa consulta no devuelve la pila. `GET /api/salud` es público y solo informa si la API y la base responden.
- Sentry es opcional (`SENTRY_DSN`) y se inicializa con `sendDefaultPii: false`.



Pantalla Estado del sistema del panel, con el estado de la base de datos, las migraciones aplicadas, los servicios configurados y el contador de errores del servidor

Figura 1. Pantalla "Estado del sistema" (solo DISTRITAL): estado de la base de datos, servicios externos y errores del servidor registrados en las últimas 24 horas y 7 días.

15. Auditoría

Las acciones sensibles se registran en la tabla de auditoría (modelo `app/models/auditoria.model.js`) con `id_empleado`, `accion`, `tabla_afectada`, `registro_afectado_id` y `detalles`.

- **Atomicidad.** `registrarAuditoria` (`app/utils/auditoria.js`) acepta la transacción de la operación: la fila de auditoría se confirma o se revierte junto con ella, por lo que no queda constancia de algo que falló ni una operación sin su registro.
- **Acceso restringido.** `GET /api/auditoria` y `POST /api/auditoria` exigen el puesto DISTRITAL (`app/routes/auditoria.routes.js`). No existen endpoints para modificar ni eliminar registros.
- **Autoría no falsificable.** En una auditoría manual, `id_empleado` se toma siempre del token y no del cuerpo de la petición.
- **Sin secretos.** Los detalles describen la acción (por ejemplo, "Empleado #5 cambió su contraseña.") y nunca incluyen contraseñas, tokens ni códigos.

Algunas acciones registradas relacionadas con la seguridad:

Acción	Origen
`EMPLEADO_REGISTRO_INICIAL`, `EMPLEADO_REGISTRADO`	Alta de empleados (`auth.controller.js`)
`EMPLEADO_ACTUALIZADO`	Cambios de datos, puesto, sucursal o estado, con el detalle de los cambios (`empleado.controller.js`)
`EMPLEADO_PASSWORD_CAMBIADA`, `EMPLEADO_PASSWORD_RESTABLECIDA`	Contraseñas de empleados
`CLIENTE_CODIGO_WEB_GENERADO`, `CLIENTE_CUENTA_WEB_ACTIVADA`	Código de activación de caja
`CLIENTE_RESTABLECIMIENTO_SOLICITADO`, `CLIENTE_PASSWORD_RESTABLECIDA`, `CLIENTE_PASSWORD_CAMBIADA`	Contraseñas de clientas (`tienda_auth.controller.js`)
`CLIENTE_ACTUALIZADO`	Cambios a la ficha de la clienta (`cliente.controller.js`)
`PAGO_INTENTO_FALLIDO`, `PAGO_REQUIERE_REEMBOLSO`	Webhook y cobros que no corresponden
`REEMBOLSO_CREADO`, `REEMBOLSO_COMPLETADO`, `REEMBOLSO_FALLIDO`	Reembolsos (`app/utils/reembolsos.js`)

16. Secretos y configuración

- Los secretos se leen de variables de entorno. `server.js` carga `.env.production` o `.env.development` según `NODE_ENV`.
- El `.gitignore` del backend excluye `.env`, `.env.development` y `.env.production`; los de `frontend/` y `tienda/` excluyen `.env` y `.env.*`, salvo `.env.example`.
- `.env.example` documenta todas las variables sin valores reales. Las sensibles son `DB_PASSWORD`, `JWT_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `CLOUDINARY_API_SECRET`, `SMTP_PASS` y `SENTRY_DSN`.
- La conexión a PostgreSQL usa SSL salvo que `DB_SSL` sea `false` (necesario solo para una base local sin SSL).
- El modo de pagos simulado se activa únicamente con el valor de prueba `sk_test_clave_de_prueba_universidad` de `STRIPE_SECRET_KEY`; con una clave real, los cobros pasan por Stripe.

17. Dependencias con avisos de seguridad pendientes

Al momento de redactar este documento, dos dependencias tienen avisos publicados cuya versión corregida aún no se ha instalado. En ambos casos el código aplica una mitigación que evita la condición vulnerable; la actualización sigue pendiente.

Paquete	Versión instalada	Corregido en	Avisos	Mitigación aplicada
`nodemailer`	10.1.0 (fijada en `package.json`)	10.1.4	GHSA-8c8m-p3m8-cjg8, GHSA-m943-m63m-w62w, GHSA-85x2-q946-cp5h (fallos del analizador de direcciones que pueden desviar un correo a otro destinatario)	`DIRECCION_SIMPLE` en `app/utils/correo.js`: solo se envía a direcciones simples sin comillas, comentarios, `;`, `<` ni `>`, y la misma regla valida los correos al registrarlos.
`body-parser`	2.3.0 (también la usa Express 5.2.1)	2.4.0	GHSA-27r3-jxmh-p828 (denegación de servicio con cuerpos comprimidos)	`inflate: false` en `bodyParser.json`, `bodyParser.urlencoded` (`server.js`) y `express.raw` del webhook (`app/routes/webhook.routes.js`); los cuerpos comprimidos se rechazan con `415`.

La acción recomendada es actualizar ambos paquetes a las versiones corregidas y volver a ejecutar las pruebas automatizadas (`npm run test:todo`). Las mitigaciones pueden mantenerse después de la actualización, ya que no afectan el funcionamiento.

18. Limitaciones conocidas

Se documentan con fines de transparencia los siguientes aspectos que no forman parte de las medidas anteriores:

- **Almacenamiento del token en el navegador.** El panel y la tienda guardan la sesión en `localStorage` (`frontend/src/services/session.js`, `tienda/src/services/session.js`), accesible para el código JavaScript del mismo origen. El efecto de un token expuesto se acota con la expiración de 24 horas y con `token_version`, que permite cerrar las sesiones cambiando la contraseña.
- **Mensajes de error internos en algunos controladores.** El manejador final, `responderError` de `tienda.controller.js` y los flujos `olvide`, `restablecer` y `cambiarPassword` de `tienda_auth.controller.js` ocultan los detalles internos, pero varios controladores del panel (por ejemplo, `empleado.controller.js` y `auth.controller.js`) y las funciones `registro`, `activar`, `login`, `perfil` y `actualizarPerfil` de `tienda_auth.controller.js` responden en sus errores 500 el texto de la excepción (`error.message`). Entre ellas hay rutas que no exigen sesión: el login del panel, el registro inicial de empleados y el registro, la activación y el login de la tienda.
- **Mensajes de Stripe.** Cuando falla la comunicación con Stripe, la respuesta `502` incluye el mensaje de error de Stripe.
- **Duración de la sesión fija.** Los tokens se firman con `expiresIn: 86400` en el código; la variable `JWT_EXPIRES_IN` de `.env.example` y `app/config/auth.config.js` no intervienen en la firma.
- **Límite por correo dependiente de la base.** Si la base no responde, `contarIntento` devuelve `null` y no se aplica el límite por correo de «olvidé mi contraseña»; el límite por IP sigue activo con el contador en memoria.
- **Cabeceras HTTP de seguridad.** La API no agrega cabeceras adicionales como las que proporciona `helmet`.

- **Costo de bcrypt.** Las contraseñas de clientas usan costo 8 y las de empleados costo 10.

19. Resumen: riesgo, medida y archivo

Riesgo	Medida	Archivo
Robo de contraseñas desde la base	Hash bcrypt (costo 10 empleados, 8 clientas); el hash nunca se devuelve	`app/utils/empleados.js`, `app/controllers/tienda_auth.controller.js`, `app/models/cliente.model.js`
Contraseñas que "funcionan" con cualquier sufijo	Límite de 72 bytes, mínimo 8 caracteres	`app/utils/empleados.js`, `app/controllers/tienda_auth.controller.js`
Sesión válida tras cambio de contraseña, desactivación o cambio de puesto/sucursal	`token_version` comparado en cada petición	`app/middlewares/authJwt.js`, `app/controllers/empleado.controller.js`, `app/controllers/cliente.controller.js`
Token de clienta usado en el panel (o al revés)	Campo `tipo: "CLIENTE"` y rechazo cruzado	`app/middlewares/authJwt.js`
Puesto obsoleto en el token	Empleado y puesto leídos de la base en cada petición	`app/middlewares/authJwt.js`
Acceso a datos de otra sucursal	`exigirAlcance`, `whereSucursal`; `404` fuera de alcance	`app/utils/alcance.js`, `app/routes/*.routes.js`
Acceso a pedidos de otra clienta	`pedidoPropio` responde `404`	`app/controllers/tienda.controller.js`
Ids malformados que provocan errores internos	`validarId` / `esIdValido`	`app/middlewares/validarId.js`
Inyección SQL	Consultas parametrizadas y `replacements`; interpolación solo de constantes o `Number()`	`app/middlewares/limitador.js`, `app/utils/alcance.js`, `app/utils/monitoreo.js`
Fuerza bruta y abuso de correos	`limitarIntentos` en la tabla `IntentoLimite`; ruta normalizada, IPv6 `/64`, límite por correo	`app/middlewares/limitador.js`, `app/migraciones/0002-intentos-limite.js`, `app/routes/auth.routes.js`, `app/routes/tienda.routes.js`
Enumeración de correos	Login en tiempo constante con hash ficticio y mensajes uniformes	`app/controllers/auth.controller.js`, `app/controllers/tienda_auth.controller.js`
Toma de cuenta con enlace de restablecimiento	Token aleatorio de 32 bytes, hash ligado al correo, 1 h, un solo uso, `timingSafeEqual`	`app/controllers/tienda_auth.controller.js`
Toma de una ficha de mostrador	Activación solo con código de caja (hash, 72 h, un solo uso) o por correo	`app/controllers/cliente.controller.js`, `app/controllers/tienda_auth.controller.js`
Peticiones desde sitios no autorizados	CORS con lista de orígenes	`server.js`

Riesgo	Medida	Archivo
Notificaciones de pago falsificadas	Firma del webhook sobre el cuerpo crudo	`app/routes/webhook.routes.js`, `app/controllers/webhook.controller.js`, `server.js`
Pedido marcado como pagado desde el navegador	Confirmación consultando el `PaymentIntent` en Stripe	`app/controllers/tienda.controller.js`
Filtración de claves secretas	Claves solo en el backend; frontends solo con variables `VITE_` públicas	`app/config/stripe.config.js`, `app/config/cloudinary.config.js`, `tienda/src/config/env.js`, `.gitignore`
Subidas arbitrarias a Cloudinary	Subida firmada con carpeta y formatos fijos; firma solo para DISTRITAL	`app/config/cloudinary.config.js`, `app/routes/imagen.routes.js`
Venta doble de la misma unidad	Transacción y bloqueo de lotes en orden fijo	`app/utils/facturacion.js`
Pago doble o mayor al total	Bloqueo Pago → Pedido, saldo comprometido, reutilización del `PaymentIntent`, reembolso automático del excedente	`app/controllers/pago.controller.js`, `app/controllers/tienda.controller.js`, `app/utils/facturacion.js`
Reembolso duplicado	Reserva de fila, búsqueda por metadata, `idempotencyKey`, escritura condicional	`app/utils/reembolsos.js`
HTML inyectado en correos	`escapar` en la plantilla	`app/utils/correo.js`
Desvío de correos por fallos de nodemailer	`DIRECCION_SIMPLE`	`app/utils/correo.js`
Denegación de servicio con cuerpos comprimidos	`inflate: false`, respuesta `415`	`server.js`, `app/routes/webhook.routes.js`, `app/utils/monitoreo.js`
Exposición de la pila o de detalles internos	Manejador final con mensaje genérico y referencia	`app/utils/monitoreo.js`, `app/controllers/tienda.controller.js`
Datos privados en los registros	Sin cuerpo ni query; retención de 30 días; Sentry sin PII	`app/utils/monitoreo.js`, `app/migraciones/0003-errores-servidor.js`
Acciones sin trazabilidad	Auditoría en la misma transacción, solo DISTRITAL, autor tomado del token	`app/utils/auditoria.js`, `app/routes/auditoria.routes.js`, `app/controllers/auditoria.controller.js`

20. Verificación en las pruebas automatizadas

Las pruebas descritas en Pruebas incluyen verificaciones específicas de estas medidas. Entre ellas:

- `tests/api/cuentas.test.js`: solo se guarda el hash del enlace y vence en aproximadamente 1 hora; el enlace es de un solo uso y el anterior deja de valer al pedir otro; enlace vencido, de otra cuenta o con formato inválido responden `400`; contraseña de más de 72 bytes responde `400`; cambiar o restablecer la contraseña cierra las otras sesiones; desactivar a una clienta cierra sus sesiones aunque luego se

reactive; corregir el correo invalida el enlace enviado al anterior; más de 3 enlaces por hora al mismo correo no se envían; las variantes de mayúsculas o de barra final comparten el contador; el cuerpo comprimido se rechaza con `415`; y «olvidé mi contraseña» responde `429` al superar el límite.

- `tests/api/monitoreo.test.js`: el 21.º intento desde la misma IP responde `429` con `Retry-After`; las rutas `/api` inexistentes responden `404` en JSON; los errores se registran sin la query; y el detalle del sistema exige token.
- `tests/api/modulo2.test.js` y `tests/api/tienda.test.js`: el webhook con firma inválida responde `400`, y un webhook tardío es idempotente (sin reembolso ni cobro doble).

Pruebas

Este documento describe cómo se verifica el sistema Distribuidora: la estrategia de pruebas de extremo a extremo, la infraestructura desechable sobre la que se ejecutan, los requisitos y comandos para correrlas, la ubicación de los resultados, el contenido de cada suite y el último resultado obtenido. Se describe además la revisión adversarial de código aplicada durante el desarrollo y las categorías de defectos que permitió encontrar y corregir. Todo lo descrito corresponde al contenido de la carpeta `tests/` del repositorio.

Contenido

1. Objetivo y alcance
2. Estrategia de pruebas
3. Requisitos
4. Ejecución
5. Resultados y artefactos
6. Suites de API
7. Suites de navegador
8. Último resultado
9. Revisión adversarial de código
10. Cómo agregar una suite
11. Documentos relacionados

---

1. Objetivo y alcance

Las pruebas comprueban que el sistema completo se comporta según las reglas de negocio descritas en Reglas de negocio y los permisos descritos en Roles y permisos. No se limitan a funciones aisladas: cada suite arranca el servidor real (`server.js`) sobre una base PostgreSQL real y lo ejercita por HTTP, como lo harían el panel, la tienda o Stripe.

Grupo	Carpeta	Qué se ejercita	Suites	Verificaciones
API	`tests/api/`	La API REST por HTTP, la base de datos, las migraciones, los scripts de respaldo y la colección de Postman	13	1006
Navegador	`tests/navegador/`	El panel de empleados y la tienda en línea en Chrome o Edge sin interfaz visible, conectados a la API	6	206
Total			**19**	**1212**

Una **verificación** es una comprobación individual con nombre (por ejemplo, «pago mayor al saldo -> 400»). Cada suite cuenta sus verificaciones correctas y fallidas y las informa al ejecutor.

---

2. Estrategia de pruebas

2.1 Pruebas de extremo a extremo

Cada suite es un programa Node.js independiente que:

1. Comprueba que sus puertos estén libres; si alguno está ocupado, se detiene con un mensaje que empieza por `ABORTADO` en lugar de interactuar con un proceso ajeno. Esta comprobación está presente en 16 de las 19 suites; `api/modulo2`, `api/tienda` y `navegador/tienda` no la realizan.
2. Borra y recrea el esquema `public` de la base de pruebas (`DROP SCHEMA public CASCADE; CREATE SCHEMA public;`).
3. Arranca `server.js` como proceso hijo con variables de entorno propias de la prueba. El servidor aplica las migraciones al arrancar, igual que en producción (ver Arquitectura).
4. Inserta los datos mínimos (sucursales, tipos de prenda, prendas, inventario) mediante SQL directo o mediante la propia API, y obtiene tokens de cada puesto (DISTRITAL, ADMIN, CAJERO, BODEGUERO) y de clientas.
5. Ejecuta los escenarios por HTTP y comprueba tanto la respuesta (código de estado y cuerpo) como el estado resultante en la base de datos (stock por lote, movimientos, pagos, auditoría).
6. Detiene sus procesos e imprime la línea `RESULTADO: n ok, m fallos`.

Este enfoque permite verificar invariantes que solo se observan con todas las piezas integradas, por ejemplo que `Pedido.total` coincida con la suma de los subtotales menos el descuento más el envío, que ningún lote quede con stock negativo o que ningún pedido tenga reembolsos por más de lo cobrado.

2.2 Base de datos desechable

El módulo `tests/soporte/postgres.js` administra un clúster PostgreSQL exclusivo para las pruebas:

Elemento	Valor
Carpeta de datos	`tests/.pgdata` (excluida del repositorio por `.gitignore`)
Puerto	`54329`
Usuario	`tester`, autenticación `trust` (sin contraseña)
Base	`distrib_test`
Codificación	UTF8, `--locale=C`
Registro del servidor	`tests/salidas/postgres.log`

Si el clúster no existe, se crea con `initdb`; luego se arranca con `pg_ctl start`, se espera hasta 30 segundos a que acepte conexiones y se crea la base con `createdb`. Si el puerto 54329 ya está escuchando, el ejecutor reutiliza ese servidor y no lo detiene al terminar; si lo arrancó él mismo, lo detiene con `pg_ctl stop -m fast`.

Las pruebas **nunca** se conectan a la base configurada en los archivos `.env` del proyecto: cada suite fija `DB_HOST=localhost`, `DB_PORT=54329`, `DB_USER=tester` y `DB_NAME=distrib_test` en el entorno del servidor que arranca. La suite de Postman agrega una comprobación de identidad: antes de ejecutar la colección consulta `GET /api/sucursales` y aborta si la respuesta no es una lista vacía, lo que confirma que el servidor es el de la base recién vaciada.

2.3 Puertos propios

Ninguna suite usa los puertos de desarrollo del proyecto (8080 para la API, 8081 para el panel y 8082 para la tienda). Cada una tiene puertos fijos en el rango 18070-18299 (incluidos el Cloudinary falso y el servidor de imágenes), y los servidores falsos de Stripe usan los puertos 12111, 12131 y 12133.

Suite	Puertos
`api/cuentas`	API 18072
`api/fotos`	API 18091, API sin Cloudinary 18092, Cloudinary falso 18093
`api/gestion`	API 18085
`api/migraciones`	API 18074 y 18075 (dos servidores simultáneos)
`api/modulo2`	API 18080
`api/monitoreo`	API 18078
`api/postman`	API 18090
`api/promociones`	API 18079
`api/reembolsos`	API 18070, Stripe falso 12131
`api/sucursales`	API 18089
`api/tienda`	API 18080, API en modo simulado 18081, Stripe falso 12111
`api/vencimientos`	API 18076 y 18077, Stripe falso 12133
`api/ventas`	API 18088
`navegador/fotos`	API 18094, panel 18195, Cloudinary falso 18096
`navegador/panel`	API 18086, panel 18187
`navegador/promociones`	API 18071, panel 18171, tienda 18271
`navegador/sucursales`	API 18098, panel 18199, tienda 18299
`navegador/tienda`	API 18280, tienda 18282, tienda con clave pública de Stripe 18283, servidor de imágenes 18290
`navegador/ventas`	API 18096, panel 18197, tienda 18297

Las suites se ejecutan una tras otra, por lo que dos suites pueden compartir un puerto (por ejemplo, `api/modulo2` y `api/tienda` usan 18080) sin conflicto.

2.4 Servicios externos simulados

Ninguna prueba se comunica con servicios reales. Cada integración se sustituye por un equivalente local:

Servicio	Sustituto en las pruebas	Mecanismo
Stripe (cobros)	Servidor HTTP local que simula `payment_intents` (crear, consultar, cancelar)	El backend acepta `STRIPE_API_HOST`, `STRIPE_API_PORT` y `STRIPE_API_PROTOCOL` (en `app/config/stripe.config.js`) para apuntar a otro servidor sin cambiar código
Stripe (reembolsos)	Servidor local propio de la suite `api/reembolsos`, que simula `payment_intents` (crear, consultar) y `refunds` (crear, consultar, listar por PaymentIntent) y respeta la cabecera `Idempotency-Key`	La suite puede forzar respuestas `pending`, `succeeded`, error 500, rechazo 400 o «respuesta perdida»
Stripe (modo simulado)	Clave `sk_test_clave_de_prueba_universidad`	Con esa clave el backend completa el pago al instante sin llamar a Stripe (`STRIPE_SIMULADO`)
Webhook de Stripe	Eventos firmados con `stripe.webhooks.generateTestHeaderString` y el secreto de prueba	Se verifica la firma real sobre el cuerpo crudo
Cloudinary	`tests/soporte/cloudinary-falso.js`	Atiende `image/upload` e `image/destroy`, verifica las firmas igual que el servicio real (SHA-1 de los parámetros ordenados más el secreto) y registra subidas, borrados y rechazos; el backend lo usa mediante `CLOUDINARY_API_BASE`
Correo (SMTP)	Modo `archivo` del módulo `app/utils/correo.js`	Con `CORREO_MODO=archivo` cada correo se guarda como JSON en `CORREO_CARPETA`; la suite lee esos archivos para comprobar destinatario, etiqueta, asunto, texto, HTML y enlace del botón
Imágenes del catálogo	Servidor HTTP local en `navegador/tienda`	Sirve imágenes de prueba, incluida una inexistente para comprobar el recuadro de respaldo

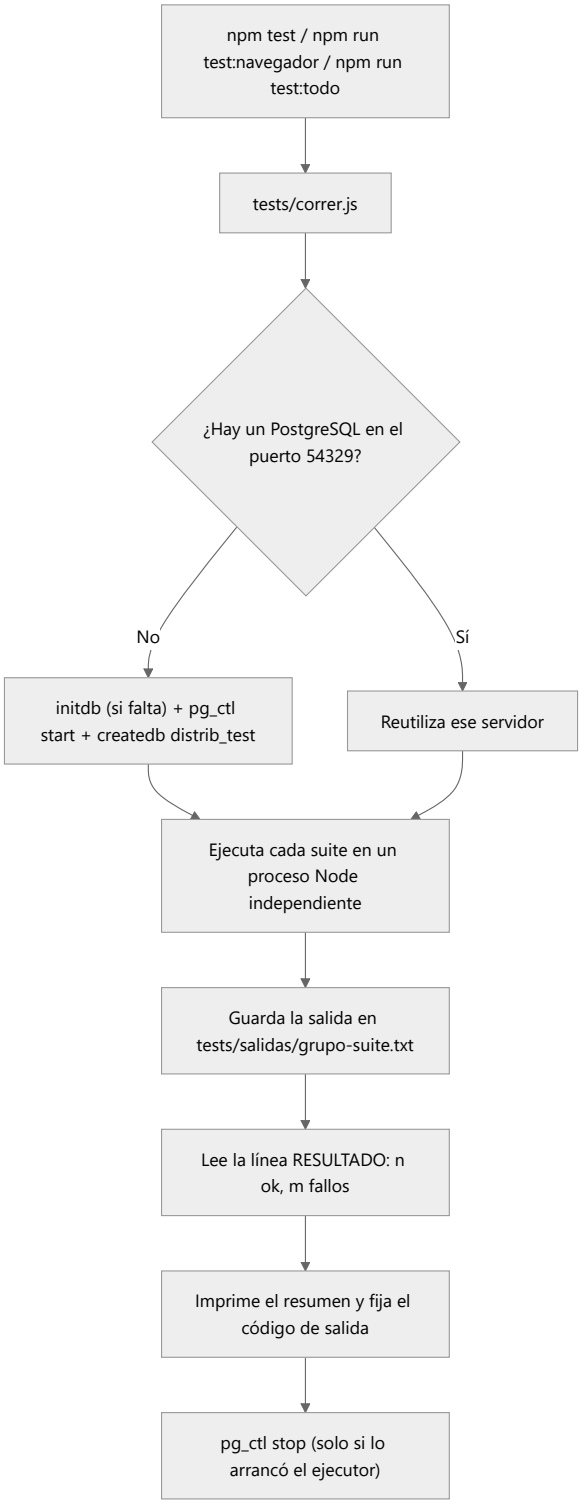
La única verificación que depende de internet es la de `navegador/tienda` que comprueba que el formulario de tarjeta de Stripe (`CardElement`) se monta, porque carga el script desde `js.stripe.com`. La suite de Postman descarga `newman@6.2.1` mediante `npx` la primera vez que se ejecuta.

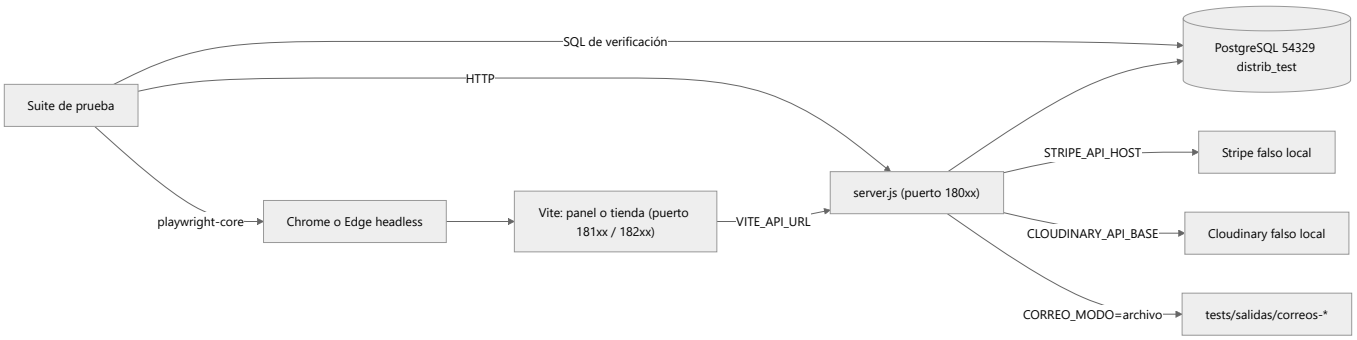
2.5 Pruebas en navegador

Las suites de `tests/navegador/` usan `playwright-core` (versión 1.63.0, dependencia de desarrollo) para controlar un Chrome o Edge ya instalado en modo *headless*, sin descargar navegadores propios. Cada suite:

- arranca la API, y el panel (`frontend/`) o la tienda (`tienda/`) con `npx vite --port <puerto> --strictPort`, pasando `VITE_API_URL` para que apunten a la API de prueba;
- abre contextos con resolución de escritorio (1280 x 900, `locale` es-GT) y, en varias suites, contextos de móvil (por ejemplo 390 x 844 con `isMobile`) para comprobar que la página no tenga desplazamiento horizontal;
- registra los errores de JavaScript (`pageerror`) y los errores de consola, y termina con una verificación de ausencia de errores de JavaScript (por ejemplo, «sin errores de JavaScript en consola»);
- toma capturas de pantalla en `tests/salidas/capturas/` .

2.6 Flujo de una ejecución





---

3. Requisitos

Requisito	Detalle
Node.js y dependencias	Ejecutar `npm install` en la raíz (incluye `playwright-core`) y en `frontend/` y `tienda/` para las suites de navegador
PostgreSQL instalado	Se usan sus programas `initdb`, `pg_ctl` y `createdb`, además de `pg_dump` y `pg_restore` en la suite de migraciones y respaldos
`PG_BIN` (opcional)	Carpeta `bin` de PostgreSQL. Si no se define, en Windows se usa la versión más reciente encontrada en `C:/Program Files/PostgreSQL/<versión>/bin`; si no existe, se usan los programas del `PATH`
Chrome o Edge	Solo para las suites de navegador
`CHROME_PATH` (opcional)	Ruta del ejecutable del navegador. Si no se define, se buscan las rutas habituales de Chrome en Windows (64 y 32 bits), Edge en Windows, `google-chrome`, `chromium` y `chromium-browser` en Linux, y Chrome en macOS
Puertos libres	Los indicados en la sección 2.3 y el 54329
Internet	Solo para la verificación del formulario de tarjeta de Stripe y para la primera descarga de `newman`

No se necesita ningún archivo `.env` ni credencial real: cada suite define sus propias variables con valores de prueba.

---

4. Ejecución

Comando	Qué ejecuta
`npm test`	Las 13 suites de `tests/api/`
`npm run test:navegador`	Las 6 suites de `tests/navegador/`
`npm run test:todo`	Las 19 suites
`node tests/correr.js api/ventas`	Solo las suites cuyo nombre contiene `api/ventas`
`node tests/correr.js navegador`	Todas las suites cuyo nombre contiene `navegador`

Comando	Qué ejecuta
`node tests/correr.js tienda`	`api/tienda` y `navegador/tienda`

Cuando se indica un filtro (un argumento que no empieza por `--`), el ejecutor busca en ambos grupos, `api` y `navegador`, y conserva las suites cuya ruta relativa contiene el texto. Las suites se ejecutan en orden alfabético dentro de cada grupo.

Durante la ejecución, por cada suite se muestran su nombre, las primeras 15 líneas que indiquen un fallo (líneas que empiezan por `FAIL` o contienen `Error:` o `excepción`) y su línea de resultado. Al final se imprime un resumen como el siguiente:

```
── Resumen
✓ api/cuentas.test.js              50 ok    0 fallos  …s
✓ api/fotos.test.js                19 ok    0 fallos  …s
…
Total: 1212 verificaciones correctas, 0 fallos. Detalle en tests/salidas/.
```

Una suite que termina con un código de salida distinto de cero se cuenta con al menos un fallo aunque no haya impreso su línea `RESULTADO`. El ejecutor termina con código 1 si hay cualquier fallo, lo que permite usarlo en procesos automáticos.

---

5. Resultados y artefactos

Todo lo que generan las pruebas queda en `tests/salidas/`, carpeta excluida del repositorio por `.gitignore`.

Ubicación	Contenido
`tests/salidas/api-<suite>.txt` y `navegador-<suite>.txt`	Salida completa de cada suite, con cada verificación marcada `ok` o `FAIL`
`tests/salidas/*-servidor*.log`, `server-*.log`, `migraciones-*.log`, `monitoreo-*.log`, `ui-*.log`	Registro del servidor de la API o de Vite arrancado por cada suite
`tests/salidas/postgres.log`	Registro del PostgreSQL de pruebas
`tests/salidas/postman.txt`	Informe completo de `newman`
`tests/salidas/correos-<suite>/`	Correos generados en modo archivo, uno por archivo JSON
`tests/salidas/capturas/`	Capturas de pantalla de las suites de navegador (catálogo, carrito, pedido pagado, diálogos del panel, reportes, vistas móviles) y los PDF de reportes descargados
`tests/salidas/respaldos-prueba/`	Respaldos creados por la suite de migraciones (nunca la carpeta `respaldos/` del proyecto)

Cuando una verificación falla, su línea `FAIL` incluye un extracto de la respuesta o del estado observado, lo que suele bastar para localizar la causa sin volver a ejecutar.

---

6. Suites de API

Cada suite de `tests/api/` arranca su propio servidor y verifica un área funcional. La tabla resume su alcance; los apartados siguientes detallan lo que cubre cada una.

Suite	Qué verifica	Verificaciones
`cuentas`	Cuentas de la tienda: restablecimiento de contraseña, cierre de sesiones y correos de cada estado del pedido	50
`fotos`	Fotos de prendas con Cloudinary: firma, subida, cambio, eliminación y funcionamiento sin configuración	19
`gestion`	Gestión del panel: empleados, tipos de prenda, sucursales, bajas de stock, auditoría y proveedores	133
`migraciones`	Migraciones con Umzug, respaldo y restauración	30
`modulo2`	Facturación dinámica y estados del pedido: FIFO, totales, pagos, transiciones, devoluciones y webhook	118
`monitoreo`	Salud, monitoreo de errores, paginación y límite de intentos persistido	45
`postman`	Colección de Postman completa ejecutada con `newman` (118 peticiones)	137
`promociones`	Costo de envío, cupones, IVA, reembolsos con descuento y reportes	65
`reembolsos`	Reembolsos por Stripe y en caja, reintentos, idempotencia y concurrencia	80
`sucursales`	Alcance distrital frente a sucursal, catálogo distrital, devoluciones (una vez, 24 h) y reportes	117
`tienda`	Tienda en línea: registro, catálogo, pedidos, pago con tarjeta, cancelaciones y devoluciones	91
`vencimientos`	Cancelación automática de pedidos pendientes de pago	27
`ventas`	Ventas de mostrador, cobros, listado de pedidos, clientes, código de activación web y proveedores	94

6.1 `cuentas`

- **Olvidé mi contraseña:** misma respuesta exista o no la cuenta; el correo lleva un enlace a `TIENDA_URL` con un token de 64 caracteres hexadecimales; en la base solo se guarda su hash con vencimiento de aproximadamente una hora; pedir otro enlace invalida el anterior; el enlace es de un solo uso y vence.
- **Validaciones:** contraseña corta, contraseña de más de 72 bytes, token de otra cuenta o con formato inválido (400 y no 500).

- **Sesiones:** al restablecer o cambiar la contraseña se cierran las sesiones anteriores (`token_version`); desactivar y reactivar a una clienta no revive sus sesiones; un enlace pendiente deja de valer si cambia la contraseña o si caja corrige el correo.
- **Clienta de mostrador:** crea su contraseña con el enlace y conserva su ficha; una cuenta desactivada no recibe enlaces.
- **Correos del pedido:** pagado (en línea y en caja), enviado, entregado (con el plazo de una sola solicitud), cancelado y solicitud de devolución; un pago rechazado no envía correo de pagado; el HTML de los correos escapa los datos.
- **Seguridad:** direcciones con comillas o `>` rechazadas, máximo 3 enlaces por hora al mismo correo, variantes de mayúsculas o barra final de la ruta comparten el contador de intentos, cuerpos comprimidos rechazados (415) y límite de intentos (429).

6.2 `fotos`

- `GET /api/imagenes/config` informa si la subida está habilitada y el tamaño máximo (5 MB).
- Solo el DISTRITAL obtiene firma de subida; la firma incluye carpeta `distribuidora/prendas`, formatos `jpg,jpeg,png,webp` y nunca el secreto.
- El Cloudinary falso rechaza una firma alterada y acepta la del backend.
- Cambiar o quitar la foto de una prenda borra la anterior de Cloudinary, salvo que otra variante la siga usando; los enlaces externos o de otra cuenta de Cloudinary nunca se borran; descartar solo elimina las subidas que ninguna prenda usa; eliminar una prenda sin historial borra su foto.
- Sin las variables `CLOUDINARY_*` la subida queda deshabilitada (firma 503 con explicación) y se puede seguir pegando un enlace.

6.3 `gestion`

- **Migración automática** sobre una base envejecida: agrega `Empleado.activo` y el valor `AJUSTE_SALIDA`.
- **Registro y sesión de empleados:** registro inicial simultáneo (solo uno queda con el puesto inicial), correo en minúsculas, mismo mensaje para correo inexistente y clave incorrecta, longitud de contraseña entre 8 y 72, cambio de contraseña con cierre de las demás sesiones y hash bcrypt de costo 10.
- **Empleados:** permisos por puesto, lista blanca de campos editables, un administrador no puede desactivarse ni quitarse el puesto, cambiar el puesto o desactivar cierra la sesión, restablecimiento de contraseña, fichas sin acceso y unicidad de correo sin distinguir mayúsculas en la base; desactivación cruzada simultánea que mantiene al menos un DISTRITAL con acceso.
- **Tipos de prenda y sucursales:** validaciones de nombre y longitud, duplicados sin distinguir mayúsculas (400 y no 500), eliminación bloqueada (409) si hay prendas, empleados o inventario asociados, altas simultáneas «Gorras»/«GORRAS».
- **Bajas de stock:** validación de cantidades, motivos y filas; detección de `cantidad_actual` desactualizada (409 sin aplicar cambios); un movimiento `AJUSTE_SALIDA` por fila con empleado y motivo; bajas simultáneas y venta simultánea con baja sin stock negativo ni doble descuento.
- **Auditoría y proveedores:** acceso restringido por puesto y registro a nombre del empleado del token.

6.4 `migraciones`

- Sobre una base vacía, el servidor aplica todas las migraciones de `app/migraciones/` antes de escuchar y las registra en `SequelizeMeta`; en un segundo arranque informa que la base está al día.
- Sobre una base anterior a las migraciones agrega lo que falta sin borrar columnas que no están en los modelos (lo que habría hecho `sync alter`).
- Si un modelo cambia sin migración, el servidor avisa en el registro sin modificar la base; `migrar.js estado` lo informa sin mostrar la contraseña.
- `migrar.js deshacer` revierte solo la última migración y rechaza deshacer la línea base; `migrar.js aplicar` las reaplica.
- Dos servidores que arrancan a la vez sobre una base vacía aplican cada migración una sola vez gracias al candado.
- **Respaldo y restauración:** el respaldo rota según `RESPALDOS_MAX` y no deja archivos `.parcial`; restaurar exige `--base` con el nombre correcto, guarda antes un respaldo de seguridad, funciona aunque se haya creado después una tabla con llave foránea, y rechaza archivos que no son respaldos sin tocar la base.

6.5 `modulo2`

- **Creación de pedidos:** estado inicial `PENDIENTE` aunque el cuerpo indique otro, precio tomado del catálogo, `id_empleado` tomado del token, descuento FIFO por lote con movimientos `VENTA`, rechazo completo (sin pedido ni stock descontado) si falla cualquier línea.
- **Detalles:** agregar y quitar líneas recalcula el total y devuelve el stock con movimiento `CANCELACION`; cinco altas concurrentes dejan un total consistente.
- **Pagos y transiciones:** pagos parciales, confirmación, exceso por 0,01 rechazado, paso automático a `PROCESADO` al completar el total, y la secuencia obligatoria `PROCESADO -> ENVIADO -> ENTREGADO`; `ENTREGADO` y `CANCELADO` son estados finales.
- **Devoluciones** sobre pedidos entregados con su historial de movimientos.
- **Webhook de Stripe:** firma inválida (400), `payment_failed`, evento firmado que completa el pago y cobro sobre un pedido ya pagado que se marca `REEMBOLSADO` con reembolso a la tarjeta.
- **Regresiones de la revisión adversarial** (ver sección 9).
- **Invariantes finales:** `Pedido.total = SUM(subtotal)` en todos los pedidos y ningún lote con stock negativo.

6.6 `monitoreo`

- `GET /api/salud` es público y no expone datos internos; el detalle solo es visible para el DISTRITAL e incluye migraciones, pendientes y modo de los servicios sin secretos.
- Cada respuesta lleva `X-Request-Id`; las rutas inexistentes y el JSON mal formado responden en JSON.
- Un error 500 queda registrado en `ErrorServidor` con ruta, estado, usuario e identificador de petición, sin la cadena de consulta.
- Parámetros de filtro o paginación inválidos responden 400 y no llenan el registro de errores.

- Paginación de clientes y pedidos (25 por página, búsqueda en el servidor, comodines de `LIKE` tratados literalmente, alcance por sucursal); sin `?pagina` se conserva la respuesta como arreglo completo.
- **Límite de intentos:** el intento 21 desde la misma IP recibe 429 con `Retry-After`; los contadores se guardan en la tabla `IntentoLimite` sin la IP en claro, sobreviven a un reinicio, se liberan pasada la ventana de 15 minutos y, si la tabla no responde, se usa un contador en memoria.
- El servidor arranca aunque `SENTRY_DSN` esté definido sin el paquete instalado.

6.7 `postman`

Ejecuta la colección `postman/distribuidora-backend.postman_collection.json` completa con `newman` contra un servidor sobre la base vacía. El último resultado fue de 118 peticiones y 137 aserciones sin fallos. El número de verificaciones informado corresponde a las aserciones correctas; los fallos suman peticiones y aserciones fallidas. La colección se describe en API.

6.8 `promociones`

- **Envío:** costo de 25 a domicilio, gratis desde 300 calculado después del descuento, y sin costo con retiro en sucursal.
- **Cupones:** validaciones, código en mayúsculas, compra mínima, vigencia, usos, sucursal, cupón del 100 %, monto fijo; dos compras simultáneas con un cupón de un uso (solo una lo obtiene); el uso se libera al cancelar el pedido; un cupón usado no cambia de código ni se elimina; editar un cupón no altera pedidos ya creados.
- **Desglose:** subtotal, descuento, envío e IVA incluido en la tienda y en el panel; en mostrador no hay envío; agregar o quitar líneas recalcula descuento y envío.
- **Reembolsos:** se devuelve lo efectivamente pagado por la prenda (descuento prorrateado) y nunca el envío.
- **Reportes:** descuentos, envíos, IVA, reembolsos entregados, ventas netas y ventas por cupón.

6.9 `reembolsos`

- Reintegrar una devolución de un pedido pagado con Stripe crea un reembolso sobre el PaymentIntent correcto, por el monto en centavos y con clave de idempotencia, y lo informa por correo.
- Estados de Stripe `pending`, `succeeded`, error 500 y rechazo 400: el reembolso queda `PENDIENTE` o `FALLIDO` según corresponda, sin revertir la devolución; un reembolso fallido puede entregarse en efectivo en caja.
- Reintentos: dos reintentos simultáneos producen una sola llamada a Stripe, todos los intentos usan la misma clave de idempotencia y, si la respuesta se perdió, el reintento encuentra el reembolso existente por sus metadatos y no devuelve dos veces.
- Caja: reembolsos en efectivo, reversión en POS y PayPal; pagos mixtos (300 por POS y 100 en efectivo); dos cajeros simultáneos (solo uno lo entrega).
- Cambio por una variante más barata: se reembolsa la diferencia acordada al solicitarlo; un cambio por una variante más cara se rechaza.
- Alcance por sucursal, permisos por puesto, validación de filtros e identificadores.
- Invariante final: ningún pedido tiene reembolsos por más de lo cobrado.

6.10 `sucursales`

- Reglas de alta y edición de empleados según puesto: el ADMIN solo gestiona cajeros y bodegueros de su sucursal; el DISTRITAL gestiona a todos.
- Catálogo (tipos de prenda y prendas), sucursales y auditoría reservados al DISTRITAL.
- Alcance por sucursal de inventario, clientes, pedidos, pagos y trazabilidad (los recursos de otra sucursal responden 404 o 403); una clienta de otra sucursal se encuentra por correo exacto.
- Devoluciones: una sola solicitud por pedido (incluso con solicitudes simultáneas y aunque la primera se rechace) dentro de las 24 horas posteriores a la entrega, en el panel y en la tienda.
- Reportes: ventas del periodo, ticket promedio, prendas vendidas, métodos de pago, saldo por cobrar, devoluciones; días calculados en hora de Guatemala; validación de periodos.
- Correcciones al arrancar: completar `fecha_entrega` desde la auditoría, promover un ADMIN si no existe ningún DISTRITAL y recuperar la sucursal de clientes que no la tienen.

6.11 `tienda`

- Registro e inicio de sesión de clientas, perfil, separación entre tokens de clienta y de empleado.
- Catálogo agrupado por variantes activas, stock disponible por sucursal activa, filtros por categoría y búsqueda por color.
- Pedidos: carrito vacío, máximo 20 unidades por prenda, sucursal o prenda inactiva, máximo 3 pedidos pendientes por clienta, total calculado por el servidor.
- Pago con tarjeta contra el Stripe falso: creación y reutilización del PaymentIntent, confirmación, estado `processing`, webhook tardío idempotente, PaymentIntent cancelado en Stripe, cancelación del pedido con pago en curso, cobro del resto cuando ya hay pagos en caja.
- Devoluciones y cambios desde la tienda, con historial `SOLICITADA -> APROBADA -> RECIBIDA -> ACEPTADA -> CAMBIADA`.
- Validación de `imagen_url`, CORS para la tienda, modo de pago simulado y límite de intentos de registro.
- Invariantes finales de total y de cobro.

6.12 `vencimientos`

Se arranca el servidor con `PLAZO_PEDIDO_PENDIENTE_HORAS=1` y `VENCIMIENTO_INTERVALO_MIN=0.02` (revisión aproximadamente cada segundo).

- La tienda y el panel informan hasta cuándo se puede pagar (`pago_vence`).
- Un pedido vencido sin pagos se cancela, devuelve el stock, queda auditado y se notifica por correo.
- Un intento de tarjeta reciente (menos de 30 minutos) o un cobro en proceso en Stripe impiden la cancelación; si Stripe ya cobró, se registra el cobro y el pedido pasa a `PROCESADO`.
- Pagos parciales en efectivo y PayPal por confirmar mantienen el pedido `PENDIENTE` para que lo resuelva caja.
- El stock se devuelve y el correo se envía una sola vez por pedido; `PLAZO_PEDIDO_PENDIENTE_HORAS=0` desactiva el vencimiento.

6.13 `ventas`

- Migración automática sobre una base envejecida (métodos `EFECTIVO` y `TARJETA`, columnas del código de activación e índice de correo).
- Clientes: alta, edición, validación de correo, permisos por puesto, listado sin hashes ni códigos.
- Venta de mostrador y cobros con `EFECTIVO`, `TARJETA` (POS) y `PAYPAL` (por confirmar), saldos pagado, por confirmar y por cobrar; paso a `PROCESADO`; despacho por el BODEGUERO.
- Cancelación y devolución de pagos: no se cancela con cobros activos, devolución concurrente del mismo pago aplicada una sola vez, PayPal no se devuelve desde caja.
- Filtros del listado de pedidos (origen, sucursal, fecha en hora de Guatemala, cliente y estado).
- Código de activación web `XXXX-XXXX`: solo se guarda su hash, vence, se invalida al generar otro o al desactivar, no se reutiliza y permite a la clienta de mostrador ver sus compras en la tienda.
- Proveedores: alta, edición, permisos y auditoría.

---

7. Suites de navegador

Suite	Qué verifica	Verificaciones
`fotos`	Subir, cambiar, quitar y cancelar fotos de prendas en el panel con el Cloudinary falso	15
`panel`	Tipos de prenda, sucursales, bajas de stock, empleados y Mi cuenta, por puesto y en móvil	60
`promociones`	Promociones, Estado del sistema, paginación de clientes, checkout con cupón y retiro en sucursal, venta con cupón	17
`sucursales`	Distrital frente a gerente de sucursal, reportes con PDF, alcance por sucursal y regla de devoluciones	35
`tienda`	Recorrido completo de compra en la tienda, en escritorio y móvil, y montaje del formulario de Stripe	29
`ventas`	Pedidos, Nueva venta, cobro, estados, clientes con código web, proveedores y activación en la tienda	50

7.1 `navegador/fotos`

Comprueba en el formulario de prenda del panel que se rechacen archivos que no son imagen, que al subir aparezcan la vista previa (en versión optimizada) y las acciones «Cambiar foto» y «Quitar foto», que cambiar antes de guardar o cancelar la edición descarte la subida no usada, que al guardar se borre la foto anterior y que también se pueda pegar un enlace. Verifica además que Cloudinary nunca reciba una firma inválida.

7.2 `navegador/panel`

- Menú según puesto: el ADMIN ve Sucursales y Empleados; el BODEGUERO ve Sucursales sin acciones de edición y tipos de prenda en solo lectura; el CAJERO no ve Sucursales ni Empleados y recibe un aviso de permisos si entra por URL.

- Tipos de prenda: cantidad de prendas que usan cada tipo, edición con error de nombre repetido dentro del diálogo, explicación cuando un tipo en uso no puede eliminarse.
- Sucursales: alta, edición, desactivación y explicación cuando una sucursal con historial no puede eliminarse.
- Bajas de stock: selección de filas, resumen recalculado, motivo obligatorio, cantidad mayor a la actual bloqueada, aplicación en la base, error claro si el stock cambió mientras se editaba, y registro «Baja por ajuste» en la trazabilidad.
- Empleados y Mi cuenta: alta con confirmación de contraseña, edición de puesto, bloqueo de puesto y estado al editarse a sí mismo, restablecimiento y cambio de contraseña.
- Sesión: un empleado desactivado con sesión abierta vuelve al inicio de sesión con aviso; una sesión guardada se actualiza con el puesto vigente (`/yo`).
- Vistas móviles sin desplazamiento horizontal y ausencia de errores de JavaScript.

7.3 `navegador/promociones`

El DISTRITAL crea un cupón en Promociones y consulta Estado del sistema (base, migraciones y servicios). La lista de clientes se pagina de 25 en 25 con búsqueda en el servidor. En la tienda, el checkout muestra el envío a domicilio y el IVA incluido, el retiro en sucursal sin envío ni dirección, el aviso de un cupón inválido y el descuento de uno válido; el pedido resultante muestra el desglose. En el panel, Nueva venta aplica el cupón sin envío y el detalle del pedido muestra subtotal, descuento e IVA.

7.4 `navegador/sucursales`

- El DISTRITAL ve los reportes de todas las sucursales con desglose por sucursal, filtra por sucursal y periodo, y descarga un PDF válido.
- El ADMIN de sucursal ve el reporte solo de su sucursal (cuadrado contra la base), descarga su PDF, ve el catálogo en solo lectura con stock de su sucursal, solo empleados de su sucursal y solo puede dar de alta cajeros o bodegueros.
- La cajera de otra sucursal no ve a la clienta en su lista, pero la encuentra por correo.
- La regla de devoluciones (una solicitud, 24 horas) se muestra y se aplica en Nueva solicitud del panel y en la tienda, incluso si se entra directo al formulario.
- Vistas móviles sin desplazamiento horizontal.

7.5 `navegador/tienda`

Recorre la compra completa: catálogo con prendas agrupadas y recuadro de respaldo para imágenes rotas, filtros, ficha con talla sin stock y aviso de otra sucursal, carrito con total, registro desde el checkout con regreso al pago, dirección precargada, pedido pendiente, pago aprobado en modo simulado, cancelación, pedido entregado con sus cuatro pasos, solicitud de cambio de talla y lista de mis pedidos. Repite el catálogo y la ficha en un contexto móvil y, con una clave pública de Stripe de prueba, comprueba que el formulario de tarjeta se monte y que el botón de pago permanezca deshabilitado hasta completar la tarjeta.

7.6 `navegador/ventas`

- CAJERO: alta de cliente («Mostrador»), generación del código `XXXX-XXXX`, Nueva venta en su sucursal fija con límite de stock, cobro en efectivo con cálculo del vuelto, pago parcial con tarjeta, confirmación de PayPal, agregar y quitar prendas, cancelación, explicación cuando un cobro impide cancelar y devolución del cobro en efectivo.
- Listado de pedidos con filtro por estado en la URL y búsqueda por cliente.
- BODEGUERO: marca pedidos como enviados y entregados, gestiona proveedores sin poder eliminarlos y no ve acciones de cobro.
- ADMIN: elimina proveedores y ve todo el menú.
- Tienda: el registro con un correo de caja sugiere activar la cuenta; el código incorrecto muestra error; la cuenta activada ve su compra de mostrador.

---

8. Último resultado

Resultado de la última ejecución completa (`npm run test:todo`), tomado de las salidas en `tests/salidas/`:

Grupo	Suite	Verificaciones correctas	Fallos
API	`cuentas`	50	0
API	`fotos`	19	0
API	`gestion`	133	0
API	`migraciones`	30	0
API	`modulo2`	118	0
API	`monitoreo`	45	0
API	`postman`	137	0
API	`promociones`	65	0
API	`reembolsos`	80	0
API	`sucursales`	117	0
API	`tienda`	91	0
API	`vencimientos`	27	0
API	`ventas`	94	0
Navegador	`fotos`	15	0
Navegador	`panel`	60	0
Navegador	`promociones`	17	0
Navegador	`sucursales`	35	0
Navegador	`tienda`	29	0

Grupo	Suite	Verificaciones correctas	Fallos
Navegador	`ventas`	50	0
Subtotal API		**1006**	**0**
Subtotal navegador		**206**	**0**
Total		**1212**	**0**

---

9. Revisión adversarial de código

Además de las pruebas, durante el desarrollo se aplicó una revisión adversarial del código. El procedimiento tuvo dos papeles separados:

1. **Revisores independientes**, que leían el código de un módulo con el objetivo explícito de encontrar defectos y describían, para cada uno, un escenario concreto de fallo (datos de entrada, orden de operaciones y resultado incorrecto).
2. **Verificadores**, que intentaban refutar cada hallazgo leyendo el código o reproduciéndolo. Solo los hallazgos que resistían ese intento se consideraban confirmados.

Los hallazgos confirmados se corrigieron y se agregaron verificaciones que reproducen el escenario original, para que el defecto no reaparezca. En las suites `api/modulo2` y `api/tienda` esas verificaciones están agrupadas en secciones que la salida titula «Regresiones de la revisión» (en el código, «Regresiones de la revisión adversarial») y llevan en su nombre el número del hallazgo, por ejemplo `[#0]`, `[#4]` o `[#13]`. En las demás suites, las comprobaciones de concurrencia y de seguridad cumplen la misma función.

Los defectos encontrados y corregidos se agrupan en cuatro categorías. Los ejemplos corresponden a verificaciones presentes en las suites:

Categoría	Tipo de defecto	Ejemplos verificados
Concurrencia	Operaciones simultáneas que duplicaban efectos o se bloqueaban mutuamente	Tres eliminaciones simultáneas del mismo detalle devuelven el stock una sola vez; ventas cruzadas simultáneas sin interbloqueo (*deadlock*); pagar y cancelar a la vez sin error 500; bajas simultáneas sin doble descuento; un cupón de un uso no se otorga dos veces; dos reintegros o dos entregas de reembolso simultáneos se aplican una sola vez; dos servidores no aplican dos veces una migración
Dinero	Montos, saldos o estados de pago incorrectos	Montos con tres decimales rechazados; pagos pendientes que sumados exceden el total; quitar una línea que deja el total por debajo de lo pagado; cancelar un pedido que Stripe ya cobró (409 y registro del cobro); cobro con tarjeta solo por el resto; reembolsos con descuento prorrateado y sin el envío; ningún pedido con reembolsos por más de lo cobrado

Categoría	Tipo de defecto	Ejemplos verificados
Seguridad	Accesos indebidos, exposición de datos o abuso de servicios	Permisos por puesto y por sucursal en cada recurso; sesiones cerradas al cambiar contraseña, puesto o estado; enlaces y códigos de un solo uso guardados como hash; direcciones de correo que podrían alterar las cabeceras; escape de HTML en correos; variantes de ruta que eludían el límite de intentos; cuerpos comprimidos; detalle de salud y registro de errores sin secretos ni consultas
Interfaz	Comportamientos de las pantallas que contradecían las reglas o confundían al usuario	Botones que no deben mostrarse según el puesto; puesto y estado bloqueados al editarse a sí mismo; explicación cuando una acción no está permitida (tipo en uso, sucursal con historial, plazo de devolución vencido); detección de stock cambiado durante una baja; vistas móviles sin desplazamiento horizontal; ausencia de errores de JavaScript

La revisión adversarial no garantiza la ausencia de defectos; reduce la probabilidad de los que se pueden encontrar leyendo el código con intención crítica, y las verificaciones de regresión aseguran que los ya corregidos sigan corregidos.

---

10. Cómo agregar una suite

Las convenciones que el ejecutor espera se deducen de `tests/correr.js` y de las suites existentes:

1. Crear un archivo `tests/api/<nombre>.test.js` o `tests/navegador/<nombre>.test.js`; el ejecutor toma cualquier archivo terminado en `.test.js`.
2. Elegir puertos propios que no usen otras suites ni los de desarrollo (8080, 8081 y 8082), y abortar si están ocupados.
3. Conectarse al PostgreSQL de pruebas (`localhost:54329`, usuario `tester`, base `distrib_test`), vaciar el esquema y arrancar `server.js` con variables de entorno de prueba, sin depender de archivos `.env`.
4. Usar servicios simulados: el Stripe falso o la clave del modo simulado, `tests/soporte/cloudinary-falso.js` y `CORREO_MODO=archivo` con una carpeta dentro de `tests/salidas/`.
5. Registrar cada comprobación con una función `check(nombre, condición, información)` que imprima `ok` o `FAIL`.
6. Imprimir al final la línea `RESULTADO: <n> ok, <m> fallos` y terminar con código distinto de cero si hubo fallos.
7. Detener en un bloque `finally` todos los procesos arrancados.

---

11. Documentos relacionados

- Arquitectura: componentes, migraciones al arrancar y servicios externos.
- Modelo de datos: tablas que las pruebas consultan para verificar el estado.
- Roles y permisos: permisos por puesto y alcance por sucursal que las suites comprueban.
- Reglas de negocio: pedidos, pagos, devoluciones, reembolsos, cupones, envío, IVA y vencimientos.

- API: endpoints y colección de Postman.
- Seguridad: medidas cuyo funcionamiento se verifica en las suites.
- Despliegue y operación: migraciones, respaldos y monitoreo en producción.

Despliegue y operación

Este documento describe cómo está publicado el sistema Distribuidora en Render, el procedimiento para volver a publicarlo desde cero o para publicar una actualización, todas las variables de entorno que el sistema reconoce, las limitaciones del plan gratuito y las tareas de operación: migraciones de la base de datos, respaldos y restauración, monitoreo, tareas periódicas del servidor y solución de los problemas más frecuentes. Todo lo descrito corresponde al código del repositorio (`server.js` , `app/utils/migrador.js` , `app/utils/esquema.js` , `app/migraciones/` , `scripts/` , `app/utils/monitoreo.js` , `app/controllers/salud.controller.js` y la configuración de los dos frontends).

Contenido

1. Arquitectura de despliegue
2. Requisitos previos
3. Publicación en Render paso a paso
4. Variables de entorno
5. Limitaciones del plan gratuito
6. Migraciones de la base de datos
7. Respaldos y restauración
8. Monitoreo
9. Tareas periódicas
10. Solución de problemas frecuentes
11. Documentos relacionados

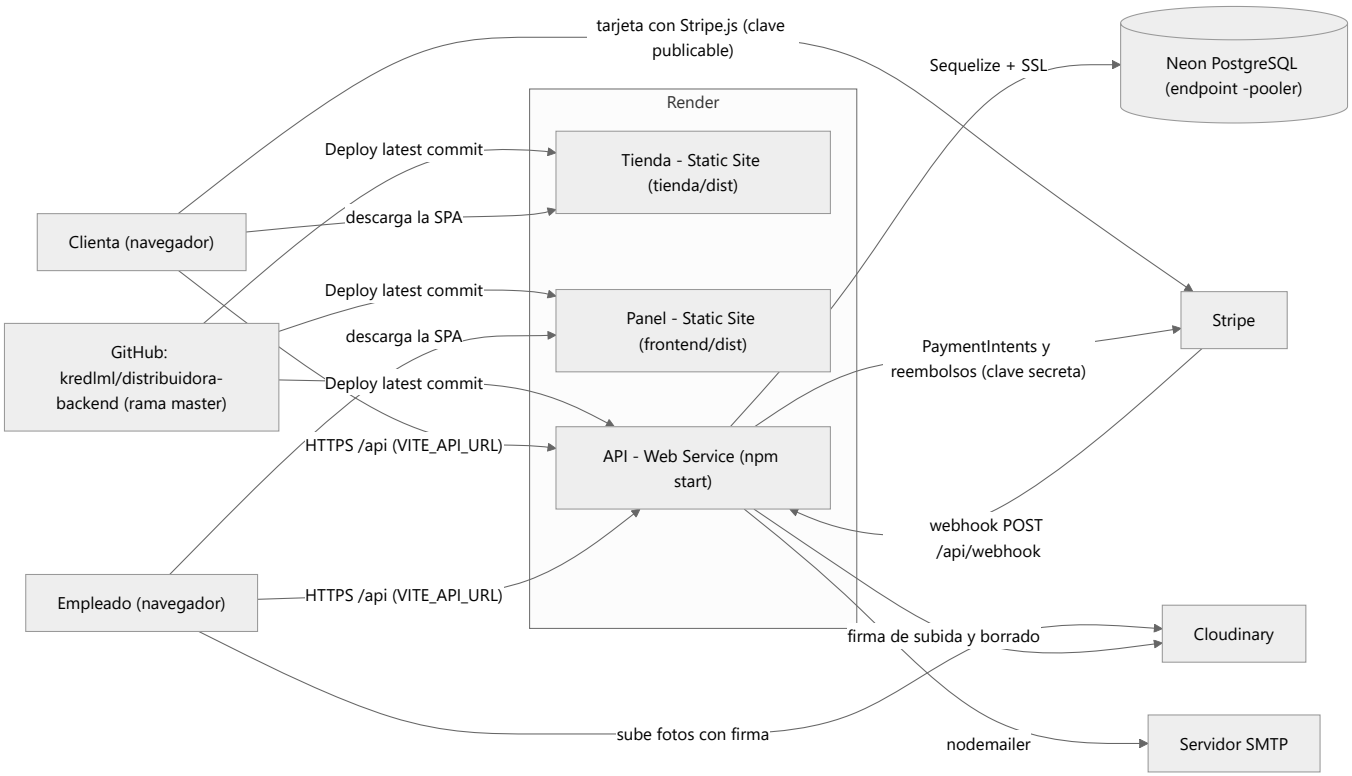
---

1. Arquitectura de despliegue

El sistema se publica como tres servicios de Render construidos a partir del mismo repositorio público (`https://github.com/kredlml/distribuidora-backend` , rama `master`). Cada servicio usa una carpeta distinta del repositorio como directorio raíz.

Servicio	Tipo en Render	Directorio raíz	Dirección pública
API REST	Web Service (Node)	raíz del repositorio	https://distribuidora-api-r6rw.onrender.com
Panel de empleados	Static Site	`frontend`	https://distribuidora-panel.onrender.com
Tienda en línea	Static Site	`tienda`	https://distribuidora-tienda.onrender.com

Los dos frontends son archivos estáticos (HTML, CSS y JavaScript generados por `vite build`) que el navegador descarga y que después se comunican directamente con la API. La API es el único componente que se conecta a la base de datos y que usa claves secretas de los servicios externos.

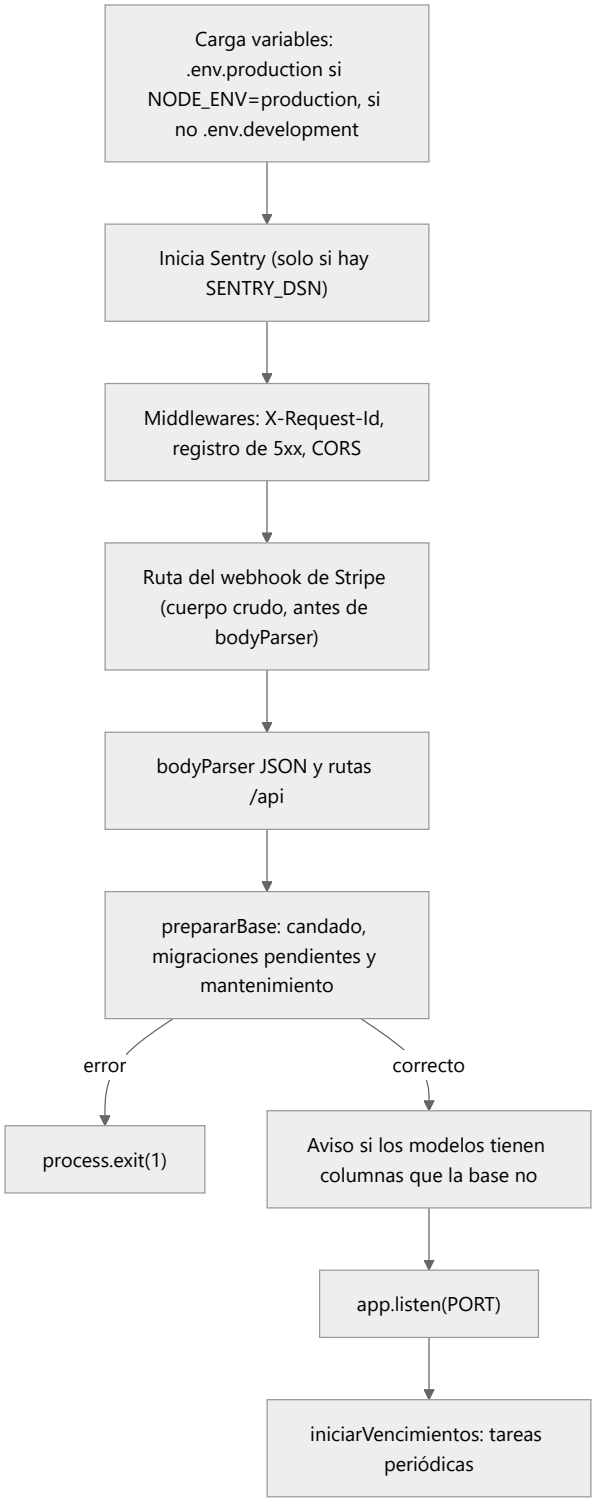


Puntos relevantes de esta arquitectura:

- **Neon con pooler.** La API se conecta al endpoint con pooler (PgBouncer en modo transacción) que Neon publica con el sufijo `-pooler` en el nombre del host. Por esa razón los candados de PostgreSQL del sistema son candados de transacción (`pg_advisory_xact_lock`), que funcionan detrás de un pooler, y los respaldos usan la conexión directa (ver Respaldos y restauración).
- **Proxy de Render.** `server.js` establece `app.set("trust proxy", 1)` para que `req.ip` sea la IP real del cliente y no la del proxy de Render; el límite de intentos de inicio de sesión y registro depende de ese valor.
- **Fotos.** El navegador del empleado sube la foto directamente a Cloudinary con una firma generada por la API (`app/config/cloudinary.config.js`); la clave secreta de Cloudinary nunca sale del servidor y la base solo guarda la URL.
- **Pagos.** La tienda usa Stripe.js con la clave publicable; la clave secreta y el secreto del webhook solo existen en la API.

1.1 Arranque de la API

Al iniciarse con `npm start` (`node server.js`), la API ejecuta la siguiente secuencia. Si la base de datos no queda al día, el proceso termina con código 1 y no atiende peticiones.



En Render no existe un archivo `.env.production`: `dotenv` no encuentra el archivo y las variables se toman de las definidas en la sección *Environment* del servicio. `dotenv` no sobrescribe variables que ya existen en el entorno.

---

2. Requisitos previos

Requisito	Uso
Cuenta de GitHub con acceso al repositorio público	Origen del código de los tres servicios.
Cuenta de Render	Hospedaje de la API, el panel y la tienda.
Proyecto en Neon con una base PostgreSQL	Base de datos. Se necesitan el host con pooler, el usuario, la contraseña y el nombre de la base.
Cuenta de Stripe (modo de pruebas)	Clave secreta `sk_test_…`, clave publicable `pk_test_…` y secreto del webhook `whsec_…`. Opcional: con la clave simulada `sk_test_clave_de_prueba_universidad` los cobros se aprueban al instante sin llamar a Stripe.
Cuenta de Cloudinary	Nombre de la nube, API key y API secret. Opcional: sin estas tres variables el panel solo permite pegar un enlace de imagen.
Servidor SMTP	Envío de correos a las clientas. Opcional: sin `SMTP_HOST` los correos solo se muestran en la consola.
Node.js 18 o superior	Express 5 lo requiere y el código usa `fetch` global (borrado de fotos en Cloudinary).
PostgreSQL cliente (`pg_dump`, `pg_restore`) en el equipo de operación	Solo para respaldos y restauración.

---

3. Publicación en Render paso a paso

El orden recomendado es: base de datos, API, frontends, enlace entre servicios (CORS y enlaces de correo), webhook de Stripe, primer usuario y verificación.

3.1 Base de datos en Neon

1. Crear un proyecto en Neon y una base de datos.
2. Copiar los datos de conexión del endpoint **con pooler** (host con `-pooler` en el nombre). Esos valores se usan en `DB_HOST`, `DB_USER`, `DB_PASSWORD` y `DB_NAME`.
3. No es necesario crear tablas: la API aplica las migraciones al arrancar por primera vez (ver Migraciones).

3.2 API (Web Service)

1. En Render, elegir **New > Web Service** y conectar el repositorio público `https://github.com/kredlml/distribuidora-backend`.
2. Configurar el servicio:

Campo de Render	Valor
Branch	`master`
Root Directory	(vacío: raíz del repositorio)

Campo de Render	Valor
Runtime	Node
Build Command	`npm install`
Start Command	`npm start`
Health Check Path (Advanced)	`/api/salud`
Instance Type	Free

`package-lock.json` está excluido en `.gitignore`, por lo que el comando de construcción es `npm install` y no `npm ci`.

3. En **Environment**, definir las variables de la API (la lista completa y sus valores por defecto están en Variables de entorno). Como mínimo:

Variable	Valor en producción
`NODE_ENV`	`production`
`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`	Datos del endpoint con pooler de Neon.
`JWT_SECRET`	Cadena larga y aleatoria. Firma las sesiones de empleados y clientas.
`CORS_ORIGIN`	`https://distribuidora-panel.onrender.com,https://distribuidora-tienda.onrender.com`
`TIENDA_URL`	`https://distribuidora-tienda.onrender.com`
`STRIPE_SECRET_KEY`	Clave secreta de Stripe (o la clave simulada).
`STRIPE_WEBHOOK_SECRET`	Secreto del webhook (se obtiene en el paso 3.5).
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Credenciales de Cloudinary.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `CORREO_REMITENTE`	Datos del servidor de correo.

`PORT` no se define: Render la asigna y `server.js` la lee (`process.env.PORT || 8080`).

4. Crear el servicio. En el registro (*Logs*) del primer despliegue deben aparecer las líneas `[migración] aplicada 0001-esquema-base.js …` hasta `0005-cupon-en-pedido.js` y, al final, `Servidor corriendo en el puerto … [Ambiente: production].`
5. Verificar `https://distribuidora-api-r6rw.onrender.com/api/salud`: debe responder con `"estado": "ok"` y `"base": "ok"`.

3.3 Panel y tienda (Static Sites)

Los dos frontends se publican igual; solo cambian el directorio raíz y las variables.

1. En Render, elegir **New > Static Site** y conectar el mismo repositorio.

2. Configurar cada sitio:

Campo de Render	Panel de empleados	Tienda en línea
Branch	`master`	`master`
Root Directory	`frontend`	`tienda`
Build Command	`npm install && npm run build`	`npm install && npm run build`
Publish Directory	`dist`	`dist`

3. Definir las variables de entorno de construcción:

Variable	Panel	Tienda	Valor
`VITE_API_URL`	Sí	Sí	`https://distribuidora-api-r6rw.onrender.com` (sin `/` final; el código la elimina si se escribe).
`VITE_STRIPE_PUBLISHABLE_KEY`	No	Sí	Clave publicable `pk_test_…` de la misma cuenta que `STRIPE_SECRET_KEY`. Puede quedar vacía si la API usa la clave simulada.

Las variables `VITE_*` se incorporan al código durante `vite build`: cambiarlas no tiene efecto hasta volver a construir el sitio.

4. En la pestaña **Redirects/Rewrites** de cada sitio, agregar la regla:

Source	Destination	Action
`/*`	`/index.html`	Rewrite

Ambas aplicaciones usan `BrowserRouter` de React Router: rutas como `/pedidos/15` o `/perfil` no existen como archivos y, sin esta regla, Render responde 404 al recargar la página o al abrir un enlace directo.

`vite.config.js` de ambos frontends define además `preview: { host: true, allowedHosts: true }`, lo que permite, como alternativa, servirlos como Web Service con `npm run preview` bajo un dominio `*.onrender.com`. La forma recomendada es Static Site, que no consume horas de instancia ni se suspende.

3.4 Enlace entre servicios: CORS_ORIGIN y TIENDA_URL

Una vez creados los dos sitios se conocen sus direcciones definitivas, que deben registrarse en la API:

- **`CORS_ORIGIN`**: lista separada por comas de los orígenes que pueden llamar a la API. Cada origen se compara de forma exacta (esquema y dominio, sin `/` final); los espacios alrededor de las comas se eliminan. Si no se define, la API solo acepta `http://localhost:8081` y `http://localhost:8082`.
- **`TIENDA_URL`**: dirección pública de la tienda. Los botones de los correos (seguimiento del pedido, restablecer contraseña) apuntan a ella. En producción, si falta, la API escribe en el registro `[correo] ATENCIÓN: falta TIENDA_URL…`.

Después de guardar los cambios, la API debe volver a desplegarse para que los lea.

3.5 Webhook de Stripe

El webhook es necesario cuando se usa una clave real de Stripe (`sk_test_…` o `sk_live_…`). Con la clave simulada no se usa.

1. En el panel de Stripe, ir a **Developers > Webhooks > Add endpoint**.
2. URL del endpoint: `https://distribuidora-api-r6rw.onrender.com/api/webhook` .
3. Seleccionar los eventos que procesa `app/controllers/webhook.controller.js` :

Evento	Efecto en el sistema
`payment_intent.succeeded`	Registra el cobro exitoso del pago (estado PENDIENTE o FALLIDO pasa a COMPLETADO) y el pedido avanza.
`payment_intent.payment_failed`	Deja constancia en la auditoría (`PAGO_INTENTO_FALLIDO`); el pago sigue PENDIENTE porque la clienta puede reintentar con otra tarjeta.
`payment_intent.canceled`	Marca como FALLIDO el pago que seguía PENDIENTE, porque ese PaymentIntent ya no podrá cobrarse.
`refund.created` , `refund.updated` , `refund.failed` , `charge.refund.updated`	Vuelve a consultar en Stripe el reembolso pendiente correspondiente y actualiza su estado.

4. Copiar el **Signing secret** (`whsec_…`) en la variable `STRIPE_WEBHOOK_SECRET` de la API y volver a desplegarla.

La ruta del webhook se registra en `server.js` antes de `bodyParser.json` y usa `express.raw` , porque la firma de Stripe se valida sobre el cuerpo sin procesar. Una firma inválida se responde con 400.

3.6 Primer usuario DISTRITAL

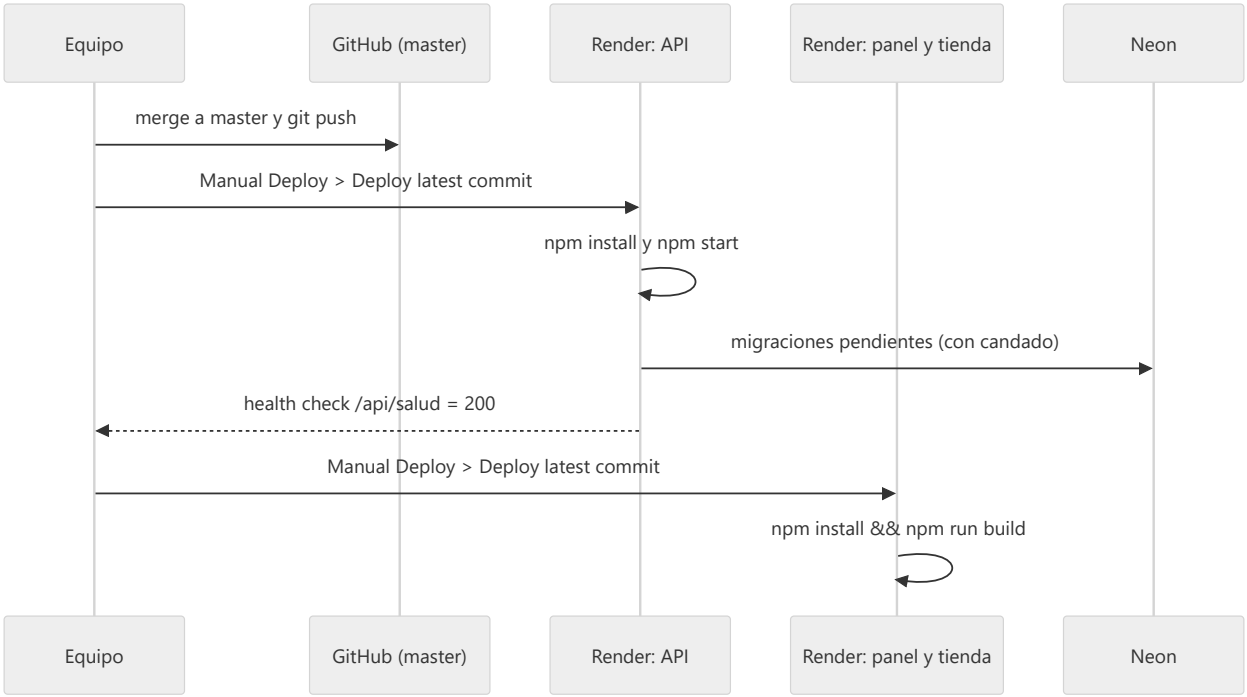
Con la base vacía, la ruta `POST /api/auth/registro` acepta un registro sin sesión y el primer empleado registrado queda siempre con puesto DISTRITAL y sin sucursal (`app/controllers/auth.controller.js`). Se puede enviar desde la colección de Postman (`postman/`) con `nombre` , `apellido` , `email` y `password` . A partir de ese momento los demás empleados se registran desde la pantalla Empleados del panel. Además, si no existe ningún DISTRITAL activo, en cada arranque el mantenimiento promueve a DISTRITAL a los ADMIN sin sucursal o, si no hay, al ADMIN activo más antiguo (ver Tareas periódicas).

3.7 Verificación

Verificación	Resultado esperado
`GET https://distribuidora-api-r6rw.onrender.com/api/salud`	`{"estado":"ok","base":"ok",…}`
Abrir la tienda y recargar una ruta interna	La página carga (regla de reescritura activa).
Iniciar sesión en el panel	Sin errores de CORS en la consola del navegador.
Panel > Estado del sistema (DISTRITAL)	Base "Responde", 0 migraciones pendientes y los servicios con el estado esperado.

Verificación	Resultado esperado
Compra de prueba con la tarjeta `4242 4242 4242 4242` (modo de pruebas de Stripe)	El pedido pasa a pagado; en Stripe el evento del webhook figura como entregado.

3.8 Publicar una actualización



Se despliega primero la API, para que las migraciones y los endpoints nuevos estén disponibles antes de que los frontends los usen. Antes de una actualización con migraciones se recomienda un respaldo (`npm run respaldo`).

---

4. Variables de entorno

La plantilla de la API es `.env.example` (sin valores reales). En desarrollo se copia a `.env.development` ; en un servidor propio, a `.env.production` . Ambos archivos están excluidos en `.gitignore` . En Render las variables se definen en la sección *Environment* de cada servicio.

4.1 API

Variable	Descripción	Valor por defecto
Servidor		
`NODE_ENV`	Ambiente. Con `production` se carga `.env.production` y se activan los avisos de correo sin configurar.	`development` (se lee `.env.development`)

Variable	Descripción	Valor por defecto
`PORT`	Puerto HTTP. Render lo asigna.	`8080`
`CORS_ORIGIN`	Orígenes permitidos (panel y tienda), separados por coma.	`http://localhost:8081,http://localhost:8082`
Base de datos		
`DB_HOST`	Host de PostgreSQL (en Neon, el endpoint con `-pooler`).	— (obligatoria)
`DB_PORT`	Puerto de PostgreSQL.	`5432`
`DB_USER`	Usuario.	— (obligatoria)
`DB_PASSWORD`	Contraseña.	—
`DB_NAME`	Nombre de la base.	— (obligatoria)
`DB_DIALECT`	Dialecto de Sequelize.	`postgres`
`DB_SSL`	SSL activado salvo que valga `false` (bases locales sin SSL).	activado
`DB_POOL_MAX`	Conexiones máximas del pool de Sequelize.	`5`
`DB_LOG_SQL`	`true` muestra cada consulta SQL en consola (depuración).	`false`
`DB_HOST_DIRECTO`	Host sin pooler para `pg_dump` / `pg_restore`. En Neon se deduce quitando `-pooler` del host.	deducido de `DB_HOST`
Sesiones		
`JWT_SECRET`	Secreto con el que se firman y verifican los tokens de empleados y clientas.	— (obligatoria)
`JWT_EXPIRES_IN`	Presente en `.env.example` y en `app/config/auth.config.js`; en la versión actual los tokens se emiten con una vigencia fija de 86400 s (24 h) en `app/utils/empleados.js` y `app/controllers/tienda_auth.controller.js`, por lo que esta variable no altera la vigencia.	`86400` en la plantilla
Stripe		
`STRIPE_SECRET_KEY`	Clave secreta. El valor `sk_test_clave_de_prueba_universidad` activa el modo simulado (cobros aprobados al instante sin llamar a Stripe).	—
`STRIPE_WEBHOOK_SECRET`	Secreto de firma del webhook (`whsec_…`).	—
`STRIPE_API_HOST`, `STRIPE_API_PORT`, `STRIPE_API_PROTOCOL`	Opcionales: apuntan el cliente de Stripe a un servidor de pruebas (p. ej. stripe-mock). No van en producción.	API real de Stripe
Cloudinary		

Variable	Descripción	Valor por defecto
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Credenciales para firmar subidas y borrar fotos de la carpeta `distribuidora/prendas`. Sin las tres, el panel solo admite pegar un enlace.	—
`CLOUDINARY_API_BASE`	Opcional: base de la API de Cloudinary (usada por las pruebas).	`https://api.cloudinary.com`
Correo		
`SMTP_HOST`	Servidor SMTP. Sin él no se envía ningún correo.	—
`SMTP_PORT`	Puerto SMTP; con `465` se usa conexión segura directa.	`587`
`SMTP_USER`, `SMTP_PASS`	Credenciales SMTP (si no hay usuario, se conecta sin autenticación).	—
`CORREO_REMITENTE`	Remitente de los correos.	`Distribuidora <no-responder@distribuidora.local>`
`TIENDA_URL`	Dirección pública de la tienda para los enlaces de los correos.	`http://localhost:8082`
`CORREO_MODO`	`smtp`, `consola` o `archivo` (guarda cada correo como JSON; lo usan las pruebas).	`smtp` si hay `SMTP_HOST`; si no, `consola`
`CORREO_CARPETA`	Carpeta del modo `archivo`.	`correos/` en el directorio de trabajo
Reglas del negocio		
`HORAS_PLAZO_DEVOLUCION`	Horas desde la entrega para solicitar un cambio o devolución (valores menores o iguales a 0 se ignoran).	`24`
`PLAZO_PEDIDO_PENDIENTE_HORAS`	Horas para pagar un pedido pendiente antes de cancelarlo; `0` desactiva el vencimiento.	`24`
`VENCIMIENTO_INTERVALO_MIN`	Minutos entre revisiones de pedidos vencidos y reembolsos pendientes.	`10`
`ENVIO_COSTO`	Costo del envío a domicilio de la tienda en línea.	`25`
`ENVIO_GRATIS_DESDE`	Monto (después del descuento) desde el que el envío es gratis; `0` = nunca gratis.	`300`
`IVA_PORCENTAJE`	IVA incluido en los precios, desglosado en pedidos y reportes.	`12`
`ZONA_HORARIA_NOMBRE`	Zona horaria de reportes y vigencia de cupones.	`America/Guatemala`

Variable	Descripción	Valor por defecto
`ZONA_HORARIA_TIENDA`	Desfase horario usado en los filtros por fecha de pedidos.	`-06:00`
Monitoreo		
`SENTRY_DSN`	DSN de Sentry. Requiere el paquete `@sentry/node`, que no forma parte de las dependencias.	— (desactivado)
Respaldos		
`PG_BIN`	Carpeta `bin` de PostgreSQL si `pg_dump` no está en el PATH. En Windows se busca la versión más nueva en `C:/Program Files/PostgreSQL`.	búsqueda automática
`RESPALDOS_DIR`	Carpeta de los respaldos.	`respaldos/` en la raíz del proyecto
`RESPALDOS_MAX`	Cantidad de respaldos normales que se conservan (mínimo 1).	`14`

4.2 Panel y tienda

Ambos frontends leen su configuración en `src/config/env.js`; las plantillas son `frontend/.env.example` y `tienda/.env.example`.

Variable	Aplicación	Descripción	Valor por defecto
`VITE_API_URL`	Panel y tienda	URL base de la API, sin `/` final.	`http://localhost:8080`
`VITE_STRIPE_PUBLISHABLE_KEY`	Tienda	Clave publicable de Stripe de la misma cuenta que `STRIPE_SECRET_KEY`. Puede quedar vacía con la clave simulada.	vacía

Ninguna clave secreta debe definirse con prefijo `VITE_`: todo lo que lleva ese prefijo queda visible en el código que descarga el navegador.

---

5. Limitaciones del plan gratuito

Limitación	Consecuencia en el sistema	Cómo se maneja
El Web Service gratuito se suspende tras 15 minutos sin peticiones.	La primera petición después de la suspensión tarda en responder mientras la instancia vuelve a arrancar (incluye la revisión de migraciones). El panel o la tienda pueden mostrar un error de conexión en ese intervalo.	Abrir `/api/salud` antes de una demostración y esperar la respuesta; luego usar el sistema normalmente.
Mientras la API está suspendida no se ejecuta ningún temporizador.	Los pedidos vencidos y los reembolsos pendientes no se procesan durante la suspensión.	Se procesan en la primera revisión tras el arranque, unos 5 segundos después de que la API vuelve a escuchar (ver Tareas periódicas).

Limitación	Consecuencia en el sistema	Cómo se maneja
Despliegues manuales.	Un `git push` a `master` no publica por sí solo en este proyecto.	Cada servicio se actualiza con **Manual Deploy > Deploy latest commit**, empezando por la API.
Sistema de archivos efímero.	Lo escrito en disco por la API (p. ej. correos en modo `archivo`) se pierde en cada reinicio o despliegue.	En producción se usa `CORREO_MODO smtp`; los respaldos se generan desde un equipo de operación, no en Render.

Los Static Sites no se suspenden: el panel y la tienda cargan de inmediato aunque la API esté despertando.

---

6. Migraciones de la base de datos

El esquema de la base se maneja con migraciones controladas mediante Umzug (`app/utils/migrador.js`). Reemplazan a `sequelize.sync({ alter: true })`, que en cada arranque comparaba modelos y base y podía borrar columnas o cambiar tipos sin aviso.

6.1 Funcionamiento

- Cada archivo de `app/migraciones/` cuyo nombre cumple el patrón `NNNN-nombre.js` (cuatro dígitos, guion y nombre) es una migración. Se ordenan por nombre y cada una se aplica una sola vez.
- Las migraciones aplicadas quedan anotadas en la tabla `SequelizeMeta` (una fila por nombre de archivo).
- El servidor aplica las pendientes en cada arranque, antes de escuchar peticiones (`prepararBase`). Si una falla, el servidor no arranca.
- **Candado.** Todo ocurre bajo un candado de transacción de PostgreSQL (`pg_advisory_xact_lock` con la clave `74210531`) tomado en una conexión dedicada. Si arrancan dos instancias a la vez, la segunda espera a la primera hasta 300 segundos (`lock_timeout`). Al ser un candado de transacción, se libera con `COMMIT` y funciona detrás del pooler de Neon.
- Bajo el mismo candado se ejecuta el mantenimiento de datos (`app/utils/mantenimiento.js`).
- Después se comparan los modelos con la base (`diferencias` en `app/utils/esquema.js`). Si un modelo tiene una columna o un valor de ENUM que la base no tiene, se escribe en el registro `[migración] ATENCIÓN: los modelos tienen columnas que la base no: …`. La base no se modifica en ese caso.

Migración	Contenido
`0001-esquema-base.js`	Línea base: crea las tablas, columnas y valores de ENUM que falten a partir de los modelos, e índices únicos sin distinguir mayúsculas (correo de empleado, correo de cliente y nombre de tipo de prenda). No se puede deshacer.
`0002-intentos-limite.js`	Tabla `IntentoLimite` para el límite de intentos.

Migración	Contenido
`0003-errores-servidor.js`	Tabla `ErrorServidor` para el registro de errores.
`0004-envio-cupones.js`	Tabla `Cupon` y columnas `costo_envio`, `descuento` e `id_cupon` en `Pedido`.
`0005-cupon-en-pedido.js`	Copia del cupón en el pedido (`cupon_tipo`, `cupon_valor`, `cupon_minimo`) y relleno de los pedidos existentes.

6.2 Comandos

Los comandos usan la misma configuración que el servidor: `.env.production` si `NODE_ENV=production`; si no, `.env.development`.

Comando	Acción
`npm run migrar`	Aplica las migraciones pendientes y el mantenimiento (lo mismo que hace el servidor al arrancar).
`npm run migrar:estado`	Lista las migraciones aplicadas y pendientes, y avisa si algún modelo tiene columnas sin migración.
`node scripts/migrar.js deshacer`	Revierte la **última** migración aplicada, si tiene función `down`. Conviene hacer un respaldo antes. La línea base `0001` no se puede deshacer.

En PowerShell, para operar sobre la configuración de producción: `$env:NODE_ENV="production"; npm run migrar:estado`.

6.3 Escribir una migración nueva

Toda migración debe ser **idempotente**: debe poder ejecutarse dos veces sin fallar. En una base nueva la línea base `0001` ya crea el esquema de los modelos actuales y después se ejecutan las demás, que encontrarán sus columnas ya creadas. Para eso `app/utils/esquema.js` ofrece ayudas "si no existe":

Ayuda	Efecto
`asegurarColumna(sequelize, tabla, campo, definicion)`	Agrega la columna solo si no existe.
`asegurarValoresEnum(sequelize, tabla, campo, valores)`	Agrega a la columna ENUM los valores que falten.
`asegurarModelo(sequelize, modelo)`	Crea la tabla de un modelo o le agrega las columnas que falten. Nunca borra ni cambia tipos.

Procedimiento:

1. Modificar el modelo en `app/models/`.

2. Crear el archivo con el número siguiente; por ejemplo, para una columna hipotética `horario` en `Sucursal`, `app/migraciones/0006-horario-sucursal.js`:

```
// Horario de atención de la sucursal (ejemplo). Idempotente.
const { asegurarColumna } = require("../utils/esquema");

exports.up = async ({ sequelize, Sequelize }) => {
  await asegurarColumna(sequelize, "Sucursal", "horario", { type: Sequelize.STRING(100), allowNull:
true });
};

exports.down = async ({ sequelize }) => {
  await sequelize.query(`ALTER TABLE "Sucursal" DROP COLUMN IF EXISTS horario`);
};
```

3. Ejecutar `npm run migrar:estado` para ver la migración como pendiente, y `npm run migrar` (o reiniciar el servidor) para aplicarla.

Reglas a respetar:

- La función `up` recibe `{ db, sequelize, qi, Sequelize }` (modelos, conexión, `QueryInterface` y la clase Sequelize).
- Una columna `NOT NULL` agregada a una tabla con datos necesita valor por defecto.
- Las migraciones no se ejecutan dentro de una transacción (`ALTER TYPE … ADD VALUE` no lo permite en versiones antiguas de PostgreSQL).
- Una migración ya aplicada en producción no se edita: cualquier corrección va en una migración nueva.
- `down` es opcional; sin ella, `deshacer` responde con un error.

---

7. Respaldos y restauración

Los scripts `scripts/respaldo.js` y `scripts/restaurar.js` usan `pg_dump` y `pg_restore` con la misma configuración de base que el servidor (`scripts/pg.js`). La contraseña se entrega por la variable `PGPASSWORD`, nunca en la línea de comandos, y el modo SSL sigue a `DB_SSL`.

Conexión directa. `pg_dump` y `pg_restore` no deben pasar por un pooler en modo transacción, porque sus ajustes de sesión podrían aplicarse en otra conexión. Por eso los scripts usan `DB_HOST_DIRECTO` si está definido o, si el host es de Neon (`*.neon.tech`) y contiene `-pooler.`, el mismo host sin `-pooler`. Si el host resultante sigue pareciendo un pooler, se muestra un aviso.

La versión de `pg_dump` del equipo debe ser igual o más reciente que la versión de PostgreSQL de la base.

7.1 Respaldo

```
npm run respaldo
```

- Genera `respaldos/<base>-AAAAMMDD-HHMMSS.dump` en formato *custom* de `pg_dump` (comprimido, con `--no-owner --no-privileges`).
- El archivo se escribe primero como `.dump.parcial` y solo se renombra al terminar correctamente, de modo que un respaldo interrumpido no se confunde con uno válido.
- **Rotación:** se conservan los últimos `RESPALDOS_MAX` respaldos normales de esa base (14 por defecto); los más antiguos se borran. También se eliminan restos `.parcial` de más de 6 horas. Los respaldos con prefijo `antes-de-restaurar-` no entran en la rotación.
- La carpeta `respaldos/` está excluida del repositorio.

Para respaldar la base de producción desde PowerShell: `$env:NODE_ENV="production"; npm run respaldo` (requiere un `.env.production` local con los datos de Neon).

7.2 Programar el respaldo diario en Windows

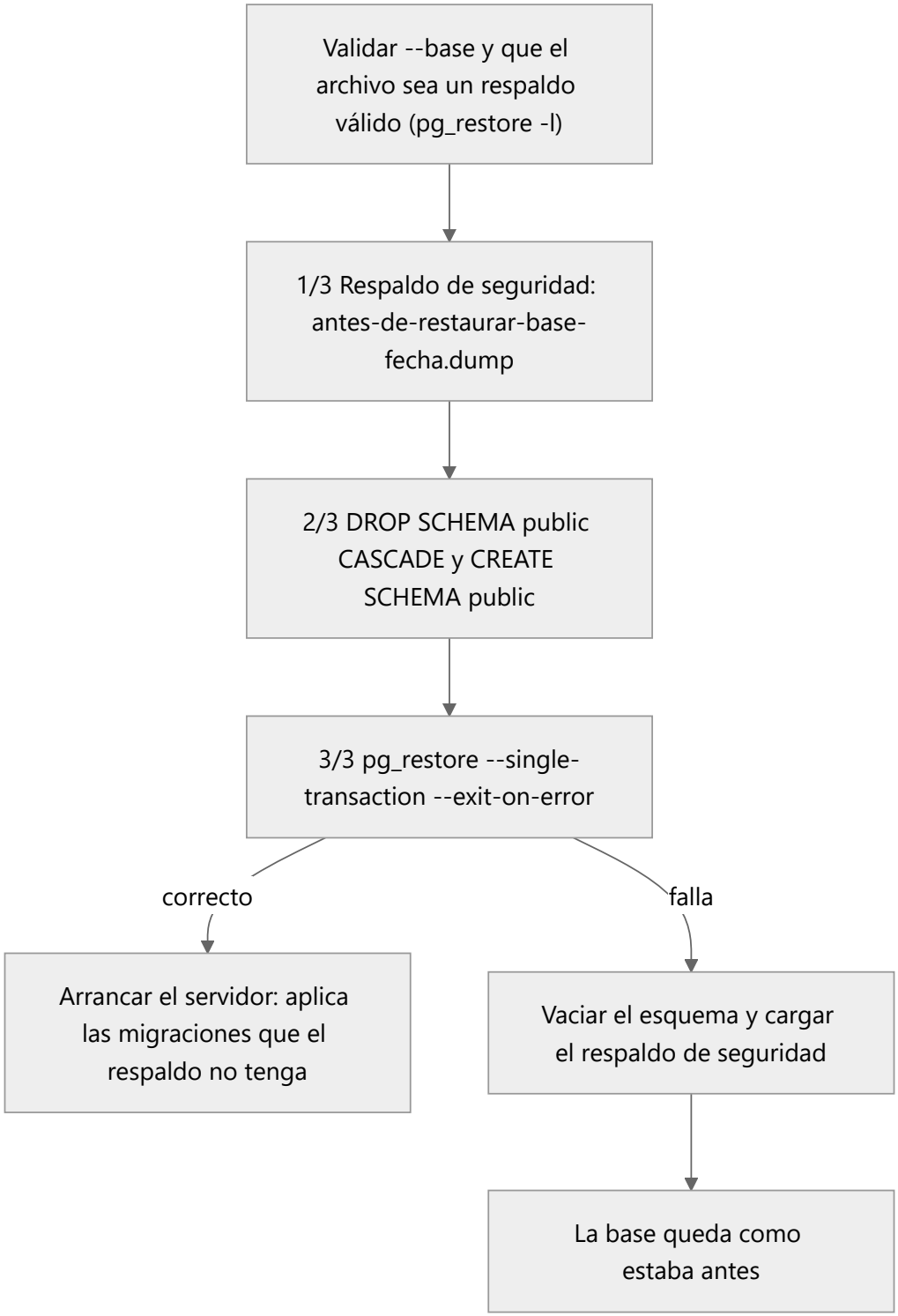
```
schtasks /Create /SC DAILY /ST 23:00 /TN "Respaldo distribuidora" /TR "cmd /c cd /d <carpeta del proyecto> && npm run respaldo"
```

La tarea se ejecuta todos los días a las 23:00 en el equipo donde se registra; ese equipo debe estar encendido y tener `pg_dump` disponible.

7.3 Restauración

```
npm run restaurar -- respaldos/<archivo>.dump --base <DB_NAME>
```

La restauración **reemplaza todo el contenido** de la base. El parámetro `--base` debe coincidir con `DB_NAME` de la configuración activa; si no coincide, el script termina sin tocar nada. Antes de ejecutarla se debe detener el servidor.



Al vaciar el esquema también se elimina lo que el respaldo no contiene (tablas nuevas y la propia `SequelizeMeta`), por lo que, al arrancar, el servidor detecta y aplica las migraciones posteriores al respaldo.

---

8. Monitoreo

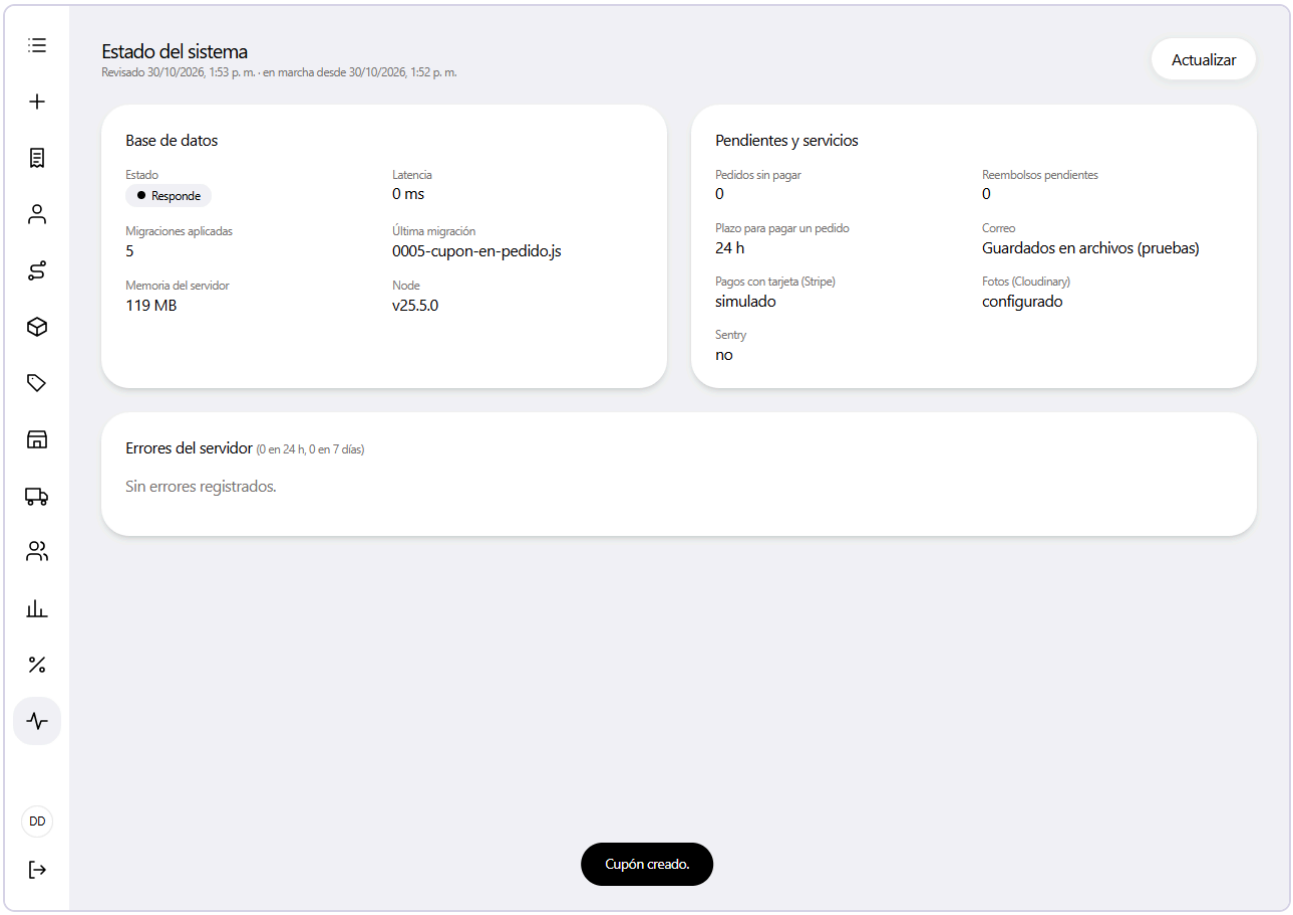
8.1 Endpoints de salud

Endpoint	Acceso	Respuesta
`GET /api/salud`	Público (lo usa el health check de Render)	200 con `estado`, `base`, `latencia_base_ms` y `hora` si la base responde a `SELECT 1`; 503 con `"estado": "degradado"` si no. No expone datos internos.
`GET /api/salud/detalle`	DISTRITAL	Hora de arranque, versión de Node, memoria, latencia de la base, migraciones (aplicadas, pendientes, última), columnas faltantes, errores de las últimas 24 h y 7 días, pedidos y reembolsos pendientes, reembolsos fallidos y estado de los servicios (correo, Stripe, Cloudinary, Sentry, plazo de vencimiento).
`GET /api/salud/errores?pagina=N`	DISTRITAL	Errores registrados en `ErrorServidor`, del más reciente al más antiguo, paginados.

El servicio Stripe se informa como `simulado`, `real (pruebas)`, `real (producción)` (clave `sk_live_`) o `sin configurar`.

8.2 Pantalla Estado del sistema

El panel ofrece al DISTRITAL la pantalla **Estado del sistema** (`frontend/src/pages/Sistema/Sistema.jsx`), que consulta los dos endpoints protegidos y muestra avisos cuando la base no responde, hay migraciones sin aplicar, faltan columnas en la base, no hay SMTP configurado o existen reembolsos rechazados por Stripe que esperan entrega en efectivo.



Pantalla Estado del sistema del panel con el estado de la base, migraciones, pendientes, servicios y errores del servidor

Estado del sistema: la base responde, hay 5 migraciones aplicadas (última `0005-cupon-en-pedido.js`), no hay pedidos sin pagar ni reembolsos pendientes, el plazo para pagar es de 24 h, el correo está en modo archivo (pruebas), Stripe en modo simulado, Cloudinary configurado, Sentry desactivado y no hay errores registrados.

8.3 Registro de errores (ErrorServidor)

`app/utils/monitoreo.js` instala en `server.js` las siguientes piezas:

Pieza	Función
`idDePeticion`	Asigna un UUID a cada petición y lo devuelve en la cabecera `X-Request-Id` (expuesta por CORS para que los frontends puedan leerla).
`registrarRespuestas5xx`	Guarda en `ErrorServidor` toda respuesta con estado 500 o superior: fecha, id de petición, método, ruta sin la cadena de consulta, estado, mensaje, pila (si existe) y usuario (`empleado:N` o `cliente:N`). Nunca guarda el cuerpo de la petición, que puede contener contraseñas.
`noEncontrado`	Responde en JSON a las rutas `/api` inexistentes (404).
`manejadorDeErrores`	Convierte los errores no controlados en respuestas JSON: 400 para JSON inválido, 415 para cuerpos comprimidos, 413 para cuerpos demasiado grandes y 500 con el texto `Error interno del servidor (referencia <id>)`. La pila nunca se envía al cliente.

Pieza	Función
`vigilarProceso`	Registra promesas rechazadas sin manejar y excepciones no controladas. Ante una excepción no controlada registra el error y termina el proceso, que Render vuelve a iniciar.

La referencia que recibe el usuario en un error 500 coincide con el `request_id` guardado, lo que permite localizar el registro exacto en la pantalla Estado del sistema. Los registros de más de 30 días se eliminan automáticamente.

8.4 Sentry (opcional)

Si se define `SENTRY_DSN`, el servidor inicia Sentry antes que Express y envía cada error registrado como mensaje de nivel `error`, con el `request_id` como etiqueta, sin datos personales (`sendDefaultPii: false`) y sin trazas de rendimiento. El paquete `@sentry/node` no está en las dependencias: para activarlo en Render se debe agregar al `package.json` (`npm install @sentry/node`) y volver a desplegar. Si el DSN está definido pero falta el paquete, el servidor sigue funcionando y lo advierte en el registro.

8.5 Registro de Render

Los mensajes de operación aparecen en *Logs* del servicio de la API con prefijos que facilitan su búsqueda: `[migración]`, `[mantenimiento]`, `[vencimientos]`, `[reembolsos]`, `[correo]`, `[monitoreo]` y `[cloudinary]`.

---

9. Tareas periódicas

La API ejecuta tareas en segundo plano sin servicios externos de programación. Todas se detienen mientras la instancia de Render está suspendida.

Tarea	Frecuencia	Detalle	Código
Mantenimiento de datos	En cada arranque, después de las migraciones	Completa `fecha_entrega` de pedidos ENTREGADOS antiguos a partir de la auditoría; garantiza que exista un DISTRITAL activo (promueve a los ADMIN sin sucursal o, si no hay, al ADMIN activo más antiguo, y lo registra en la auditoría); asigna sucursal a clientes de mostrador que no la tenían.	`app/utils/mantenimiento.js`
Vencimiento de pedidos pendientes	Cada `VENCIMIENTO_INTERVALO_MIN` minutos (10); la primera revisión unos 5 s después del arranque	Cancela hasta 200 pedidos PENDIENTES por revisión cuyo plazo (`PLAZO_PEDIDO_PENDIENTE_HORAS`) venció, libera el stock reservado, cancela el cobro iniciado en Stripe y notifica a la clienta. Omite pedidos con algún pago completado, con pagos de caja por confirmar o con un cobro con tarjeta iniciado hace menos de 30 minutos; si Stripe ya había cobrado, registra el cobro en lugar de cancelar.	`app/utils/vencimientos.js`
Reintento de reembolsos a tarjeta	En la misma revisión que el vencimiento	Vuelve a consultar en Stripe hasta 50 reembolsos PENDIENTES creados hace más de 5 minutos.	`app/utils/reembolsos.js`

Tarea	Frecuencia	Detalle	Código
Limpieza de `ErrorServidor`	Cada 6 horas	Elimina los errores de más de 30 días.	`app/utils/monitoreo.js`
Limpieza de `IntentoLimite`	Cada hora, desde el primer uso del limitador	Elimina los contadores de intentos de más de un día.	`app/middlewares/limitador.js`

La revisión de vencimientos y reembolsos usa un candado de transacción con la clave `74210532` en modo "intentar": si hubiera varias instancias de la API, solo una revisa y las demás no esperan, sino que lo intentan en la siguiente vuelta.

---

10. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
En la consola del navegador aparece *blocked by CORS policy* y el panel o la tienda no cargan datos.	El origen del sitio no está en `CORS_ORIGIN`, tiene `/` final, usa `http` en lugar de `https`, o la API no se volvió a desplegar después del cambio.	Definir `CORS_ORIGIN=https://distribuidora-panel.onrender.com,https://distribuidora-tienda.onrender.com` (sin `/` final) y volver a desplegar la API.
Al recargar una página interna del panel o la tienda aparece *Not Found* (404).	Falta la regla de reescritura del Static Site.	Agregar en *Redirects/Rewrites* la regla `/*` → `/index.html` con acción **Rewrite**.
Un cambio subido a GitHub no aparece en el sistema publicado.	El cambio está en otra rama (por ejemplo `documentacion`) o no se ejecutó el despliegue manual.	Verificar que el servicio use la rama `master`, unir el cambio a `master` y ejecutar **Manual Deploy > Deploy latest commit**.
La primera petición tarda mucho o el panel muestra un error de conexión y luego funciona.	La API estaba suspendida por inactividad (plan gratuito).	Abrir `https://distribuidora-api-r6rw.onrender.com/api/salud`, esperar la respuesta y reintentar.
Las variables `VITE_API_URL` o `VITE_STRIPE_PUBLISHABLE_KEY` se cambiaron, pero el sitio sigue usando los valores anteriores.	Las variables de Vite se fijan al construir.	Ejecutar un nuevo despliegue del Static Site.

Síntoma	Causa probable	Solución
Las clientas no reciben correos.	No hay `SMTP_HOST` (modo `consola`), `CORREO_MODO` está forzado a `consola` o `archivo`, las credenciales SMTP son incorrectas, o el proveedor de hospedaje restringe los puertos SMTP salientes en el plan contratado.	Revisar el servicio "Correo" en Estado del sistema y el registro (`[correo] ATENCIÓN: no hay SMTP configurado…`); definir `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` y quitar `CORREO_MODO`; verificar que el plan de Render permita conexiones salientes al puerto SMTP usado.
Los enlaces de los correos llevan a `localhost`.	Falta `TIENDA_URL`.	Definir `TIENDA_URL=https://distribuidora-tienda.onrender.com` y volver a desplegar la API.
La API no arranca y el registro muestra `[migración] No se pudo preparar la base de datos: …`.	Datos de conexión incorrectos, la base no responde o una migración falló.	Revisar las variables `DB_*` y el mensaje; corregir la migración en un archivo nuevo si es necesario y volver a desplegar.
Los pagos con tarjeta quedan pendientes y Stripe muestra fallas de entrega del webhook (400).	`STRIPE_WEBHOOK_SECRET` no corresponde al endpoint, o la URL del webhook es incorrecta.	Copiar de nuevo el *Signing secret* del endpoint `…/api/webhook` y volver a desplegar la API.
`npm run respaldo` informa que no se pudo ejecutar `pg_dump` o que la versión del servidor no coincide.	`pg_dump` no está instalado o es más antiguo que la base.	Instalar una versión de PostgreSQL igual o superior a la de Neon o indicar su carpeta `bin` en `PG_BIN`.
Estado del sistema advierte "Faltan en la base: …".	Se modificó un modelo sin escribir la migración correspondiente.	Escribir una migración idempotente (sección 6.3) y desplegar.

---

11. Documentos relacionados

- Arquitectura: componentes, capas del backend y flujo de una compra.
- Modelo de datos: tablas que crean las migraciones.
- Reglas de negocio: vencimiento de pedidos, reembolsos, envío e IVA.
- API: endpoints, incluidos los de salud y el webhook.
- Seguridad: manejo de secretos, sesiones y límite de intentos.
- Pruebas: ejecución de las pruebas antes de desplegar.